

软件源代码缺陷检测

TDinternal

检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:31:49



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

12 C/C++缺陷类型-不匹配的类型转换

12.1 分项汇总表

序号	C/C++缺陷类型-不匹配的类型转换	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4749	长整数变量赋值给短整数变量必须强制转换	82	严重
2	WK4753	禁止使用无实质作用的类型转换	265	严重
3	WK4750	double 型变量赋值给 float 型变量必须强制转换	2	严重
4	WK4794	指针或引用类型的转换中禁止移除 const 或 volatile 属性	77	严重
5	WK4793	禁止将不相关指针类型强制转换为对象指针类型	13	严重

12.2 缺陷/安全漏洞详解

12.2.1 严重类

12.2.1.1 WK4749 长整数变量赋值给短整数变量必须强制转换

缺陷数量：82 个		
缺陷描述： 长整数变量赋值给短整数变量必须强制转换。		
修复意见： 无		
1	缺陷发生位置： /tools/shell/src/shellWebsocket.c	行号： 25
	步骤描述： 从'unsigned long'到类型'int'的缩小转换	
	代码片段： 23: int shell_conn_ws_server(bool first) { 24: char cuttedDsn[SHELL_WS_DSN_BUFF] = {0}; 25: int dsnLen = strlen(shell.args.dsn); 26: snprintf(cuttedDsn, 27: ((dsnLen-SHELL_WS_DSN_MASK) > SHELL_WS_DSN_BUFF)?	
	污染路径： 无	
2	缺陷发生位置： /utils/test/c/tmq_taosx_ci.c	行号： 895
	步骤描述： 从'long'到类型'int'的缩小转换	
	代码片段： 893: g_conf.dropTable = true; 894: } else if (strcmp(argv[i], "-sv") == 0) { 895: g_conf.srcVgroups = atol(argv[++i]); 896: } else if (strcmp(argv[i], "-dv") == 0) { 897: g_conf.dstVgroups = atol(argv[++i]);	
	污染路径： 无	

3	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1317
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1315: 1316: // show 1317: int count = strlen(part); 1318: shellInsertStr(cmd, part, count); 1319: cntDel = count; // next press tab delete current append count	
	污染路径: 无	
4	缺陷发生位置: /source/common/src/trow.c	行号: 574
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 572: switch (nOffset) { 573: case 0: 574: *pDestByte = ((*pDestByte) & 0x7F) (valType << 7); 575: // set the value and clear other partitions for offset 0 576: // *pDestByte = (valType << 7);	
	污染路径: 无	
5	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 700
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 698: while (*p != 0 && *p++ != c) { 699: } 700: return p - a; 701: } 702: static int idxCacheJsonTermCompareImpl(char* a, char* b) {	
	污染路径: 无	
6	缺陷发生位置:	行号:

	/source/util/test/trefTest.c	38
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 36: for (int i = 0; i < pSpace->steps; ++i) { 37: printf("a"); 38: id = taosRand() % pSpace->refNum; 39: if (pSpace->rid[id] <= 0) { 40: pSpace->p[id] = taosMemoryMalloc(128);	
	污染路径: 无	
7	缺陷发生位置: /source/os/src/osTimezone.c	行号: 820
	步骤描述: 从'int'到类型'int8_t' (aka 'signed char')的缩小转换	
	代码片段: 818: tz += daylight; 819: sprintf(outTimezoneStr, "%s (%s, %s%02d00)", buf, tzname[daylight], tz >= 0 ? "+" : "-", abs(tz)); 820: *outDaylight = daylight; 821: 822: #endif	
	污染路径: 无	
8	缺陷发生位置: /source/common/src/tdatablock.c	行号: 333
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 331: uint8_t* p = (uint8_t*)pSource->>nullbitmap; 332: pColumnInfoData->>nullbitmap[BitmapLen(numOfRow1) - 1] &= (0B11111111 << shiftBits); // clear remind bits 333: pColumnInfoData->>nullbitmap[BitmapLen(numOfRow1) - 1] = (p[0] >> remindBits); // copy remind bits 334: 335: if (BitmapLen(numOfRow1) == BitmapLen(total)) {	
	污染路径: 无	
9	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 113

	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 111: int32_t streamSnapHandleInit(SStreamSnapHandle* pHandle, char* path, int64_t chkpld, void* pMeta) { 112: // impl later 113: int len = strlen(path); 114: char* tdir = taosMemoryCalloc(1, len + 256); 115: memcpy(tdir, path, len); </pre>	
	污染路径: 无	
10	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 201
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 199: // insert a new word 200: bool deleteWord(STire* tire, char* word) { 201: int len = strlen(word); 202: if (len >= MAX_WORD_LEN) { 203: return false; </pre>	
	污染路径: 无	
11	缺陷发生位置: /source/common/src/tglobal.c	行号: 1004
	步骤描述: 从'bool'到类型'char'的缩小转换	
	代码片段: <pre> 1002: tsRpcQueueMemoryAllowed = cfgGetItem(pCfg, "rpcQueueMemoryAllowed")->i64; 1003: 1004: tsSIMDBuiltins = (bool)cfgGetItem(pCfg, "SIMD- builtins")->bval; 1005: tsTagFilterCache = (bool)cfgGetItem(pCfg, "tagFilterCache")->bval; 1006: </pre>	
	污染路径: 无	
12	缺陷发生位置: /source/util/src/tdes.c	行号: 143

	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 141: int32_t i; 142: for (i = 0; i < 8; i++) { 143: char shift_byte = 0x01 << (7 - i); 144: if (shift_byte & input) { 145: printf("1");	
	污染路径: 无	
13	缺陷发生位置:	行号:
	/source/common/src/tglobal.c	1005
	步骤描述: 从'bool'到类型'char'的缩小转换	
	代码片段: 1003: 1004: tsSIMDBuiltins = (bool)cfgGetItem(pCfg, "SIMD- builtins")->bval; 1005: tsTagFilterCache = (bool)cfgGetItem(pCfg, "tagFilterCache")->bval; 1006: 1007: tsEnableMonitor = cfgGetItem(pCfg, "monitor")->bval;	
14	污染路径: 无	
	缺陷发生位置:	行号:
	/include/util/tencode.h	556
	步骤描述: 从'uint64_t' (aka 'unsigned long')到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 554: static FORCE_INLINE int32_t tPutI32v(uint8_t* p, int32_t v) { return tPutU32v(p, ZIGZAGE(int32_t, v)); } 555: 556: static FORCE_INLINE int32_t tPutU64v(uint8_t* p, uint64_t v) { tPutV(p, v); } 557: 558: static FORCE_INLINE int32_t tPutI64v(uint8_t* p, int64_t v) { return tPutU64v(p, ZIGZAGE(int64_t, v)); }	
15	污染路径: expanded from macro 'tPutV'	
	缺陷发生位置:	行号:

	/source/libs/scalar/test/scalar/scalarTests.cpp	1731
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1729: leftv[i][2] = 'a' + i; 1730: leftv[i][3] = 'b' + i; 1731: leftv[i][4] = '0' + i; 1732: varDataSetLen(leftv[i], 3); 1733: }	
	污染路径: 无	
16	缺陷发生位置: /include/util/tencode.h	行号: 552
	步骤描述: 从'uint32_t' (aka 'unsigned int')到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 550: static FORCE_INLINE int32_t tPutl16v(uint8_t* p, int16_t v) { return tPutU16v(p, ZIGZAGE(int16_t, v)); } 551: 552: static FORCE_INLINE int32_t tPutU32v(uint8_t* p, uint32_t v) { tPutV(p, v); } 553: 554: static FORCE_INLINE int32_t tPutl32v(uint8_t* p, int32_t v) { return tPutU32v(p, ZIGZAGE(int32_t, v)); }	
	污染路径: expanded from macro 'tPutV'	
17	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1056
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1054: } 1055: int defaultKeyDecode(void* k, char* buf) { 1056: int len = strlen(buf); 1057: memcpy(k, buf, len); 1058: return len;	
	污染路径: 无	
18	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1051

	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1049: } 1050: int defaultKeyEncode(void* k, char* buf) { 1051: int len = strlen((char*)k); 1052: memcpy(buf, (char*)k, len); 1053: return len;	
	污染路径: 无	
19	缺陷发生位置:	行号:
	/source/util/src/tenv.c	20
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 18: #include "tconfig.h" 19: 20: static char toLowChar(char c) { return (c > 'Z' c < 'A' ? c : (c - 'A' + 'a')); } 21: 22: int32_t taosEnvNameToCfgName(const char *envNameStr, char *cfgNameStr, int32_t cfgNameMaxLen) {	
20	污染路径: 无	
	缺陷发生位置:	行号:
	/include/util/tencode.h	233
	步骤描述: 从'int'到类型'uint16_t' (aka 'unsigned short')的缩小转换	
21	代码片段: 231: static FORCE_INLINE int32_t tEncodeU16v(SEncoder* pCoder, uint16_t val) { TD_ENCODE_VARIANT_MACRO(pCoder, val); } 232: static FORCE_INLINE int32_t tEncodeI16v(SEncoder* pCoder, int16_t val) { 233: return tEncodeU16v(pCoder, ZIGZAGE(int16_t, val)); 234: } 235: // 32v	
	污染路径: expanded from macro 'ZIGZAGE'	
	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tq/tqRead.c	933
	步骤描述: 从'bool'到类型'char'的缩小转换	
	代码片段: <pre> 931: bool currentRowAssigned = ! COL_VAL_IS_NONE(&colVal); 932: if (currentRowAssigned != assigned[j]) { 933: assigned[j] = currentRowAssigned; 934: buildNew = true; 935: } </pre>	
22	污染路径: 无	
	缺陷发生位置: /source/util/src/tcompression.c	行号: 519
	步骤描述: 从'int'到类型'char'的缩小转换	
23	代码片段: <pre> 517: t = ((uint8_t)1 << (2 * (i % ele_per_byte))) - 1; 518: /* t = (~(((uint8_t)1) << (7-i%BITS_PER_BYTE))); */ 519: output[pos] &= t; 520: } else if (input[i] == TSDB_DATA_BOOL_NULL) { 521: t = ((uint8_t)2 << (2 * (i % ele_per_byte))); </pre>	
	污染路径: 无	
	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 386
24	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: <pre> 384: *dst = taosMemoryCalloc(1, bufSize + 1); 385: idxInt2str(*(int64_t*)src, *dst, -1); 386: tlen = strlen(*dst); 387: break; 388: case TSDB_DATA_TYPE_BOOL: </pre>	
	污染路径: 无	
24	缺陷发生位置: /source/util/src/tcompression.c	行号: 175
	步骤描述: 从'int'到类型'char'的缩小转换	

	代码片段: <pre> 173: elems + 1 <= selector_to_elems[(int32_t) (bit_to_selector[(int32_t)tmp_bit]))] { 174: // If can hold another one. 175: selector = selector > bit_to_selector[(int32_t)tmp_bit] ? selector : bit_to_selector[(int32_t)tmp_bit]; 176: elems++; 177: bit = bit_per_integer[(int32_t)selector]; </pre>	
	污染路径: 无	
25	缺陷发生位置: /include/util/tcoding.h	行号: 247
	步骤描述: 从'int'到类型'uint16_t' (aka 'unsigned short')的缩小转换	
	代码片段: <pre> 245: // ---- Variant I16 246: static FORCE_INLINE int32_t taosEncodeVariantI16(void **buf, int16_t value) { 247: return taosEncodeVariantU16(buf, ZIGZAGE(int16_t, value)); 248: } 249: </pre>	
	污染路径: expanded from macro 'ZIGZAGE'	
26	缺陷发生位置: /source/util/src/tdes.c	行号: 213
	步骤描述: 从'int'到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: <pre> 211: } 212: 213: key_sets[0].d[3] = (key_sets[0].k[6] & 0x0F) << 4; 214: 215: for (i = 1; i < 17; i++) { </pre>	
	污染路径: 无	
27	缺陷发生位置: /source/os/src/osTime.c	行号: 627
	步骤描述:	

	从'long'到类型'int'的缩小转换	
	代码片段: 625: } 626: 627: result->tm_mday = days + 1; /* Add 1 since our 'days' is zero-based. */ 628: result->tm_year -= 1900; /* Surprisingly tm_year is year-1900. */ 629: #else	
	污染路径: 无	
28	缺陷发生位置: /source/common/src/trow.c	行号: 591
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 589: break; 590: case 4: 591: *pDestByte = ((*pDestByte) & 0xF7) (valType << 3); 592: // *pDestByte = (valType << 3); 593: break;	
	污染路径: 无	
29	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 71
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 69: while (val != 0) { 70: new_val = val / 10; 71: *--p = '0' + (char)(val - new_val * 10); 72: val = new_val; 73: }	
	污染路径: 无	
30	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1729
	步骤描述: 从'int'到类型'char'的缩小转换	

	代码片段: 1727: char rightv[3][5] = {0}; 1728: for (int32_t i = 0; i < 5; ++i) { 1729: leftv[i][2] = 'a' + i; 1730: leftv[i][3] = 'b' + i; 1731: leftv[i][4] = '0' + i; 	
	污染路径: 无	
31	缺陷发生位置: /source/os/src/osTime.c	行号: 604
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 602: * where sunday = 0, so to calculate the day of the week we have to add 4 603: * and take the modulo by 7. */ 604: result->tm_wday = (days + 4) % 7; 605: 606: /* Calculate the current year. */	
	污染路径: 无	
32	缺陷发生位置: /source/util/src/tbase64.c	行号: 38
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 36: if (vlen > 1) oval = value[1] >> 4; 37: *out++ = basis_64[oval]; 38: *out++ = (vlen < 2) ? '=' : basis_64[(value[1] << 2) & 0x3C]; 39: *out++ = '='; 40: }	
	污染路径: 无	
33	缺陷发生位置: /source/os/src/osTime.c	行号: 597
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 595:	

	596: result->tm_isdst = dst; 597: result->tm_hour = seconds / secsHour; 598: result->tm_min = (seconds % secsHour) / secsMin; 599: result->tm_sec = (seconds % secsHour) % secsMin;	
	污染路径: 无	
34	缺陷发生位置: /source/util/src/tcompare.c	行号: 1073
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1071: } 1072: 1073: char rejectList[2] = {toupper(c), tolower(c)}; 1074: 1075: str += nMatchChar;	
	污染路径: 无	
35	缺陷发生位置: /source/os/src/osString.c	行号: 319
	步骤描述: 从'int'到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 317: ln = src[i + 1] > '9' ? src[i + 1] - 'a' + 10 : src[i + 1] - '0'; 318: 319: out = (hn << 4) ln; 320: memcpy(dst + j, &out, 1); 321: }	
	污染路径: 无	
36	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 883
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 881: 882: char* serialized = walMetaSerialize(pWal); 883: int len = strlen(serialized); 884: if (len != taosWriteFile(pMetaFile, serialized, len)) {	

	885: wError("vgld:%d, failed to write file due to %s. file: %s", pWal->cfg.vgld, strerror(errno), tmpFnameStr);	
	污染路径: 无	
37	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 897
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 895: g_conf.srcVgroups = atol(argv[++i]); 896: } else if (strcmp(argv[i], "-dv") == 0) { 897: g_conf.dstVgroups = atol(argv[++i]); 898: } else if (strcmp(argv[i], "-t") == 0) { 899: g_conf.subTable = true;	
	污染路径: 无	
38	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 180
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 178: static __thread int id = 0; 179: if (id != 0) return id; 180: id = syscall(SYS_gettid); 181: return id; 182: }	
	污染路径: 无	
39	缺陷发生位置: /source/common/src/trow.c	行号: 583
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 581: break; 582: case 2: 583: *pDestByte = ((*pDestByte) & 0xDF) (valType << 5); 584: // *pDestByte = (valType << 5); 585: break;	

	污染路径: 无	
40	缺陷发生位置: /source/util/src/tqueue.c	行号: 124
	步骤描述: 从'EQLtype'到类型'int8_t' (aka 'signed char')的缩小转换	
	代码片段: 122: pNode->dataSize = dataSize; 123: pNode->size = size; 124: pNode->itype = itype; 125: pNode->timestamp = taosGetTimestampUs(); 126:	
	污染路径: 无	
41	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1166
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 1164: p = taosDecodeFixedU64(p, &sess->key.groupId); 1165: p = taosDecodeFixedI64(p, &sess->opNum); 1166: return p - buf; 1167: } 1168: int stateSessionKeyToString(void* k, char* buf) {	
	污染路径: 无	
42	缺陷发生位置: /source/libs/tdb/src/db/tdbDb.c	行号: 28
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 26: *ppDb = NULL; 27: 28: dsize = strlen(dbname); 29: zsize = sizeof(*pDb) + dsize * 2 + strlen(TDB_JOURNAL_NAME) + 3; 30:	
	污染路径: 无	
43	缺陷发生位置:	行号:

	/source/os/src/osTime.c	599
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 597: result->tm_hour = seconds / secsHour; 598: result->tm_min = (seconds % secsHour) / secsMin; 599: result->tm_sec = (seconds % secsHour) % secsMin; 600: 601: /* 1/1/1970 was a Thursday, that is, day 4 from the POV of the tm structure	
44	污染路径: 无	
	缺陷发生位置: /contrib/test/cos/main.c	行号: 1299
	步骤描述: 从'int'到类型'char'的缩小转换	
45	代码片段: 1297: str = (char *)cos_palloc(p, len + 1); 1298: for (; i < len; i++) { 1299: str[i] = 'a' + rand() % 32; 1300: } 1301: str[len] = '\0';	
	污染路径: 无	
	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 349
46	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 347: // enum first layer 348: for (int i = 0; i < CHAR_CNT; i++) { 349: pre[0] = FIRST_ASCII + i; 350: 351: // each node	
	污染路径: 无	
46	缺陷发生位置: /include/util/tcoding.h	行号: 261
	步骤描述: 从'unsigned int'到类型'uint8_t' (aka 'unsigned char')的缩小转换	

	代码片段: 259: int32_t i = 0; 260: while (value >= ENCODE_LIMIT) { 261: if (buf != NULL) ((uint8_t *) (*buf))[i] = (value ENCODE_LIMIT); 262: value >>= 7; 263: i++;	
	污染路径: 无	
47	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 908
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 906: static int32_t sortBlocksToExtSource(SSortHandle* pHandle, SArray* aBlk, SBlockOrderInfo* order, SArray* aExtSrc) { 907: int32_t code = TSDB_CODE_SUCCESS; 908: int pgHeaderSz = sizeof(int32_t) + sizeof(int32_t) * taosArrayGetSize(pHandle->pDataBlock->pDataBlock); 909: int32_t rowCap = blockDataGetCapacityInRow(pHandle->pDataBlock, pHandle->pageSize, pgHeaderSz); 910: blockDataEnsureCapacity(pHandle->pDataBlock, rowCap);	
	污染路径: 无	
48	缺陷发生位置: /source/os/src/osString.c	行号: 448
	步骤描述: 从'uint32_t' (aka 'unsigned int')到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 446: ASSERT(tmp <= UCHAR_MAX); 447: #endif 448: return tmp; 449: } 450:	
	污染路径: 无	
49	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 834

	步骤描述: 从'bool'到类型'char'的缩小转换	
	代码片段: <pre> 832: bool currentRowAssigned = ! COL_VAL_IS_NONE(&colVal); 833: if (currentRowAssigned != assigned[j]) { 834: assigned[j] = currentRowAssigned; 835: buildNew = true; 836: }</pre>	
	污染路径: 无	
50	缺陷发生位置: /include/util/tcoding.h	行号: 268
	步骤描述: 从'uint32_t' (aka 'unsigned int')到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: <pre> 266: 267: if (buf != NULL) { 268: ((uint8_t *) (*buf))[i] = value; 269: *buf = POINTER_SHIFT(*buf, i + 1); 270: }</pre>	
	污染路径: 无	
51	缺陷发生位置: /source/util/src/tcompression.c	行号: 523
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 521: t = ((uint8_t)2 << (2 * (i % ele_per_byte))); 522: /* t = (~(((uint8_t)1) << (7-i%BITS_PER_BYTE))); */ 523: output[pos] = t; 524: } else { 525: uError("Invalid compress bool value:%d", output[pos]);</pre>	
	污染路径: 无	
52	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1106
	步骤描述: 从'long'到类型'int'的缩小转换	

	代码片段: 1104: p = taosDecodeFixedl64(p, &key->key.ts); 1105: p = taosDecodeFixedl64(p, &key->opNum); 1106: return p - buf; 1107: } 1108:	
	污染路径: 无	
53	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 805
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 803: for (int i = 0; i < 7; ++i) { 804: // sprintf(&key[count - 2], "%c", i); 805: key[count - 2] = '0' + i; 806: 807: ret = tdbTblInsert(pDb, key, count, NULL, 0, txn);	
	污染路径: 无	
54	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 829
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 827: tColDataGetValue(pCol, i, &colVal); 828: if (curRow == 0) { 829: assigned[j] = !COL_VAL_IS_NONE(&colVal); 830: buildNew = true; 831: } else {	
	污染路径: 无	
55	缺陷发生位置: /source/common/src/tdatablock.c	行号: 332
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 330: 331: uint8_t* p = (uint8_t*)pSource->>nullbitmap; 332: pColumnInfoData->>nullbitmap[BitmapLen(numOfRow1)	

	- 1] &= (0B11111111 << shiftBits); // clear remind bits 333: pColumnInfoData->nullbitmap[BitmapLen(numOfRow1) - 1] = (p[0] >> remindBits); // copy remind bits 334:	
	污染路径: 无	
56	缺陷发生位置: /source/common/src/tdatablock.c	行号: 1546
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1544: int mid = start + (end - start) / 2; 1545: // data size + var data type columns offset + fixed data type columns bitmap len 1546: int midSize = rowSize * mid + numVarCols * sizeof(int32_t) * mid + numFixCols * BitmapLen(mid); 1547: if (midSize > payloadSize) { 1548: result = mid;	
	污染路径: 无	
57	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1551
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1549: for (int32_t i = 0; i < 5; ++i) { 1550: rightv[i][2] = rightv[i][3] = '0'; 1551: rightv[i][4] = '0' + i; 1552: varDataSetLen(rightv[i], 3); 1553: }	
	污染路径: 无	
58	缺陷发生位置: /source/os/src/osTime.c	行号: 598
	步骤描述: 从'long'到类型'int'的缩小转换	
	代码片段: 596: result->tm_isdst = dst; 597: result->tm_hour = seconds / secsHour; 598: result->tm_min = (seconds % secsHour) / secsMin;	

	599: result->tm_sec = (seconds % secsHour) % secsMin; 600:	
	污染路径: 无	
59	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 366
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 364: int ret = 0; 365: char* p = idxPackJsonData(tem); 366: int sz = strlen(p); 367: 368: int64_t st = taosGetTimestampUs();	
	污染路径: 无	
60	缺陷发生位置: /source/util/src/tdes.c	行号: 209
	步骤描述: 从'int'到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 207: 208: for (i = 0; i < 3; i++) { 209: key_sets[0].d[i] = (key_sets[0].k[i + 3] & 0x0F) << 4; 210: key_sets[0].d[i] = (key_sets[0].k[i + 4] & 0xF0) >> 4; 211: }	
	污染路径: 无	
61	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 237
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 235: STireNode* c; 236: char word[MAX_WORD_LEN]; 237: int len = strlen(prefix); 238: for (int i = 0; i < CHAR_CNT; i++) { 239: c = nodes[i];	
	污染路径:	

	无	
62	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1730
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1728: for (int32_t i = 0; i < 5; ++i) { 1729: leftv[i][2] = 'a' + i; 1730: leftv[i][3] = 'b' + i; 1731: leftv[i][4] = '0' + i; 1732: varDataSetLen(leftv[i], 3);	
	污染路径: 无	
63	缺陷发生位置: /include/util/tencode.h	行号: 548
	步骤描述: 从'uint16_t' (aka 'unsigned short')到类型'uint8_t' (aka 'unsigned char')的缩小转换	
	代码片段: 546: } 547: 548: static FORCE_INLINE int32_t tPutU16v(uint8_t* p, uint16_t v) { tPutV(p, v); } 549: 550: static FORCE_INLINE int32_t tPutI16v(uint8_t* p, int16_t v) { return tPutU16v(p, ZIGZAGE(int16_t, v)); }	
	污染路径: expanded from macro 'tPutV'	
64	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 366
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 364: sprintf(secondLine, " * The %s CLI supports tab completion for a variety of items, ", shell.info.cusName); 365: printf("%s", secondLine); 366: int secondLineLen = strlen(secondLine); 367: while (87 - (secondLineLen++) > 0) { 368: printf(" ");	
	污染路径: 无	

65	缺陷发生位置: /include/util/tencode.h	行号: 550
	步骤描述: 从'int'到类型'uint16_t' (aka 'unsigned short')的缩小转换	
	代码片段: 548: static FORCE_INLINE int32_t tPutU16v(uint8_t* p, uint16_t v) { tPutV(p, v); } 549: 550: static FORCE_INLINE int32_t tPutI16v(uint8_t* p, int16_t v) { return tPutU16v(p, ZIGZAGE(int16_t, v)); } 551: 552: static FORCE_INLINE int32_t tPutU32v(uint8_t* p, uint32_t v) { tPutV(p, v); }	
	污染路径: expanded from macro 'ZIGZAGE'	
66	缺陷发生位置: /source/util/test/trefTest.c	行号: 74
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 72: printf("a"); 73: 74: id = taosRand() % pSpace->refNum; 75: void *p = taosAcquireRef(pSpace->rsetId, (int64_t)pSpace->p[id]); 76: if (p) {	
	污染路径: 无	
67	缺陷发生位置: /include/util/tcoding.h	行号: 106
	步骤描述: 从'int'到类型'uint16_t' (aka 'unsigned short')的缩小转换	
	代码片段: 104: // ---- Fixed I16 105: static FORCE_INLINE int32_t taosEncodeFixedI16(void **buf, int16_t value) { 106: return taosEncodeFixedU16(buf, ZIGZAGE(int16_t, value)); 107: } 108:	

	污染路径: expanded from macro 'ZIGZAGE'	
68	缺陷发生位置: /source/common/src/trow.c	行号: 579
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 577: break; 578: case 1: 579: *pDestByte = ((*pDestByte) & 0xBF) (valType << 6); 580: // *pDestByte = (valType << 6); 581: break;	
	污染路径: 无	
69	缺陷发生位置: /source/common/src/trow.c	行号: 587
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 585: break; 586: case 3: 587: *pDestByte = ((*pDestByte) & 0xEF) (valType << 4); 588: // *pDestByte = (valType << 4); 589: break;	
	污染路径: 无	
70	缺陷发生位置: /source/util/test/trefTest.c	行号: 55
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: 53: for (int i = 0; i < pSpace->steps; ++i) { 54: printf("d"); 55: id = taosRand() % pSpace->refNum; 56: if (pSpace->rid[id] > 0) { 57: code = taosRemoveRef(pSpace->rsetId, pSpace->rid[id]);	
	污染路径:	

	无	
71	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 890
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 888: if (pCollInfoData->varmeta.offset[row] != -1) { 889: char* p = colDataGetData(pCollInfoData, row); 890: sz += varDataTLen(p); 891: } 892:	
	污染路径: expanded from macro 'varDataTLen'	
72	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 130
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 128: // insert a new word 129: bool insertWord(STire* tire, char* word) { 130: int len = strlen(word); 131: if (len >= MAX_WORD_LEN) { 132: return false; 133: }	
	污染路径: 无	
73	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1512
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 1510: for (int32_t i = 0; i < 5; ++i) { 1511: rightv[i][2] = rightv[i][3] = '0'; 1512: rightv[i][4] = '0' + i; 1513: varDataSetLen(rightv[i], 3); 1514: }	
	污染路径: 无	
74	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 392
	步骤描述:	

	从'unsigned long'到类型'int'的缩小转换	
	代码片段: 390: *dst = taosMemoryCalloc(1, bufSize + 1); 391: idxInt2str(*(uint8_t*)src, *dst, 1); 392: tlen = strlen(*dst); 393: break; 394: case TSDB_DATA_TYPE_TINYINT:	
	污染路径: 无	
75	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1344
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 1342: 1343: // show 1344: int count = strlen(part); 1345: shellInsertStr(cmd, part, count); 1346: cntDel = count; // next press tab delete current append count	
	污染路径: 无	
76	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 66
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 64: *p = '\0'; 65: new_val = (int64_t)(uval / 10); 66: *--p = '0' + (char)(uval - (uint64_t)new_val * 10); 67: val = new_val; 68:	
	污染路径: 无	
77	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 928
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: 926: tRowGet(pRow, pTSchema, j, &colVal);	

	<pre> 927: if (curRow == 0) { 928: assigned[j] = !COL_VAL_IS_NONE(&colVal); 929: buildNew = true; 930: } else { </pre>	
	污染路径: 无	
78	缺陷发生位置: /source/util/src/tcompression.c	行号: 515
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 513: if (input[i] == 1) { 514: t = (((uint8_t)1) << (2 * (i % ele_per_byte))); 515: output[pos] = t; 516: } else if (input[i] == 0) { 517: t = ((uint8_t)1 << (2 * (i % ele_per_byte))) - 1; </pre>	
	污染路径: 无	
79	缺陷发生位置: /source/os/src/osString.c	行号: 499
	步骤描述: 从'unsigned int'到类型'int'的缩小转换	
	代码片段: <pre> 497: uint8_t *byte = tmp + *size - 1; 498: 499: for (int i = n - 1; i >= 0; i--) { 500: if (z[i] >= 'a') { 501: *byte = ((uint8_t)(10 + (z[i] - 'a'))) << (num * 4)); </pre>	
	污染路径: 无	
80	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 248
	步骤描述: 从'int'到类型'char'的缩小转换	
	代码片段: <pre> 246: memset(word, 0, tListLen(word)); 247: strncpy(word, prefix, len); 248: word[len] = FIRST_ASCII + i; // append current char 249: 250: // chain middle node </pre>	

	污染路径: 无	
81	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 291
	步骤描述: 从'unsigned long'到类型'int'的缩小转换	
	代码片段: 289: uint64_t l = *(uint64_t*)a; 290: uint64_t r = *(uint64_t*)b; 291: return l - r; 292: } 293: int32_t idxConvertData(void* src, int8_t type, void** dst) {	
	污染路径: 无	
82	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1027
	步骤描述: 从'size_t' (aka 'unsigned long')到类型'int'的缩小转换	
	代码片段: 1025: 1026: int defaultKeyComp(void* state, const char* aBuf, size_t aLen, const char* bBuf, size_t bLen) { 1027: int len = aLen < bLen ? aLen : bLen; 1028: int ret = memcmp(aBuf, bBuf, len); 1029: if (ret == 0) {	
	污染路径: 无	

12.2.1.2 WK4753 禁止使用无实质作用的类型转换

缺陷数量：265 个		
缺陷描述: 禁止使用无实质作用的类型转换。		
修复意见: 无		
1	缺陷发生位置:	行号:

	/source/util/src/tskiplist.c		680
	步骤描述: 强制转换为相同类型.		
	代码片段: 678: 679: if (hasDup && (dupMode != SL_ALLOW_DUP_KEY)) { 680: if (dupMode == SL_UPDATE_DUP_KEY) { 681: if (isForward) { 682: pNode = SL_NODE_GET_FORWARD_POINTER(direction[0], 0);		
	污染路径: expanded from macro 'SL_UPDATE_DUP_KEY'		
2	缺陷发生位置: /source/libs/transport/src/thttp.c		行号: 113
	步骤描述: 强制转换为相同类型.		
	代码片段: 111: 112: gzipStream.next_in = (Bytef*)pSrc; 113: gzipStream.avail_in = (uLong)srcLen; 114: gzipStream.next_out = (Bytef*)pDest; 115: gzipStream.avail_out = (uLong)(destLen);		
	污染路径: 无		
3	缺陷发生位置: /source/os/src/osAtomic.c		行号: 560
	步骤描述: 强制转换为相同类型.		
	代码片段: 558: return __sync_fetch_and_add((ptr), (val)); 559: #else 560: return __atomic_fetch_add((void**)(ptr), (size_t)(val), __ATOMIC_SEQ_CST); 561: #endif 562: }		
	污染路径: 无		
4	缺陷发生位置: /source/util/src/tskiplist.c		行号: 522
	步骤描述:		

	强制转换为相同类型.	
	代码片段: 520: int32_t level = pNode->level; 521: uint8_t dupMode = SL_DUP_MODE(pSkipList); 522: ASSERT(dupMode != SL_DISCARD_DUP_KEY && dupMode != SL_UPDATE_DUP_KEY); 523: 524: for (int32_t j = level - 1; j >= 0; --j) {	
	污染路径: expanded from macro 'SL_DISCARD_DUP_KEY'	
5	缺陷发生位置: /source/common/src/tname.c	行号: 309
	步骤描述: 强制转换为相同类型.	
	代码片段: 307: taosStringBuilderAppendStringLen(&sb, tagKv->value, tagKv->length); 308: } else { 309: taosStringBuilderAppendStringLen(&sb, (char*)(&(tagKv->value)), tagKv->length); 310: } 311: }	
	污染路径: 无	
6	缺陷发生位置: /source/util/src/talgo.c	行号: 35
	步骤描述: 强制转换为相同类型.	
	代码片段: 33: } 34: 35: if (comparFn(elePtrAt(src, size, mid), elePtrAt(src, size, e), param) > 0) { 36: doswap(elePtrAt(src, size, mid), elePtrAt(src, size, s), size, buf); 37: doswap(elePtrAt(src, size, mid), elePtrAt(src, size, e), size, buf);	
	污染路径: expanded from macro 'elePtrAt'	
7	缺陷发生位置:	行号:

	/source/util/src/tworker.c	195
	步骤描述: 强制转换为相同类型.	
	代码片段: 193: 194: while (1) { 195: if (taosReadQitemFromQset(pool->qset, (void **)&msg, &qinfo) == 0) { 196: uInfo("worker:%s:%d qset:%p, got no message and exiting, thread:%08" PRIu64, pool->name, worker->id, pool->qset, 197: worker->pid);	
	污染路径: 无	
8	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 562
	步骤描述: 强制转换为相同类型.	
	代码片段: 560: } 561: 562: fnInfo("udf init succeeded. name %s type %d context %p", udf->name, udf->scriptType, (void *)udf->scriptUdfCtx); 563: return 0; 564: }	
	污染路径: 无	
9	缺陷发生位置: /source/util/src/tides.c	行号: 359
	步骤描述: 强制转换为相同类型.	
	代码片段: 357: column = ((er[1] & 0xE0) >> 5); 358: 359: ser[0] = (uint8_t)S2[row * 16 + column]; 360: 361: // Byte 2	
	污染路径: 无	
10	缺陷发生位置:	行号:

	/utils/test/c/varbinary_test.c	410
	步骤描述: 强制转换为相同类型.	
	代码片段: 408: params[j].is_null = ((i == j) ? &is_null : 0); 409: } 410: v.b = (int8_t)i % 2; 411: 412: for (int j = 0; j < vbinLen; ++j) {	
	污染路径: 无	
11	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 49
	步骤描述: 强制转换为相同类型.	
	代码片段: 47: 48: for (int32_t i = 100; i < 150; ++i) { 49: taosHashRemove(hashTable, (const char*)&i, sizeof(int32_t)); 50: } 51:	
	污染路径: 无	
12	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 3782
	步骤描述: 强制转换为相同类型.	
	代码片段: 3780: if (pSupp->tbCountSlotId != -1) { 3781: SColumnInfoData* colInfoData = taosArrayGet(pRes->pDataBlock, pSupp->tbCountSlotId); 3782: colDataSetVal(colInfoData, 0, (char*)&count, false); 3783: } 3784: pRes->info.rows = 1;	
	污染路径: 无	
13	缺陷发生位置: /source/util/src/tmd5.c	行号: 112
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 110: if ((mdContext->i[0] + ((uint32_t)inLen << 3)) < mdContext->i[0]) mdContext->i[1]++; 111: mdContext->i[0] += ((uint32_t)inLen << 3); 112: mdContext->i[1] += ((uint32_t)inLen >> 29); 113: 114: while (inLen--) {	
	污染路径: 无	
14	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbRead.c	5452
	步骤描述: 强制转换为相同类型.	
	代码片段: 5450: 5451: while (pStatus->pTableIter != NULL) { 5452: STableBlockScanInfo* pBlockScanInfo = *(STableBlockScanInfo**)pStatus->pTableIter; 5453: 5454: STbData* d = NULL;	
	污染路径: 无	
15	缺陷发生位置:	行号:
	/source/util/src/tdes.c	389
	步骤描述: 强制转换为相同类型.	
	代码片段: 387: column = ((er[3] & 0x78) >> 3); 388: 389: ser[2] = ((uint8_t)S5[row * 16 + column] << 4); 390: 391: row = 0;	
	污染路径: 无	
16	缺陷发生位置:	行号:
	/source/libs/index/src/indexComm.c	71
	步骤描述: 强制转换为相同类型.	
	代码片段:	

	<pre> 69: while (val != 0) { 70: new_val = val / 10; 71: *--p = '0' + (char)(val - new_val * 10); 72: val = new_val; 73: }</pre>	
	污染路径: 无	
17	缺陷发生位置: /source/util/src/tskiplist.c	行号: 72
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 70: } 71: 72: if (SL_IS_THREAD_SAFE(pSkipList)) { 73: pSkipList->lock = (TdThreadRwlock *)taosMemoryCalloc(1, sizeof(TdThreadRwlock)); 74: if (pSkipList->lock == NULL) {</pre>	
	污染路径: expanded from macro 'SL_IS_THREAD_SAFE' expanded from macro 'SL_THREAD_SAFE'	
18	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 1956
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 1954: int32_t len = snprintf(userConn, sizeof(userConn), "%s@", pCxt->pParseCxt->pUser); 1955: taosGetFqdn(userConn + len); 1956: char* pUserConn = taosStrdup((void*)userConn); 1957: if (NULL == pUserConn) { 1958: return TSDB_CODE_OUT_OF_MEMORY;</pre>	
	污染路径: 无	
19	缺陷发生位置: /source/util/src/tmd5.c	行号: 111
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 109: /* update number of bits */</pre>	

	110: if ((mdContext->i[0] + ((uint32_t)inLen << 3)) < mdContext->i[0]) mdContext->i[1]++; 111: mdContext->i[0] += ((uint32_t)inLen << 3); 112: mdContext->i[1] += ((uint32_t)inLen >> 29); 113:	
	污染路径: 无	
20	缺陷发生位置: /source/util/src/tdigest.c	行号: 124
	步骤描述: 强制转换为相同类型.	
	代码片段: 122: taosSort(unmerged_centroids, num_unmerged, sizeof(SCentroid), cmpCentroid); 123: memset(&args, 0, sizeof(SMergeArgs)); 124: args.centroids = (SCentroid *)taosMemoryMalloc((size_t)(sizeof(SCentroid) * t->size)); 125: memset(args.centroids, 0, (size_t)(sizeof(SCentroid) * t->size)); 126:	
	污染路径: 无	
21	缺陷发生位置: /include/common/tdatablock.h	行号: 149
	步骤描述: 强制转换为相同类型.	
	代码片段: 147: pColumnInfoData->info.type == TSDB_DATA_TYPE_USMALLINT); 148: char* p = pColumnInfoData->pData + pColumnInfoData->info.bytes * rowIndex; 149: *(int16_t*)p = *(int16_t*)v; 150: } 151:	
	污染路径: 无	
22	缺陷发生位置: /include/common/tdatablock.h	行号: 155
	步骤描述: 强制转换为相同类型.	

	代码片段: 153: ASSERT(pColumnInfoData->info.type == TSDB_DATA_TYPE_INT pColumnInfoData->info.type == TSDB_DATA_TYPE_UINT); 154: char* p = pColumnInfoData->pData + pColumnInfoData->info.bytes * rowIndex; 155: *(int32_t*)p = *(int32_t*)v; 156: } 157:	
	污染路径: 无	
23	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 1178
	步骤描述: 强制转换为相同类型.	
	代码片段: 1176: addr.sin_family = AF_INET; 1177: addr.sin_addr.s_addr = ipaddr; 1178: addr.sin_port = (uint16_t)htons(pList->port); 1179: 1180: tTrace("%s conn %p try to connect to %s", pTransInst->label, conn, pList->dst);	
	污染路径: 无	
24	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 508
	步骤描述: 强制转换为相同类型.	
	代码片段: 506: int64_t et2 = taosGetTimestampUs(); 507: tsdbDebug("load block index for %d/%d tables completed, elapsed time:%.2f ms, set BrinBlk:%.2f ms, size: %.2f Kb %s", 508: numOfTables, (int32_t)pBlkArray->size, (et1 - st) / 1000.0, (et2 - et1) / 1000.0, 509: pBlkArray->size * sizeof(SBrinBlk) / 1024.0, pReader->idStr); 510:	
	污染路径: 无	

	污染路径: 无	
28	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 151
	步骤描述: 强制转换为相同类型.	
	代码片段: 149: 150: static int32_t tlvEncodeI64(STlvEncoder* pEncoder, int16_t type, int64_t value) { 151: value = htonl(value); 152: return tlvEncodeImpl(pEncoder, type, &value, sizeof(value)); 153: }	
	污染路径: expanded from macro 'htonll'	
29	缺陷发生位置: /source/os/src/osFile.c	行号: 469
	步骤描述: 强制转换为相同类型.	
	代码片段: 467: 468: while (nleft > 0) { 469: nwritten = write(pFile->fd, (void *)tbuf, (uint32_t)nleft); 470: if (nwritten < 0) { 471: if (errno == EINTR) {	
	污染路径: 无	
30	缺陷发生位置: /tests/script/api/stmtBatchTest.c	行号: 730
	步骤描述: 强制转换为相同类型.	
	代码片段: 728: 729: unsigned long long endtime = getCurrentTime(); 730: unsigned long long totalRows = (uint32_t) (totalRowsPerTbl * tableNum); 731: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL,	

	(endtime-starttime)/totalRows); 732:	
	污染路径: 无	
31	缺陷发生位置: /source/util/src/talgo.c	行号: 31
	步骤描述: 强制转换为相同类型.	
	代码片段: 29: int32_t mid = ((int32_t)(e - s) >> 1u) + (int32_t)s; 30: 31: if (comparFn(elePtrAt(src, size, mid), elePtrAt(src, size, s), param) > 0) { 32: doswap(elePtrAt(src, size, mid), elePtrAt(src, size, s), size, buf); 33: }	
	污染路径: expanded from macro 'elePtrAt'	
32	缺陷发生位置: /source/util/src/talgo.c	行号: 42
	步骤描述: 强制转换为相同类型.	
	代码片段: 40: } 41: 42: ASSERT(comparFn(elePtrAt(src, size, mid), elePtrAt(src, size, s), param) <= 0 && 43: comparFn(elePtrAt(src, size, s), elePtrAt(src, size, e), param) <= 0); 44: }	
	污染路径: expanded from macro 'elePtrAt'	
33	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 5379
	步骤描述: 强制转换为相同类型.	
	代码片段: 5377: pTableBlockInfo->numOfFiles += 1; 5378: 5379: int32_t numOfTables =	

	(int32_t)tSimpleHashGetSize(pStatus->pTableMap); 5380: int defaultRows = 4096; 5381:	
	污染路径: 无	
34	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 559
	步骤描述: 强制转换为相同类型.	
	代码片段: 557: if (len < 0) { 558: sclError("charset:%s to %s. val:%s convert ncharTobinary failed.", DEFAULT_UNICODE_ENCODEC, tsCharset, 559: (char *)varDataVal(buf)); 560: taosMemoryFree(t); 561: return NULL;	
	污染路径: 无	
35	缺陷发生位置: /source/util/src/tcrc32c.c	行号: 947
	步骤描述: 强制转换为相同类型.	
	代码片段: 945: next += 4; 946: } while (next < end); 947: crc0 = shift_crc(long_shifts, (uint32_t)(crc0)) ^ crc1; 948: crc0 = shift_crc(long_shifts, (uint32_t)(crc0)) ^ crc2; 949: next += 2 * LONG_SHIFT;	
	污染路径: 无	
36	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 100
	步骤描述: 强制转换为相同类型.	
	代码片段: 98: } 99: int kvDeserial(KV *kv, char *buf) { 100: char *p1 = (char *)buf; 101: // p1 += 1;	

	102: p1 = taosDecodeFixedU64(p1, &kv->k1);	
	污染路径: 无	
37	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 1929
	步骤描述: 强制转换为相同类型.	
	代码片段: 1927: 1928: static int32_t rewriteClentVersionFunc(STranslateContext* pCxt, SNode** pNode) { 1929: char* pVer = taosStrdup((void*)version); 1930: if (NULL == pVer) { 1931: return TSDB_CODE_OUT_OF_MEMORY;	
	污染路径: 无	
38	缺陷发生位置: /source/util/src/tlog.c	行号: 451
	步骤描述: 强制转换为相同类型.	
	代码片段: 449: taosWriteFile(tsLogObj.logHandle->pFile, name, (uint32_t)strlen(name)); 450: sprintf(name, " new log file \n"); 451: taosWriteFile(tsLogObj.logHandle->pFile, name, (uint32_t)strlen(name)); 452: sprintf(name, "===== =====\n"); 453: taosWriteFile(tsLogObj.logHandle->pFile, name, (uint32_t)strlen(name));	
	污染路径: 无	
39	缺陷发生位置: /source/libs/function/src/detail/tminmax.c	行号: 565
	步骤描述: 强制转换为相同类型.	
	代码片段: 563: for (int32_t i = start; i < num + start; ++i) {	

	564: char* p = colDataGetData(pCol, i); 565: if (memcmp((void*)tval, p, pCol->info.bytes) == 0) { 566: return i; 567: }	
	污染路径: 无	
40	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 6212
	步骤描述: 强制转换为相同类型.	
	代码片段: 6210: yymssp += yysize+1; 6211: yypParser->yptos = yymssp; 6212: yymssp->stateno = (YYACTIONTYPE)yyact; 6213: yymssp->major = (YYCODETYPE)yygoto; 6214: yyTraceShift(yypParser, yyact, "... then shift");	
	污染路径: 无	
41	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 540
	步骤描述: 强制转换为相同类型.	
	代码片段: 538: int32_t* offset = (int32_t*)((char*)pPage + startOffset); 539: columnLen = (int32_t)((char*)pPage + startOffset + sizeof(int32_t) * pInfo->rowCapacity); 540: char* data = (char*)((char*)columnLen + sizeof(int32_t)); 541: 542: if (colDataIsNull_s(pCollInfoData, j)) {	
	污染路径: 无	
42	缺陷发生位置: /source/util/src/tdes.c	行号: 379
	步骤描述: 强制转换为相同类型.	
	代码片段: 377: column = ((er[2] & 0x1E) >> 1); 378:	

	379: ser[1] = (uint8_t)S4[row * 16 + column]; 380: 381: // Byte 3	
	污染路径: 无	
43	缺陷发生位置: /source/os/src/osSocket.c	行号: 772
	步骤描述: 强制转换为相同类型.	
	代码片段: 770: } 771: /* bind socket to server address */ 772: if (bind(pSocket->fd, (struct sockaddr *)&serverAdd, sizeof(serverAdd)) < 0) { 773: // printf("bind tcp server socket failed, 0x%x: %hu(%s)", ip, port, strerror(errno)); 774: taosCloseSocket(&pSocket);	
	污染路径: 无	
44	缺陷发生位置: /include/common/tdatablock.h	行号: 142
	步骤描述: 强制转换为相同类型.	
	代码片段: 140: pColumnInfoData->info.type == TSDB_DATA_TYPE_UTINYINT pColumnInfoData->info.type == TSDB_DATA_TYPE_BOOL); 141: char* p = pColumnInfoData->pData + pColumnInfoData->info.bytes * rowIndex; 142: *(int8_t*)p = *(int8_t*)v; 143: } 144:	
	污染路径: 无	
45	缺陷发生位置: /source/util/src/thash.c	行号: 724
	步骤描述: 强制转换为相同类型.	
	代码片段: 722: // release the pNode, return next pNode, and lock the	

	current entry 723: static void *taosHashReleaseNode(SHashObj *pHashObj, void *p, int *slot) { 724: SHashNode *pOld = (SHashNode *)GET_HASH_PNODE(p); 725: SHashNode *prevNode = NULL; 726:	
	污染路径: 无	
46	缺陷发生位置: /source/libs/index/test/jsonUT.cc	行号: 25
	步骤描述: 强制转换为相同类型.	
	代码片段: 23: char buf[256] = {0}; 24: int16_t sz = nColVal; 25: memcpy(buf, (uint16_t*)&sz, 2); 26: memcpy(buf + 2, colVal, nColVal); 27: if (colType == TSDB_DATA_TYPE_BINARY colType == TSDB_DATA_TYPE_GEOMETRY) {	
	污染路径: 无	
47	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 2931
	步骤描述: 强制转换为相同类型.	
	代码片段: 2929: colDataSetVal(pCollInfo, rowIndex, str, false); 2930: } else if (FUNCTION_TYPE_TBUID == pExprInfo->pExpr->_function.functionType) { 2931: colDataSetVal(pCollInfo, rowIndex, (char*)&pUidTagInfo->uid, false); 2932: } else if (FUNCTION_TYPE_VGID == pExprInfo->pExpr->_function.functionType) { 2933: colDataSetVal(pCollInfo, rowIndex, (char*)&pOperator->pTaskInfo->id.vgId, false);	
	污染路径: 无	
48	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 165

	步骤描述: 强制转换为相同类型.	
	代码片段: 163: } 164: 165: if (!isdigit(tz[i])) { 166: return false; 167: }	
	污染路径: expanded from macro 'isdigit' expanded from macro '___isctype'	
49	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1051
	步骤描述: 强制转换为相同类型.	
	代码片段: 1049: } 1050: int defaultKeyEncode(void* k, char* buf) { 1051: int len = strlen((char*)k); 1052: memcpy(buf, (char*)k, len); 1053: return len;	
	污染路径: 无	
50	缺陷发生位置: /source/common/src/ttszip.c	行号: 309
	步骤描述: 强制转换为相同类型.	
	代码片段: 307: } else if (pBlock->tag.nType != TSDB_DATA_TYPE_NULL) { 308: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &pBlock->tag.nLen, sizeof(pBlock->tag.nLen)); 309: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &pBlock->tag.i, (size_t)pBlock->tag.nLen); 310: } else { 311: trueLen = 0;	
	污染路径: 无	
51	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 145

	步骤描述: 强制转换为相同类型.	
	代码片段: 143: 144: char* pData = colDataGetData(p1, 3); 145: printf("the second row of binary:%s, length:%d\n", (char*)varDataVal(pData), varDataLen(pData)); 146: 147: SArray* pOrderInfo = taosArrayInit(3, sizeof(SBlockOrderInfo));	
	污染路径: 无	
52	缺陷发生位置:	行号:
	/source/libs/stream/src/tstreamFileState.c	456
	步骤描述: 强制转换为相同类型.	
	代码片段: 454: memcpy(buf, val, len); 455: buf[len] = 0; 456: maxCheckPointId = atol((char*)buf); 457: taosMemoryFree(val); 458: }	
53	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/function/src/functionMgt.c	279
	步骤描述: 强制转换为相同类型.	
	代码片段: 277: void fmFuncMgtDestroy() { 278: void* m = gFunMgtService.pFuncNameHashTable; 279: if (m != NULL && atomic_val_compare_exchange_ptr((void**)&gFunMgtService.p FuncNameHashTable, m, 0) == m) { 280: taosHashCleanup(m); 281: }	
54	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/src/tdes.c	370

	步骤描述: 强制转换为相同类型.	
	代码片段: 368: column = ((er[2] & 0x80) >> 7); 369: 370: ser[1] = ((uint8_t)S3[row * 16 + column] << 4); 371: 372: row = 0;	
	污染路径: 无	
55	缺陷发生位置:	行号:
	/source/util/src/tcache.c	301
	步骤描述: 强制转换为相同类型.	
	代码片段: 299: SCacheObj *pCacheObj = ps->pCacheObj; 300: 301: if ((int64_t)pNode->expireTime < ps->time && T_REF_VAL_GET(pNode) <= 0) { 302: taosCacheReleaseNode(pCacheObj, pNode); 303:	
56	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/parser/src/parTokenizer.c	519
	步骤描述: 强制转换为相同类型.	
57	代码片段: 517: * .123e4 518: */ 519: if (isdigit(z[1])) { 520: for (i = 2; isdigit(z[i]); i++) { 521: }	
	污染路径: expanded from macro 'isdigit' expanded from macro ' __isctype'	
	缺陷发生位置:	行号:
	/source/libs/function/src/builtinsimpl.c	4098
	步骤描述: 强制转换为相同类型.	

	代码片段: 4096: } 4097: 4098: z += m * hllSigma(buckethisto[0] / (double)m); 4099: double E = (double)llroundl(HLL_ALPHA_INF * m * m / z); 4100:	
	污染路径: 无	
58	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 1488
	步骤描述: 强制转换为相同类型.	
	代码片段: 1486: } 1487: pVgHash = taosHashInit(16, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), true, HASH_NO_LOCK); 1488: taosHashPut(pVgHash, (const char*)&vgData.vgld, sizeof(vgData.vgld), (char*)&vgData, sizeof(vgData)); 1489: 1490: code = rawBlockBindData(pQuery, pTableMeta, pData, NULL, NULL, 0, false);	
	污染路径: 无	
59	缺陷发生位置: /include/common/tatablock.h	行号: 162
	步骤描述: 强制转换为相同类型.	
	代码片段: 160: ASSERT(type == TSDB_DATA_TYPE_BIGINT type == TSDB_DATA_TYPE_UBIGINT type == TSDB_DATA_TYPE_TIMESTAMP); 161: char* p = pColumnInfoData->pData + pColumnInfoData->info.bytes * rowIndex; 162: *(int64_t*)p = *(int64_t*)v; 163: } 164:	
	污染路径: 无	

60	缺陷发生位置： /source/libs/transport/test/cliBench.c	行号： 42
	步骤描述： 强制转换为相同类型.	
	代码片段： 40: tsAsyncLog = 0; 41: // idxDebugFlag = 143; 42: strcpy(tsLogDir, (char *)logDir); 43: taosRemoveDir(tsLogDir); 44: taosMkDir(tsLogDir);	
	污染路径： 无	
61	缺陷发生位置： /source/libs/parser/src/parTokenizer.c	行号： 524
	步骤描述： 强制转换为相同类型.	
	代码片段： 522: 523: if ((z[i] == 'e' z[i] == 'E') && 524: (isdigit(z[i + 1]) ((z[i + 1] == '+' z[i + 1] == '-') && isdigit(z[i + 2])))) { 525: i += 2; 526: while (isdigit(z[i])) {	
	污染路径： expanded from macro 'isdigit' expanded from macro ' __isctype'	
62	缺陷发生位置： /source/libs/executor/src/cachescanoperator.c	行号： 272
	步骤描述： 强制转换为相同类型.	
	代码片段： 270: SExprSupp* pSup = &pInfo->pseudoExprSup; 271: 272: STableKeyInfo* pKeyInfo = &((STableKeyInfo*)pList) [0]; 273: pInfo->pRes->info.id.groupId = pKeyInfo-> groupId; 274:	
	污染路径： 无	

63	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 526
	步骤描述: 强制转换为相同类型.	
	代码片段: 524: - fstStateTotalTransSize(s, version, sizes, nTrans) - (nTrans * oSizes) - oSizes; 525: uint8_t* data = fstSliceData(slice, NULL); 526: return unpackUint64(data + at, (uint8_t)oSizes); 527: } 528: uint64_t fstStateFindInput(FstState* s, FstNode* node, uint8_t b, bool* null) {	
	污染路径: 无	
64	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 504
	步骤描述: 强制转换为相同类型.	
	代码片段: 502: c = taosGetConsoleChar(); 503: 504: if (c == (char)EOF) { 505: return c; 506: }	
	污染路径: 无	
65	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 377
	步骤描述: 强制转换为相同类型.	
	代码片段: 375: struct timeval systemTime; 376: taosGetTimeOfDay(&systemTime); 377: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 378: } 379:	
	污染路径: 无	
66	缺陷发生位置:	行号:

	/source/util/src/tutil.c	28
	步骤描述: 强制转换为相同类型.	
	代码片段: 26: 27: limit = haystack + hlen - nlen + 1; 28: while ((haystack = (char *)memchr(haystack, needle[0], limit - haystack)) != NULL) { 29: if (memcmp(haystack, needle, nlen) == 0) { 30: return (void *)haystack;	
67	污染路径: 无	
	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 85
	步骤描述: 强制转换为相同类型.	
68	代码片段: 83: uint64_t v = 0; 84: GET_TYPED_DATA(v, uint64_t, pVar->nType, &pVar->u); 85: colDataSetVal(pDst, rowIndex, (char*)&v, isNull); 86: } else if (pDst->info.type == TSDB_DATA_TYPE_TIMESTAMP) { 87: colDataSetVal(pDst, rowIndex, (const char*)¤tKey, isNull);	
	污染路径: 无	
	缺陷发生位置: /tests/script/api/apitest.c	行号: 310
69	步骤描述: 强制转换为相同类型.	
	代码片段: 308: params[j].is_null = ((i == j) ? &is_null : 0); 309: } 310: v.b = (int8_t)i % 2; 311: v.v1 = (int8_t)i; 312: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
69	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1377

	步骤描述: 强制转换为相同类型.	
	代码片段: 1375: 1376: SSortHandle* pHandle = (SSortHandle*)param; 1377: SArray* orderInfo = (SArray*)pHandle->pSortInfo; 1378: uint32_t colNum = blockDataGetNumOfCols(pHandle->pDataBlock); 1379: for (int32_t i = 0; i < orderInfo->size; ++i) {	
	污染路径: 无	
70	缺陷发生位置: /include/common/tmsg.h	行号: 2894
	步骤描述: 强制转换为相同类型.	
	代码片段: 2892: if (tEncodeI32(pEncoder, pKv->key) < 0) return -1; 2893: if (tEncodeI32(pEncoder, pKv->valueLen) < 0) return -1; 2894: if (tEncodeBinary(pEncoder, (uint8_t*)pKv->value, pKv->valueLen) < 0) return -1; 2895: return 0; 2896: }	
	污染路径: 无	
71	缺陷发生位置: /source/util/src/tdes.c	行号: 349
	步骤描述: 强制转换为相同类型.	
	代码片段: 347: column = ((er[0] & 0x78) >> 3); 348: 349: ser[0] = ((uint8_t)S1[row * 16 + column] << 4); 350: 351: row = 0;	
	污染路径: 无	
72	缺陷发生位置: /source/common/src/tmsg.c	行号: 5449
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 5447: int32_t tDeserializeSMqHbReq(void *buf, int32_t bufLen, SMqHbReq *pReq) { 5448: SDecoder decoder = {0}; 5449: tDecoderInit(&decoder, (char *)buf, bufLen); 5450: 5451: if (tStartDecode(&decoder) < 0) return -1;	
73	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/src/tcompression.c	521
	步骤描述: 强制转换为相同类型.	
	代码片段: 519: output[pos] &= t; 520: } else if (input[i] == TSDB_DATA_BOOL_NULL) { 521: t = ((uint8_t)2 << (2 * (i % ele_per_byte))); 522: /* t = (~(((uint8_t)1) << (7-i%BITS_PER_BYTE))); */ 523: output[pos] = t;	
	污染路径: 无	
74	缺陷发生位置:	行号:
	/source/util/src/tworker.c	81
	步骤描述: 强制转换为相同类型.	
	代码片段: 79: 80: while (1) { 81: if (taosReadQitemFromQset(pool->qset, (void **)&msg, &qinfo) == 0) { 82: ulInfo("worker:%s:%d qset:%p, got no message and exiting, thread:%08" PRIu64, pool->name, worker->id, pool->qset, 83: worker->pid);	
	污染路径: 无	
	缺陷发生位置:	行号:
75	/source/os/src/osSocket.c	291
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 289: return setsockopt(pSocket->fd, level, optname, optval, optlen); 290: #else 291: return setsockopt(pSocket->fd, level, optname, optval, (int)optlen); 292: #endif 293: }	
	污染路径: 无	
76	缺陷发生位置: /source/client/src/clientEnv.c	行号: 106
	步骤描述: 强制转换为相同类型.	
	代码片段: 104: pRequest->metric.planCostUs, pRequest->metric.execCostUs); 105: 106: atomic_add_fetch_64((int64_t *)&pActivity->queryElapsedTime, duration); 107: reqType = SLOW_LOG_TYPE_QUERY; 108: }	
	污染路径: 无	
77	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 168
	步骤描述: 强制转换为相同类型.	
	代码片段: 166: SName name = {0}; 167: tNameFromString(&name, stbFullName, T_NAME_ACCT T_NAME_DB T_NAME_TABLE); 168: pCreateTbReq->ctb.stbName = taosStrdup((char*)tNameGetTableName(&name)); // taosStrdup(stbFullName); 169: 170: // set tag content	
	污染路径: 无	

78	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 3670
	步骤描述: 强制转换为相同类型.	
	代码片段: 3668: 3669: int32_t getHistogramInfoSize() { 3670: return (int32_t)sizeof(SHistoFuncInfo) + HISTOGRAM_MAX_BINS_NUM * sizeof(SHistoFuncBin); 3671: } 3672:	
	污染路径: 无	
79	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 1224
	步骤描述: 强制转换为相同类型.	
	代码片段: 1222: 1223: nodesRewriteExprsPostOrder(groupNew, doTranslateTagExpr, &mr); 1224: char* isNull = (char*)keyBuf; 1225: char* pStart = (char*)keyBuf + sizeof(int8_t) * LIST_LENGTH(pGroupNode); 1226:	
	污染路径: 无	
80	缺陷发生位置: /source/common/src/ttypes.c	行号: 140
	步骤描述: 强制转换为相同类型.	
	代码片段: 138: case TSDB_DATA_TYPE_VARBINARY: 139: case TSDB_DATA_TYPE_GEOMETRY: 140: varDataCopy(val, src); 141: break; 142: case TSDB_DATA_TYPE_NCHAR:	
	污染路径: expanded from macro 'varDataCopy'	
81	缺陷发生位置:	行号:

	/source/util/src/tarray.c	426
	步骤描述: 强制转换为相同类型.	
	代码片段: 424: int32_t taosArraySearchIdx(const SArray* pArray, const void* key, __compar_fn_t comparFn, int32_t flags) { 425: void* item = taosArraySearch(pArray, key, comparFn, flags); 426: return item == NULL ? -1 : (int32_t)((char*)item - (char*)pArray->pData) / pArray->elemSize; 427: } 428:	
	污染路径: 无	
82	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 902
	步骤描述: 强制转换为相同类型.	
	代码片段: 900: } 901: varDataSetLen(tagTypeStr, tagTypeLen); 902: colDataSetVal(pCollInfoData, numOfRows, (char*)tagTypeStr, false); 903: 904: STagVal tagVal = {0};	
	污染路径: 无	
83	缺陷发生位置: /utils/test/c/sml_test.c	行号: 974
	步骤描述: 强制转换为相同类型.	
	代码片段: 972: char *tmp = (char *)taosMemoryCalloc(1024, 1); 973: memcpy(tmp, sql[0], strlen(sql[0])); 974: *(char *)(tmp + 44) = 0; 975: int32_t totalRows = 0; 976: pRes = taos_schemaless_insert_raw(taos, tmp, strlen(sql[0]), &totalRows, TSDB_SML_LINE_PROTOCOL,	
	污染路径: 无	

84	缺陷发生位置： /tests/script/api/batchprepare.c	行号： 371
	步骤描述： 强制转换为相同类型.	
	代码片段： 369: struct timeval systemTime; 370: taosGetTimeOfDay(&systemTime); 371: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 372: } 373:	
	污染路径： 无	
85	缺陷发生位置： /tests/script/api/apitest.c	行号： 509
	步骤描述： 强制转换为相同类型.	
	代码片段： 507: is_null[i] = 0; 508: 509: v.b[i] = (int8_t)i % 2; 510: v.v1[i] = (int8_t)i; 511: v.v2[i] = (int16_t)(i * 2);	
	污染路径： 无	
86	缺陷发生位置： /source/libs/nodes/src/nodesMsgFuncs.c	行号： 156
	步骤描述： 强制转换为相同类型.	
	代码片段： 154: 155: static int32_t tlvEncodeValueI64(STlvEncoder* pEncoder, int64_t value) { 156: value = htonl(value); 157: return tlvEncodeValueImpl(pEncoder, &value, sizeof(value)); 158: }	
	污染路径： expanded from macro 'htonll'	
87	缺陷发生位置：	行号：

	/source/libs/nodes/src/nodesMsgFuncs.c	179
	步骤描述: 强制转换为相同类型.	
	代码片段: 177: 178: static int32_t tlvEncodeU64(STlvEncoder* pEncoder, int16_t type, uint64_t value) { 179: value = htonl(value); 180: return tlvEncodeImpl(pEncoder, type, &value, sizeof(value)); 181: }	
	污染路径: expanded from macro 'htonll'	
88	缺陷发生位置: /source/libs/index/src/index.c	行号: 311
	步骤描述: 强制转换为相同类型.	
	代码片段: 309: len = idxConvertDataToStr((void*)colVal, IDX_TYPE_GET_TYPE(colType), (void**)&buf); 310: } else if (colVal == NULL) { 311: buf = strdup(INDEX_DATA_NULL_STR, (int32_t)strlen(INDEX_DATA_NULL_STR)); 312: len = (int32_t)strlen(INDEX_DATA_NULL_STR); 313: } else {	
	污染路径: 无	
89	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 1454
	步骤描述: 强制转换为相同类型.	
	代码片段: 1452: taosFprintfFile(1453: g_fp, "Spent %.3f seconds to poll msgs: %" PRIu64 " with %d thread(s), throughput: %.3f msgs/s, %.1f MB/s\n\n", 1454: tlnMs, totalMsgs, g_stConflInfo.numOfThread, (double)(totalMsgs / tlnMs), 1455: (double)totalLenOfMsgs / (1024 * 1024) / tlnMs); 1456:	
	污染路径:	

	无	
90	缺陷发生位置： /source/libs/index/src/indexComm.c	行号： 377
	步骤描述： 强制转换为相同类型.	
	代码片段： 375: int32_t idxConvertDataToStr(void* src, int8_t type, void** dst) { 376: if (src == NULL) { 377: *dst = strdup(INDEX_DATA_NULL_STR, (int)strlen(INDEX_DATA_NULL_STR)); 378: return (int32_t)strlen(INDEX_DATA_NULL_STR); 379: }	
	污染路径： 无	
91	缺陷发生位置： /source/util/src/tcrc32c.c	行号： 966
	步骤描述： 强制转换为相同类型.	
	代码片段： 964: next += 4; 965: } while (next < end); 966: crc0 = shift_crc(short_shifts, (uint32_t)(crc0)) ^ crc1; 967: crc0 = shift_crc(short_shifts, (uint32_t)(crc0)) ^ crc2; 968: next += 2 * SHORT_SHIFT;	
	污染路径： 无	
92	缺陷发生位置： /source/client/src/clientEnv.c	行号： 63
	步骤描述： 强制转换为相同类型.	
	代码片段： 61: 62: int32_t total = atomic_add_fetch_64((int64_t *)&pSummary->totalRequests, 1); 63: int32_t currentInst = atomic_add_fetch_64((int64_t *)&pSummary->currentRequests, 1); 64: tscDebug("0x%" PRIx64 " new Request from connObj:0x%" PRIx64 65: ", current:%d, app current:%d, total:%d, reqId:0x%" PRIx64,	

	污染路径: 无	
93	缺陷发生位置: /source/libs/command/src/explain.c	行号: 1778
	步骤描述: 强制转换为相同类型.	
	代码片段: 1776: 1777: rsp->completed = 1; 1778: rsp->numOfRows = htobe64((int64_t)rowNum); 1779: 1780: int32_t len = blockEncode(pBlock, rsp->data, taosArrayGetSize(pBlock->pDataBlock));	
	污染路径: expanded from macro 'htobe64'	
94	缺陷发生位置: /source/client/src/clientImpl.c	行号: 1204
	步骤描述: 强制转换为相同类型.	
	代码片段: 1202: atomic_add_fetch_64((int64_t*)&pActivity->numOfInsertsReq, 1); 1203: } else if (QUERY_NODE_SELECT_STMT == pQuery->pRoot->type) { 1204: atomic_add_fetch_64((int64_t*)&pActivity->numOfQueryReq, 1); 1205: } 1206: }	
	污染路径: 无	
95	缺陷发生位置: /source/libs/function/src/detail/tminmax.c	行号: 537
	步骤描述: 强制转换为相同类型.	
	代码片段: 535: // AVX version to speedup the loop 536: if (tsAVXEnable && tsSIMDBuiltins) { 537: *val = (double)doubleVectorCmpAVX(pData, numOfRows, isMinFunc); 538: } else {	

	539: if (!pBuf->assign) {	
	污染路径: 无	
96	缺陷发生位置: /source/libs/index/src/index.c	行号: 248
	步骤描述: 强制转换为相同类型.	
	代码片段: 246: SArray* trslt = NULL; 247: idxTermSearch(index, qterm, &trslt); 248: taosArrayPush(iRslts, (void*)&trslt); 249: } 250: idxMergeFinalResults(iRslts, opera, result);	
	污染路径: 无	
97	缺陷发生位置: /source/util/src/tskiplist.c	行号: 679
	步骤描述: 强制转换为相同类型.	
	代码片段: 677: SSkipListNode *pNode = NULL; 678: 679: if (hasDup && (dupMode != SL_ALLOW_DUP_KEY)) { 680: if (dupMode == SL_UPDATE_DUP_KEY) { 681: if (isForward) {	
	污染路径: expanded from macro 'SL_ALLOW_DUP_KEY'	
98	缺陷发生位置: /tests/script/api/stmtBatchTest.c	行号: 221
	步骤描述: 强制转换为相同类型.	
	代码片段: 219: 220: unsigned long long endtime = getCurrentTime(); 221: unsigned long long totalRows = (uint32_t) (totalRowsPerTbl * tableNum); 222: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows);	

	223:	
	污染路径: 无	
99	缺陷发生位置: /tests/script/api/stmtBatchTest.c	行号: 406
	步骤描述: 强制转换为相同类型.	
	代码片段: 404: 405: unsigned long long endtime = getCurrentTime(); 406: unsigned long long totalRows = (uint32_t)(totalRowsPerTbl * tableNum); 407: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 408:	
	污染路径: 无	
100	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 381
	步骤描述: 强制转换为相同类型.	
	代码片段: 379: } 380: 381: pApp->lastAccessTimeMs = (uint64_t)taosGetTimestampMs(); 382: 383: mTrace("app %" PRIx64 " acquired from cache", appId);	
	污染路径: 无	
101	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 357
	步骤描述: 强制转换为相同类型.	
	代码片段: 355: case '\f': 356: case '\r': { 357: for (i = 1; isspace(z[i]); i++) {	

	358: } 359: *tokenId = TK_NK_SPACE;	
	污染路径: expanded from macro 'isspace' expanded from macro ' __isctype'	
102	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1355
	步骤描述: 强制转换为相同类型.	
	代码片段: 1353: len += taosEncodeFixedl64((void*)&buf, key.unixTimestamp); 1354: len += taosEncodeFixedl32((void*)&buf, key.len); 1355: len += taosEncodeBinary((void*)&buf, (char*)value, vlen); 1356: *dest = p; 1357: } else {	
	污染路径: 无	
103	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 196
	步骤描述: 强制转换为相同类型.	
	代码片段: 194: pStart += dataLen; 195: } else if (IS_VAR_DATA_TYPE(pCol->type)) { 196: varDataCopy(pStart, val); 197: pStart += varDataTLen(val); 198: } else {	
	污染路径: expanded from macro 'varDataCopy'	
104	缺陷发生位置: /source/os/src/osSocket.c	行号: 1084
	步骤描述: 强制转换为相同类型.	
	代码片段: 1082: #else // Linux like systems 1083: uint32_t conn_timeout_ms = timeout; 1084: if (0 != setsockopt(fd, IPPROTO_TCP, TCP_USER_TIMEOUT, (char *)&conn_timeout_ms,	

	sizeof(conn_timeout_ms))) { 1085: taosCloseSocketNoCheck1(fd); 1086: return -1;	
	污染路径: 无	
105	缺陷发生位置: /source/util/test/arrayTest.cpp	行号: 55
	步骤描述: 强制转换为相同类型.	
	代码片段: 53: */ 54: TEST(arrayTest, array_search_test) { 55: SArray* pa = (SArray*)taosArrayInit(4, sizeof(int32_t)); 56: 57: for (int32_t i = 10; i < 20; ++i) {	
	污染路径: 无	
106	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 258
	步骤描述: 强制转换为相同类型.	
	代码片段: 256: memcpy(dest, data, dataLen); 257: } else if (IS_VAR_DATA_TYPE(pCollInfoData->info.type)) { 258: varDataCopy(dest, data); 259: } else { 260: memcpy(dest, data, pCollInfoData->info.bytes);	
	污染路径: expanded from macro 'varDataCopy'	
107	缺陷发生位置: /source/util/src/thashutil.c	行号: 48
	步骤描述: 强制转换为相同类型.	
	代码片段: 46: uint32_t hash = 5381; 47: for (uint32_t i = 0; i < len; i++) { 48: hash = ((hash << 5) + hash) + (uint8_t)key[i]; /* hash * 33 + c */ 49: }	

	50: return hash;	
	污染路径: 无	
108	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 486
	步骤描述: 强制转换为相同类型.	
	代码片段: 484: TFileValue* v1 = genTFileValue("ab"); 485: 486: SArray* data = (SArray*)taosArrayInit(4, sizeof(void*)); 487: 488: taosArrayPush(data, &v1);	
	污染路径: 无	
109	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 614
	步骤描述: 强制转换为相同类型.	
	代码片段: 612: } 613: 614: static void *topicNameDup(void *p) { return taosStrdup((char *)p); } 615: 616: static void freeItem(void *param) {	
	污染路径: 无	
110	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 409
	步骤描述: 强制转换为相同类型.	
	代码片段: 407: 408: SBackendFileItem item; 409: item.name = taosStrdup((char*)ROCKSDB_CURRENT); 410: item.type = ROCKSDB_CURRENT_TYPE; 411: taosArrayPush(list, &item);	
	污染路径:	

	/source/client/src/clientImpl.c	1534
	步骤描述: 强制转换为相同类型.	
	代码片段: 1532: if (pEpSet != NULL) { 1533: tEpSet = taosMemoryCalloc(1, sizeof(SEpSet)); 1534: memcpy((void*)tEpSet, (void*)pEpSet, sizeof(SEpSet)); 1535: } 1536:	
115	污染路径: 无	
	缺陷发生位置: /source/client/src/clientEnv.c	行号: 98
	步骤描述: 强制转换为相同类型.	
	代码片段: 96: duration, pRequest->metric.parseCostUs, pRequest->metric.ctgCostUs, pRequest->metric.analyseCostUs, 97: pRequest->metric.planCostUs, pRequest->metric.execCostUs); 98: atomic_add_fetch_64((int64_t *)&pActivity->insertElapsedTime, duration); 99: reqType = SLOW_LOG_TYPE_INSERT; 100: } else if (QUERY_NODE_SELECT_STMT == pRequest->stmtType) {	
116	污染路径: 无	
	缺陷发生位置: /include/common/tatablock.h	行号: 168
	步骤描述: 强制转换为相同类型.	
	代码片段: 166: ASSERT(pColumnInfoData->info.type == TSDB_DATA_TYPE_FLOAT); 167: char* p = pColumnInfoData->pData + pColumnInfoData->info.bytes * rowIndex; 168: *(float*)p = *(float*)v; 169: } 170:	

	污染路径: 无	
117	缺陷发生位置: /source/common/src/ttszip.c	行号: 317
	步骤描述: 强制转换为相同类型.	
	代码片段: 315: taosWriteFile(pTSBuf->pFile, &pBlock->numOfElem, sizeof(pBlock->numOfElem)); 316: taosWriteFile(pTSBuf->pFile, &pBlock->compLen, sizeof(pBlock->compLen)); 317: taosWriteFile(pTSBuf->pFile, pBlock->payload, (size_t)pBlock->compLen); 318: taosWriteFile(pTSBuf->pFile, &pBlock->compLen, sizeof(pBlock->compLen)); 319:	
	污染路径: 无	
118	缺陷发生位置: /source/util/src/thash.c	行号: 412
	步骤描述: 强制转换为相同类型.	
	代码片段: 410: } 411: 412: if ((atomic_load_64((int64_t *)&pHashObj->size) == 0)) { 413: return NULL; 414: }	
	污染路径: 无	
119	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 315
	步骤描述: 强制转换为相同类型.	
	代码片段: 313: keywordTable[i].len = (uint8_t)strlen(keywordTable[i].name); 314: void* ptr = &keywordTable[i]; 315: taosHashPut(keywordHashTable,	

	keywordTable[i].name, keywordTable[i].len, (void*)&ptr, POINTER_BYTES); 316: } 317: }	
	污染路径: 无	
120	缺陷发生位置: /source/os/src/osString.c	行号: 482
	步骤描述: 强制转换为相同类型.	
	代码片段: 480: if(n % 2 != 0) return false; 481: for(size_t i = HEX_PREFIX_LEN; i < n; i++){ 482: if(isxdigit(z[i]) == 0){ 483: return false; 484: }	
	污染路径: expanded from macro 'isxdigit' expanded from macro '__isctype'	
121	缺陷发生位置: /include/common/trow.h	行号: 301
	步骤描述: 强制转换为相同类型.	
	代码片段: 299: int32_t tdSRowResetBuf(SRowBuilder *pBuilder, void *pBuf); 300: static FORCE_INLINE void tdSRowEnd(SRowBuilder *pBuilder) { 301: STSRow *pRow = (STSRow *)pBuilder->pBuf; 302: if (pBuilder->hasNull pBuilder->hasNone) { 303: pRow->statis = 1;	
	污染路径: 无	
122	缺陷发生位置: /source/util/src/tdigest.c	行号: 49
	步骤描述: 强制转换为相同类型.	
	代码片段: 47: void tdigestAutoFill(TDigest *t, int32_t compression) { 48: t->centroids = (SCentroid *)((char *)t + sizeof(TDigest));	

	49: t->buffered_pts = (SPt *)((char *)t + sizeof(TDigest) + sizeof(SCentroid) * (int32_t)GET_CENTROID(compression)); 50: } 51:	
	污染路径: 无	
123	缺陷发生位置: /source/util/src/tutil.c	行号: 73
	步骤描述: 强制转换为相同类型.	
	代码片段: 71: 72: int32_t delta = 0; 73: while (isspace(z[j])) { 74: ++j; 75: }	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
124	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 575
	步骤描述: 强制转换为相同类型.	
	代码片段: 573: char buf[256] = {0}; 574: int16_t sz = nColVal; 575: memcpy(buf, (uint16_t*)&sz, 2); 576: memcpy(buf + 2, colVal, nColVal); 577: if (colType == TSDB_DATA_TYPE_BINARY colType == TSDB_DATA_TYPE_GEOMETRY) {	
	污染路径: 无	
125	缺陷发生位置: /source/util/src/tutil.c	行号: 88
	步骤描述: 强制转换为相同类型.	
	代码片段: 86: if (isspace(z[j]) && stop == 0) { 87: stop = j; 88: } else if (!isspace(z[j]) && stop != 0) {	

	89: stop = 0; 90: }	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
126	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 996
	步骤描述: 强制转换为相同类型.	
	代码片段: 994: } 995: varDataSetLen(colTypeStr, colTypeLen); 996: colDataSetVal(pColumnInfoData, numOfRows, (char*)colTypeStr, false); 997: 998: pColumnInfoData = taosArrayGet(dataBlock->pDataBlock, 5);	
	污染路径: 无	
127	缺陷发生位置: /tests/script/api/api_with_reqid_test.c	行号: 379
	步骤描述: 强制转换为相同类型.	
	代码片段: 377: is_null[i] = 0; 378: 379: v.b[i] = (int8_t)i % 2; 380: v.v1[i] = (int8_t)i; 381: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
128	缺陷发生位置: /tests/script/api/demoapi.c	行号: 164
	步骤描述: 强制转换为相同类型.	
	代码片段: 162: "values(%" PRIu64 " , %f, %" PRIu64 " , " 163: ""%c%d', '%s%c%d', '%c%d')", 164: i, 1650000000000+j+1, (float)j, j, 165: 'a'+(int)j%25, rand(),	

	166: "数据", 'z' - (int)j%25, rand(),	
	污染路径: 无	
129	缺陷发生位置: /source/os/src/osString.c	行号: 505
	步骤描述: 强制转换为相同类型.	
	代码片段: 503: *byte = ((uint8_t)(10 + (z[i] - 'A')) << (num * 4)); 504: } else { 505: *byte = ((uint8_t)(z[i] - '0') << (num * 4)); 506: } 507: if (num == 1) {	
	污染路径: 无	
130	缺陷发生位置: /tests/script/api/apitest.c	行号: 768
	步骤描述: 强制转换为相同类型.	
	代码片段: 766: is_null[i] = 0; 767: 768: v.b[i] = (int8_t)i % 2; 769: v.v1[i] = (int8_t)i; 770: v.v2[i] = (int16_t)(i * 2);	
	污染路径: 无	
131	缺陷发生位置: /source/common/src/ttszip.c	行号: 306
	步骤描述: 强制转换为相同类型.	
	代码片段: 304: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &pBlock->tag.nLen, sizeof(pBlock->tag.nLen)); 305: float tfloat = (float)pBlock->tag.d; 306: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &tfloat, (size_t)pBlock->tag.nLen); 307: } else if (pBlock->tag.nType != TSDB_DATA_TYPE_NULL) { 308: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile,	

	&pBlock->tag.nLen, sizeof(pBlock->tag.nLen));	
	污染路径: 无	
132	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 505
	步骤描述: 强制转换为相同类型.	
	代码片段: 503: char buf[256] = {0}; 504: int16_t sz = colVal.size(); 505: memcpy(buf, (uint16_t*)&sz, 2); 506: memcpy(buf + 2, colVal.c_str(), colVal.size()); 507: SIndexTerm* term =	
	污染路径: 无	
133	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 4811
	步骤描述: 强制转换为相同类型.	
	代码片段: 4809: 4810: while (pStatus->pTableIter != NULL) { 4811: STableBlockScanInfo* pBlockScanInfo = *(STableBlockScanInfo**)pStatus->pTableIter; 4812: 4813: STbData* d = NULL;	
	污染路径: 无	
134	缺陷发生位置: /source/libs/index/src/index.c	行号: 315
	步骤描述: 强制转换为相同类型.	
	代码片段: 313: } else { 314: static const char* emptyStr = " "; 315: buf = strdup(emptyStr, (int32_t)strlen(emptyStr)); 316: len = (int32_t)strlen(emptyStr); 317: }	
	污染路径:	

	无	
135	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 190
	步骤描述: 强制转换为相同类型.	
	代码片段: 188: static int32_t tlvEncodeDouble(STlvEncoder* pEncoder, int16_t type, double value) { 189: int64_t temp = *(int64_t*)&value; 190: temp = htonl(temp); 191: return tlvEncodeImpl(pEncoder, type, &temp, sizeof(temp)); 192: }	
	污染路径: expanded from macro 'htonl'	
136	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 2358
	步骤描述: 强制转换为相同类型.	
	代码片段: 2356: colDataSetNULL(pOutputData, 0); 2357: } else { 2358: colDataSetVal(pOutputData, 0, (char*)&val, false); 2359: } 2360:	
	污染路径: 无	
137	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 571
	步骤描述: 强制转换为相同类型.	
	代码片段: 569: 570: SRow* pRow = NULL; 571: code = tRowBuild(pVals, (STSchema*)pTSchema, &pRow); 572: if (code != TSDB_CODE_SUCCESS) { 573: tDestroySubmitTbData(pTableData, TSDB_MSG_FLG_ENCODE);	
	污染路径:	

	无	
138	缺陷发生位置: /source/common/src/tmsg.c	行号: 5342
	步骤描述: 强制转换为相同类型.	
	代码片段: 5340: int32_t tDeserializeSBatchRsp(void *buf, int32_t bufLen, SBatchRsp *pRsp) { 5341: SDecoder decoder = {0}; 5342: tDecoderInit(&decoder, (char *)buf, bufLen); 5343: 5344: if (tStartDecode(&decoder) < 0) return -1;	
	污染路径: 无	
139	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 787
	步骤描述: 强制转换为相同类型.	
	代码片段: 785: 786: static int tfileStrCompare(const void* a, const void* b) { 787: int ret = strcmp((char*)a, (char*)b); 788: if (ret == 0) { 789: return ret;	
	污染路径: 无	
140	缺陷发生位置: /source/util/src/talgo.c	行号: 43
	步骤描述: 强制转换为相同类型.	
	代码片段: 41: 42: ASSERT(comparFn(elePtrAt(src, size, mid), elePtrAt(src, size, s), param) <= 0 && 43: comparFn(elePtrAt(src, size, s), elePtrAt(src, size, e), param) <= 0); 44: } 45:	
	污染路径: expanded from macro 'elePtrAt'	

141	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 204
	步骤描述: 强制转换为相同类型.	
	代码片段: 202: int32_t step = isLtrim ? 1 : -1; 203: for (int32_t i = startPos; i < len i >= 0; i += step) { 204: if (!isspace(str[i])) { 205: break; 206: }	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
142	缺陷发生位置: /source/common/src/ttszip.c	行号: 302
	步骤描述: 强制转换为相同类型.	
	代码片段: 300: pBlock->tag.nType == TSDB_DATA_TYPE_NCHAR pBlock->tag.nType == TSDB_DATA_TYPE_GEOMETRY) { 301: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &pBlock->tag.nLen, sizeof(pBlock->tag.nLen)); 302: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, pBlock->tag.pz, (size_t)pBlock->tag.nLen); 303: } else if (pBlock->tag.nType == TSDB_DATA_TYPE_FLOAT) { 304: metaLen += (int32_t)taosWriteFile(pTSBuf->pFile, &pBlock->tag.nLen, sizeof(pBlock->tag.nLen));	
	污染路径: 无	
143	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 119
	步骤描述: 强制转换为相同类型.	
	代码片段: 117: struct timeval systemTime; 118: rtGetTimeOfDay(&systemTime); 119: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 120: } 121:	

	污染路径: 无	
144	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 136
	步骤描述: 强制转换为相同类型.	
	代码片段: 134: case 5: { 135: for (int32_t i = 1; i < len; ++i) { 136: if (!isdigit(tz[i])) { 137: return false; 138: } 139: }	
	污染路径: expanded from macro 'isdigit' expanded from macro '___isctype'	
145	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 1516
	步骤描述: 强制转换为相同类型.	
	代码片段: 1514: taosFprintfFile(g_fp, 1515: "Spent %.3f seconds to poll msgs: %"PRlu64 " with %d thread(s), throughput: %.3f msgs/second\n\n", 1516: tlnMs, totalMsgs, g_stConflInfo.numOfThread, (double)(totalMsgs / tlnMs)); 1517: 1518: taosFprintfFile(g_fp, "==== close tmqlog ==== \n");	
	污染路径: 无	
146	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 1906
	步骤描述: 强制转换为相同类型.	
	代码片段: 1904: STagVal getJsonValue(char *json, char *key, bool *isExist) { 1905: STagVal val = {.pKey = key}; 1906: if (json == NULL tTagIsJson((const STag *)json) == false) {	

	1907: terrno = TSDB_CODE_QRY_JSON_NOT_SUPPORT_ERROR; 1908: if (isExist) {	
	污染路径: 无	
147	缺陷发生位置: /source/util/src/tmempool.c	行号: 50
	步骤描述: 强制转换为相同类型.	
	代码片段: 48: pool_p->blockSize = blockSize; 49: pool_p->numOfBlock = numOfBlock; 50: pool_p->pool = (char *)taosMemoryMalloc((size_t) (blockSize * numOfBlock)); 51: pool_p->freeList = (int32_t *)taosMemoryMalloc(sizeof(int32_t) * (size_t)numOfBlock); 52:	
	污染路径: 无	
148	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 553
	步骤描述: 强制转换为相同类型.	
	代码片段: 551: pBlock->info.rows = 0; 552: } else if (status == FILTER_RESULT_PARTIAL_QUALIFIED) { 553: trimDataBlock(pBlock, pBlock->info.rows, (bool*)pIndicator); 554: } else { 555: qError("unknown filter result type: %d", status);	
	污染路径: 无	
149	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 662
	步骤描述: 强制转换为相同类型.	
	代码片段: 660: colVal.c_str(), colVal.size()); 661: SIndexTermQuery query = {term, QUERY_TERM};	

	662: SArray* ret = (SArray*)taosArrayInit(4, sizeof(suid)); 663: STermValueType valType; 664:	
	污染路径: 无	
150	缺陷发生位置: /source/util/src/thash.c	行号: 632
	步骤描述: 强制转换为相同类型.	
	代码片段: 630: 631: for (int32_t i = 0; i < inc; ++i) { 632: pHashObj->hashList[i + pHashObj->capacity] = (void *)((char *)p + i * sizeof(SHashEntry)); 633: } 634:	
	污染路径: 无	
151	缺陷发生位置: /source/util/src/tcompression.c	行号: 203
	步骤描述: 强制转换为相同类型.	
	代码片段: 201: break; 202: case TSDB_DATA_TYPE_BIGINT: 203: curr_value = (int64_t)((int64_t *)input + i)); 204: break; 205: }	
	污染路径: 无	
152	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 2049
	步骤描述: 强制转换为相同类型.	
	代码片段: 2047: } else if (IS_FLOAT_TYPE(type)) { 2048: double *out = (double *)pOutputData->pData; 2049: *(double *)out = *out / (double)count; 2050: }	

	2051: }	
	污染路径: 无	
153	缺陷发生位置: /source/common/src/ttszip.c	行号: 213
	步骤描述: 强制转换为相同类型.	
	代码片段: 211: if (pTSBuf->numOfAlloc <= pTSBuf->numOfGroups) { 212: uint32_t newSize = (uint32_t)(pTSBuf->numOfAlloc * 1.5); 213: ASSERT((int32_t)newSize > pTSBuf->numOfAlloc); 214: 215: STSGroupBlockInfoEx* tmp =	
	污染路径: expanded from macro 'ASSERT' expanded from macro 'ASSERTS'	
154	缺陷发生位置: /include/util/tcoding.h	行号: 407
	步骤描述: 强制转换为相同类型.	
	代码片段: 405: 406: static FORCE_INLINE void *taosDecodeBinary(const void *buf, void **value, int32_t valueLen) { 407: *value = taosMemoryMalloc((size_t)valueLen); 408: if (*value == NULL) return NULL; 409: memcpy(*value, buf, (size_t)valueLen);	
	污染路径: 无	
155	缺陷发生位置: /include/util/tcoding.h	行号: 401
	步骤描述: 强制转换为相同类型.	
	代码片段: 399: *buf = POINTER_SHIFT(*buf, valueLen); 400: } 401: tlen += (int32_t)valueLen; 402: 403: return tlen;	

	污染路径: 无	
156	缺陷发生位置: /include/util/tencode.h	行号: 489
	步骤描述: 强制转换为相同类型.	
	代码片段: 487: // PUT 488: static FORCE_INLINE int32_t tPutU8(uint8_t* p, uint8_t v) { 489: if (p) ((uint8_t*)p)[0] = v; 490: return sizeof(uint8_t); 491: }	
	污染路径: 无	
157	缺陷发生位置: /source/common/src/tdataformat.c	行号: 3426
	步骤描述: 强制转换为相同类型.	
	代码片段: 3424: break; 3425: case 2: 3426: val = ((uint8_t *)pColData->pData)[iVal]; 3427: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3428: break;	
	污染路径: 无	
158	缺陷发生位置: /include/util/tencode.h	行号: 562
	步骤描述: 强制转换为相同类型.	
	代码片段: 560: // GET 561: static FORCE_INLINE int32_t tGetU8(uint8_t* p, uint8_t* v) { 562: if (v) *v = ((uint8_t*)p)[0]; 563: return sizeof(uint8_t); 564: }	

	污染路径: 无	
159	缺陷发生位置: /source/util/src/tmd5.c	行号: 110
	步骤描述: 强制转换为相同类型.	
	代码片段: 108: 109: /* update number of bits */ 110: if ((mdContext->i[0] + ((uint32_t)inLen << 3)) < mdContext->i[0]) mdContext->i[1]++; 111: mdContext->i[0] += ((uint32_t)inLen << 3); 112: mdContext->i[1] += ((uint32_t)inLen >> 29);	
	污染路径: 无	
160	缺陷发生位置: /source/util/src/talgo.c	行号: 38
	步骤描述: 强制转换为相同类型.	
	代码片段: 36: doswap(elePtrAt(src, size, mid), elePtrAt(src, size, s), size, buf); 37: doswap(elePtrAt(src, size, mid), elePtrAt(src, size, e), size, buf); 38: } else if (comparFn(elePtrAt(src, size, s), elePtrAt(src, size, e), param) > 0) { 39: doswap(elePtrAt(src, size, s), elePtrAt(src, size, e), size, buf); 40: }	
	污染路径: expanded from macro 'elePtrAt'	
161	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 663
	步骤描述: 强制转换为相同类型.	
	代码片段: 661: if (IS_VAR_DATA_TYPE(left->colValType)) { 662: if (!IS_VAR_DATA_TYPE(right->colValType)) { 663: NUM_TO_STRING(right->colValType, right->condValue, sizeof(buf) - 2, buf + VARSTR_HEADER_SIZE); 664: varDataSetLen(buf, strlen(buf) +	

	VARSTR_HEADER_SIZE)); 665: param.val = buf;	
	污染路径: expanded from macro 'NUM_TO_STRING'	
162	缺陷发生位置: /tests/script/api/stmtBatchTest.c	行号: 528
	步骤描述: 强制转换为相同类型.	
	代码片段: 526: 527: unsigned long long endtime = getCurrentTime(); 528: unsigned long long totalRows = (uint32_t) (totalRowsPerTbl * tableNum); 529: printf("insert total %d records, used %u seconds, avg: %u useconds per record\n", totalRows, (endtime-starttime)/1000000UL, (endtime-starttime)/totalRows); 530:	
	污染路径: 无	
163	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 31
	步骤描述: 强制转换为相同类型.	
	代码片段: 29: 30: for (int32_t i = 0; i < 200; ++i) { 31: char* p = (char*)taosHashGet(hashTable, (const char*)&i, sizeof(int32_t)); 32: ASSERT_TRUE(p != nullptr); 33: ASSERT_EQ(*reinterpret_cast<int32_t*>(p), i);	
	污染路径: 无	
164	缺陷发生位置: /source/os/src/osString.c	行号: 280
	步骤描述: 强制转换为相同类型.	
	代码片段: 278: 279: int32_t taosUcs4len(TdUcs4 *ucs4) {	

	280: TdUcs4 *wstr = (TdUcs4 *)ucs4; 281: if (NULL == wstr) { 282: return 0;	
	污染路径: 无	
165	缺陷发生位置: /source/client/src/clientEnv.c	行号: 81
	步骤描述: 强制转换为相同类型.	
	代码片段: 79: SAppClusterSummary *pActivity = &pTscObj->pAppInfo->summary; 80: 81: int32_t currentInst = atomic_sub_fetch_64((int64_t *)&pActivity->currentRequests, 1); 82: int32_t num = atomic_sub_fetch_32(&pTscObj->numOfReqs, 1); 83: int32_t reqType = SLOW_LOG_TYPE_OTHERS;	
	污染路径: 无	
166	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 4774
	步骤描述: 强制转换为相同类型.	
	代码片段: 4772: 4773: while ((p = tSimpleHashIterate(pStatus->pTableMap, p, &iter)) != NULL) { 4774: STableBlockScanInfo* pInfo = *(STableBlockScanInfo**)p; 4775: 4776: pInfo->iterInit = false;	
	污染路径: 无	
167	缺陷发生位置: /source/util/src/tlog.c	行号: 449
	步骤描述: 强制转换为相同类型.	
	代码片段: 447:	

	<pre> 448: sprintf(name, "===== =====\\n"); 449: taosWriteFile(tsLogObj.logHandle->pFile, name, (uint32_t)strlen(name)); 450: sprintf(name, " new log file \\n"); 451: taosWriteFile(tsLogObj.logHandle->pFile, name, (uint32_t)strlen(name)); </pre>	
	污染路径: 无	
168	缺陷发生位置: /source/client/src/clientImpl.c	行号: 1202
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 1200: if (QUERY_NODE_VNODE_MODIFY_STMT == pQuery- >pRoot->type && 1201: (0 == ((SVnodeModifyOpStmt*)pQuery->pRoot)- >sqlNodeType)) { 1202: atomic_add_fetch_64((int64_t*)&pActivity- >numOfInsertsReq, 1); 1203: } else if (QUERY_NODE_SELECT_STMT == pQuery- >pRoot->type) { 1204: atomic_add_fetch_64((int64_t*)&pActivity- >numOfQueryReq, 1); </pre>	
	污染路径: 无	
169	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 179
	步骤描述: 强制转换为相同类型.	
	代码片段: <pre> 177: 178: static int32_t compareWinKey(void* pKey, void* data, int32_t index) { 179: void* pDataPos = taosArrayGet((SArray*)data, index); 180: return winKeyCmprImpl(pKey, pDataPos); 181: } </pre>	
	污染路径: 无	

170	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1158
	步骤描述: 强制转换为相同类型.	
	代码片段: 1156: int64_t ts = mr.me.ctbEntry.btime; 1157: pCollInfoData = taosArrayGet(p->pDataBlock, 2); 1158: colDataSetVal(pCollInfoData, numOfRows, (char*)&ts, false); 1159: 1160: SMetaReader mr1 = {0};	
	污染路径: 无	
171	缺陷发生位置: /source/util/src/thash.c	行号: 287
	步骤描述: 强制转换为相同类型.	
	代码片段: 285: 286: for (int32_t i = 0; i < pHashObj->capacity; ++i) { 287: pHashObj->hashList[i] = (void *)((char *)p + i * sizeof(SHashEntry)); 288: pHashObj->hashList[i]->num = 0; 289: pHashObj->hashList[i]->latch = 0;	
	污染路径: 无	
172	缺陷发生位置: /source/util/test/taosbsearchTest.cpp	行号: 12
	步骤描述: 强制转换为相同类型.	
	代码片段: 10: void *pRet = NULL; 11: 12: pRet = taosbsearch((void *)&key, NULL, 0, sizeof(int), compareFunc, TD_EQ); 13: ASSERT_EQ(pRet, nullptr); 14:	
	污染路径: 无	
173	缺陷发生位置:	行号:

	/source/util/src/tcrc32c.c		876
	步骤描述: 强制转换为相同类型.		
	代码片段: 874: to an eight-byte boundary */ 875: while (len && ((uintptr_t)next & 7) != 0) { 876: crc0 = _mm_crc32_u8((uint32_t)(crc0), *next); 877: ++next; 878: --len;		
	污染路径: 无		
174	缺陷发生位置: /source/util/test/hashTest.cpp		行号: 25
	步骤描述: 强制转换为相同类型.		
	代码片段: 23: // put 400 elements in the hash table 24: for (int32_t i = -200; i < 200; ++i) { 25: taosHashPut(hashTable, (const char*)&i, sizeof(int32_t), (char*)&i, sizeof(int32_t)); 26: } 27:		
	污染路径: 无		
175	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c		行号: 473
	步骤描述: 强制转换为相同类型.		
	代码片段: 471: 472: TSKEY ts; 473: ts = atol((char*)buf); 474: if (ts < mark) { 475: // statekey winkey.ts < mark		
	污染路径: 无		
176	缺陷发生位置: /source/libs/scalar/src/scvector.c		行号: 134
	步骤描述: 强制转换为相同类型.		

	代码片段: 132: int64_t getVectorBigintValue_INT(void *src, int32_t index) { return (int64_t) * ((int32_t *)src + index); } 133: int64_t getVectorBigintValue_UINT(void *src, int32_t index) { return (int64_t) * ((uint32_t *)src + index); } 134: int64_t getVectorBigintValue_BIGINT(void *src, int32_t index) { return (int64_t) * ((int64_t *)src + index); } 135: int64_t getVectorBigintValue_UBIGINT(void *src, int32_t index) { return (int64_t) * ((uint64_t *)src + index); } 136: int64_t getVectorBigintValue_FLOAT(void *src, int32_t index) { return (int64_t) * ((float *)src + index); }	
	污染路径: 无	
177	缺陷发生位置: /source/libs/command/src/command.c	行号: 40
	步骤描述: 强制转换为相同类型.	
	代码片段: 38: (*pRsp)->compressed = 0; 39: (*pRsp)->compLen = 0; 40: (*pRsp)->numOfRows = htobe64((int64_t)pBlock->info.rows); 41: (*pRsp)->numOfCols = htonl(numOfCols); 42:	
	污染路径: expanded from macro 'htobe64'	
178	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 37
	步骤描述: 强制转换为相同类型.	
	代码片段: 35: 36: for (int32_t i = 1000; i < 2000; ++i) { 37: taosHashRemove(hashTable, (const char*)&i, sizeof(int32_t)); 38: } 39:	
	污染路径: 无	
179	缺陷发生位置:	行号:

	/source/client/src/clientRawBlockWrite.c		1401
	步骤描述: 强制转换为相同类型.		
	代码片段: 1399: } 1400: pVgHash = taosHashInit(16, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), true, HASH_NO_LOCK); 1401: taosHashPut(pVgHash, (const char*)&vgData.vgId, sizeof(vgData.vgId), (char*)&vgData, sizeof(vgData)); 1402: // uError("td23101 1vgId:%d, numEps:%d, name:%s, uid:%"PRIu64, vgData.vgId, vgData.epSet.numOfEps, tbname, 1403: // pTableMeta->uid);		
	污染路径: 无		
180	缺陷发生位置: /source/common/src/ttime.c		行号: 769
	步骤描述: 强制转换为相同类型.		
	代码片段: 767: if (IS_CALENDAR_TIME_DURATION(pInterval->slidingUnit)) { 768: 769: start /= (int64_t)(TSDB_TICK_PER_SECOND(precision)); 770: struct tm tm; 771: time_t tt = (time_t)start;		
	污染路径: 无		
181	缺陷发生位置: /tests/script/api/stmtTest.c		行号: 212
	步骤描述: 强制转换为相同类型.		
	代码片段: 210: v.c5 = (double)0.000000001; 211: for (int j = 0; j < sizeof(v.c6); j++) { 212: v.c6[j] = (char)('a'); 213: } 214: v.c7 = 32767;		
	污染路径: 无		

182	缺陷发生位置: /source/util/src/tjson.c	行号: 368
	步骤描述: 强制转换为相同类型.	
	代码片段: 366: if (!jsonKey) return false; 367: for (size_t j = 0; j < strlen(jsonKey); ++j) { 368: if (isprint(jsonKey[j]) == 0) return false; 369: } 370:	
	污染路径: expanded from macro 'isprint' expanded from macro '___isctype'	
183	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 2582
	步骤描述: 强制转换为相同类型.	
	代码片段: 2580: 2581: // load the last data block of current table 2582: STableBlockScanInfo* pScanInfo = *(STableBlockScanInfo**)pStatus->pTableIter; 2583: if (pScanInfo == NULL) { 2584: tsdbError("table Iter is null, invalid pScanInfo, try next table %s", pReader->idStr);	
	污染路径: 无	
184	缺陷发生位置: /source/common/src/ttime.c	行号: 738
	步骤描述: 强制转换为相同类型.	
	代码片段: 736: } else { 737: skey /= (int64_t)(TSDB_TICK_PER_SECOND(precision)); 738: ekey /= (int64_t)(TSDB_TICK_PER_SECOND(precision)); 739: 740: struct tm tm;	
	污染路径: 无	
185	缺陷发生位置:	行号:

	/source/libs/function/src/udfd.c	794
	步骤描述: 强制转换为相同类型.	
	代码片段: 792: uv_mutex_unlock(&global.udfsMutex); 793: if (unloadUdf) { 794: fnInfo("udf teardown. udf name: %s type %d: context %p", udf->name, udf->scriptType, (void *)(udf->scriptUdfCtx)); 795: uv_cond_destroy(&udf->condReady); 796: uv_mutex_destroy(&udf->lock);	
186	污染路径: 无	
	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 294
	步骤描述: 强制转换为相同类型.	
187	代码片段: 292: bool newItem = true; 293: while (*(dup + i)) { 294: if (isspace(*(dup + i))) { 295: *(dup + i) = 0; 296: ++i;	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
	缺陷发生位置: /include/util/tencode.h	行号: 390
188	步骤描述: 强制转换为相同类型.	
	代码片段: 388: if (TD_CODER_CHECK_CAPACITY_FAILED(pCoder, length)) return -1; 389: if (val) { 390: *val = (uint8_t*)TD_CODER_CURRENT(pCoder); 391: } 392:	
	污染路径: 无	
188	缺陷发生位置: /source/util/src/tutil.c	行号: 384

	步骤描述: 强制转换为相同类型.	
	代码片段: 382: // char *by; 383: 384: len = (int32_t)strlen((char *)hexstr) / 2; 385: 386: for (i = 0; i < len; i++) {	
	污染路径: 无	
189	缺陷发生位置:	行号:
	/source/os/src/osAtomic.c	506
	步骤描述: 强制转换为相同类型.	
	代码片段: 504: return __sync_add_and_fetch((ptr), (val)); 505: #else 506: return __atomic_add_fetch((void**)(ptr), (size_t)(val), __ATOMIC_SEQ_CST); 507: #endif 508: }	
190	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/test/taosbsearchTest.cpp	19
	步骤描述: 强制转换为相同类型.	
191	代码片段: 17: 18: key = 1; 19: pRet = taosbsearch((void *)&key, (void *)array1, 1, sizeof(int), compareFunc, TD_EQ); 20: ASSERT_EQ(pRet, nullptr); 21:	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/common/src/tdatablock.c	1658
	步骤描述: 强制转换为相同类型.	

	代码片段: 1656: } else { 1657: int32_t bytes = pCollInfoData->info.bytes; 1658: memmove(pCollInfoData->pData, ((char*)pCollInfoData->pData + n * bytes), (total - n) * bytes); 1659: doShiftBitmap(pCollInfoData->nullbitmap, n, total); 1660: }	
	污染路径: 无	
192	缺陷发生位置: /source/common/src/ttypes.c	行号: 143
	步骤描述: 强制转换为相同类型.	
	代码片段: 141: break; 142: case TSDB_DATA_TYPE_NCHAR: 143: varDataCopy(val, src); 144: break; 145: default: {	
	污染路径: expanded from macro 'varDataCopy'	
193	缺陷发生位置: /source/util/src/tutil.c	行号: 86
	步骤描述: 强制转换为相同类型.	
	代码片段: 84: int32_t stop = 0; 85: while (z[j] != 0) { 86: if (isspace(z[j]) && stop == 0) { 87: stop = j; 88: } else if (!isspace(z[j]) && stop != 0) {	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
194	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1151
	步骤描述: 强制转换为相同类型.	
	代码片段: 1149: // vglD	

	1150: pCollInfoData = taosArrayGet(p->pDataBlock, 6); 1151: colDataSetVal(pCollInfoData, numOfRows, (char*)&vgId, false); 1152: 1153: int32_t tableType = mr.me.type;	
	污染路径: 无	
195	缺陷发生位置: /source/common/src/tvariant.c	行号: 247
	步骤描述: 强制转换为相同类型.	
	代码片段: 245: case TSDB_DATA_TYPE_JSON: 246: case TSDB_DATA_TYPE_GEOMETRY: 247: return (char *)pVar->pz; 248: case TSDB_DATA_TYPE_NCHAR: 249: return (char *)pVar->ucs4;	
	污染路径: 无	
196	缺陷发生位置: /source/util/src/thash.c	行号: 313
	步骤描述: 强制转换为相同类型.	
	代码片段: 311: return 0; 312: } 313: return (int32_t)atomic_load_64((int64_t *)&pHashObj->size); 314: } 315:	
	污染路径: 无	
197	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 77
	步骤描述: 强制转换为相同类型.	
	代码片段: 75: double v = 0; 76: GET_Typed_DATA(v, double, pVar->nType, &pVar->d); 77: colDataSetVal(pDst, rowIndex, (char*)&v, isNull);	

	78: } else if (IS_SIGNED_NUMERIC_TYPE(pDst->info.type)) { 79: int64_t v = 0;	
	污染路径: 无	
198	缺陷发生位置: /source/util/src/tcrc32c.c	行号: 948
	步骤描述: 强制转换为相同类型.	
	代码片段: 946: } while (next < end); 947: crc0 = shift_crc(long_shifts, (uint32_t)(crc0)) ^ crc1; 948: crc0 = shift_crc(long_shifts, (uint32_t)(crc0)) ^ crc2; 949: next += 2 * LONG_SHIFT; 950: len -= 3 * LONG_SHIFT;	
	污染路径: 无	
199	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 303
	步骤描述: 强制转换为相同类型.	
	代码片段: 301: if (type == TSDB_DATA_TYPE_VARCHAR) { 302: for (int32_t i = 0; i < charLen; ++i) { 303: if (!isspace(*(varDataVal(input) + i))) { 304: break; 305: }	
	污染路径: expanded from macro 'isspace' expanded from macro '__isctype'	
200	缺陷发生位置: /source/common/src/tglobal.c	行号: 1005
	步骤描述: 强制转换为相同类型.	
	代码片段: 1003: 1004: tsSIMDBuiltins = (bool)cfgGetItem(pCfg, "SIMD- builtins")->bval; 1005: tsTagFilterCache = (bool)cfgGetItem(pCfg, "tagFilterCache")->bval; 1006:	

	1007: tsEnableMonitor = cfgGetItem(pCfg, "monitor")->bval;	
	污染路径: 无	
201	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 2404
	步骤描述: 强制转换为相同类型.	
	代码片段: 2402: } else { 2403: double result = max - min; 2404: colDataSetVal(pOutputData, 0, (char *)&result, false); 2405: } 2406:	
	污染路径: 无	
202	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 130
	步骤描述: 强制转换为相同类型.	
	代码片段: 128: candidate = (readSize - (haystack - buf)) > 0 ? haystack : NULL; 129: } else { 130: candidate = tmemmem(haystack, readSize - (haystack - buf), (char*)&magic, sizeof(magic)); 131: } 132:	
	污染路径: 无	
203	缺陷发生位置: /source/util/src/tcrc32c.c	行号: 846
	步骤描述: 强制转换为相同类型.	
	代码片段: 844: --length; 845: } 846: return (uint32_t)crc ^ 0xffffffff; 847: } 848: #if !defined(_TD_ARM_) && !defined(_TD_MIPS_) && ! defined(_TD_LOONGARCH_)	

	污染路径: 无	
204	缺陷发生位置: /source/client/src/clientEnv.c	行号: 62
	步骤描述: 强制转换为相同类型.	
	代码片段: 60: SAppClusterSummary *pSummary = &pTscObj->pAppInfo->summary; 61: 62: int32_t total = atomic_add_fetch_64((int64_t *)&pSummary->totalRequests, 1); 63: int32_t currentInst = atomic_add_fetch_64((int64_t *)&pSummary->currentRequests, 1); 64: tscDebug("0x%" PRIx64 " new Request from connObj:0x%" PRIx64	
	污染路径: 无	
205	缺陷发生位置: /source/client/src/clientImpl.c	行号: 1046
	步骤描述: 强制转换为相同类型.	
	代码片段: 1044: atomic_add_fetch_64((int64_t*)&pActivity->numOfInsertsReq, 1); 1045: } else if (QUERY_NODE_SELECT_STMT == pQuery->pRoot->type) { 1046: atomic_add_fetch_64((int64_t*)&pActivity->numOfQueryReq, 1); 1047: } 1048: }	
	污染路径: 无	
206	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 1948
	步骤描述: 强制转换为相同类型.	
	代码片段: 1946: return TSDB_CODE_RPC_NETWORK_UNAVAIL; 1947: }	

	1948: char* pStatus = taosStrdup((void*)"1"); 1949: return rewriteFuncToValue(pCxt, pStatus, pNode); 1950: }	
	污染路径: 无	
207	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 109
	步骤描述: 强制转换为相同类型.	
	代码片段: 107: if (pi->length >= size) { 108: offset = pi->offset; 109: pi->offset += (int32_t)size; 110: pi->length -= (int32_t)size; 111:	
	污染路径: 无	
208	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 4752
	步骤描述: 强制转换为相同类型.	
	代码片段: 4750: 4751: while ((p = tSimpleHashIterate(pStatus->pTableMap, p, &iter)) != NULL) { 4752: STableBlockScanInfo* pInfo = *(STableBlockScanInfo**)p; 4753: 4754: pInfo->iterInit = false;	
	污染路径: 无	
209	缺陷发生位置: /tests/script/api/stmtTest.c	行号: 210
	步骤描述: 强制转换为相同类型.	
	代码片段: 208: v.c3 = (int64_t)2147483648; 209: v.c4 = (float)0.1; 210: v.c5 = (double)0.000000001; 211: for (int j = 0; j < sizeof(v.c6); j++) {	

	212: v.c6[j] = (char)('a');	
	污染路径: 无	
210	缺陷发生位置: /examples/c/prepare.c	行号: 179
	步骤描述: 强制转换为相同类型.	
	代码片段: 177: params[j].is_null = ((i == j) ? &is_null : 0); 178: } 179: v.b = (int8_t)i % 2; 180: v.v1 = (int8_t)i; 181: v.v2 = (int16_t)(i * 2);	
	污染路径: 无	
211	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1872
	步骤描述: 强制转换为相同类型.	
	代码片段: 1870: 1871: SStateKey sKey = {.key = *key, .opNum = pState->number}; 1872: STREAM_STATE_PUT_ROCKSDB(pState, "state", &sKey, (void*)value, vLen); 1873: return code; 1874: }	
	污染路径: expanded from macro 'STREAM_STATE_PUT_ROCKSDB'	
212	缺陷发生位置: /source/util/src/tref.c	行号: 68
	步骤描述: 强制转换为相同类型.	
	代码片段: 66: taosThreadOnce(&tsRefModuleInit, taosInitRefModule); 67: 68: nodeList = taosMemoryCalloc(sizeof(SRefNode *), (size_t)max); 69: if (nodeList == NULL) { 70: terrno = TSDB_CODE_OUT_OF_MEMORY;	

	污染路径: 无	
213	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 304
	步骤描述: 强制转换为相同类型.	
	代码片段: 302: } 303: 304: if (streamSnapHandleInit(&pReader->handle, (char*)path, chkpld, pMeta) < 0) { 305: taosMemoryFree(pReader); 306: return -1;	
	污染路径: 无	
214	缺陷发生位置: /source/util/test/taosbsearchTest.cpp	行号: 23
	步骤描述: 强制转换为相同类型.	
	代码片段: 21: 22: key = 6; 23: pRet = taosbsearch((void *)&key, (void *)array1, 1, sizeof(int), compareFunc, TD_EQ); 24: ASSERT_EQ(pRet, nullptr); 25:	
	污染路径: 无	
215	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 66
	步骤描述: 强制转换为相同类型.	
	代码片段: 64: *p = '\0'; 65: new_val = (int64_t)(uval / 10); 66: *--p = '0' + (char)(uval - (uint64_t)new_val * 10); 67: val = new_val; 68:	
	污染路径:	

	无	
216	缺陷发生位置: /source/common/src/tmsg.c	行号: 5393
	步骤描述: 强制转换为相同类型.	
	代码片段: 5391: int32_t tDeserializeSMqAskEpReq(void *buf, int32_t bufLen, SMqAskEpReq *pReq) { 5392: SDecoder decoder = {0}; 5393: tDecoderInit(&decoder, (char *)buf, bufLen); 5394: 5395: if (tStartDecode(&decoder) < 0) return -1;	
	污染路径: 无	
217	缺陷发生位置: /source/util/src/tcompression.c	行号: 517
	步骤描述: 强制转换为相同类型.	
	代码片段: 515: output[pos] = t; 516: } else if (input[i] == 0) { 517: t = ((uint8_t)1 << (2 * (i % ele_per_byte))) - 1; 518: /* t = (~(((uint8_t)1) << (7-i%BITS_PER_BYTE))); */ 519: output[pos] &= t;	
	污染路径: 无	
218	缺陷发生位置: /source/util/src/tlog.c	行号: 596
	步骤描述: 强制转换为相同类型.	
	代码片段: 594: if (c >= 16) { 595: temp[pos++] = '\n'; 596: taosWriteFile(tsLogObj.logHandle->pFile, temp, (uint32_t)pos); 597: c = 0; 598: pos = 0;	
	污染路径: 无	
219	缺陷发生位置:	行号:

	/source/os/src/osFile.c		394
	步骤描述: 强制转换为相同类型.		
	代码片段: 392: readbytes = _read(pFile->fd, (void *)tbuf, (uint32_t)leftbytes); 393: #else 394: readbytes = read(pFile->fd, (void *)tbuf, (uint32_t)leftbytes); 395: #endif 396: if (readbytes < 0) {		
	污染路径: 无		
220	缺陷发生位置: /source/client/src/clientMain.c		行号: 195
	步骤描述: 强制转换为相同类型.		
	代码片段: 193: const char *taos_errstr(TAOS_RES *res) { 194: if (res == NULL TD_RES_TMQ_META(res)) { 195: return (const char *)tstrerror(terrno); 196: } 197:		
	污染路径: 无		
221	缺陷发生位置: /source/libs/executor/src/scanoperator.c		行号: 3285
	步骤描述: 强制转换为相同类型.		
	代码片段: 3283: 3284: int32_t dumpQueryTableCond(const SQueryTableDataCond* src, SQueryTableDataCond* dst) { 3285: memcpy((void*)dst, (void*)src, sizeof(SQueryTableDataCond)); 3286: dst->colList = taosMemoryCalloc(src->numOfCols, sizeof(SColumnInfo)); 3287: for (int i = 0; i < src->numOfCols; i++) {		
	污染路径: 无		

222	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 333
	步骤描述: 强制转换为相同类型.	
	代码片段: 331: if (type == TSDB_DATA_TYPE_VARCHAR) { 332: for (int32_t i = charLen - 1; i >= 0; --i) { 333: if (!isspace(*(varDataVal(input) + i))) { 334: break; 335: } }	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
223	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 258
	步骤描述: 强制转换为相同类型.	
	代码片段: 256: 257: if (tTagIsJson(pTag)) { 258: STag* p = (STag*)pTag; 259: if (p->nTag == 0) { 260: goto end; }	
	污染路径: 无	
224	缺陷发生位置: /source/common/src/ttime.c	行号: 737
	步骤描述: 强制转换为相同类型.	
	代码片段: 735: if (order == TSDB_ORDER_DESC && ret * interval < (ekey - skey)) ret += 1; 736: } else { 737: skey /= (int64_t)(TSDB_TICK_PER_SECOND(precision)); 738: ekey /= (int64_t)(TSDB_TICK_PER_SECOND(precision)); 739:	
	污染路径: 无	
225	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 1063

	步骤描述: 强制转换为相同类型.	
	代码片段: 1061: // number of columns 1062: pColumnInfoData = taosArrayGet(p->pDataBlock, 3); 1063: colDataSetVal(pColumnInfoData, numOfRows, (char*)&pm->colNum, false); 1064: 1065: for (int32_t j = 4; j <= 8; ++j) {	
	污染路径: 无	
226	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbRead.c	3181
	步骤描述: 强制转换为相同类型.	
	代码片段: 3179: 3180: // load the last data block of current table 3181: STableBlockScanInfo* pScanInfo = *(STableBlockScanInfo**)pStatus->pTableIter; 3182: if (pScanInfo == NULL) { 3183: tsdbError("table iter is null, invalid pScanInfo, try next table %s", pReader->idStr);	
	污染路径: 无	
227	缺陷发生位置:	行号:
	/source/libs/function/src/builtinsimpl.c	5879
	步骤描述: 强制转换为相同类型.	
	代码片段: 5877: } 5878: 5879: return (duration > 0) ? ((double)diff) / (duration / tickPerSec) : 0.0; 5880: } 5881:	
	污染路径: 无	
228	缺陷发生位置:	行号:
	/source/common/src/tglobal.c	1004

	步骤描述: 强制转换为相同类型.	
	代码片段: 1002: tsRpcQueueMemoryAllowed = cfgGetItem(pCfg, "rpcQueueMemoryAllowed")->i64; 1003: 1004: tsSIMDBuiltins = (bool)cfgGetItem(pCfg, "SIMD-builtins")->bval; 1005: tsTagFilterCache = (bool)cfgGetItem(pCfg, "tagFilterCache")->bval; 1006:	
	污染路径: 无	
229	缺陷发生位置:	行号:
	/source/util/test/taosbsearchTest.cpp	35
	步骤描述: 强制转换为相同类型.	
	代码片段: 33: 34: key = 1; 35: pRet = taosbsearch((void *)&key, (void *)array2, 2, sizeof(int), compareFunc, TD_EQ); 36: ASSERT_EQ(pRet, nullptr); 37:	
	污染路径: 无	
230	缺陷发生位置:	行号:
	/source/common/src/tddataformat.c	3415
	步骤描述: 强制转换为相同类型.	
	代码片段: 3413: if (HAS_VALUE == pColData->flag) { 3414: for (int32_t iVal = 0; iVal < pColData->nVal; iVal++) { 3415: val = ((uint8_t *)pColData->pData)[iVal]; 3416: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3417: } 	
	污染路径: 无	

231	缺陷发生位置： /source/libs/function/src/builtinsimpl.c	行号： 4092
	步骤描述： 强制转换为相同类型.	
	代码片段： 4090: hllBucketHisto(buckets, buckethisto); 4091: 4092: double z = m * hllTau((m - buckethisto[HLL_DATA_BITS + 1]) / (double)m); 4093: for (int j = HLL_DATA_BITS; j >= 1; --j) { 4094: z += buckethisto[j];	
	污染路径： 无	
232	缺陷发生位置： /source/util/src/tarray.c	行号： 244
	步骤描述： 强制转换为相同类型.	
	代码片段： 242: 243: int32_t remain = (int32_t)(pArray->size - index); 244: memmove((char*)dst + pArray->elemSize, (char*)dst, pArray->elemSize * remain); 245: memcpy(dst, pData, pArray->elemSize); 246:	
	污染路径： 无	
233	缺陷发生位置： /source/libs/transport/src/transCli.c	行号： 1599
	步骤描述： 强制转换为相同类型.	
	代码片段： 1597: addr.sin_family = AF_INET; 1598: addr.sin_addr.s_addr = ipaddr; 1599: addr.sin_port = (uint16_t)htons(port); 1600: 1601: tGTrace("%s conn %p try to connect to %s", pTransInst->label, conn, conn->dstAddr);	
	污染路径： 无	
234	缺陷发生位置：	行号：

	/source/client/src/clientStmt.c	949
	步骤描述: 强制转换为相同类型.	
	代码片段: 947: 948: if (stmt == NULL NULL == pStmt->exec.pRequest) { 949: return (char*)tstrerror(terrno); 950: } 951:	
	污染路径: 无	
235	缺陷发生位置: /source/util/src/thashutil.c	行号: 39
	步骤描述: 强制转换为相同类型.	
	代码片段: 37: uint32_t result = 0x55555555; 38: for (uint32_t i = 0; i < len; i++) { 39: result ^= (uint8_t)key[i]; 40: result = ROTL32(result, 5); 41: }	
	污染路径: 无	
236	缺陷发生位置: /source/client/src/clientTmq.c	行号: 446
	步骤描述: 强制转换为相同类型.	
	代码片段: 444: static int32_t tmqCommitCb(void* param, SDataBuf* pBuf, int32_t code) { 445: SMqCommitCbParam* pParam = (SMqCommitCbParam*)param; 446: SMqCommitCbParamSet* pParamSet = (SMqCommitCbParamSet*)pParam->params; 447: 448: // taosMemoryFree(pParam->pOffset);	
	污染路径: 无	
237	缺陷发生位置: /source/libs/scalar/src/scvector.c	行号: 211

	步骤描述: 强制转换为相同类型.	
	代码片段: 209: int64_t minValue = tDataTypes[pOut->columnData->info.type].minValue; 210: int64_t maxValue = tDataTypes[pOut->columnData->info.type].maxValue; 211: int64_t value = (int64_t)taosStr2Int64(buf, NULL, 10); 212: if (value > maxValue) { 213: *overflow = 1;	
	污染路径: 无	
238	缺陷发生位置:	行号:
	/tests/script/api/dbTableRoute.c	113
	步骤描述: 强制转换为相同类型.	
	代码片段: 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: } 115:	
239	污染路径: 无	
	缺陷发生位置:	行号:
	/utils/test/c/tmqSim.c	345
	步骤描述: 强制转换为相同类型.	
240	代码片段: 343: int len = 0; 344: char* p = str; 345: while (*p != '\0' && isspace(*p)) { 346: ++p; 347: ++len;	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
	缺陷发生位置:	行号:
	/source/os/src/osString.c	501

	步骤描述: 强制转换为相同类型.	
	代码片段: 499: for (int i = n - 1; i >= 0; i--) { 500: if (z[i] >= 'a') { 501: *byte = ((uint8_t)(10 + (z[i] - 'a'))) << (num * 4)); 502: } else if (z[i] >= 'A') { 503: *byte = ((uint8_t)(10 + (z[i] - 'A'))) << (num * 4));	
	污染路径: 无	
241	缺陷发生位置:	行号:
	/tools/shell/src/shellWebsocket.c	354
	步骤描述: 强制转换为相同类型.	
	代码片段: 352: et = taosGetTimestampUs(); 353: double total_time = (et - st)/1E3; 354: double net_time = total_time - (double)execute_time; 355: printf("Query Ok, %d of %d row(s) in database\n", numOfRows, numOfRows); 356: printf("Execute: %.2f ms Network: %.2f ms Total: %.2f ms\n",	
242	污染路径: 无	
	缺陷发生位置:	行号:
	/utils/test/c/tmqSim.c	1459
	步骤描述: 强制转换为相同类型.	
243	代码片段: 1457: printf("Spent %.3f seconds to cons rows: %" PRIu64 " msgs: %" PRIu64 1458: " with %d thread(s), throughput: %.3f msgs/s, %.1f MB/s\n\n", 1459: tInMs, totalRows, totalMsgs, g_stConflInfo.numOfThread, (double)(totalMsgs / tInMs), 1460: (double)totalLenOfMsgs / (1024 * 1024) / tInMs); 1461:	
	污染路径: 无	
	缺陷发生位置:	行号:

246	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 1305
	步骤描述: 强制转换为相同类型.	
	代码片段: 1303: int64_t delta = currentTs - startTs; 1304: if (delta < SEND_TIME_UNIT * 1000) { 1305: int64_t sleepLen = (int32_t)(SEND_TIME_UNIT * 1000 - delta); 1306: // printf("sleep %" PRId64 " us, use time: %" PRId64 " us\n", sleepLen, delta); 1307: taosUsleep((int32_t)sleepLen);	
	污染路径: 无	
247	缺陷发生位置: /source/os/src/osFile.c	行号: 131
	步骤描述: 强制转换为相同类型.	
	代码片段: 129: size += bytes; 130: 131: if (taosWriteFile(pFileTo, (void *)buffer, bytes) < bytes) goto _err; 132: if (bytes < sizeof(buffer)) break; 133: }	
	污染路径: 无	
248	缺陷发生位置: /source/libs/index/test/indexBench.cc	行号: 88
	步骤描述: 强制转换为相同类型.	
	代码片段: 86: char buf[256] = {0}; 87: int16_t sz = nColVal; 88: memcpy(buf, (uint16_t *)&sz, 2); 89: memcpy(buf + 2, colVal, nColVal); 90: if (colType == TSDB_DATA_TYPE_BINARY colType == TSDB_DATA_TYPE_GEOMETRY) {	
	污染路径: 无	

249	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 981
	步骤描述: 强制转换为相同类型.	
	代码片段: 979: data->smallData[i] = (int16_t)i; 980: data->usmallData[i] = (uint16_t)(i+1); 981: data->intData[i] = (int32_t)i; 982: data->uintData[i] = (uint32_t)(i+1); 983: data->bigData[i] = (int64_t)i;	
	污染路径: 无	
250	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1361
	步骤描述: 强制转换为相同类型.	
	代码片段: 1359: len += taosEncodeFixedl64((void**)&buf, key.unixTimestamp); 1360: len += taosEncodeFixedl32((void**)&buf, key.len); 1361: len += taosEncodeBinary((void**)&buf, (char*)value, vlen); 1362: } 1363: return len;	
	污染路径: 无	
251	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 1889
	步骤描述: 强制转换为相同类型.	
	代码片段: 1887: tm.tm_year = mon / 12; 1888: tm.tm_mon = mon % 12; 1889: tw->skey = convertTimePrecision((int64_t)taosMktime(&tm) * 1000LL, TSDB_TIME_PRECISION_MILLI, pInterval->precision); 1890: 1891: tw->ekey = taosTimeAdd(tw->skey, pInterval->interval, pInterval->intervalUnit, pInterval->precision) - 1;	
	污染路径:	

	无	
252	缺陷发生位置: /source/os/src/osSocket.c	行号: 739
	步骤描述: 强制转换为相同类型.	
	代码片段: 737: // printf("open tcp server socket:0x%x:%hu", ip, port); 738: 739: bzero((char *)&serverAdd, sizeof(serverAdd)); 740: serverAdd.sin_family = AF_INET; 741: #ifdef WINDOWS	
	污染路径: 无	
253	缺陷发生位置: /source/common/src/tmsg.c	行号: 4359
	步骤描述: 强制转换为相同类型.	
	代码片段: 4357: if (tEncode18(&encoder, pReq->isTsm) < 0) return -1; 4358: if (pReq->isTsm) { 4359: uint32_t tsmLen = (uint32_t)(htonl(((SMsgHead *)pReq->pTsm)->contLen)); 4360: if (tEncodeBinary(&encoder, (const uint8_t *)pReq->pTsm, tsmLen) < 0) return -1; 4361: }	
	污染路径: 无	
254	缺陷发生位置: /source/os/src/osSocket.c	行号: 746
	步骤描述: 强制转换为相同类型.	
	代码片段: 744: serverAdd.sin_addr.s_addr = ip; 745: #endif 746: serverAdd.sin_port = (uint16_t)htons(port); 747: 748: fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);	
	污染路径: 无	
255	缺陷发生位置:	行号:

	/source/util/test/taosbsearchTest.cpp		27
	步骤描述: 强制转换为相同类型.		
	代码片段: 25: 26: key = 5; 27: pRet = taosbsearch((void *)&key, (void *)array1, 1, sizeof(int), compareFunc, TD_EQ); 28: ASSERT_NE(pRet, nullptr); 29: ASSERT_EQ(*(int *)pRet, key);		
	污染路径: 无		
256	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c		行号: 1052
	步骤描述: 强制转换为相同类型.		
	代码片段: 1050: int defaultKeyEncode(void* k, char* buf) { 1051: int len = strlen((char*)k); 1052: memcpy(buf, (char*)k, len); 1053: return len; 1054: }		
	污染路径: 无		
257	缺陷发生位置: /source/util/src/tcompression.c		行号: 152
	步骤描述: 强制转换为相同类型.		
	代码片段: 150: break; 151: case TSDB_DATA_TYPE_BIGINT: 152: curr_value = (int64_t)((int64_t *)input + j)); 153: break; 154: }		
	污染路径: 无		
258	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c		行号: 418
	步骤描述: 强制转换为相同类型.		

	代码片段: 416: SName name = {0}; 417: tNameFromString(&name, stbFullName, T_NAME_ACCT T_NAME_DB T_NAME_TABLE); 418: pCreateTbReq->ctb.stbName = taosStrdup((char*)tNameGetTableName(&name)); 419: 420: STagVal tagVal = { .cid = numOfCols, .type = TSDB_DATA_TYPE_UBIGINT, .i64 = pDataBlock-> info.id.groupId};	
	污染路径: 无	
259	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 30
	步骤描述: 强制转换为相同类型.	
	代码片段: 28: } 29: 30: rslt = (SArray *)taosArrayInit(10, sizeof(uint64_t)); 31: } 32: virtual void TearDown() {	
	污染路径: 无	
260	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 709
	步骤描述: 强制转换为相同类型.	
	代码片段: 707: elements->timestamp = sql; 708: while (sql < sqlEnd) { 709: if (unlikely(isspace(*sql))) { 710: break; 711: }	
	污染路径: expanded from macro 'isspace' expanded from macro '___isctype'	
261	缺陷发生位置: /source/os/src/osString.c	行号: 503
	步骤描述:	

	强制转换为相同类型.	
	代码片段: 501: *byte = ((uint8_t)(10 + (z[i] - 'a')) << (num * 4)); 502: } else if (z[i] >= 'A') { 503: *byte = ((uint8_t)(10 + (z[i] - 'A')) << (num * 4)); 504: } else { 505: *byte = ((uint8_t)(z[i] - '0') << (num * 4));	
	污染路径: 无	
262	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 496
	步骤描述: 强制转换为相同类型.	
	代码片段: 494: (*pRsp)->compressed = 0; 495: (*pRsp)->compLen = 0; 496: (*pRsp)->numOfRows = htobe64((int64_t)pBlock->info.rows); 497: (*pRsp)->numOfCols = htonl(SHOW_VARIABLES_RESULT_COLS); 498:	
	污染路径: expanded from macro 'htobe64'	
263	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 520
	步骤描述: 强制转换为相同类型.	
	代码片段: 518: */ 519: if (isdigit(z[1])) { 520: for (i = 2; isdigit(z[i]); i++) { 521: } 522:	
	污染路径: expanded from macro 'isdigit' expanded from macro ' __isctype'	
264	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 184
	步骤描述: 强制转换为相同类型.	

	代码片段: 182: 183: static int32_t tlvEncodeValueU64(STlvEncoder* pEncoder, uint64_t value) { 184: value = htonl(value); 185: return tlvEncodeValueImpl(pEncoder, &value, sizeof(value)); 186: }	
	污染路径: expanded from macro 'htonll'	
265	缺陷发生位置: /source/common/src/tmsg.c	行号: 5547
	步骤描述: 强制转换为相同类型.	
	代码片段: 5545: if (tEncodeCStrWithLen(&encoder, pReq->sql, pReq-> sqlLen) < 0) return -1; 5546: if (tEncodeU32(&encoder, pReq->msgLen) < 0) return -1; 5547: if (tEncodeBinary(&encoder, (uint8_t *)pReq->msg, pReq->msgLen) < 0) return -1; 5548: 5549: tEndEncode(&encoder);	
	污染路径: 无	

12.2.1.3 WK4750 double 型变量赋值给 float 型变量必须强

制转换

缺陷数量： 2 个		
缺陷描述: double 型变量赋值给 float 型变量必须强制转换。		
修复意见: 无		
1	缺陷发生位置: /source/libs/parser/src/parlInsertSql.c	行号: 486

	步骤描述: double 型变量 dv 赋值给 float 型变量必须强制转换	
	代码片段: 484: return buildSyntaxErrMsg(pMsgBuf, "illegal float data", pToken->z); 485: } 486: *(float*)&val->i64 = dv; 487: break; 488: }	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/source/os/src/osSysinfo.c	457
	步骤描述: double 型变量 atof 赋值给 float 型变量必须强制转换	
	代码片段: 455: } else if (((done & 2) == 0) && strcmp(line, "cpu cores", 9) == 0) { 456: const char *v = strchr(line, ':') + 2; 457: *numOfCores = atof(v); 458: done = 2; 459: }	
	污染路径:	
	无	

12.2.1.4 WK4794 指针或引用类型的转换中禁止移除 const 或

volatile 属性

缺陷数量： 77 个		
缺陷描述: 指针或引用类型的转换中禁止移除 const 或 volatile 属性。		
修复意见: 无		
1	缺陷发生位置:	行号:
	/source/common/src/ttime.c	68
步骤描述:		

	A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 66: 67: bool checkTzPresent(const char* str, int32_t len) { 68: char* seg = forwardToTimeStringEnd((char*)str); 69: int32_t seg_len = len - (int32_t)(seg - str); 70:	
	污染路径: 无	
2	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 1796
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1794: // ignore the table not exists error, since this table may have been dropped during the scan procedure. 1795: if (code != TSDB_CODE_SUCCESS && code != TSDB_CODE_PAR_TABLE_NOT_EXIST) { 1796: blockDataFreeRes((SSDataBlock*)pBlock); 1797: T_LONG_JMP(pTaskInfo->env, code); 1798: }	
	污染路径: 无	
3	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 76
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 74: 75: char *p1 = (char *)aBuf; 76: char *p2 = (char *)bBuf; 77: // p1 += 1; 78: // p2 += 1;	
	污染路径: 无	
4	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 365

	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 363: if (ptr == NULL) return; 364: 365: return free((void*)ptr); 366: } 367:	
	污染路径: 无	
5	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndDef.c	495
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 493: // #endif 494: 495: return (void *)buf; 496: } 497:	
6	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/transport/test/transUT.cpp	51
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
7	代码片段: 49: memset(&rpclnit_, 0, sizeof(rpclnit_)); 50: rpclnit_.localPort = 0; 51: rpclnit_.label = (char *)label; 52: rpclnit_.numOfThreads = nThread; 53: rpclnit_.cfp = processResp;	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/transport/test/transUT.cpp	118
步骤描述:		

	A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 116: memcpy(rpcInit_.localFqdn, "localhost", strlen("localhost")); 117: rpcInit_.localPort = port; 118: rpcInit_.label = (char *)label; 119: rpcInit_.numOfThreads = 5; 120: rpcInit_.cfp = processReq;	
	污染路径: 无	
8	缺陷发生位置:	行号:
	/source/libs/transport/test/cliBench.c	42
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 40: tsAsyncLog = 0; 41: // idxDebugFlag = 143; 42: strcpy(tsLogDir, (char *)logDir); 43: taosRemoveDir(tsLogDir); 44: taosMkDir(tsLogDir);	
9	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/parser/test/parAlterToBalanceTest.cpp	658
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
10	代码片段: 656: clearAlterTbReq(); 657: 658: setAlterTableOptions("t1", -1, (char*)"test"); 659: run("ALTER TABLE t1 COMMENT 'test'"); 660: clearAlterTbReq();	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/sync/test/syncRaftCfgTest.cpp	65
	步骤描述:	

	A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 63: void test3() { 64: SSyncCfg* pCfg = createSyncCfg(); 65: char* s = (char*)"./test3_raft_cfg.json"; 66: 67: if (taosCheckExistFile(s)) {	
	污染路径: 无	
11	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 1081
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1079: rpcSendRecv(pConn->pTrans, &pConn->mgmtEps, &rpcMsg, &rpcRsp); 1080: 1081: CTG_ERR_RET(ctgProcessRspMsg(out, reqType, rpcRsp.pCont, rpcRsp.contLen, rpcRsp.code, (char*)user)); 1082: 1083: rpcFreeCont(rpcRsp.pCont);	
	污染路径: 无	
12	缺陷发生位置: /source/common/src/ttime.c	行号: 62
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 60: return parseTimeWithTz(timestr, utime, timePrec, 0); 61: } else { 62: return parseLocaltimeDst((char*)timestr, len, utime, timePrec, 0); 63: } 64: }	
	污染路径: 无	
13	缺陷发生位置:	行号:

	/source/libs/executor/src/exchangeoperator.c	569
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 567: if (pColList == NULL) { // data from other sources 568: blockDataCleanup(pRes); 569: *pNextStart = (char*)blockDecode(pRes, pData); 570: } else { // extract data according to pColList 571: char* pStart = pData;	
	污染路径: 无	
14	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1131
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1129: memset(&w2, 0, sizeof(w2)); 1130: 1131: char* p1 = (char*)aBuf; 1132: char* p2 = (char*)bBuf; 1133:	
	污染路径: 无	
15	缺陷发生位置: /source/common/src/tmsg.c	行号: 192
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 190: buf = taosDecodeStringTo(buf, pEp->eps[i].fqdn); 191: } 192: return (void *)buf; 193: } 194:	
	污染路径: 无	
16	缺陷发生位置: /source/libs/sync/test/syncTest.cpp	行号: 32

	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 30: void test1() { 31: sDebugFlag = DEBUG_TRACE + DEBUG_SCREEN + DEBUG_FILE; 32: logTest((char*)__FUNCTION__); 33: } 34:	
	污染路径: 无	
17	缺陷发生位置: /source/dnode/mnode/impl/src/mndSma.c	行号: 679
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 677: SName smaName = {0}; 678: if (tNameFromString(&smaName, pCreate->name, T_NAME_ACCT T_NAME_DB T_NAME_TABLE) < 0) return -1; 679: if (*(char *)tNameGetTableName(&smaName) == 0) return -1; 680: 681: terrno = 0;	
	污染路径: 无	
18	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 599
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 597: data = tTagValToData((const STagVal*)p, false); 598: } else { 599: data = (char*)p; 600: } 601:	
	污染路径: 无	

19	缺陷发生位置: /source/common/src/ttime.c	行号: 56
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 54: return parseTimeWithTz(timestr, utime, timePrec, 'T'); 55: } else { 56: return parseLocaltimeDst((char*)timestr, len, utime, timePrec, 'T'); 57: } 58: } else {	
	污染路径: 无	
20	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1077
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1075: 1076: char* p1 = (char*)aBuf; 1077: char* p2 = (char*)bBuf; 1078: 1079: p1 = taosDecodeFixedU64(p1, &key1.key.groupId);	
	污染路径: 无	
21	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 78
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 76: size_t size = 0; 77: 78: code = doGeomFromText(varDataVal(input), &outputGeom, &size); 79: if (code != TSDB_CODE_SUCCESS) { 80: goto _exit;	
	污染路径:	

	expanded from macro 'varDataVal'	
22	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 121
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 119: rpclnit_.numOfThreads = 5; 120: rpclnit_.cfp = processReq; 121: rpclnit_.user = (char *)user; 122: rpclnit_.connType = TAOS_CONN_SERVER; 123: taosVersionStrToInt(version, &(rpclnit_.compatibilityVer));	
	污染路径: 无	
23	缺陷发生位置: /source/util/src/thash.c	行号: 592
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 590: int32_t num = 0; 591: 592: taosHashRLock((SHashObj *)pHashObj); 593: for (int32_t i = 0; i < pHashObj->size; ++i) { 594: SHashEntry *pEntry = pHashObj->hashList[i];	
	污染路径: 无	
24	缺陷发生位置: /source/common/src/tatablock.c	行号: 1789
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1787: taosArrayPush(pBlock->pDataBlock, &data); 1788: } 1789: return (void*)buf; 1790: } 1791:	
	污染路径:	

	无	
25	缺陷发生位置: /source/util/src/tcache.c	行号: 917
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre> 915: SCachelter *taosCacheCreatelter(const SCacheObj *pCacheObj) { 916: SCachelter *plter = taosMemoryCalloc(1, sizeof(SCachelter)); 917: plter->pCacheObj = (SCacheObj *)pCacheObj; 918: plter->entryIndex = -1; 919: plter->index = -1; </pre>	
	污染路径: 无	
26	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 522
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre> 520: for (int i = 0; i < sizeof(sql) / sizeof(sql[0]); i++) { 521: SSmlLineInfo elements = {0}; 522: int ret = smlParseTelnetString(info, (char*)sql[i], (char*)(sql[i] + strlen(sql[i])), &elements); 523: // printf("i:%d\n", i); 524: ASSERT_EQ(ret, 0); </pre>	
	污染路径: 无	
27	缺陷发生位置: /source/libs/sync/test/syncTest.cpp	行号: 42
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre> 40: void test3() { 41: sDebugFlag = DEBUG_INFO + DEBUG_SCREEN + DEBUG_FILE; 42: logTest((char*)__FUNCTION__); 43: } </pre>	

	44:	
	污染路径: 无	
28	缺陷发生位置: /include/common/tmsg.h	行号: 704
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 702: pSW->pSchema = NULL; 703: } 704: return (void*)buf; 705: } 706:	
	污染路径: 无	
29	缺陷发生位置: /source/libs/sync/test/syncTest.cpp	行号: 47
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 45: void test4() { 46: sDebugFlag = DEBUG_ERROR + DEBUG_SCREEN + DEBUG_FILE; 47: logTest((char*)__FUNCTION__); 48: } 49:	
	污染路径: 无	
30	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1132
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1130: 1131: char* p1 = (char*)aBuf; 1132: char* p2 = (char*)bBuf; 1133:	

	1134: p1 = taosDecodeFixedl64(p1, &w1.key.win.skey);	
	污染路径: 无	
31	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 575
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 573: break; 574: } 575: return (void *)buf; 576: } 577:	
	污染路径: 无	
32	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 1233
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1231: char dbFName[TSDB_DB_FNAME_LEN]; 1232: tNameGetFullDbName(pTableName, dbFName); 1233: SBuildTableInput blInput = {.vgId = vgroupInfo->vgId, .dbFName = dbFName, .tbName = (char*)pTableName->tname}; 1234: 1235: SEp* pEp = &vgroupInfo->epSet.eps[vgroupInfo->epSet.inUse];	
	污染路径: 无	
33	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 71
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 69: 70: // make input as a zero ending string	

	71: char *end = varDataVal(input) + varDataLen(input); 72: char endValue = *end; 73: *end = 0;	
	污染路径: expanded from macro 'varDataVal'	
34	缺陷发生位置: /include/common/tmsg.h	行号: 658
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 656: buf = taosDecodeFixedI16(buf, &pSchema->colId); 657: buf = taosDecodeStringTo(buf, pSchema->name); 658: return (void*)buf; 659: } 660:	
	污染路径: 无	
35	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 234
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 232: } 233: buf = taosDecodeSEpSet(buf, &pVgEp->epSet); 234: return (void *)buf; 235: } 236:	
	污染路径: 无	
36	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 503
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 501: for (int i = 0; i < sizeof(sql) / sizeof(sql[0]); i++) { 502: SSmlLineInfo elements = {0}; 503: int ret = smlParseTelnetString(info, (char*)sql[i],	

	<pre>(char*)(sql[i] + strlen(sql[i])), &elements); 504: // printf("i:%d\n", i); 505: ASSERT_NE(ret, 0);</pre>	
	污染路径: 无	
37	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 393
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre>391: taosMemoryFree(newBuf); 392: } else { 393: memcpy(varDataVal(output) + *dataLen, varDataVal(input), varDataLen(input)); 394: *dataLen += varDataLen(input); 395: }</pre>	
	污染路径: expanded from macro 'varDataVal'	
38	缺陷发生位置: /source/util/src/tarray.c	行号: 505
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre>503: taosArrayPush(*pArray, &data); 504: } 505: return (void*)buf; 506: } 507:</pre>	
	污染路径: 无	
39	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 54
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: <pre>52: rpclnit_.numOfThreads = nThread; 53: rpclnit_.cfp = processResp;</pre>	

	54: rpclnit_.user = (char *)user; 55: rpclnit_.parent = this; 56: rpclnit_.connType = TAOS_CONN_CLIENT;	
	污染路径: 无	
40	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1076
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1074: memset(&key2, 0, sizeof(key2)); 1075: 1076: char* p1 = (char*)aBuf; 1077: char* p2 = (char*)bBuf; 1078:	
	污染路径: 无	
41	缺陷发生位置: /source/os/src/osString.c	行号: 339
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 337: size_t i; 338: 339: if (!*pattern) return (char *)str; 340: 341: for (; *str; str++) {	
	污染路径: 无	
42	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 515
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 513: void *decodeUdfSetupRequest(const void *buf, SUdfSetupRequest *request) { 514: buf = taosDecodeBinaryTo(buf, request->udfName,	

	TSDB_FUNC_NAME_LEN); 515: return (void *)buf; 516: } 517:	
	污染路径: 无	
43	缺陷发生位置: /source/util/src/tutil.c	行号: 151
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 149: for(int32_t i = 0; i < len; ++i) { 150: if (haystack[i] == needle) { 151: return (TdUcs4*) &haystack[i]; 152: } 153: }	
	污染路径: 无	
44	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 382
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 380: } 381: 382: pPBuf->prefix = (char*)dir; 383: pPBuf->emptyDummyIdList = taosArrayInit(1, sizeof(int32_t)); 384:	
	污染路径: 无	
45	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 412
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 410: buf = taosDecodeFixedI32(buf, &pConsumer-	

	>resetOffsetCfg); 411: } 412: return (void *)buf; 413: } 414:	
	污染路径: 无	
46	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 586
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 584: void *decodeUdfTeardownRequest(const void *buf, SUdfTeardownRequest *teardown) { 585: buf = taosDecodeFixedl64(buf, &teardown->udfHandle); 586: return (void *)buf; 587: } 588:	
	污染路径: 无	
47	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 384
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 382: TdUcs4 *newBuf = taosMemoryCalloc((varDataLen(input) + 1) * TSDB_NCHAR_SIZE, 1); 383: int32_t len = varDataLen(input); 384: bool ret = taosMbsToUcs4(varDataVal(input), len, newBuf, (varDataLen(input) + 1) * TSDB_NCHAR_SIZE, &len); 385: if (!ret) { 386: taosMemoryFree(newBuf);	
	污染路径: expanded from macro 'varDataVal'	
48	缺陷发生位置: /source/common/src/tname.c	行号: 265

	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 263: if ((type & T_NAME_TABLE) == T_NAME_TABLE) { 264: dst->type = TSDB_TABLE_NAME_T; 265: char* start = (char*)((p == NULL) ? str : (p + 1)); 266: 267: // too long account id or too long db name	
	污染路径: 无	
49	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 859
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 857: rpcSendRecv(pConn->pTrans, &pConn->mgmtEps, &rpcMsg, &rpcRsp); 858: 859: CTG_ERR_RET(ctgProcessRspMsg(out, reqType, rpcRsp.pCont, rpcRsp.contLen, rpcRsp.code, (char*)dbFName)); 860: 861: rpcFreeCont(rpcRsp.pCont);	
	污染路径: 无	
50	缺陷发生位置: /source/client/src/clientStmt.c	行号: 949
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 947: 948: if (stmt == NULL NULL == pStmt->exec.pRequest) { 949: return (char*)tstrerror(terrno); 950: } 951:	
	污染路径: 无	

	39:	
	污染路径: 无	
54	缺陷发生位置: /source/common/src/tname.c	行号: 244
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 242: if ((type & T_NAME_DB) == T_NAME_DB) { 243: dst->type = TSDB_DB_NAME_T; 244: char* start = (char*)((p == NULL) ? str : (p + 1)); 245: 246: int32_t len = 0;	
	污染路径: 无	
55	缺陷发生位置: /source/util/src/tutil.c	行号: 141
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 139: 140: if (haystack[i] == needle) { 141: return (char *)&haystack[i]; 142: } 143: }	
	污染路径: 无	
56	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1188
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1186: memset(&w2, 0, sizeof(w2)); 1187: 1188: char* p1 = (char*)aBuf; 1189: char* p2 = (char*)bBuf; 1190:	

	污染路径: 无	
57	缺陷发生位置: /source/common/src/ttime.c	行号: 238
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 236: 237: int64_t fraction = 0; 238: str = forwardToTimeStringEnd((char*)timestr); 239: 240: if ((str[0] == 'Z' str[0] == 'z') && str[1] == '\0') {	
	污染路径: 无	
58	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 28
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 26: if (pFD->pFD == NULL) { 27: int errsv = errno; 28: const char *object_name = taosDirEntryBaseName((char *)path); 29: long s3_size = tsS3Enabled ? s3Size(object_name) : 0; 30: if (tsS3Enabled && !strncmp(path + strlen(path) - 5, ".data", 5) && s3_size > 0) {	
	污染路径: 无	
59	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 1168
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1166: 1167: SBuildTableInput blInput = { 1168: .vgld = vgroupInfo->vgld, .dbFName =	

	dbFName, .tbName = (char*)tNameGetTableName(pTableName)}; 1169: char* msg = NULL; 1170: int32_t msgLen = 0;	
	污染路径: 无	
60	缺陷发生位置: /source/os/src/osString.c	行号: 344
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 342: if (toupper(*str) == toupper(*pattern)) { 343: for (i = 1;; i++) { 344: if (!pattern[i]) return (char *)str; 345: if (toupper(str[i]) != toupper(pattern[i])) break; 346: }	
	污染路径: 无	
61	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 530
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 528: buf = taosDecodeFixedI32(buf, &state->bufLen); 529: buf = taosDecodeBinary(buf, (void **)&state->buf, state->bufLen); 530: return (void *)buf; 531: } 532:	
	污染路径: 无	
62	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 736
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 734: }	

	735: 736: bool taosIsKeyWordToken(const char* z, int32_t len) { return (tKeywordCode((char*)z, len) != TK_NK_ID); } 737: 738: void taosCleanupKeywordsTable() {	
	污染路径: 无	
63	缺陷发生位置: /source/libs/parser/test/parInitialCTest.cpp	行号: 1096
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1094: snprintf(expect.name, sizeof(expect.name), "0.%s", pTopicName); 1095: expect.igExists = igExists; 1096: expect.sql = (char*)pSql; 1097: expect.withMeta = withMeta; 1098: if (nullptr != pTbname) {	
	污染路径: 无	
64	缺陷发生位置: /source/client/src/clientEnv.c	行号: 165
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 163: rpclnit.sessions = 1024; 164: rpclnit.connType = TAOS_CONN_CLIENT; 165: rpclnit.user = (char *)user; 166: rpclnit.idleTime = tsShellActivityTimer * 1000; 167: rpclnit.compressSize = tsCompressMsgSize;	
	污染路径: 无	
65	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 1026
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段:	

	1024: rpcSendRecv(pConn->pTrans, &pConn->mgmtEps, &rpcMsg, &rpcRsp); 1025: 1026: CTG_ERR_RET(ctgProcessRspMsg(out, reqType, rpcRsp.pCont, rpcRsp.contLen, rpcRsp.code, (char*)funcName)); 1027: 1028: rpcFreeCont(rpcRsp.pCont);	
	污染路径: 无	
66	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 418
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 416: SName name = {0}; 417: tNameFromString(&name, stbFullName, T_NAME_ACCT T_NAME_DB T_NAME_TABLE); 418: pCreateTbReq->ctb.stbName = taosStrdup((char*)tNameGetTableName(&name)); 419: 420: STagVal tagVal = { .cid = numOfCols, .type = TSDB_DATA_TYPE_UBIGINT, .i64 = pDataBlock->info.id.groupId};	
	污染路径: 无	
67	缺陷发生位置: /source/dnode/mnode/impl/src/mndDef.c	行号: 656
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 654: pSub->qmsg = taosStrdup(""); 655: } 656: return (void *)buf; 657: } 658:	
	污染路径: 无	

68	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 75
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 73: memset(&w2, 0, sizeof(w2)); 74: 75: char *p1 = (char *)aBuf; 76: char *p2 = (char *)bBuf; 77: // p1 += 1;	
	污染路径: 无	
69	缺陷发生位置: /source/common/src/ttszip.c	行号: 824
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 822: 823: // compared with the last vnode id 824: int32_t id = tsBufGetLastGroupInfo((STSBuf*)pSrcBuf)->info.id; 825: if (id != tsBufGetLastGroupInfo(pDestBuf)->info.id) { 826: int32_t oldSize = pDestBuf->numOfGroups;	
	污染路径: 无	
70	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1196
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 1194: 1195: // insert new 1196: shellInsertStr(cmd, (char*)str, strlen); 1197: } 1198:	
	污染路径: 无	

71	缺陷发生位置： /source/util/src/thash.c	行号： 603
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 601: } 602: 603: taosHashRUnlock((SHashObj *)pHashObj); 604: return num; 605: }	
	污染路径： 无	
72	缺陷发生位置： /source/libs/parser/test/parInitialCTest.cpp	行号： 1103
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 1101: } else if (nullptr != pAst) { 1102: expect.subType = TOPIC_SUB_TYPE__COLUMN; 1103: expect.ast = (char*)pAst; 1104: } else { 1105: expect.subType = TOPIC_SUB_TYPE__DB;	
	污染路径： 无	
73	缺陷发生位置： /source/libs/tdb/src/db/tdbBtree.c	行号： 142
	步骤描述： A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段： 140: pBt->info.nLevel = 1; 141: pBt->info.nData = 0; 142: pBt->tname = (char *)tname; 143: 144: ret = tdbTbInsert(pPager->pEnv->pMainDb, tname, strlen(tname) + 1, &pBt->info, sizeof(pBt->info), txn);	
	污染路径： 无	

74	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 623
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 621: buf = decodeUdfTeardownRequest(buf, &request->teardown); 622: } 623: return (void *)buf; 624: } 625:	
	污染路径: 无	
75	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 293
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 291: if (disorderTs) { 292: tRowSort(tbData.aRowP); 293: if ((terrno = tRowMerge(tbData.aRowP, (STSchema*)pTSchema, 0)) != 0) { 294: goto _end; 295: } 	
	污染路径: 无	
76	缺陷发生位置: /source/util/src/thash.c	行号: 313
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 311: return 0; 312: } 313: return (int32_t)atomic_load_64((int64_t *)&pHashObj->size); 314: } 315:	

	污染路径: 无	
77	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 409
	步骤描述: A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	
	代码片段: 407: 408: SBackendFileItem item; 409: item.name = taosStrdup((char*)ROCKSDB_CURRENT); 410: item.type = ROCKSDB_CURRENT_TYPE; 411: taosArrayPush(list, &item);	
	污染路径: 无	

12.2.1.5 WK4793 禁止将不相关指针类型强制转换为对象指针

类型

缺陷数量：13 个		
缺陷描述: 禁止将不相关指针类型强制转换为对象指针类型。		
修复意见: 无		
1	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 32
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 30: const TdWchar* pattern3 = L"%_"; 31: const TdWchar* str3 = L"88"; 32: ret = wcsPatternMatch(reinterpret_cast<const TdUcs4*>(pattern3), 2, reinterpret_cast<const TdUcs4*>(str3), wcslen(str3), &pInfo); 33: ASSERT_EQ(ret, TSDB_PATTERN_MATCH); 34:	

	污染路径: 无	
2	缺陷发生位置: /source/os/test/osMathTests.cpp	行号: 40
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: <pre> 38: int cmpFunc(const void* a, const void* b) { 39: const TestStruct* pa = reinterpret_cast<const TestStruct*>(a); 40: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); 41: if (pa->a < pb->a) { 42: return -1; </pre>	
	污染路径: 无	
3	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 37
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: <pre> 35: const TdWchar* pattern4 = L"%__"; 36: const TdWchar* str4 = L"88"; 37: ret = wcsPatternMatch(reinterpret_cast<const TdUcs4*>(pattern4), 4, reinterpret_cast<const TdUcs4*>(str4), wcslen(str4), &pInfo); 38: ASSERT_EQ(ret, TSDB_PATTERN_NOWILDCARDMATCH); 39: </pre>	
	污染路径: 无	
4	缺陷发生位置: /source/os/test/osMathTests.cpp	行号: 39
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: <pre> 37: // Define a custom comparison function for testing 38: int cmpFunc(const void* a, const void* b) { 39: const TestStruct* pa = reinterpret_cast<const TestStruct*>(a); 40: const TestStruct* pb = reinterpret_cast<const TestStruct*>(b); </pre>	

	41: if (pa->a < pb->a) {	
	污染路径: 无	
5	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 19
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 17: 18: const TdWchar* str0 = L"14"; 19: ret = wcsPatternMatch(reinterpret_cast<const TdUcs4*>(pattern), 2, reinterpret_cast<const TdUcs4*>(str0), wcslen(str0), &pInfo); 20: ASSERT_EQ(ret, TSDB_PATTERN_NOWILDCARDMATCH); 21:	
	污染路径: 无	
6	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 102
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 100: taosRandStr(varDataVal(str), size); 101: varDataSetLen(str, size); 102: colDataSetVal(pColumnInfo, i, reinterpret_cast<const char*>(str), false); 103: pBlock->info.hasVarCol = true; 104: printf("binary: %s\n", varDataVal(str));	
	污染路径: 无	
7	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 27
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 25: 26: const TdWchar* str2 = L"41"; 27: ret = wcsPatternMatch(reinterpret_cast<const TdUcs4*>(pattern), 2, reinterpret_cast<const TdUcs4*>(str2), wcslen(str2), &pInfo);	

	28: ASSERT_EQ(ret, TSDB_PATTERN_MATCH); 29:	
	污染路径: 无	
8	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 107
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 105: } else if (pInfo->type == TSDB_DATA_TYPE_DOUBLE pInfo->type == TSDB_DATA_TYPE_FLOAT) { 106: double v = rand_f2(); 107: colDataSetVal(pColumnInfo, i, reinterpret_cast<const char*>(&v), false); 108: printf("float: %f\n", v); 109: } else {	
	污染路径: 无	
9	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 155
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 153: 154: ts = (++pInfo->tsStart); 155: colDataSetVal(pColumnInfo, i, reinterpret_cast<const char*>(&ts), false); 156: 157: SColumnInfoData* pColumnInfo1 = static_cast<SColumnInfoData*>(TARRAY_GET_ELEM(pBlock- ->pDataBlock, 1));	
	污染路径: 无	
10	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 94
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 92: } 93: varDataSetLen(str, len);	

	94: colDataSetVal(pColInfo, i, reinterpret_cast<const char*>(str), false); 95: pBlock->info.hasVarCol = true; 96: printf("nchar: %s\n", strOri);	
	污染路径: 无	
11	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 166
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 164: } 165: 166: colDataSetVal(pColInfo1, i, reinterpret_cast<const char*>(&v), false); 167: 168: // sprintf(buf, "this is %d row", i);	
	污染路径: 无	
12	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 23
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 21: 22: const TdWchar* str1 = L"11"; 23: ret = wcsPatternMatch(reinterpret_cast<const TdUcs4*>(pattern), 2, reinterpret_cast<const TdUcs4*>(str1), wcslen(str1), &pInfo); 24: ASSERT_EQ(ret, TSDB_PATTERN_MATCH); 25:	
	污染路径: 无	
13	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 110
	步骤描述: 不要使用 reinterpret_cast	
	代码片段: 108: } 109:	

	<pre>110: colDataSetVal(pCollInfo, i, reinterpret_cast<const char*>(&v), false); 111: 112: // sprintf(buf, "this is %d row", i);</pre>
	污染路径: 无

13 C/C++缺陷类型-比较的问题

13.1 分项汇总表

序号	C/C++缺陷类型-比较的问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4759	禁止对指针进行大于或小于的逻辑比较	22	严重
2	WK4761	禁止对无符号数进行大于等于零或小于零的比较	13	严重
3	WK4762	禁止无符号数和有符号数之间的直接比较	88	严重

13.2 缺陷/安全漏洞详解

13.2.1 严重类

13.2.1.1 WK4759 禁止对指针进行大于或小于的逻辑比较

缺陷数量：22 个		
缺陷描述： 禁止对指针进行大于或小于的逻辑比较。		
修复意见： 无		
1	缺陷发生位置： /tools/shell/src/shellAuto.c	行号： 1382
	步骤描述： 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段： 1380: // like select count(' ' 1381: return false; 1382: } else if (p1 && p2 && p1 > p2) { 1383: // like select sum(age), count(' ' 1384: return false;	
	污染路径： 无	
2	缺陷发生位置： /source/client/src/clientSmlLine.c	行号： 267
	步骤描述： 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段： 265: bool keyEscaped = false; 266: size_t keyLenEscaped = 0; 267: while (*sql < sqlEnd) { 268: if (unlikely(IS_SPACE(*sql) IS_COMMA(*sql))) { 269: smlBuildInvalidDataMsg(&info->msgBuf, "invalid data", *sql);	
	污染路径： 无	

3	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 61
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 59: 60: static void smlParseTelnetElement(char **sql, char *sqlEnd, char **data, int32_t *len) { 61: while (*sql < sqlEnd) { 62: if (unlikely((**sql != SPACE && !(*data)))) { 63: *data = *sql;	
	污染路径: 无	
4	缺陷发生位置: /utils/tsim/src/simExe.c	行号: 404
	步骤描述: 禁止对指针 p 进行大于或小于的逻辑比较	
	代码片段: 402: *(end + interval) = 0; 403: 404: for (char *p = end; p >= begin; p--) { 405: *(p + interval) = *p; 406: }	
	污染路径: 无	
5	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 2940
	步骤描述: 禁止对指针 pParser 进行大于或小于的逻辑比较	
	代码片段: 2938: void ParseFinalize(void *p){ 2939: yyParser *pParser = (yyParser*)p; 2940: while(pParser->yynos>pParser->yystack) yy_pop_parser_stack(pParser); 2941: #if YYSTACKDEPTH<=0 2942: if(pParser->yystack!=&pParser->yystk0) free(pParser->yystack);	
	污染路径: 无	
6	缺陷发生位置:	行号:

	/source/client/src/clientSmlLine.c		476
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较		
	代码片段: 474: bool keyEscaped = false; 475: size_t keyLenEscaped = 0; 476: while (*sql < sqlEnd) { 477: if (unlikely(IS_SPACE(*sql) IS_COMMA(*sql))) { 478: smlBuildInvalidDataMsg(&info->msgBuf, "invalid data", *sql);		
	污染路径: 无		
7	缺陷发生位置: /source/client/src/clientSmlLine.c		行号: 466
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较		
	代码片段: 464: } 465: 466: while (*sql < sqlEnd) { 467: if (unlikely(IS_SPACE(*sql))) { 468: break;		
	污染路径: 无		
8	缺陷发生位置: /source/client/src/clientSmlTelnet.c		行号: 114
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较		
	代码片段: 112: taosArrayClear(preLineKV); 113: const char *sql = data; 114: while (sql < sqlEnd) { 115: JUMP_SPACE(sql, sqlEnd) 116: if (unlikely(*sql == '\0')) break;		
	污染路径: 无		
9	缺陷发生位置: /source/libs/parser/src/sql.c		行号: 2923
	步骤描述: 禁止对指针 pParser 进行大于或小于的逻辑比较		

	代码片段: 2921: yyStackEntry *yytos; 2922: assert(pParser->yytos!=0); 2923: assert(pParser->yytos > pParser->yystack); 2924: yytos = pParser->yytos--; 2925: #ifndef NDEBUG	
	污染路径: 无	
10	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1412
	步骤描述: 禁止对指针 plast 进行大于或小于的逻辑比较	
	代码片段: 1410: 1411: // like 'select count(*),sum(age) fr' need return true 1412: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1413: return true; 1414: }	
	污染路径: 无	
11	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 294
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 292: bool valueEscaped = false; 293: size_t valueLenEscaped = 0; 294: while (*sql < sqlEnd) { 295: // parse value 296: if (unlikely(IS_SPACE(*sql) IS_COMMA(*sql))) {	
	污染路径: 无	
12	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 257
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 255: taosArrayClearEx(preLineKV, freeSSmlKv); 256:	

	257: while (*sql < sqlEnd) { 258: if (unlikely(IS_SPACE(*sql))) { 259: break;	
	污染路径: 无	
13	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1365
	步骤描述: 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段: 1363: 1364: if (p1 && p2) { 1365: return p1 > p2 ? p1 + 1 : p2 + 1; 1366: } else if (p1) { 1367: return p1 + 1;	
	污染路径: 无	
14	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 4473
	步骤描述: 禁止对指针 yypParser 进行大于或小于的逻辑比较	
	代码片段: 4471: } 4472: #else 4473: if(yypParser->yynos=>&yypParser->yystack[yypParser->yystksz-1]){ 4474: if(yyGrowStack(yypParser)){ 4475: yyStackOverflow(yypParser);	
	污染路径: 无	
15	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1412
	步骤描述: 禁止对指针 p1 进行大于或小于的逻辑比较	
	代码片段: 1410: 1411: // like 'select count(*),sum(age) fr' need return true 1412: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1413: return true;	

	1414: }	
	污染路径: 无	
16	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 3174
	步骤描述: 禁止对指针 yypParser 进行大于或小于的逻辑比较	
	代码片段: 3172: } 3173: #else 3174: if(yypParser->yynos=>&yypParser->yystack[yypParser->yystksz]){ 3175: if(yyGrowStack(yypParser)){ 3176: yypParser->yynos--;	
	污染路径: 无	
17	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1412
	步骤描述: 禁止对指针 p2 进行大于或小于的逻辑比较	
	代码片段: 1410: 1411: // like 'select count(*),sum(age) fr' need return true 1412: if (plast && plast > p3 && p2 > p1 && plast > p2 && p1 > p3) { 1413: return true; 1414: }	
	污染路径: 无	
18	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 122
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 120: 121: // parse key 122: while (sql < sqlEnd) { 123: if (unlikely(*sql == SPACE)) { 124: smlBuildInvalidDataMsg(msg, "invalid data", sql);	

	污染路径: 无	
19	缺陷发生位置: /source/libs/sync/src/syncRaftLog.c	行号: 209
	步骤描述: 禁止对指针 index 进行大于或小于的逻辑比较	
	代码片段: 207: int64_t tsElapsed = tsWriteEnd - tsWriteBegin; 208: 209: if (index < 0) { 210: int32_t err = terrno; 211: const char* errStr = strerror(err);	
	污染路径: 无	
20	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 3120
	步骤描述: 禁止对指针 yypParser 进行大于或小于的逻辑比较	
	代码片段: 3118: } 3119: #endif 3120: while(yypParser->yptos>yypParser->yystack) yy_pop_parser_stack(yypParser); 3121: /* Here code is inserted which will execute if the parser 3122: ** stack every overflows */	
	污染路径: 无	
21	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 147
	步骤描述: 禁止对指针 sql 进行大于或小于的逻辑比较	
	代码片段: 145: const char *value = sql; 146: size_t valueLen = 0; 147: while (sql < sqlEnd) { 148: // parse value 149: if (unlikely(*sql == SPACE)) {	
	污染路径: 无	

22	缺陷发生位置： /source/client/src/clientSml.c	行号： 1576
	步骤描述： 禁止对指针 rawLine 进行大于或小于的逻辑比较	
	代码片段： 1574: } else if (rawLine) { 1575: tmp = rawLine; 1576: while (rawLine < rawLineEnd) { 1577: if (*(rawLine++) == '\n') { 1578: break; 1579: } 1580: } 1581: }	
	污染路径： 无	

13.2.1.2 WK4761 禁止对无符号数进行大于等于零或小于零的比较

缺陷数量： 13 个		
缺陷描述： 禁止对无符号数进行大于等于零或小于零的比较。		
修复意见： 无		
1	缺陷发生位置： /source/libs/scalar/src/filter.c	行号： 2701
	步骤描述： 禁止无符号数与 0 比较。	
	代码片段： 2699: } 2700: 2701: FLT_CHK_JMP(colNum <= 0); 2702: 2703: info->colRangeNum = colNum;	
	污染路径： 无	
2	缺陷发生位置： /source/util/src/tarray.c	行号： 306
	步骤描述：	

	禁止无符号数与 0 比较.	
	代码片段: 304: 305: SArray* taosArrayFromList(const void* src, size_t size, size_t elemSize) { 306: if (elemSize <= 0) { 307: terrno = TSDB_CODE_INVALID_PARA; 308: return NULL;	
	污染路径: 无	
3	缺陷发生位置: /source/client/src/clientHb.c	行号: 709
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 707: } 708: 709: if (stbNum <= 0) { 710: taosMemoryFree(stbs); 711: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
4	缺陷发生位置: /source/client/src/clientHb.c	行号: 628
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 626: } 627: 628: if (userNum <= 0) { 629: taosMemoryFree(users); 630: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
5	缺陷发生位置: /source/client/src/clientHb.c	行号: 665
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 663: }	

	664: 665: if (dbNum <= 0) { 666: taosMemoryFree(dbs); 667: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
6	缺陷发生位置: /source/client/src/clientImpl.c	行号: 39
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 37: 38: size_t len = strlen(str); 39: if (len <= 0 len > maxsize) { 40: return false; 41: }	
	污染路径: 无	
7	缺陷发生位置: /source/client/src/clientStmt.c	行号: 585
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 583: STMT_ERR_RET(stmtSwitchStatus(pStmt, STMT_PREPARE)); 584: 585: if (length <= 0) { 586: length = strlen(sql); 587: }	
	污染路径: 无	
8	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 1423
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 1421: 1422: size_t metaNum = taosArrayGetSize(slot->meta); 1423: if (metaNum <= 0) { 1424: qDebug("no meta in slot:%d, type:%d", ridx, mgmt-	

	>type); 1425: *num = 0;	
	污染路径: 无	
9	缺陷发生位置: /source/common/src/tname.c	行号: 184
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 182: int32_t tNameAddTbName(SName* dst, const char* tbName, size_t nameLen) { 183: // too long account id or too long db name 184: if (nameLen >= tListLen(dst->tname) nameLen <= 0) { 185: return -1; 186: }	
	污染路径: 无	
10	缺陷发生位置: /source/common/src/tdatablock.c	行号: 1166
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 1164: static int32_t doEnsureCapacity(SColumnInfoData* pColumn, const SDataBlockInfo* pBlockInfo, uint32_t numOfRows, 1165: bool clearPayload) { 1166: if (numOfRows <= 0 numOfRows <= pBlockInfo- >capacity) { 1167: return TSDB_CODE_SUCCESS; 1168: }	
	污染路径: 无	
11	缺陷发生位置: /source/common/src/tdatablock.c	行号: 496
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 494: 495: size_t numOfCols = taosArrayGetSize(pDataBlock-	

	>pDataBlock); 496: if (numOfCols <= 0) { 497: return -1; 498: }	
	污染路径: 无	
12	缺陷发生位置: /source/common/src/tname.c	行号: 172
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 170: int32_t tNameSetDbName(SName* dst, int32_t acct, const char* dbName, size_t nameLen) { 171: // too long account id or too long db name 172: if (nameLen <= 0 nameLen >= tListLen(dst->dbname)) { 173: return -1; 174: }	
	污染路径: 无	
13	缺陷发生位置: /source/common/src/tglobal.c	行号: 1552
	步骤描述: 禁止无符号数与 0 比较.	
	代码片段: 1550: } 1551: 1552: if (tsServerPort <= 0) { 1553: uError("invalid server port:%u, dnode can not be initialized", tsServerPort); 1554: return -1;	
	污染路径: 无	

13.2.1.3 WK4762 禁止无符号数和有符号数之间的直接比较

缺陷数量：88 个

缺陷描述： 禁止无符号数和有符号数之间的直接比较。		
修复意见： 无		
1	缺陷发生位置： /include/util/tencode.h	行号： 225
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：pCoder,val	
	代码片段： 223: static FORCE_INLINE int32_t tEncodeI16(SEncoder* pCoder, int16_t val) { TD_ENCODE_MACRO(pCoder, val, int16_t, 16); } 224: // 32 225: static FORCE_INLINE int32_t tEncodeU32(SEncoder* pCoder, uint32_t val) { TD_ENCODE_MACRO(pCoder, val, uint32_t, 32); } 226: static FORCE_INLINE int32_t tEncodeI32(SEncoder* pCoder, int32_t val) { TD_ENCODE_MACRO(pCoder, val, int32_t, 32); } 227: // 64	
	污染路径： expanded from macro 'TD_ENCODE_MACRO' expanded from macro 'TD_CODER_CHECK_CAPACITY_FAILED'	
2	缺陷发生位置： /source/util/src/tcompare.c	行号： 326
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：left,right	
	代码片段： 324: uint16_t right = GET_UINT16_VAL(pRight); 325: if (left > right) return 1; 326: if (left < right) return -1; 327: return 0; 328: }	
	污染路径： 无	
3	缺陷发生位置： /include/common/tdatablock.h	行号： 125
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,start	
	代码片段： 123: static FORCE_INLINE void	

	colDataSetNNULL(SColumnInfoData* pColumnInfoData, uint32_t start, size_t nRows) { 124: if (IS_VAR_DATA_TYPE(pColumnInfoData->info.type)) { 125: for (int32_t i = start; i < start + nRows; ++i) { 126: colDataSetNull_var(pColumnInfoData, i); // it is a null value of VAR type. 127: } 	
	污染路径: 无	
4	缺陷发生位置: /source/common/src/tdformat.c	行号: 1216
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: n,pTag	
	代码片段: 1214: printf("%s:%d >>> STAG === %s:%s, len: %d, nTag: %d, sver:%d\n", tag, ln, isjson ? "json" : "normal", 1215: isLarge ? "large" : "small", (int32_t)pTag->len, (int32_t)pTag->nTag, pTag->ver); 1216: for (uint16_t n = 0; n < pTag->nTag; ++n) { 1217: if (isLarge) { 1218: offset = ((int16_t *)pTag->idx)[n];	
	污染路径: 无	
5	缺陷发生位置: /source/util/src/tbloomfilter.c	行号: 79
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,pBF	
	代码片段: 77: const register uint64_t size = pBF->numBits; 78: uint64_t cbHash = h1; 79: for (uint64_t i = 0; i < pBF->hashFunctions; ++i) { 80: hasChange = setBit(pBF->buffer, cbHash % size); 81: cbHash += h2;	
	污染路径: 无	
6	缺陷发生位置: /source/util/src/ttimer.c	行号: 256
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: wheelIdx,wheels	
	代码片段:	

	254: static bool removeFromWheel(tmr_obj_t* timer) { 255: uint8_t wheelIdx = timer->wheel; 256: if (wheelIdx >= tListLen(wheels)) { 257: return false; 258: }	
	污染路径: 无	
7	缺陷发生位置: /source/os/src/osString.c	行号: 481
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,n	
	代码片段: 479: bool isValidHex(const char* z, uint32_t n){ 480: if(n % 2 != 0) return false; 481: for(size_t i = HEX_PREFIX_LEN; i < n; i++){ 482: if(isxdigit(z[i]) == 0){ 483: return false;	
	污染路径: 无	
8	缺陷发生位置: /source/libs/index/src/indexFstRegister.c	行号: 57
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: end,sz	
	代码片段: 55: static void fstRegistryCellPromote(SArray* arr, uint32_t start, uint32_t end) { 56: size_t sz = taosArrayGetSize(arr); 57: if (start >= sz && end >= sz) { 58: return; 59: }	
	污染路径: 无	
9	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 2742
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: n,num	
	代码片段: 2740: printf("%d - expired stableNum:%d\n", i, num); 2741: if (stable) { 2742: for (int32_t n = 0; n < num; ++n) {	

	2743: printf("suid:%" PRId64 " , dbFName:%s, stbName:%s, sversion:%d, tversion:%d\n", stable[n].suid, 2744: stable[n].dbFName, stable[n].stbName, stable[n].sversion, stable[n].tversion);	
	污染路径: 无	
10	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 809
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: defaultPgsz,rowSize	
	代码片段: 807: *defaultPgsz = 4096; 808: uint32_t last = *defaultPgsz; 809: while (*defaultPgsz < rowSize * 4) { 810: *defaultPgsz <= 1u; 811: if (*defaultPgsz < last) {	
	污染路径: 无	
11	缺陷发生位置: /source/client/src/clientHb.c	行号: 633
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,userNum	
	代码片段: 631: } 632: 633: for (int32_t i = 0; i < userNum; ++i) { 634: SUserAuthVersion *user = &users[i]; 635: user->version = htonl(user->version);	
	污染路径: 无	
12	缺陷发生位置: /source/util/src/tcompare.c	行号: 317
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: left,right	
	代码片段: 315: int8_t left = GET_INT8_VAL(pLeft); 316: uint8_t right = GET_UINT8_VAL(pRight); 317: if (left > right) return 1; 318: if (left < right) return -1;	

	319: return 0;	
	污染路径: 无	
13	缺陷发生位置: /source/libs/index/src/indexFstRegex.c	行号: 30
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,strlen	
	代码片段: 28: // construct insts based on str 29: SArray *insts = taosArrayInit(256, sizeof(uint8_t)); 30: for (int i = 0; i < strlen(str); i++) { 31: uint8_t v = str[i]; 32: taosArrayPush(insts, &v);	
	污染路径: 无	
14	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 1094
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,bindNum	
	代码片段: 1092: } 1093: 1094: for (int32_t i = 0; i < bindNum; ++i) { 1095: data->tsData[i] = bpTs + tblIdx*gCurCase->rowNum + rand()%gCurCase->rowNum; 1096: data->boolData[i] = (bool)(tblIdx*gCurCase->rowNum + rand() % gCurCase->rowNum);	
	污染路径: 无	
15	缺陷发生位置: /source/util/src/tcompare.c	行号: 318
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: left,right	
	代码片段: 316: uint8_t right = GET_UINT8_VAL(pRight); 317: if (left > right) return 1; 318: if (left < right) return -1; 319: return 0; 320: }	

	污染路径: 无	
16	缺陷发生位置: /source/libs/parser/src/parInsertSql.c	行号: 345
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: size,bytes	
	代码片段: 343: } 344: 345: if (size + VARSTR_HEADER_SIZE > bytes) { 346: taosMemoryFree(data); 347: return TSDB_CODE_PAR_VALUE_TOO_LONG;	
	污染路径: 无	
17	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 994
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,allRowNum	
	代码片段: 992: } 993: 994: for (int b = 0; b < (allRowNum/gCurCase->bindRowNum); b++) { 995: for (int c = 0; c < gCurCase->bindColNum; ++c) { 996: prepareColData(BP_BIND_COL, data, b*gCurCase->bindColNum+c, b*gCurCase->bindRowNum, c);	
	污染路径: 无	
18	缺陷发生位置: /source/os/src/osDir.c	行号: 344
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: strlen,maxlen	
	代码片段: 342: if (realpath(dirname, tmp) != NULL) { 343: #endif 344: if (strlen(tmp) < maxlen) { 345: if (realPath == NULL) { 346: strncpy(dirname, tmp, maxlen);	
	污染路径: 无	

19	缺陷发生位置: /source/os/src/osString.c	行号: 578
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,n	
	代码片段: 576: *(tmp++) = '\\'; 577: *(tmp++) = 'x'; 578: for(int i = 0; i < n; i ++){ 579: uint8_t val = z[i]; 580: tmp[i*2] = valueOf(val >> 4);	
	污染路径: 无	
20	缺陷发生位置: /source/util/test/pageBufferTest.cpp	行号: 170
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,pPg	
	代码片段: 168: bool checkBufVarData(SFilePage* pPg, const char* varData, uint32_t varLen) { 169: const char* start = pPg->data + sizeof(SFilePage); 170: for (uint32_t i = 0; i < (pPg->num - sizeof(SFilePage)) / varLen; ++i) { 171: if (0 != strncmp(start + 6 * i + 3, varData, varLen - 3)) { 172: using namespace std;	
	污染路径: 无	
21	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 97
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: taosQueueItemSize,capacity	
	代码片段: 95: static bool allocBuf(SDataDeleterHandle* pDeleter, const SInputData* pInput, SDataDeleterBuf* pBuf) { 96: uint32_t capacity = pDeleter->pManager-> cfg.maxDataBlockNumPerQuery; 97: if (taosQueueItemSize(pDeleter->pDataBlocks) > capacity) { 98: qError("SinkNode queue is full, no capacity, max:%d, current:%d, no capacity", capacity,	

	99: taosQueueItemSize(pDeleter->pDataBlocks));	
	污染路径: 无	
22	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 1037
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,bindNum	
	代码片段: 1035: } 1036: 1037: for (int32_t i = 0; i < bindNum; ++i) { 1038: data->tsData[i] = bpTs + tblIdx*gCurCase->rowNum + rand()%gCurCase->rowNum; 1039: data->boolData[i] = (bool)(tblIdx*gCurCase->rowNum + rand() % gCurCase->rowNum);	
	污染路径: 无	
23	缺陷发生位置: /include/common/t datablock.h	行号: 129
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,start	
	代码片段: 127: } 128: } else { 129: for (int32_t i = start; i < start + nRows; ++i) { 130: colDataSetNull_f(pColumnInfoData->nullbitmap, i); 131: }	
	污染路径: 无	
24	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 1212
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,len	
	代码片段: 1210: int32_t len; 1211: uint8_t* data = fstSliceData(key, &len); 1212: for (uint32_t i = 0; i < len; i++) { 1213: uint8_t b = data[i]; 1214: uint64_t res = 0;	

	污染路径: 无	
25	缺陷发生位置: /source/common/src/tdataformat.c	行号: 3427
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: min,val	
	代码片段: 3425: case 2: 3426: val = ((uint8_t *)pColData->pData)[iVal]; 3427: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3428: break; 3429: default:	
	污染路径: expanded from macro 'CALC_SUM_MAX_MIN'	
26	缺陷发生位置: /source/util/src/tcompression.c	行号: 1731
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCmprsor,BIT_TO_SELECTOR	
	代码片段: 1729: if (pCmprsor->i_nEle + 1 <= SELECTOR_TO_ELEMS[pCmprsor->i_selector] && 1730: pCmprsor->i_nEle + 1 <= SELECTOR_TO_ELEMS[BIT_TO_SELECTOR[nBit]]) { 1731: if (pCmprsor->i_selector < BIT_TO_SELECTOR[nBit]) { 1732: pCmprsor->i_selector = BIT_TO_SELECTOR[nBit]; 1733: }	
	污染路径: 无	
27	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 111
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,len	
	代码片段: 109: un->last = fstLastTransitionCreate(data[0], out); 110: 111: for (uint64_t i = 1; i < len; i++) { 112: FstBuilderNode* n = taosMemoryMalloc(sizeof(FstBuilderNode));	

	113: n->isFinal = false;	
	污染路径: 无	
28	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 530
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: nRows,limitRowNum	
	代码片段: 528: size_t rightRowsSize = taosArrayGetSize(pRightRows); 529: for (j = rightRowIdx; j < rightRowsSize; ++j) { 530: if (*nRows >= limitRowNum) { 531: *pReachThreshold = true; 532: break;	
	污染路径: 无	
29	缺陷发生位置: /source/common/src/tdformat.c	行号: 3416
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: max,val	
	代码片段: 3414: for (int32_t iVal = 0; iVal < pColData->nVal; iVal++) { 3415: val = ((uint8_t *)pColData->pData)[iVal]; 3416: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3417: } 3418: } else {	
	污染路径: expanded from macro 'CALC_SUM_MAX_MIN'	
30	缺陷发生位置: /source/common/src/ttszip.c	行号: 213
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: newSize,pTSBuf	
	代码片段: 211: if (pTSBuf->numOfAlloc <= pTSBuf->numOfGroups) { 212: uint32_t newSize = (uint32_t)(pTSBuf->numOfAlloc * 1.5); 213: ASSERT((int32_t)newSize > pTSBuf->numOfAlloc); 214:	

	215: STSGroupBlockInfoEx* tmp =	
	污染路径: 无	
31	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 355
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,nSubmitTbData	
	代码片段: 353: int64_t btimeMs = taosGetTimestampMs(); 354: int64_t ctimeMs = btimeMs; 355: for (int32_t i = 0; i < nSubmitTbData; i++) { 356: code = vnodePreProcessSubmitTbData(pVnode, pCoder, btimeMs, ctimeMs); 357: TSDB_CHECK_CODE(code, lino, _exit);	
	污染路径: 无	
32	缺陷发生位置: /source/common/src/tmsg.c	行号: 7852
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,nColData	
	代码片段: 7850: } 7851: 7852: for (int32_t i = 0; i < nColData; ++i) { 7853: pCoder->pos += tGetColData(pCoder->data + pCoder->pos, taosArrayReserve(pSubmitTbData->aCol, 1)); 7854: }	
	污染路径: 无	
33	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 139
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr2	
	代码片段: 137: uint64_t arr2[] = {8, 10, 12, 13}; 138: f = (SArray *)taosArrayGetP(src, 1); 139: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 140: taosArrayPush(f, &arr2[i]); 141: }	

	污染路径: 无	
34	缺陷发生位置: /source/util/src/tlosetree.c	行号: 60
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,numOfSources	
	代码片段: 58: #endif 59: 60: for (int32_t i = totalEntries - 1; i >= numOfSources; i--) { 61: tMergeTreeAdjust(pTreeInfo, i); 62: }	
	污染路径: 无	
35	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 425
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pBlock,uid	
	代码片段: 423: if (pBlock->uid->data[j] == uid) { 424: return true; 425: } else if (pBlock->uid->data[j] > uid) { 426: while (j >= 0 && pBlock->suid->data[j] == suid) { 427: if (pBlock->uid->data[j] == uid) {	
	污染路径: 无	
36	缺陷发生位置: /source/libs/index/src/indexFstRegister.c	行号: 57
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: start,sz	
	代码片段: 55: static void fstRegistryCellPromote(SArray* arr, uint32_t start, uint32_t end) { 56: size_t sz = taosArrayGetSize(arr); 57: if (start >= sz && end >= sz) { 58: return; 59: }	
	污染路径: 无	
37	缺陷发生位置:	行号:

	/include/util/tencode.h	290
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCoder,val	
	代码片段: <pre> 288: // 8 289: static FORCE_INLINE int32_t tDecodeU8(SDecoder* pCoder, uint8_t* val) { 290: if (TD_CODER_CHECK_CAPACITY_FAILED(pCoder, sizeof(*val))) return -1; 291: tGet(uint8_t, TD_CODER_CURRENT(pCoder), *val); 292: TD_CODER_MOVE_POS(pCoder, sizeof(*val)); </pre>	
	污染路径: expanded from macro 'TD_CODER_CHECK_CAPACITY_FAILED'	
38	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 1033
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,len	
	代码片段: <pre> 1031: SArray* nodes = (SArray*)taosArrayInit(len, sizeof(FstNode*)); 1032: taosArrayPush(nodes, &root); 1033: for (uint32_t i = 0; i < len; i++) { 1034: uint8_t inp = data[i]; 1035: Output res = 0; </pre>	
	污染路径: 无	
39	缺陷发生位置: /source/client/src/clientHb.c	行号: 670
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,dbNum	
	代码片段: <pre> 668: } 669: 670: for (int32_t i = 0; i < dbNum; ++i) { 671: SDbCacheInfo *db = &dbbs[i]; 672: tscDebug("the %dth expired dbFName:%s, dbId:%" PRId64 " , vgVersion:%d, cfgVersion:%d, numOfTable:%d, startTs:%" PRId64, </pre>	
	污染路径: 无	

40	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 497
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,len	
	代码片段: 495: memcpy(s, ptr, len); 496: 497: for (int32_t i = 0; i < len; ++i) { 498: if (!syncUtilCanPrint(s[i])) { 499: s[i] = '.';	
	污染路径: 无	
41	缺陷发生位置: /source/dnode/vnode/test/tsdbSmaTest.cpp	行号: 274
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,taosArrayGetSize	
	代码片段: 272: SArray *pUids = metaGetSmaTbUids(pMeta, false); 273: assert(pUids != NULL); 274: for (uint32_t i = 0; i < taosArrayGetSize(pUids); ++i) { 275: printf("metaGetSmaTbUids: uid[%d] = %" PRIu32 " " PRli64 "\n", i, *(tb_uid_t *)taosArrayGet(pUids, i)); 276: // printf("metaGetSmaTbUids: index[%d] = %s", i, (char *)taosArrayGet(pUids, i));	
	污染路径: 无	
42	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 168
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,ssz	
	代码片段: 166: } 167: if (addPrefix != 0) { 168: if (i + 1 < ssz) { 169: FstBuilderNodeUnfinished* unf = taosArrayGet(node->stack, i + 1); 170: fstBuilderNodeUnfinishedAddOutputPrefix(unf, addPrefix);	
	污染路径: 无	

43	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 967
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,allRowNum	
	代码片段: 965: } 966: 967: for (int32_t i = 0; i < allRowNum; ++i) { 968: if (gCurCase->duplicateValue) { 969: data->tsData[i] = bpTs;	
	污染路径: 无	
44	缺陷发生位置: /include/util/tencode.h	行号: 222
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCoder,val	
	代码片段: 220: 221: // 16 222: static FORCE_INLINE int32_t tEncodeU16(SEncoder* pCoder, uint16_t val) { TD_ENCODE_MACRO(pCoder, val, uint16_t, 16); } 223: static FORCE_INLINE int32_t tEncodeI16(SEncoder* pCoder, int16_t val) { TD_ENCODE_MACRO(pCoder, val, int16_t, 16); } 224: // 32	
	污染路径: expanded from macro 'TD_ENCODE_MACRO' expanded from macro 'TD_CODER_CHECK_CAPACITY_FAILED'	
45	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 133
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr1	
	代码片段: 131: uint64_t arr1[] = {13, 14, 15}; 132: SArray * f = (SArray *)taosArrayGetP(src, 0); 133: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) { 134: taosArrayPush(f, &arr1[i]); 135: }	
	污染路径:	

	无	
46	缺陷发生位置： /source/libs/index/test/utilUT.cc	行号： 108
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,arr1	
	代码片段： 106: uint64_t arr1[] = {2, 3, 4, 5}; 107: SArray * f = (SArray *)taosArrayGetP(src, 0); 108: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) { 109: taosArrayPush(f, &arr1[i]); 110: }	
	污染路径： 无	
47	缺陷发生位置： /source/common/src/tmsg.c	行号： 8050
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,nCreateTbRsp	
	代码片段： 8048: } 8049: 8050: for (int32_t i = 0; i < nCreateTbRsp; ++i) { 8051: SVCCreateTbRsp *pCreateTbRsp = taosArrayReserve(pRsp->aCreateTbRsp, 1); 8052: if (tDecodeSVCCreateTbRsp(pCoder, pCreateTbRsp) < 0) {	
	污染路径： 无	
48	缺陷发生位置： /source/libs/stream/src/streamBackendRocksdb.c	行号： 241
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：strlen,sstLen	
	代码片段： 239: continue; 240: } 241: if (strlen(name) >= sstLen && strncmp(name + strlen(name) - 4, pSST, sstLen) == 0) { 242: char* p = taosStrdup(name); 243: taosHashPut(bm->pSstTbl[1 - bm->idx], name, strlen(name), &dummy, sizeof(dummy));	
	污染路径：	

	无	
49	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 152
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,lsz	
	代码片段: <pre> 150: *out = in; 151: uint64_t i = 0; 152: for (i = 0; i < lsz && i < ssz; i++) { 153: FstBuilderNodeUnfinished* un = taosArrayGet(node->stack, i); 154: </pre>	
	污染路径: 无	
50	缺陷发生位置: /source/common/src/tdatablock.c	行号: 376
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: finalNumOfRows,capacity	
	代码片段: <pre> 374: if (IS_VAR_DATA_TYPE(pColumnInfoData->info.type)) { 375: // Handle the bitmap 376: if (finalNumOfRows > (*capacity)) { 377: char* p = taosMemoryRealloc(pColumnInfoData->varmeta.offset, sizeof(int32_t) * (numOfRow1 + numOfRow2)); 378: if (p == NULL) { </pre>	
	污染路径: 无	
51	缺陷发生位置: /source/util/src/tcompare.c	行号: 391
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: left,right	
	代码片段: <pre> 389: int16_t left = GET_INT16_VAL(pLeft); 390: uint8_t right = GET_UINT8_VAL(pRight); 391: if (left > right) return 1; 392: if (left < right) return -1; 393: return 0; </pre>	
	污染路径:	

	无	
52	缺陷发生位置： /source/util/src/ttimer.c	行号： 221
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,wheels	
	代码片段： 219: // but the inaccuracy should not be too large. 220: timer->wheel = tListLen(wheels) - 1; 221: for (uint8_t i = 0; i < tListLen(wheels); i++) { 222: time_wheel_t* wheel = wheels + i; 223: if (delay < wheel->resolution * wheel->size) {	
	污染路径： 无	
53	缺陷发生位置： /source/libs/index/test/utilUT.cc	行号： 114
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,arr2	
	代码片段： 112: uint64_t arr2[] = {1, 2, 3, 5}; 113: f = (SArray *)taosArrayGetP(src, 1); 114: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 115: taosArrayPush(f, &arr2[i]); 116: }	
	污染路径： 无	
54	缺陷发生位置： /source/client/src/clientHb.c	行号： 714
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,stbNum	
	代码片段： 712: } 713: 714: for (int32_t i = 0; i < stbNum; ++i) { 715: SSTableVersion *stb = &stbs[i]; 716: stb->suid = htobe64(stb->suid);	
	污染路径： 无	
55	缺陷发生位置： /source/dnode/vnode/src/vnd/vnodeSvr.c	行号： 308
	步骤描述：	

	禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: iRow,nRow	
	代码片段: <pre> 306: } 307: 308: for (int32_t iRow = 0; iRow < nRow; ++iRow) { 309: SRow *pRow = (SRow *)(pCoder->data + pCoder->pos); 310: pCoder->pos += pRow->len; </pre>	
56	污染路径: 无	
	缺陷发生位置: /source/common/src/tdataformat.c	行号: 3416
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: min,val	
	代码片段: <pre> 3414: for (int32_t iVal = 0; iVal < pColData->nVal; iVal++) { 3415: val = ((uint8_t *)pColData->pData)[iVal]; 3416: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3417: } 3418: } else { </pre>	
57	污染路径: expanded from macro 'CALC_SUM_MAX_MIN'	
	缺陷发生位置: /source/common/src/tdataformat.c	行号: 3427
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: max,val	
	代码片段: <pre> 3425: case 2: 3426: val = ((uint8_t *)pColData->pData)[iVal]; 3427: CALC_SUM_MAX_MIN(*(uint64_t *)sum, *(uint64_t *)max, *(uint64_t *)min, val); 3428: break; 3429: default: </pre>	
58	污染路径: expanded from macro 'CALC_SUM_MAX_MIN'	
	缺陷发生位置: /include/util/tencode.h	行号: 228
步骤描述:		

	禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCoder,val 代码片段: <pre> 226: static FORCE_INLINE int32_t tEncodeI32(SEncoder* pCoder, int32_t val) { TD_ENCODE_MACRO(pCoder, val, int32_t, 32); } 227: // 64 228: static FORCE_INLINE int32_t tEncodeU64(SEncoder* pCoder, uint64_t val) { TD_ENCODE_MACRO(pCoder, val, uint64_t, 64); } 229: static FORCE_INLINE int32_t tEncodeI64(SEncoder* pCoder, int64_t val) { TD_ENCODE_MACRO(pCoder, val, int64_t, 64); } 230: // 16v </pre> 污染路径: expanded from macro 'TD_ENCODE_MACRO' expanded from macro 'TD_CODER_CHECK_CAPACITY_FAILED'	
59	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 1057
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,bindNum 代码片段: <pre> 1055: } 1056: 1057: for (int b = 0; b < bindNum; b++) { 1058: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1059: prepareColData(BP_BIND_COL, data, b*gCurCase- >bindColNum+c, b*gCurCase->bindRowNum, c); </pre>	
	污染路径: 无	
60	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 512
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,len 代码片段: <pre> 510: 511: char *p = s; 512: for (int32_t i = 0; i < len; ++i) { 513: int32_t n = sprintf(p, "%d,", ptr[i]); 514: p += n; </pre>	
	污染路径:	

	无	
61	缺陷发生位置： /source/util/src/tlruccache.c	行号： 834
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,numShards	
	代码片段： <pre> 832: taosThreadMutexLock(&cache- >shardedCache.capacityMutex); 833: 834: for (int i = 0; i < numShards; ++i) { 835: taosLRUCacheShardSetCapacity(&cache->shards[i], perShard); 836: }</pre>	
	污染路径： 无	
62	缺陷发生位置： /source/common/src/tdatablock.c	行号： 251
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,numOfRows	
	代码片段： <pre> 249: } else { 250: int32_t i = 0; 251: for (; i < numOfRows; ++i) { 252: if (BitPos(currentRow + i)) { 253: colDataSetNull_f(pColumnInfoData->>nullbitmap, currentRow + i);</pre>	
	污染路径： 无	
63	缺陷发生位置： /source/util/src/tcompare.c	行号： 325
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：left,right	
	代码片段： <pre> 323: int8_t left = GET_INT8_VAL(pLeft); 324: uint16_t right = GET_UINT16_VAL(pRight); 325: if (left > right) return 1; 326: if (left < right) return -1; 327: return 0;</pre>	
	污染路径： 无	

64	缺陷发生位置： /source/libs/executor/src/scanoperator.c	行号： 1719
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：tsEndCol,skey	
	代码片段： <pre> 1717: } 1718: 1719: if (tsEndCol[i] >= skey) { 1720: p[i] = true; 1721: } else { // this row should be removed, since it is not in this query time window, which is [skey, INT64_MAX] </pre>	
	污染路径： 无	
65	缺陷发生位置： /source/common/src/tdatablock.c	行号： 246
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,numOfRows	
	代码片段： <pre> 244: } else { 245: if (numOfRows < sizeof(char) * 2) { 246: for (int32_t i = 0; i < numOfRows; ++i) { 247: colDataSetNull_f(pColumnInfoData->nullbitmap, currentRow + i); 248: } </pre>	
	污染路径： 无	
66	缺陷发生位置： /utils/test/c/tmqSim.c	行号： 1270
	步骤描述： 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为：i,batchPerTblTimes	
	代码片段： <pre> 1268: while (totalSendLoopTimes) { 1269: int64_t startTs = taosGetTimestampUs(); 1270: for (int i = 0; i < batchPerTblTimes; ++i) { 1271: uint32_t msgsOfSql = g_stConfInfo.batchSize; 1272: if ((i == batchPerTblTimes - 1) && (0 != remainder)) { </pre>	
	污染路径： 无	
67	缺陷发生位置：	行号：

	/include/common/tmsg.h		3380
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,pSW		
	代码片段: 3378: if (pSW->tSma) { 3379: if (deepCopy) { 3380: for (uint32_t i = 0; i < pSW->number; ++i) { 3381: tDestroyTSma(pSW->tSma + i); 3382: }		
	污染路径: 无		
68	缺陷发生位置: /source/libs/executor/src/scanoperator.c		行号: 1715
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: tsStartCol,skey		
	代码片段: 1713: if (pWindow->skey != INT64_MIN) { 1714: for (int32_t i = 0; i < numOfRows; i++) { 1715: if (tsStartCol[i] < skey) { 1716: tsStartCol[i] = skey; 1717: }		
	污染路径: 无		
69	缺陷发生位置: /source/common/src/tmsg.c		行号: 7868
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: iRow,nRow		
	代码片段: 7866: } 7867: 7868: for (int32_t iRow = 0; iRow < nRow; ++iRow) { 7869: SRow **ppRow = taosArrayReserve(pSubmitTbData->aRowP, 1); 7870:		
	污染路径: 无		
70	缺陷发生位置: /source/libs/scalar/src/filter.c		行号: 2162
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,size		

	代码片段: 2160: SFilterRangeCtx *ctx = filterInitRangeCtx(type, 0); 2161: 2162: for (uint32_t i = 0; i < size; ++i) { 2163: SFilterUnit *u = taosArrayGetP(colArray, i); 2164: uint8_t optr = FILTER_UNIT_OPTR(u);	
	污染路径: 无	
71	缺陷发生位置: /source/util/src/tbloomfilter.c	行号: 95
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,pBF	
	代码片段: 93: const register uint64_t size = pBF->numBits; 94: uint64_t cbHash = h1; 95: for (uint64_t i = 0; i < pBF->hashFunctions; ++i) { 96: if (!getBit(pBF->buffer, cbHash % size)) { 97: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
72	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 120
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,arr3	
	代码片段: 118: uint64_t arr3[] = {3, 5, 10, 11}; 119: f = (SArray *)taosArrayGetP(src, 2); 120: for (int i = 0; i < sizeof(arr3) / sizeof(arr3[0]); i++) { 121: taosArrayPush(f, &arr3[i]); 122: }	
	污染路径: 无	
73	缺陷发生位置: /source/util/src/tcompression.c	行号: 1776
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCmprsor,BIT_TO_SELECTOR	
	代码片段: 1774: pCmprsor->i_selector = 0; 1775: for (int32_t iVal = pCmprsor->i_start; iVal <	

	pCmprsor->i_end; iVal = NEXT_IDX[iVal]) { 1776: if (pCmprsor->i_selector < BIT_TO_SELECTOR[pCmprsor->i_aBitN[iVal]]) { 1777: pCmprsor->i_selector = BIT_TO_SELECTOR[pCmprsor->i_aBitN[iVal]]; 1778: }	
	污染路径: 无	
74	缺陷发生位置: /source/libs/index/src/indexFstRegister.c	行号: 38
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: a,sz	
	代码片段: 36: static void fstRegistryCellSwap(SArray* arr, uint32_t a, uint32_t b) { 37: size_t sz = taosArrayGetSize(arr); 38: if (a >= sz b >= sz) { 39: return; 40: }	
	污染路径: 无	
75	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 4438
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: yyruleno,yyRuleName	
	代码片段: 4436: yymssp = yypParser->yertos; 4437: #ifndef NDEBUB 4438: if(yyTraceFILE && yyruleno<(int) (sizeof(yyRuleName)/sizeof(yyRuleName[0]))){ 4439: yysize = yyRuleInfoNRhs[yyruleno]; 4440: if(yysize){	
	污染路径: 无	
76	缺陷发生位置: /source/util/src/tlruccache.c	行号: 860
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,numShards	
	代码片段:	

	858: taosThreadMutexLock(&cache->shardedCache.capacityMutex); 859: 860: for (int i = 0; i < numShards; ++i) { 861: taosLRUCacheShardSetStrictCapacity(&cache->shards[i], strict); 862: }	
	污染路径: 无	
77	缺陷发生位置: /source/libs/index/src/indexFstRegister.c	行号: 38
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: b,sz	
	代码片段: 36: static void fstRegistryCellSwap(SArray* arr, uint32_t a, uint32_t b) { 37: size_t sz = taosArrayGetSize(arr); 38: if (a >= sz b >= sz) { 39: return; 40: }	
	污染路径: 无	
78	缺陷发生位置: /source/common/src/tatablock.c	行号: 306
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: maxNum,numOfRows	
	代码片段: 304: while (num < numOfRows) { 305: int32_t maxNum = num << 1; 306: int32_t tnum = maxNum > numOfRows ? (numOfRows - num) : num; 307: 308: memcpy(pColumnInfoData->pData + colOffset, pStart, tnum * colBytes);	
	污染路径: 无	
79	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 66
	步骤描述:	

	禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: __a,__b	
	代码片段: 64: } 65: rowSize += selectRowSize; 66: pFileState->maxRowCount = TMAX((uint64_t)memSize / rowSize, FLUSH_NUM * 2); 67: pFileState->usedBufs = tdListNew(POINTER_BYTES); 68: pFileState->freeBufs = tdListNew(POINTER_BYTES);	
	污染路径: expanded from macro 'TMAX'	
80	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndProfile.c	780
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: numOfQueries,offset	
	代码片段: 778: taosRLockLatch(&pConn->queryLock); 779: int32_t numOfQueries = taosArrayGetSize(pConn->pQueries); 780: if (NULL == pConn->pQueries numOfQueries <= offset) { 781: taosRUnlockLatch(&pConn->queryLock); 782: return 0;	
	污染路径: 无	
81	缺陷发生位置:	行号:
	/utils/test/c/tmqSim.c	1277
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: j,msgsOfSql	
	代码片段: 1275: int len = 0; 1276: len += snprintf(sqlBuf + len, MAX_SQL_LEN - len, "insert into %s values ", ctbName); 1277: for (int j = 0; j < msgsOfSql; j++) { 1278: int64_t timeStamp = taosGetTimestampNs(); 1279: len += snprintf(sqlBuf + len, MAX_SQL_LEN - len, "(%" PRIu64 ", \"%s\")", timeStamp, g_payload);	
	污染路径: 无	
82	缺陷发生位置:	行号:

	/source/libs/executor/src/scanoperator.c	1727
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: tsEndCol,ekey	
	代码片段: <pre> 1725: } else if (pWindow->ekey != INT64_MAX) { 1726: for(int32_t i = 0; i < numOfRows; ++i) { 1727: if (tsEndCol[i] > ekey) { 1728: tsEndCol[i] = ekey; 1729: } </pre>	
	污染路径: 无	
83	缺陷发生位置: /source/util/src/ttimer.c	行号: 524
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,tsMaxTmrCtrl	
	代码片段: <pre> 522: memset(&timerMap, 0, sizeof(timerMap)); 523: 524: for (uint32_t i = 0; i < tsMaxTmrCtrl - 1; ++i) { 525: tmr_ctrl_t* ctrl = tmrCtrls + i; 526: ctrl->next = ctrl + 1; </pre>	
	污染路径: 无	
84	缺陷发生位置: /include/util/tencode.h	行号: 205
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: pCoder,val	
	代码片段: <pre> 203: static FORCE_INLINE int32_t tEncodeU8(SEncoder* pCoder, uint8_t val) { 204: if (pCoder->data) { 205: if (TD_CODER_CHECK_CAPACITY_FAILED(pCoder, sizeof(val))) return -1; 206: tPut(uint8_t, TD_CODER_CURRENT(pCoder), val); 207: } </pre>	
	污染路径: expanded from macro 'TD_CODER_CHECK_CAPACITY_FAILED'	
85	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 1731
	步骤描述:	

	禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: tsStartCol,ekey	
	代码片段: <pre> 1729: } 1730: 1731: if (tsStartCol[i] <= ekey) { 1732: p[i] = true; 1733: } else { </pre>	
	污染路径: 无	
86	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 3073
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,yy_action	
	代码片段: <pre> 3071: return yy_default[stateno]; 3072: }else{ 3073: assert(i>=0 && i<sizeof(yy_action)/sizeof(yy_action[0])); 3074: return yy_action[i]; 3075: } </pre>	
	污染路径: 无	
87	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 235
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: strlen,maniLen	
	代码片段: <pre> 233: } 234: 235: if (strlen(name) >= maniLen && strncmp(name, pManifest, maniLen) == 0) { 236: taosMemoryFreeClear(bm->pManifest); 237: bm->pManifest = taosStrdup(name); </pre>	
	污染路径: 无	
88	缺陷发生位置: /source/common/src/tdatablock.c	行号: 262
	步骤描述: 禁止无符号数和有符号数之间的直接比较,两个变量名称分别为: i,numOfRows	
	代码片段:	

	<pre> 260: i += (numOfRows - i) / sizeof(char) * sizeof(char); 261: 262: for (; i < numOfRows; ++i) { 263: colDataSetNull_f(pColumnInfoData->nullbitmap, currentRow + i); 264: } </pre>
	污染路径: 无

14 C/C++缺陷类型-字符串问题

14.1 分项汇总表

序号	C/C++缺陷类型-字符串问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK5639	禁止字符串中单独使用“\”，字符串的终止必须使用\0	2	严重
2	WK4778	用于表示字符串的数组必须以\0结束	14	严重

14.2 缺陷/安全漏洞详解

14.2.1 严重类

14.2.1.1 WK5639 禁止字符串中单独使用 “\”,字符串的终止必须使用\0

缺陷数量： 2 个		
缺陷描述： 只用 “\”并不能控制字符串的终止，应该用 “\0” 控制字符串的终止。		
修复意见： 无		
1	缺陷发生位置： /tools/shell/src/shellAuto.c	行号： 387
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 385: printf("\nThe %s CLI supports the following commands:", shell.info.cusName); 386: printf(387: "\n\ 388: ----- A ----- \n\ 389: alter database <db_name> <db_options> \n\ 	
	污染路径： 无	
2	缺陷发生位置： /tools/shell/src/shellAuto.c	行号： 520
	步骤描述： 禁止字符串中单独使用\,字符串的终止必须使用\0	
	代码片段： 518: // define in getDuration() function 519: printf(520: "\n\ 521: Timestamp expression Format:\n\ 	

	522: b - nanosecond \n\
	污染路径: 无

14.2.1.2 WK4778 用于表示字符串的数组必须以 \0 结束

缺陷数量：14 个		
缺陷描述： 用于表示字符串的数组必须以 \0 结束		
修复意见： 无		
1	缺陷发生位置： /tools/shell/src/shellTire.c	行号： 344
	步骤描述： 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段： <pre> 342: // get all items from tires tree 343: void enumFromTree(STire* tire, SMatch* match) { 344: char pre[2] = {0, 0}; 345: STireNode* c; 346: </pre>	
	污染路径： 无	
2	缺陷发生位置： /source/util/src/tuuid.c	行号： 23
	步骤描述： 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段： <pre> 21: int32_t tGenIdPI32(void) { 22: if (tUUIDHashId == 0) { 23: char uid[65] = {0}; 24: int32_t code = taosGetSystemUUID(uid, sizeof(uid)); 25: uid[64] = 0; </pre>	
	污染路径： 无	
3	缺陷发生位置： /source/util/src/tcompression.c	行号： 102

	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 100: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 101: // 12 13 14 15 102: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 103: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 104: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13, 13,	
	污染路径: 无	
4	缺陷发生位置: /source/util/src/tcompression.c	行号: 259
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 257: // Selector value: 0 1 2 3 4 5 6 7 8 9 10 11 258: // 12 13 14 15 259: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 260: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 261:	
	污染路径: 无	
5	缺陷发生位置: /source/util/src/tcompression.c	行号: 104
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 102: char bit_per_integer[] = {0, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 20, 30, 60}; 103: int32_t selector_to_elems[] = {240, 120, 60, 30, 20, 15, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1}; 104: char bit_to_selector[] = {0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 11, 11, 12, 12, 12, 13, 13, 13, 13, 13, 105: 14, 14, 14, 14, 14, 14, 14, 14, 14, 14,	

	15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 106: 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15};	
	污染路径: 无	
6	缺陷发生位置: /include/util/tutil.h	行号: 72
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 70: tMD5Update(&context, inBuf, (uint32_t)len); 71: tMD5Final(&context); 72: char buf[TSDB_PASSWORD_LEN + 1]; 73: 74: buf[TSDB_PASSWORD_LEN] = 0;	
	污染路径: 无	
7	缺陷发生位置: /source/util/src/tcompare.c	行号: 1073
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 1071: } 1072: 1073: char rejectList[2] = {toupper(c), tolower(c)}; 1074: 1075: str += nMatchChar;	
	污染路径: 无	
8	缺陷发生位置: /source/util/src/tconfig.c	行号: 1062
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 1060: } 1061: 1062: char line[1024]; 1063: char **pEnv = environ; 1064: line[1023] = 0;	

	污染路径: 无	
9	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 498
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 496: } 497: 498: char quotationStr[2]; 499: quotationStr[0] = '\\"'; 500: quotationStr[1] = 0;	
	污染路径: 无	
10	缺陷发生位置: /source/os/src/osSocket.c	行号: 991
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 989: } 990: #endif 991: char hostname[1024]; 992: hostname[1023] = '\0'; 993: if (taosGetlocalhostname(hostname, 1023) == -1) {	
	污染路径: 无	
11	缺陷发生位置: /source/util/src/tconfig.c	行号: 656
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: 654: 655: int32_t cfgLoadFromEnvVar(SConfig *pConfig) { 656: char line[1024], *name, *value, *value2, *value3; 657: int32_t olen, vlen, vlen2, vlen3; 658: int32_t code = 0;	
	污染路径: 无	
12	缺陷发生位置: /source/util/src/tutil.c	行号: 369

	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: <pre> 367: int32_t taosByteArrayToHexStr(char bytes[], int32_t len, char hexstr[]) { 368: int32_t i; 369: char hexval[16] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f'}; 370: 371: for (i = 0; i < len; i++) { </pre>	
	污染路径: 无	
13	缺陷发生位置:	行号:
	/tools/shell/src/shellEngine.c	326
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: <pre> 324: } 325: 326: char quotationStr[2] = {"", 0}; 327: int32_t width; 328: </pre>	
14	污染路径: 无	
	缺陷发生位置:	行号:
	/source/common/src/ttime.c	386
	步骤描述: 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段: <pre> 384: 385: char getPrecisionUnit(int32_t precision) { 386: static char units[3] = {TIME_UNIT_MILLISECOND, TIME_UNIT_MICROSECOND, TIME_UNIT_NANOSECOND}; 387: switch (precision) { 388: case TSDB_TIME_PRECISION_MILLI: </pre>	
	污染路径: 无	

15 C/C++缺陷类型-返回值问题

15.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4729	有返回值的函数必须通过返回语句返回	44	严重
2	WK4732	函数返回值的类型必须与定义一致	376	严重
3	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有void说明	1002	严重

15.2 缺陷/安全漏洞详解

15.2.1 严重类

15.2.1.1 WK4729 有返回值的函数必须通过返回语句返回

缺陷数量： 44 个		
缺陷描述： 有返回值的函数必须通过返回语句返回。		
修复意见： 无		
1	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbRead.c	行号： 3604
步骤描述:		

	有返回值的函数必须通过返回语句返回	
	代码片段: 3602: } ERetrieveType; 3603: 3604: static ERetrieveType doReadDataFromLastFiles(STsdbReader* pReader) { 3605: int32_t code = TSDB_CODE_SUCCESS; 3606: SSDataBlock* pResBlock = pReader->resBlockInfo.pResBlock; }	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/source/client/src/clientSmlJson.c	815
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 813: } 814: 815: static int64_t smlParseTSFromJSONObj(SSmlHandle *info, cJSON *root, int32_t toPrecision) { 816: int32_t size = cJSON_GetArraySize(root); 817: if (unlikely(size != OTD_JSON_SUB_FIELDS_NUM)) {	
3	污染路径: 无	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbRead2.c	1452
	步骤描述: 有返回值的函数必须通过返回语句返回	
4	代码片段: 1450: } 1451: 1452: static FORCE_INLINE STSchema* doGetSchemaForTSRow(int32_t sversion, STsdbReader* pReader, uint64_t uid) { 1453: // always set the newest schema version in pReader->info.pSchema 1454: if (pReader->info.pSchema == NULL) {	
	污染路径: 无	
	缺陷发生位置:	行号:

	/source/libs/parser/src/parAstCreator.c	528
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 526: } 527: 528: static int32_t addParamToLogicConditionNode(SLogicConditionNode* pCond, SNode* pParam) { 529: if (QUERY_NODE_LOGIC_CONDITION == nodeType(pParam) && pCond->condType == ((SLogicConditionNode*)pParam)->condType) { 530: int32_t code = nodesListAppendList(pCond->pParameterList, ((SLogicConditionNode*)pParam)->pParameterList); } }	
	污染路径: 无	
5	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeOpen.c	行号: 31
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 29: } 30: 31: static int32_t vnodeMkDir(STfs *pTfs, const char *path) { 32: if (pTfs) { 33: return tfsMkdirRecur(pTfs, path); } }	
	污染路径: 无	
6	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 785
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 783: } 784: 785: static bool isGlobalTimeLineQuery(SNode* pStmt) { 786: if (QUERY_NODE_SELECT_STMT == nodeType(pStmt)) { 787: return TIME_LINE_GLOBAL == ((SSelectStmt*)pStmt)->timeLineResMode; } }	

	污染路径: 无	
7	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1538
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1536: } 1537: 1538: void* tsortGetValue(STupleHandle* pVHandle, int32_t collIndex) { 1539: SColumnInfoData* pCollInfo = TARRAY_GET_ELEM(pVHandle->pBlock->pDataBlock, collIndex); 1540: if (pCollInfo->pData == NULL) {	
	污染路径: 无	
8	缺陷发生位置: /source/util/src/tcompare.c	行号: 187
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 185: int32_t compareDoubleValDesc(const void *pLeft, const void *pRight) { return compareDoubleVal(pRight, pLeft); } 186: 187: int32_t compareLenPrefixedStr(const void *pLeft, const void *pRight) { 188: int32_t len1 = varDataLen(pLeft); 189: int32_t len2 = varDataLen(pRight);	
	污染路径: 无	
9	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 277
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 275: } 276: 277: SOperatorInfo* createOperator(SPhysiNode* pPhyNode, SExecTaskInfo* pTaskInfo, SReadHandle* pHandle, SNode* pTagCond, 278: SNode* pTagIndexCond, const char*	

	pUser, const char* dbname) { 279: int32_t type = nodeType(pPhyNode);	
	污染路径: 无	
10	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbRaw.c	行号: 19
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: <pre>17: #include "sdb.h" 18: 19: int32_t sdbGetIdFromRaw(SSdb *pSdb, SSdbRaw *pRaw) { 20: EKeyType keytype = pSdb->keyTypes[pRaw->type]; 21: if (keytype == SDB_KEY_INT32) {</pre>	
	污染路径: 无	
11	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSync.c	行号: 617
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: <pre>615: } 616: 617: static int32_t vnodeApplyQueueItems(const SSyncFSM *pFsm) { 618: SVnode *pVnode = pFsm->data; 619:</pre>	
	污染路径: 无	
12	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 744
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: <pre>742: 743: // SMergeTree ===== ===== 744: static FORCE_INLINE int32_t tLDataIterCmprFn(const SRBTreeNode *p1, const SRBTreeNode *p2) {</pre>	

	745: SLDataalter *plter1 = (SLDataalter *)(((uint8_t *)p1) - offsetof(SLDataalter, node)); 746: SLDataalter *plter2 = (SLDataalter *)(((uint8_t *)p2) - offsetof(SLDataalter, node));	
	污染路径: 无	
13	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 1503
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1501: } 1502: 1503: int32_t bpSetTableNameTags(BindData *data, int32_t tblIdx, char *tblName, TAOS_STMT *stmt) { 1504: int32_t code = 0; 1505: if (gCurCase->bindTagNum > 0) {	
	污染路径: 无	
14	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 1722
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1720: } 1721: 1722: static int32_t mergeFileBlockAndLastBlock(STsdbReader* pReader, SLastBlockReader* pLastBlockReader, int64_t key, 1723: STableBlockScanInfo* pBlockScanInfo, SBlockData* pBlockData) { 1724: SFileBlockDumpInfo* pDumpInfo = &pReader- >status.fBlockDumpInfo;	
	污染路径: 无	
15	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 181
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	179: } 180: 181: int32_t schProcessOnTaskFailure(SSchJob *pJob, SSchTask *pTask, int32_t errCode) { 182: if (TSDB_CODE_SCH_IGNORE_ERROR == errCode) { 183: return TSDB_CODE_SCH_IGNORE_ERROR; 	
	污染路径: 无	
16	缺陷发生位置: /source/libs/executor/src/cachescanoperator.c	行号: 149
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 147: } 148: 149: SSDataBlock* doScanCache(SOperatorInfo* pOperator) { 150: if (pOperator->status == OP_EXEC_DONE) { 151: return NULL; 	
	污染路径: 无	
17	缺陷发生位置: /source/util/src/tcompression.c	行号: 2502
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2500: } 2501: 2502: int32_t tsDecompressTinyint(void *pIn, int32_t nIn, int32_t nEle, void *pOut, int32_t nOut, uint8_t cmprAlg, void *pBuf, 2503: int32_t nBuf) { 2504: if (cmprAlg == ONE_STAGE_COMP) { 	
	污染路径: 无	
18	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 32
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 30: }	

	31: 32: int32_t tqOffsetRestoreFromFile(STqOffsetStore* pStore, const char* fname) { 33: TdFilePtr pFile = taosOpenFile(fname, TD_FILE_READ); 34: if (pFile == NULL) {	
	污染路径: 无	
19	缺陷发生位置: /source/client/src/clientSml.c	行号: 158
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 156: } 157: 158: int8_t smlGetTsTypeByLen(int32_t len) { 159: if (len == TSDB_TIME_PRECISION_SEC_DIGITS) { 160: return TSDB_TIME_PRECISION_SECONDS;	
	污染路径: 无	
20	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 3001
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2999: } ERetrieveType; 3000: 3001: static ERetrieveType doReadDataFromLastFiles(STsdbReader* pReader) { 3002: int32_t code = TSDB_CODE_SUCCESS; 3003: SSDataBlock* pResBlock = pReader->resBlockInfo.pResBlock;	
	污染路径: 无	
21	缺陷发生位置: /source/client/src/clientTmq.c	行号: 2249
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2247: } 2248:	

	2249: int32_t tmq_get_vgroup_id(TAOS_RES* res) { 2250: if (res == NULL){ 2251: return -1;	
	污染路径: 无	
22	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 397
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 395: } 396: 397: static int32_t mndApplyQueueItems(const SSyncFSM *pFsm) { 398: SMnode *pMnode = pFsm->data; 399:	
	污染路径: 无	
23	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 1192
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1190: } 1191: 1192: static int32_t mndTransExecuteActions(SMnode *pMnode, STrans *pTrans, SArray *pArray) { 1193: int32_t numOfActions = taosArrayGetSize(pArray); 1194: if (numOfActions == 0) return 0;	
	污染路径: 无	
24	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 378
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 376: } SGroupSortSourceParam; 377: 378: SSDataBlock* fetchNextGroupSortDataBlock(void* param) {	

	379: SGroupSortSourceParam* source = param; 380: SGroupSortOperatorInfo* grpSortOpInfo = source->grpSortOpInfo;	
	污染路径: 无	
25	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 2600
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2598: } 2599: 2600: static int32_t setSysTableVgroupList(STranslateContext* pCxt, SName* pName, SRealTableNode* pRealTable) { 2601: if (sysTableFromVnode(pRealTable->table.tableName)) { 2602: return setVnodeSysTableVgroupList(pCxt, pName, pRealTable); }	
	污染路径: 无	
26	缺陷发生位置: /source/util/src/tcompare.c	行号: 228
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 226: } 227: 228: int32_t compareLenBinaryVal(const void *pLeft, const void *pRight) { 229: int32_t len1 = varDataLen(pLeft); 230: int32_t len2 = varDataLen(pRight);	
	污染路径: 无	
27	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 154
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 152: } 153:	

	154: static int32_t tsdbJsonToSttLvl(STsdb *pTsdb, const cJSON *json, SSttLvl **lvl) { 155: const cJSON *item1, *item2; 156: int32_t level;	
	污染路径: 无	
28	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 1821
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1819: } 1820: 1821: static SSDataBlock* doQueueScan(SOperatorInfo* pOperator) { 1822: SExecTaskInfo* pTaskInfo = pOperator->pTaskInfo; 1823: SStorageAPI* pAPI = &pTaskInfo->storageAPI;	
	污染路径: 无	
29	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 119
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 117: } 118: 119: static int32_t doSetStreamBlock(SOperatorInfo* pOperator, void* input, size_t numOfBlocks, int32_t type, const char* id) { 120: if (pOperator->operatorType != QUERY_NODE_PHYSICAL_PLAN_STREAM_SCAN) { 121: if (pOperator->numOfDownstream == 0) {	
	污染路径: 无	
30	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 1139
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1137: }	

	1138: } 1139: bool fstBoundWithDatalsEmpty(FstBoundWithData* bound) { 1140: if (bound->type == Unbounded) { 1141: return true;	
	污染路径: expanded from macro 'bool'	
31	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 2018
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2016: } 2017: 2018: static FORCE_INLINE STSchema* doGetSchemaForTSRow(int32_t sversion, STsdbReader* pReader, uint64_t uid) { 2019: // always set the newest schema version in pReader->pSchema 2020: if (pReader->pSchema == NULL) {	
	污染路径: 无	
32	缺陷发生位置: /source/libs/stream/src/streamTask.c	行号: 395
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 393: } 394: 395: int32_t streamTaskGetNumOfDownstream(const SStreamTask* pTask) { 396: if (pTask->info.taskLevel == TASK_LEVEL__SINK) { 397: return 0;	
	污染路径: 无	
33	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 166
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	164: } 165: 166: static bool vmDataSpaceSufficient(SVnodeObj *pVnode) { 167: STfs *pTfs = pVnode->pImpl->pTfs; 168: if (pTfs) {	
	污染路径: 无	
34	缺陷发生位置: /source/client/src/clientImpl.c	行号: 1685
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1683: } 1684: 1685: void* doAsyncFetchRows(SRequestObj* pRequest, bool setupOneRowPtr, bool convertUcs4) { 1686: if (pRequest == NULL) { 1687: return NULL;	
	污染路径: 无	
35	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 1735
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1733: } 1734: 1735: static SNodeList* partTagsGetFuncs(SLogicNode* pNode) { 1736: if (QUERY_NODE_LOGIC_PLAN_PARTITION == nodeType(pNode)) { 1737: return NULL;	
	污染路径: 无	
36	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 347
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	345: } 346: 347: SArray *dmGetMsgHandles() { 348: int32_t code = -1; 349: SArray *pArray = taosArrayInit(16, sizeof(SMgmtHandle));	
	污染路径: 无	
37	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 1029
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1027: } 1028: 1029: int32_t qRestoreStreamOperatorOption(qTaskInfo_t tinfo) { 1030: SExecTaskInfo* pTaskInfo = (SExecTaskInfo*)tinfo; 1031: const char* id = GET_TASKID(pTaskInfo);	
	污染路径: 无	
38	缺陷发生位置: /source/common/src/tdformat.c	行号: 1023
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1021: return code; 1022: } 1023: static int32_t tRowKVUpsertColData(SRow *pRow, STSchema *pTSchema, SColData *aColData, int32_t nColData, int32_t flag) { 1024: int32_t code = 0; 1025:	
	污染路径: 无	
39	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 1216
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	1214: 1215: 1216: bool compareVal(const char* v, const SStateKeys* pKey) { 1217: if (IS_VAR_DATA_TYPE(pKey->type)) { 1218: if (varDataLen(v) != varDataLen(pKey->pData)) {	
	污染路径: expanded from macro 'bool'	
40	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 157
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 155: } 156: 157: int32_t udfdCPluginUdfAggProc(SUdfDataBlock *block, SUdfInterBuf *interBuf, SUdfInterBuf *newInterBuf, void *udfCtx) { 158: SUdfCPluginCtx *ctx = udfCtx; 159: if (ctx->aggProcFunc) {	
	污染路径: 无	
41	缺陷发生位置: /source/client/src/clientTmq.c	行号: 1888
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1886: } 1887: 1888: static void* tmqHandleAllRsp(tmq_t* tmq, int64_t timeout, bool pollIfReset) { 1889: tscDebug("consumer:0x%" PRIx64 " start to handle the rsp, total:%d", tmq->consumerId, tmq->qall->numOfItems); 1890:	
	污染路径: 无	
42	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 136
	步骤描述:	

	有返回值的函数必须通过返回语句返回	
	代码片段: 134: } 135: 136: int32_t udfdCPluginUdfScalarProc(SUdfDataBlock *block, SUdfColumn *resultCol, void *udfCtx) { 137: SUdfCPluginCtx *ctx = udfCtx; 138: if (ctx->scalarProcFunc) {	
	污染路径: 无	
43	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 657
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 655: } 656: 657: int32_t syncNodePropose(SSyncNode* pSyncNode, SRpcMsg* pMsg, bool isWeak, int64_t* seq) { 658: if (pSyncNode->state != TAOS_SYNC_STATE_LEADER) { 659: terrno = TSDB_CODE_SYN_NOT_LEADER;	
	污染路径: 无	
44	缺陷发生位置: /source/util/src/tcompression.c	行号: 748
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 746: } 747: 748: int32_t tsDecompressTimestampImp(const char *const input, const int32_t nelements, char *const output) { 749: ASSERTS(nelements >= 0, "nelements is negative"); 750: if (nelements == 0) return 0;	
	污染路径: 无	

15.2.1.2 WK4732 函数返回值的类型必须与定义一致

缺陷数量： 376 个		
缺陷描述： 函数返回值的类型必须与定义一致。		
修复意见： 无		
1	缺陷发生位置： /source/libs/executor/src/timewindowoperator.c	行号： 371
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 369: if ((key > blockEkey && (order == TSDB_ORDER_ASC)) (key < blockEkey && (order == TSDB_ORDER_DESC))) { 370: setNotInterpoWindowKey(pSup->pCtx, pSup->numOfExprs, RESULT_ROW_END_INTERP); 371: return false; 372: } 373:	
	污染路径： expanded from macro 'false'	
2	缺陷发生位置： /source/libs/executor/src/mergejoinoperator.c	行号： 717
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 715: if (NULL == pJoinInfo->pLeft NULL == pJoinInfo->pRight) { 716: setMergeJoinDone(pOperator); 717: return false; 718: } 719:	
	污染路径： expanded from macro 'false'	
3	缺陷发生位置： /source/util/src/tcache.c	行号： 327
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段：	

	325: 326: // this node should be remove from hash table 327: return false; 328: } 329:	
	污染路径: expanded from macro 'false'	
4	缺陷发生位置: /source/util/src/ttimer.c	行号: 464
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 462: timerDecRef(timer); 463: 464: return state == TIMER_STATE_WAITING; 465: } 466:	
	污染路径: 无	
5	缺陷发生位置: /source/common/src/tdatablock.c	行号: 782
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 780: } else { 781: if (pColumnInfoData->nullbitmap == NULL) { 782: return false; 783: } 784:	
	污染路径: expanded from macro 'false'	
6	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 743
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 741: buf->nullTupleSaved = false; 742: buf->nullTuplePos.pageld = -1; 743: return true; 744: }	

	745:	
	污染路径: expanded from macro 'true'	
7	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 2258
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 2256: --*iSkyline; 2257: } else { 2258: return false; 2259: } 2260: }	
	污染路径: expanded from macro 'false'	
8	缺陷发生位置: /source/util/src/thash.c	行号: 706
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 704: size_t taosHashGetMemSize(const SHashObj *pHashObj) { 705: if (pHashObj == NULL) { 706: return 0; 707: } 708:	
	污染路径: 无	
9	缺陷发生位置: /source/util/src/tcompression.c	行号: 745
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: 743: output[0] = 0; // Means the string is not compressed 744: memcpy(output + 1, input, nelements * LONG_BYTES); 745: return nelements * LONG_BYTES + 1; 746: } 747:	
	污染路径:	

	无	
10	缺陷发生位置： /tests/script/api/batchprepare.c	行号： 384
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 382: while (true) { 383: if (0 == pBind[i].buffer_type) { 384: return false; 385: } 386:	
	污染路径： expanded from macro 'false'	
11	缺陷发生位置： /source/libs/command/src/command.c	行号： 335
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 333: } 334: 335: return false; 336: } 337:	
	污染路径： expanded from macro 'false'	
12	缺陷发生位置： /source/libs/sync/src/syncUtil.c	行号： 78
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 76: 77: bool syncUtilSameId(const SRaftId* pld1, const SRaftId* pld2) { 78: if (pld1->addr == pld2->addr && pld1->vgld == pld2->vgld) return true; 79: if ((CID(pld1) == 0 CID(pld2) == 0) && (DID(pld1) == DID(pld2)) && pld1->vgld == pld2->vgld) return true; 80: return false;	
	污染路径： expanded from macro 'true'	

13	缺陷发生位置: /source/client/src/clientTmq.c	行号: 1537
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1535: SArray* newTopics = taosArrayInit(topicNumGet, sizeof(SMqClientTopic)); 1536: if (newTopics == NULL) { 1537: return false; 1538: } 1539:	
	污染路径: expanded from macro 'false'	
14	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 679
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 677: GenerateVarType(WT_VAR_LANGUAGE, udf_language, sizeof(udf_language) / sizeof(char*)); 678: 679: return true; 680: } 681:	
	污染路径: expanded from macro 'true'	
15	缺陷发生位置: /source/libs/stream/src/streamUpdate.c	行号: 93
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int *'.	
	代码片段: 91: 92: SUpdateInfo *updateInfoInitP(SInterval *pInterval, int64_t watermark, bool igUp) { 93: return updateInfoInit(pInterval->interval, pInterval->precision, watermark, igUp); 94: } 95:	
	污染路径: 无	

16	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 385
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 383: vgld, ctbName, suid, pReader->me.ctbEntry.suid); 384: terrno = TSDB_CODE_TDB_TABLE_IN_OTHER_STABLE; 385: return false; 386: } 387:	
	污染路径: expanded from macro 'false'	
17	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1125
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1123: return false; 1124: #else 1125: return false; 1126: #endif 1127: }	
	污染路径: expanded from macro 'false'	
18	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 159
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 157: continue; 158: } else { 159: return false; 160: } 161: } else {	
	污染路径: expanded from macro 'false'	
19	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 339
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	

	代码片段: 337: CONN_GET_NEXT_SENDMSG(conn); 338: cliSend(conn); 339: return true; 340: } 341: return false;	
	污染路径: expanded from macro 'true'	
20	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 2584
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 2582: } 2583: } 2584: return false; 2585: } 2586:	
	污染路径: expanded from macro 'false'	
21	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 240
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 238: taosCloseFile(&pFile); 239: taosMemoryFree(buf); 240: return -1; 241: } 242:	
	污染路径: 无	
22	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 371
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: 369: struct timeval systemTime; 370: taosGetTimeOfDay(&systemTime); 371: return (int64_t)systemTime.tv_sec * 1000LL +	

	(int64_t)systemTime.tv_usec/1000; 372: } 373:	
	污染路径: 无	
23	缺陷发生位置: /source/util/src/talgo.c	行号: 194
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'uint8_t*', 定义类型为'void*'.	
	代码片段: 192: return (c >= 0) ? p : (midx > 0 ? p - size : NULL); 193: } else if (flags == TD_GT) { 194: return (c < 0) ? p : (midx + 1 < nmemb ? p + size : NULL); 195: } else if (flags == TD_LT) { 196: return (c > 0) ? p : (midx > 0 ? p - size : NULL);	
	污染路径: 无	
24	缺陷发生位置: /source/os/src/osFile.c	行号: 150
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 148: /* coverity[+retval] */ 149: taosRemoveFile(to); 150: return -1; 151: #endif 152: }	
	污染路径: 无	
25	缺陷发生位置: /source/libs/sync/src/syncUtil.c	行号: 80
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 78: if (pId1->addr == pId2->addr && pId1->vgId == pId2->vgId) return true; 79: if ((CID(pId1) == 0 CID(pId2) == 0) && (DID(pId1) == DID(pId2)) && pId1->vgId == pId2->vgId) return true; 80: return false;	

	81: } 82:	
	污染路径: expanded from macro 'false'	
26	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 403
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 401: tDecoderClear(&decoder); 402: tqError("decode wal file error, msgLen:%d, ver:%" PRId64, bodyLen, ver); 403: return false; 404: } 405:	
	污染路径: expanded from macro 'false'	
27	缺陷发生位置: /source/util/src/texception.c	行号: 150
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 148: 149: void cleanupExecute(SExceptionNode* node, bool failed) { doExecuteCleanup(node, 0, failed); } 150: bool cleanupExceedLimit() { return expList->numCleanupAction >= expList->maxCleanupAction; }	
	污染路径: 无	
28	缺陷发生位置: /source/os/src/osDir.c	行号: 465
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 463: bool taosDirEntryIsDir(TdDirEntryPtr pDirEntry) { 464: if (pDirEntry == NULL) { 465: return false; 466: } 467: #ifdef WINDOWS	

	污染路径: expanded from macro 'false'	
29	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 277
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 275: 276: bool tsdbIsSameTFile(const STFile *f1, const STFile *f2) { 277: if (f1->type != f2->type) return false; 278: if (f1->did.level != f2->did.level) return false; 279: if (f1->did.id != f2->did.id) return false;	
	污染路径: expanded from macro 'false'	
30	缺陷发生位置: /include/common/tdatablock.h	行号: 78
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 76: } else { 77: if (pColumnInfoData->nullbitmap == NULL) { 78: return false; 79: } 80:	
	污染路径: expanded from macro 'false'	
31	缺陷发生位置: /source/os/src/osString.c	行号: 238
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 236: *len = (int32_t)(ucs4_max_len - outLeft); 237: if (*len < 0) { 238: return false; 239: } 240: }	
	污染路径: expanded from macro 'false'	
32	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 407

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 405: for (int32_t i = 0; i < numOfCols; i++) { 406: if (fmIsUserDefinedFunc(pFCtx[i].functionId) ! fmIsInvertible(pFCtx[i].functionId)) { 407: return false; 408: } 409: }</pre>	
	污染路径: expanded from macro 'false'	
33	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 72
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: <pre> 70: if (pSyncNode->rid < 0) { 71: syncNodeClose(pSyncNode); 72: return -1; 73: } 74:</pre>	
	污染路径: 无	
34	缺陷发生位置: /source/common/src/tdatablock.c	行号: 62
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: <pre> 60: int32_t colDataGetFullLength(const SColumnInfoData* pColumnInfoData, int32_t numRows) { 61: if (IS_VAR_DATA_TYPE(pColumnInfoData->info.type)) { 62: return pColumnInfoData->varmeta.length + sizeof(int32_t) * numRows; 63: } else { 64: return ((pColumnInfoData->info.type == TSDB_DATA_TYPE_NULL) ? 0 : pColumnInfoData->info.bytes * numOfRows) +</pre>	
	污染路径: 无	
35	缺陷发生位置:	行号:

	/tools/shell/src/shellUtil.c	37
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 35: if (!reti) { 36: regfree(&regex); 37: return true; 38: } else if (reti == REG_NOMATCH) { 39: regfree(&regex); </pre>	
	污染路径: expanded from macro 'true'	
36	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 109
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: <pre> 107: int64_t syncHbTimerDataAdd(SSyncHbTimerData *pData) { 108: pData->rid = taosAddRef(gHbDataRefId, pData); 109: if (pData->rid < 0) return -1; 110: return pData->rid; 111: } </pre>	
	污染路径: 无	
37	缺陷发生位置: /source/libs/stream/src/streamTask.c	行号: 404
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'size_t', 定义类型为'int32_t'.	
	代码片段: <pre> 402: } else { 403: SArray* vgInfo = pTask- >shuffleDispatcher.dbInfo.pVgroupInfos; 404: return taosArrayGetSize(vgInfo); 405: } 406: } </pre>	
	污染路径: 无	
38	缺陷发生位置: /source/util/src/tskiplist.c	行号: 290
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 288: 289: bool tSkipListIterNext(SSkipListIterator *iter) { 290: if (iter->pSkipList == NULL) return false; 291: 292: SSkipList *pSkipList = iter->pSkipList;	
	污染路径: expanded from macro 'false'	
39	缺陷发生位置: /source/util/src/tcache.c	行号: 313
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 311: 312: // do not remove element in hash table 313: return true; 314: } 315:	
	污染路径: expanded from macro 'true'	
40	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 391
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 389: SStatisBlk *p = &pStatisBlkArray->data[i]; 390: if (p->minTbid.suid > suid) { 391: return false; 392: } 393:	
	污染路径: expanded from macro 'false'	
41	缺陷发生位置: /source/common/src/trow.c	行号: 188
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 186: tdGetKvRowValOfCol(pVal, pRow, pBitmap, pColIdx->offset,	

	499: return false;	
	污染路径: expanded from macro 'false'	
45	缺陷发生位置: /source/common/src/systable.c	行号: 456
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 454: bool invisibleColumn(bool sysInfo, int8_t tableType, int8_t flags) { 455: if (sysInfo TSDB_SYSTEM_TABLE != tableType) { 456: return false; 457: } 458: return 0 != (flags & COL_IS_SYSINFO);	
	污染路径: expanded from macro 'false'	
46	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 41
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 39: if (unlikely(!data)) { 40: smlBuildInvalidDataMsg(&info->msgBuf, "timestamp can not be null", NULL); 41: return -1; 42: } 43: if (unlikely(len == 1 && data[0] == '0')) {	
	污染路径: 无	
47	缺陷发生位置: /source/common/src/trow.c	行号: 182
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 180: if (!pColIdx) { 181: pVal->valType = TD_VTYPE_NONE; 182: return true; 183: } 184:	

	污染路径: expanded from macro 'true'	
48	缺陷发生位置: /source/os/src/osSystem.c	行号: 253
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 251: int64_t taosGetsCmd(TdCmdPtr pCmd, int32_t maxSize, char* __restrict buf) { 252: if (pCmd == NULL buf == NULL) { 253: return -1; 254: } 255: if (fgets(buf, maxSize, (FILE*)pCmd) == NULL) {	
	污染路径: 无	
49	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 1037
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1035: } 1036: 1037: return true; 1038: } 1039:	
	污染路径: expanded from macro 'true'	
50	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 229
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 227: } 228: 229: return false; 230: } 231:	
	污染路径: expanded from macro 'false'	
51	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tsdb/tsdbRead2.c		1192
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: <pre> 1190: return pBlock->record.lastKey == pRec->firstKey; 1191: } else { 1192: return pBlock->record.firstKey == pRec->lastKey; 1193: } 1194: }</pre>		
	污染路径: 无		
52	缺陷发生位置:		行号:
	/source/libs/executor/src/executor.c		258
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.		
	代码片段: <pre> 256: if (NULL == pTaskInfo) { 257: terrno = TSDB_CODE_OUT_OF_MEMORY; 258: return NULL; 259: } 260:</pre>		
53	缺陷发生位置:		行号:
	/source/libs/scalar/src/scifunc.c		1685
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: <pre> 1683: bool getTimePseudoFuncEnv(SFunctionNode *UNUSED_PARAM(pFunc), SFuncExecEnv *pEnv) { 1684: pEnv->calcMemSize = sizeof(int64_t); 1685: return true; 1686: } 1687:</pre>		
54	缺陷发生位置:		行号:
	/source/client/src/clientTmq.c		1532
54	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		

	代码片段: 1530: int32_t topicNumGet = taosArrayGetSize(pRsp->topics); 1531: if (epoch <= tmq->epoch) { 1532: return false; 1533: } 1534:	
	污染路径: expanded from macro 'false'	
55	缺陷发生位置: /source/libs/parser/src/parCalcConst.c	行号: 388
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 386: SNotRefByOrderByCxt cxt = {.pCol = pCol, .hasThisCol = false}; 387: nodesWalkExprs(pOrderByList, notRefByOrderByImpl, &cxt); 388: return !cxt.hasThisCol; 389: } 390:	
	污染路径: 无	
56	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 371
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 369: } 370: } 371: return false; 372: } 373:	
	污染路径: expanded from macro 'false'	
57	缺陷发生位置: /source/client/src/clientSml.c	行号: 149
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段:	

	147: int64_t unit = smIToMilli[fromPrecision - TSDB_TIME_PRECISION_HOURS]; 148: if (tsInt64 != 0 && unit > INT64_MAX / tsInt64) { 149: return -1; 150: } 151: tsInt64 *= unit;	
	污染路径: 无	
58	缺陷发生位置:	行号:
	/tools/shell/src/shellEngine.c	441
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 439: TAOS_ROW row = taos_fetch_row(tres); 440: if (row == NULL) { 441: return 0; 442: } 443:	
59	污染路径: 无	
	缺陷发生位置:	行号:
	/source/common/src/tname.c	161
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
60	代码片段: 159: } 160: 161: bool tNameIsEmpty(const SName* name) { return name->type == 0 name->acctId == 0; } 162: 163: const char* tNameGetTableName(const SName* name) {	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/util/src/thash.c	719
60	步骤描述: 函数返回值类型必须与定义一致，实际返回为'char *'，定义类型为'void *'.	
	代码片段: 717: } 718:	

	719: return GET_HASH_NODE_KEY(node); 720: } 721:	
	污染路径: expanded from macro 'GET_HASH_NODE_KEY'	
61	缺陷发生位置: /source/common/src/tmsg.c	行号: 7452
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 7450: if (pLeft->type == pRight->type) { 7451: if (pLeft->type == TMQ_OFFSET_LOG) { 7452: return pLeft->version == pRight->version; 7453: } else if (pLeft->type == TMQ_OFFSET_SNAPSHOT_DATA) { 7454: return pLeft->uid == pRight->uid && pLeft->ts == pRight->ts;	
	污染路径: 无	
62	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 216
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 214: taosRealPath(nameBuf2, NULL, TMPNAME_LEN); 215: if (strncmp(nameBuf1, nameBuf2, TMPNAME_LEN) != 0) return false; 216: return true; 217: } 218:	
	污染路径: expanded from macro 'true'	
63	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 97
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 95: 96: bool fmlsBuiltinFunc(const char* pFunc) { 97: return NULL !=	

	taosHashGet(gFunMgtService.pFuncNameHashTable, pFunc, strlen(pFunc)); 98: } 99:	
	污染路径: expanded from macro 'NULL'	
64	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 398
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'char *', 定义类型为'void *'.	
	代码片段: 396: int32_t bucketIndex = index / pBuf->numPerBucket; 397: char** pBucket = taosArrayGet(pBuf->pData, bucketIndex); 398: return (*pBucket) + (index % pBuf->numPerBucket) * sizeof(STableBlockScanInfo); 399: } 400:	
	污染路径: 无	
65	缺陷发生位置: /source/libs/wal/src/walSeek.c	行号: 85
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 83: if (code != 0) { 84: terrno = TAOS_SYSTEM_ERROR(errno); 85: return -1; 86: } 87: code = taosCloseFile(&pWal->pLogFile);	
	污染路径: 无	
66	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 48
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 46: } 47: 48: return false;	

	49: } 50:	
	污染路径: expanded from macro 'false'	
67	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 346
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint64_t'.	
	代码片段: 344: bool null = false; 345: fstStateCommInput(s, &null); 346: return null ? 1 : 0; 347: } 348:	
	污染路径: 无	
68	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 184
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 182: end: 183: tDecoderClear(&coder); 184: return tbSuid == realTbSuid; 185: } 186:	
	污染路径: 无	
69	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 237
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 235: 236: initGroupedResultInfo(&pAggInfo->groupResInfo, pAggInfo->aggSup.pResultRowHashTable, 0); 237: return pBlock != NULL; 238: } 239:	

	污染路径: 无	
70	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 561
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint64_t'.	
	代码片段: 559: } 560: 561: return 0; 562: } 563:	
	污染路径: 无	
71	缺陷发生位置: /source/libs/wal/src/walRead.c	行号: 157
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 155: pReader->pWal->cfg.vgld, ret, sizeof(SWalIdxEntry)); 156: } 157: return -1; 158: } 159:	
	污染路径: 无	
72	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 35
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 33: bool filterRangeCompGe(const void *minv, const void *maxv, const void *minr, const void *maxr, __compar_fn_t cfunc) { 34: int32_t result = cfunc(maxv, minr); 35: return result > 0; 36: } 37: bool filterRangeCompLi(const void *minv, const void *maxv, const void *minr, const void *maxr, __compar_fn_t cfunc) {	

	污染路径: 无	
73	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 1131
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1129: return true; 1130: } 1131: return false; 1132: } 1133:	
	污染路径: expanded from macro 'false'	
74	缺陷发生位置: /source/common/src/tddataformat.c	行号: 1287
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1285: } 1286: 1287: bool tTagIsJson(const void *pTag) { return (((const STag *)pTag)->flags & TD_TAG_JSON); } 1288: 1289: bool tTagIsJsonNull(void *data) {	
	污染路径: 无	
75	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 168
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 166: } 167: 168: return true; 169: } 170:	
	污染路径: expanded from macro 'true'	

76	缺陷发生位置: /source/client/src/clientSml.c	行号: 143
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 141: int64_t tsInt64 = taosStr2Int64(value, &endPtr, 10); 142: if (unlikely(value + len != endPtr)) { 143: return -1; 144: } 145:	
	污染路径: 无	
77	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 310
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 308: bool shellIsReadyGo(SShellCmd *cmd) { 309: ASSERT(cmd->cursorOffset <= cmd->commandSize && cmd->endOffset >= cmd->screenOffset); 310: if(cmd->cursorOffset > cmd->commandSize cmd-> >endOffset < cmd->screenOffset) return false; 311: 312: char *total = (char *)taosMemoryCalloc(1, SHELL_MAX_COMMAND_SIZE);	
	污染路径: expanded from macro 'false'	
78	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 484
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 482: static inline bool syncUtilCanPrint(char c) { 483: if (c >= 32 && c <= 126) { 484: return true; 485: } else { 486: return false;	
	污染路径: expanded from macro 'true'	
79	缺陷发生位置:	行号:

	/source/libs/scalar/src/scalar.c	1057
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint8_t'.	
	代码片段: 1055: } 1056: 1057: return 0; 1058: } 1059:	
	污染路径: 无	
80	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 132
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 130: int len = strlen(word); 131: if (len >= MAX_WORD_LEN) { 132: return false; 133: } 134:	
	污染路径: expanded from macro 'false'	
81	缺陷发生位置: /source/common/src/tdatablock.c	行号: 777
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 775: for (int32_t i = startIndex; i < nRows; ++i) { 776: if (!colDataIsNull_var(pColumnInfoData, i)) { 777: return false; 778: } 779: }	
	污染路径: expanded from macro 'false'	
82	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 185
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段:	

	<pre> 183: bool fmlsWindowPseudoColumnFunc(int32_t funcId) { return isSpecificClassifyFunc(funcId, FUNC_MGT_WINDOW_PC_FUNC); } 184: 185: bool fmlsWindowClauseFunc(int32_t funcId) { return fmlsAggFunc(funcId) fmlsWindowPseudoColumnFunc(funcId); } 186: 187: bool fmlsIndefiniteRowsFunc(int32_t funcId) { return isSpecificClassifyFunc(funcId, FUNC_MGT_INDEFINITE_ROWS_FUNC); } </pre>	
	污染路径: 无	
83	缺陷发生位置: /source/libs/parser/src/parlInsertUtil.c	行号: 611
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 609: } 610: 611: return false; 612: } 613: </pre>	
	污染路径: expanded from macro 'false'	
84	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 28
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'float'，定义类型为'double'.	
	代码片段: <pre> 26: return b; 27: } else if (b == 0) { 28: return INFINITY; 29: } else { 30: return a / b; </pre>	
	污染路径: expanded from macro 'INFINITY'	
85	缺陷发生位置: /source/os/src/osDir.c	行号: 361
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 359: if (pDir != NULL) { 360: taosCloseDir(&pDir); 361: return true; 362: } 363: return false; </pre>	
	污染路径: expanded from macro 'true'	
86	缺陷发生位置: /source/util/src/tthread.c	行号: 42
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 40: 41: taosMemoryFree(pthread); 42: return true; 43: } 44: </pre>	
	污染路径: expanded from macro 'true'	
87	缺陷发生位置: /source/client/src/clientEnv.c	行号: 140
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 138: return false; 139: } 140: return true; 141: } else { 142: return false; </pre>	
	污染路径: expanded from macro 'true'	
88	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 646
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段: <pre> 644: 645: *tokenId = TK_NK_ILLEGAL; </pre>	

	646: return 0; 647: } 648:	
	污染路径: 无	
89	缺陷发生位置: /source/util/src/tthread.c	行号: 35
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 33: 34: bool taosDestroyThread(TdThread* pthread) { 35: if (pthread == NULL) return false; 36: if (taosThreadRunning(pthread)) { 37: taosThreadCancel(*pthread);	
	污染路径: expanded from macro 'false'	
90	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 282
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 280: if (f1->fid != f2->fid) return false; 281: if (f1->cid != f2->cid) return false; 282: return true; 283: } 284:	
	污染路径: expanded from macro 'true'	
91	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 43
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 41: bool filterRangeCompLe(const void *minv, const void *maxv, const void *minr, const void *maxr, __compar_fn_t cfunc) { 42: int32_t result = cfunc(minv, maxr); 43: return result < 0; 44: }	

	45: bool filterRangeCompII(const void *minv, const void *maxv, const void *minr, const void *maxr, __compar_fn_t cfunc) {	
	污染路径: 无	
92	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 979
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 977: return true; 978: } else { 979: return false; 980: } 981: }	
	污染路径: expanded from macro 'false'	
93	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 2242
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 2240: 2241: if (key->ts > pltemBack->ts) { 2242: return false; 2243: } else if (key->ts >= pltemFront->ts && key->ts <= pltemBack->ts) { 2244: if (key->version <= pltemFront->version (key->ts == pltemBack->ts && key->version <= pltemBack->version)) {	
	污染路径: expanded from macro 'false'	
94	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 126
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'char *', 定义类型为'void *'.	
	代码片段: 124: static void* tupleGetField(char* t, uint32_t colIdx, uint32_t colNum) { 125: if (tupleCollsNull(t, colIdx, colNum)) return NULL;	

	126: return t + *tupleOffset(t, colldx); 127: } 128:	
	污染路径: 无	
95	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 255
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 253: if (f1->type != f2.type) return false; 254: if (f1->did.level != f2.did.level) return false; 255: if (f1->did.id != f2.did.id) return false; 256: if (f1->cid != f2.cid) return false; 257: return true;	
	污染路径: expanded from macro 'false'	
96	缺陷发生位置: /source/common/src/tdataformat.c	行号: 1293
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 1291: int8_t isJson = tTagIsJson(pTag); 1292: if (!isJson) return false; 1293: return ((STag *)data)->nTag == 0; 1294: } 1295:	
	污染路径: 无	
97	缺陷发生位置: /source/common/src/tmisc.c	行号: 59
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 57: for (int32_t i = 0; i < s1->numOfEps; i++) { 58: if (s1->eps[i].port != s2->eps[i].port strncmp(s1->eps[i].fqdn, s2->eps[i].fqdn, TSDB_FQDN_LEN) != 0) 59: return false; 60: } 61: return true;	

	污染路径: expanded from macro 'false'	
98	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 257
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 255: if (f1->did.id != f2.did.id) return false; 256: if (f1->cid != f2.cid) return false; 257: return true; 258: } 259:	
	污染路径: expanded from macro 'true'	
99	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 1439
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'SArray **', 定义类型为'void **'.	
	代码片段: 1437: static void ctgFreeVgroupInfo(void* p) { taosMemoryFree(((SMetaRes*)p)->pRes); } 1438: 1439: static void* ctgCloneTableIndices(void* pSrc) { return taosArrayDup((const SArray*)pSrc, NULL); } 1440: 1441: static void ctgFreeTableIndices(void* p) { taosArrayDestroy((SArray*)((SMetaRes*)p)->pRes); }	
	污染路径: 无	
100	缺陷发生位置: /source/client/src/clientImpl.c	行号: 40
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 38: size_t len = strlen(str); 39: if (len <= 0 len > maxsize) { 40: return false; 41: } 42:	

	污染路径: expanded from macro 'false'	
101	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 81
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 79: static bool validateHourRange(int8_t hour) { 80: if (hour < 0 hour > 12) { 81: return false; 82: } 83:	
	污染路径: expanded from macro 'false'	
102	缺陷发生位置: /source/common/src/tdataformat.c	行号: 1292
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1290: STag *pTag = (STag *)data; 1291: int8_t isJson = tTagIsJson(pTag); 1292: if (!isJson) return false; 1293: return ((STag *)data)->nTag == 0; 1294: }	
	污染路径: expanded from macro 'false'	
103	缺陷发生位置: /source/util/src/ttimer.c	行号: 257
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 255: uint8_t wheelIdx = timer->wheel; 256: if (wheelIdx >= tListLen(wheels)) { 257: return false; 258: } 259: time_wheel_t* wheel = wheels + wheelIdx;	
	污染路径: expanded from macro 'false'	
104	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 89

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 87: } 88: 89: return true; 90: } 91:	
	污染路径: expanded from macro 'true'	
105	缺陷发生位置: /include/util/tencode.h	行号: 495
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 493: static FORCE_INLINE int32_t tPutl8(uint8_t* p, int8_t v) { 494: if (p) ((int8_t*)p)[0] = v; 495: return sizeof(int8_t); 496: } 497:	
	污染路径: 无	
106	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 163
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 161: } else { 162: if (memcmp(pkey->pData, val, pkey->bytes) != 0) { 163: return false; 164: } 165: }	
	污染路径: expanded from macro 'false'	
107	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 1410
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1408: static bool	

	<pre> projectCanMergeDataBlock(SProjectLogicNode* pProject) { 1409: if (GROUP_ACTION_KEEP == pProject- >node.groupAction) { 1410: return false; 1411: } 1412: if (DATA_ORDER_LEVEL_NONE == pProject- >node.resultDataOrder) { </pre>	
	污染路径: expanded from macro 'false'	
108	缺陷发生位置: /source/libs/wal/src/walWrite.c	行号: 575
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 573: if (wallInitWriteFile(pWal) < 0) { 574: taosThreadMutexUnlock(&pWal->mutex); 575: return -1; 576: } 577: } </pre>	
	污染路径: 无	
109	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1246
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 1244: 1245: if (numOfSources == 0) { 1246: return 0; 1247: } 1248: </pre>	
	污染路径: 无	
110	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 297
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 295: static bool hasSameTableAlias(SArray* pTables) { 296: if (taosArrayGetSize(pTables) < 2) { </pre>	

	297: return false; 298: } 299: STableNode* pTable0 = taosArrayGetP(pTables, 0);	
	污染路径: expanded from macro 'false'	
111	缺陷发生位置: /source/util/src/tqueue.c	行号: 77
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 75: 76: bool taosQueueEmpty(STaosQueue *queue) { 77: if (queue == NULL) return true; 78: 79: bool empty = false;	
	污染路径: expanded from macro 'true'	
112	缺陷发生位置: /include/common/tmsg.h	行号: 79
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 77: } 78: static inline bool vnodesMsgBlock(tmsg_t type) { 79: return (type == TDMT_VND_CREATE_TABLE) (type == TDMT_VND_ALTER_TABLE) (type == TDMT_VND_DROP_TABLE) 80: (type == TDMT_VND_UPDATE_TAG_VAL) (type == TDMT_VND_ALTER_CONFIRM) (type == TDMT_VND_COMMIT) 81: (type == TDMT_SYNC_CONFIG_CHANGE);	
	污染路径: 无	
113	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 977
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 975: return false; 976: }	

	977: return true; 978: } else { 979: return false;	
	污染路径: expanded from macro 'true'	
114	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 373
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 371: const TStatisBlkArray *pStatisBlkArray = pBlockLoadInfo->pSttStatisBlkArray; 372: if (TARRAY2_SIZE(pStatisBlkArray) <= 0) { 373: return true; 374: } 375: </pre>	
	污染路径: expanded from macro 'true'	
115	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 185
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 183: void* pInfo) { 184: if (func(pCxt, pSubplan, pNode, pInfo)) { 185: return true; 186: } 187: SNode* pChild; </pre>	
	污染路径: expanded from macro 'true'	
116	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 904
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 902: taosThreadMutexUnlock(&tiresMutex); 903: 904: return false; 905: } 906: </pre>	

	污染路径: expanded from macro 'false'	
117	缺陷发生位置: /source/util/src/tlruccache.c	行号: 568
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 566: static bool taosLRUCacheShardRelease(SLRUCacheShard *shard, LRUHandle *handle, bool eraselfLastRef) { 567: if (handle == NULL) { 568: return false; 569: } 570:	
	污染路径: expanded from macro 'false'	
118	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 238
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 236: static bool stbSplHasMultiTbScan(bool streamQuery, SLogicNode* pNode) { 237: if (1 != LIST_LENGTH(pNode->pChildren)) { 238: return false; 239: } 240: SNode* pChild = nodesListGetNode(pNode->pChildren, 0);	
	污染路径: expanded from macro 'false'	
119	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 1186
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1184: if ((udfCode = acquireUdfFuncHandle((char *)pCtx->udfName, &handle)) != 0) { 1185: fnError("udfAggInit error. step doSetupUdf. udf code: %d", udfCode); 1186: return false; 1187: }	

	1188: SUdfcUvSession *session = (SUdfcUvSession *)handle;	
	污染路径: expanded from macro 'false'	
120	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 1445
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1443: 1444: FORCE_INLINE bool cliIsEpsetUpdated(int32_t code, STransConnCtx* pCtx) { 1445: if (code != 0) return false; 1446: // if (pCtx->retryCnt == 0) return false; 1447: if (transEpSetIsEqual(&pCtx->epSet, &pCtx->origEpSet)) return false;	
	污染路径: expanded from macro 'false'	
121	缺陷发生位置: /source/util/src/thash.c	行号: 232
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 230: * @return 231: */ 232: static FORCE_INLINE bool taosHashTableEmpty(const SHashObj *pHashObj) { return taosHashGetSize(pHashObj) == 0; } 233: 234: SHashObj *taosHashInit(size_t capacity, _hash_fn_t fn, bool update, SHashLockTypeE type) {	
	污染路径: 无	
122	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 184
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 182: return ! scanPathOptHaveNormalCol(((SAggLogicNode*)pNode)->pGroupKeys);	

	183: } 184: return false; 185: } 186:	
	污染路径: expanded from macro 'false'	
123	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 1476
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'SArray *'，定义类型为'void *'.	
	代码片段: 1474: static void ctgFreeUserAuth(void* p) { taosMemoryFree(((SMetaRes*)p)->pRes); } 1475: 1476: static void* ctgCloneQnodeList(void* pSrc) { return taosArrayDup((const SArray*)pSrc, NULL); } 1477: 1478: static void ctgFreeQnodeList(void* p) { taosArrayDestroy((SArray*)((SMetaRes*)p)->pRes); }	
	污染路径: 无	
124	缺陷发生位置: /source/util/src/tskiplist.c	行号: 301
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 299: iter->cur = pSkipList->pTail; 300: tSkipListUnlock(pSkipList); 301: return false; 302: } 303:	
	污染路径: expanded from macro 'false'	
125	缺陷发生位置: /source/util/src/tlog.c	行号: 231
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 229: 230: static bool taosLockLogFile(TdFilePtr pFile) {	

	231: if (pFile == NULL) return false; 232: 233: if (tsLogObj.fileNum > 1) {	
	污染路径: expanded from macro 'false'	
126	缺陷发生位置: /source/util/src/talgo.c	行号: 192
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'uint8_t *'，定义类型为'void *'.	
	代码片段: 190: return (c <= 0) ? p : (midx + 1 < nmemb ? p + size : NULL); 191: } else if (flags == TD_LE) { 192: return (c >= 0) ? p : (midx > 0 ? p - size : NULL); 193: } else if (flags == TD_GT) { 194: return (c < 0) ? p : (midx + 1 < nmemb ? p + size : NULL);	
	污染路径: 无	
127	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 447
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 445: if (pFile == NULL) { 446: fprintf(stderr, "failed to open file: %s\r\n", fullname); 447: return -1; 448: } 449:	
	污染路径: 无	
128	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 656
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 654: } 655: 656: return true; 657: }	

	658:	
	污染路径: expanded from macro 'true'	
129	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 125
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 123: // add count 124: tire->count += 1; 125: return true; 126: } 127:	
	污染路径: expanded from macro 'true'	
130	缺陷发生位置: /source/libs/index/src/indexFstDfa.c	行号: 119
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 117: t->next[byte] = nxtState; 118: *result = nxtState; 119: return true; 120: } 121: return false;	
	污染路径: expanded from macro 'true'	
131	缺陷发生位置: /include/util/tencode.h	行号: 446
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 444: } 445: 446: static FORCE_INLINE bool tDecodelsEnd(SDecoder* pCoder) { return (pCoder->size == pCoder->pos); } 447: 448: static FORCE_INLINE void* tEncoderMalloc(SEncoder* pCoder, int32_t size) {	

	污染路径: 无	
132	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 114
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 112: } 113: 114: return true; 115: } 116:	
	污染路径: expanded from macro 'true'	
133	缺陷发生位置: /source/util/src/tutil.c	行号: 291
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 289: dig = num[i] - 'A' + 10; 290: } else { 291: return 0; 292: } 293: ret += dig * base;	
	污染路径: 无	
134	缺陷发生位置: /source/common/src/tname.c	行号: 139
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 137: return strlen(name->dbname) > 0; 138: } else { 139: return strlen(name->dbname) > 0 && strlen(name->tname) > 0; 140: } 141: }	
	污染路径: 无	
135	缺陷发生位置:	行号:

	/source/libs/parser/src/parser.c	32
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 30: SToken t = tStrGetToken((char*)pStr, &index, false, NULL); 31: if (TK_INSERT != t.type && TK_IMPORT != t.type) { 32: return false; 33: } 34:	
	污染路径: expanded from macro 'false'	
136	缺陷发生位置: /source/libs/parser/src/parAstParser.c	行号: 127
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 125: return (NULL != pSelect->pWindow && QUERY_NODE_INTERVAL_WINDOW == nodeType(pSelect->pWindow)); 126: } 127: return false; 128: } 129:	
	污染路径: expanded from macro 'false'	
137	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 649
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 647: pDnode->numOfVnodes = 0; 648: pDnode->numOfOtherNodes = 0; 649: return true; 650: } 651:	
	污染路径: expanded from macro 'true'	
138	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 237

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 235: qTaskInfo_t taskHandle = ctx->taskHandle; 236: if (NULL == taskHandle ! qTasksExecuting(taskHandle)) { 237: return true; 238: } 239: </pre>	
	污染路径: expanded from macro 'true'	
139	缺陷发生位置: /source/libs/parser/src/parCalcConst.c	行号: 224
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 222: nodesWalkExpr((SNode*)pProj, checkUselessCol, (void *)&ctx); 223: if (!ctx.isUseless) { 224: return false; 225: } 226: return NULL == ((SEExprNode*)pProj)->pAssociation; </pre>	
	污染路径: expanded from macro 'false'	
140	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 321
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 319: if (shellRegexMatch(total, reg_str, REG_EXTENDED REG_ICASE)) { 320: taosMemoryFree(total); 321: return true; 322: } 323: </pre>	
	污染路径: expanded from macro 'true'	
141	缺陷发生位置: /source/libs/index/src/indexFstDfa.c	行号: 121
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 119: return true; 120: } 121: return false; 122: } 123: </pre>	
	污染路径: expanded from macro 'false'	
142	缺陷发生位置: /source/common/src/ttszip.c	行号: 727
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 725: if (pTSBuf->block.numOfElem == 0) { 726: tsBufResetPos(pTSBuf); 727: return false; 728: } else { 729: return true; </pre>	
	污染路径: expanded from macro 'false'	
143	缺陷发生位置: /source/util/src/tlog.c	行号: 349
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 347: } else { 348: printf("\nfailed to open log file:%s, reason:%s\n", logFileName, strerror(errno)); 349: return true; 350: } 351: } </pre>	
	污染路径: expanded from macro 'true'	
144	缺陷发生位置: /source/util/src/tutil.c	行号: 79
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: <pre> 77: if (z[j] == 0) { </pre>	

	<pre> *(SBlockIndex*)taosArrayGet(pTableBlockScanInfo->pBlockList, *nextIndex); 1712: // tMapDataGetItemByIdx(&pTableBlockScanInfo- >mapData, pIndex->ordinalIndex, pBlock, tGetDataBlk); 1713: return true; 1714: } 1715: </pre>	
	污染路径: expanded from macro 'true'	
148	缺陷发生位置: /source/common/src/ttszip.c	行号: 770
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 768: } 769: 770: return true; 771: } 772: </pre>	
	污染路径: expanded from macro 'true'	
149	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 433
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 431: } 432: 433: return false; 434: } 435: </pre>	
	污染路径: expanded from macro 'false'	
150	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 856
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 854: return tsInt64 * smlFactorS[toPrecision]; 855: } </pre>	

	856: return -1; 857: } else if (typeLen == 2 && (type->valuestring[1] == 's' type->valuestring[1] == 'S')) { 858: switch (type->valuestring[0]) {	
	污染路径: 无	
151	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 220
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int32_t'，定义类型为'int64_t'.	
	代码片段: 218: } 219: 220: int64_t nodesReleaseAllocatorWeakRef(int64_t allocatorId) { return taosReleaseRef(g_allocatorReqRefPool, allocatorId); } 221: 222: void nodesDestroyAllocator(int64_t allocatorId) {	
	污染路径: 无	
152	缺陷发生位置: /source/common/src/tmsg.c	行号: 7462
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 7460: } 7461: } 7462: return false; 7463: } 7464:	
	污染路径: expanded from macro 'false'	
153	缺陷发生位置: /source/util/src/tutil.c	行号: 97
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 95: if (stop > 0) { 96: z[stop - delta] = 0; 97: return (stop - delta);	

	98: } else if (j != i) { 99: z[i] = 0;	
	污染路径: 无	
154	缺陷发生位置: /source/util/src/tlosertree.c	行号: 42
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned int', 定义类型为'int32_t'.	
	代码片段: 40: if (pTreeInfo == NULL) { 41: uError("allocate memory for loser-tree failed. reason: %s", strerror(errno)); 42: return TAOS_SYSTEM_ERROR(errno); 43: } 44:	
	污染路径: expanded from macro 'TAOS_SYSTEM_ERROR'	
155	缺陷发生位置: /source/client/src/clientEnv.c	行号: 138
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 136: msgType == TDMT_SCH_MERGE_FETCH msgType == TDMT_SCH_QUERY_HEARTBEAT msgType == TDMT_SCH_DROP_TASK 137: msgType == TDMT_SCH_TASK_NOTIFY) { 138: return false; 139: } 140: return true;	
	污染路径: expanded from macro 'false'	
156	缺陷发生位置: /source/os/src/osFile.c	行号: 424
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 422: int64_t taosPReadFile(TdFilePtr pFile, void *buf, int64_t count, int64_t offset) { 423: if (pFile == NULL) { 424: return 0;	

	425: } 426: #if FILE_WITH_LOCK	
	污染路径: 无	
157	缺陷发生位置: /source/libs/parser/src/parInsertUtil.c	行号: 607
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 605: for (int i = 0; i < numFields; i++) { 606: if(strcmp(pSchema->name, fields[i].name) == 0){ 607: return true; 608: } 609: }	
	污染路径: expanded from macro 'true'	
158	缺陷发生位置: /include/util/tencode.h	行号: 490
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 488: static FORCE_INLINE int32_t tPutU8(uint8_t* p, uint8_t v) { 489: if (p) ((uint8_t*)p)[0] = v; 490: return sizeof(uint8_t); 491: } 492:	
	污染路径: 无	
159	缺陷发生位置: /include/common/tmsg.h	行号: 85
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 83: 84: static inline bool syncUtilUserCommit(tmsg_t msgType) { 85: return msgType != TDMT_SYNC_NOOP && msgType != TDMT_SYNC_LEADER_TRANSFER; 86: } 87:	

	污染路径: 无	
160	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 67
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned int', 定义类型为'int32_t'.	
	代码片段: 65: taosOpenFile(pBuf->path, TD_FILE_CREATE TD_FILE_WRITE TD_FILE_READ TD_FILE_TRUNC TD_FILE_AUTO_DEL); 66: if (pBuf->pFile == NULL) { 67: return TAOS_SYSTEM_ERROR(errno); 68: } 69:	
	污染路径: expanded from macro 'TAOS_SYSTEM_ERROR'	
161	缺陷发生位置: /source/libs/stream/src/streamUpdate.c	行号: 166
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 164: bool updateInfolTableInserted(SUpdateInfo *pInfo, int64_t tbUid) { 165: void *pVal = taosHashGet(pInfo->pMap, &tbUid, sizeof(int64_t)); 166: if (pVal taosHashGetSize(pInfo->pMap) >= DEFAULT_MAP_SIZE) return true; 167: return false; 168: }	
	污染路径: expanded from macro 'true'	
162	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 447
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 445: } 446: 447: return false; 448: }	

	449:	
	污染路径: expanded from macro 'false'	
163	缺陷发生位置: /source/util/src/tbloomfilter.c	行号: 36
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'_Bool'.	
	代码片段: 34: uint64_t unitIndex = index >> UNIT_ADDR_NUM_BITS; 35: uint64_t mask = 1ULL << (index % UNIT_NUM_BITS); 36: return buf[unitIndex] & mask; 37: } 38:	
	污染路径: 无	
164	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 1126
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 1124: 1125: // tMapDataGetItemByIdx(&pTableBlockScanInfo->mapData, pIndex->ordinalIndex, pBlock, tGetDataBlk); 1126: return true; 1127: } 1128:	
	污染路径: expanded from macro 'true'	
165	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 703
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 701: FStmStRsIt* rt = stmStNextWith(tIter->st, NULL); 702: if (rt == NULL) { 703: return false; 704: } 705:	
	污染路径:	

	expanded from macro 'false'	
166	缺陷发生位置: /source/client/src/clientMain.c	行号: 317
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'void *'，定义类型为'int'.	
	代码片段: 315: pResultInfo = tmqGetNextResInfo(res, true); 316: if (pResultInfo == NULL) { 317: return NULL; 318: } 319:	
	污染路径: expanded from macro 'NULL'	
167	缺陷发生位置: /source/util/src/ttimer.c	行号: 448
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 446: const char* fmt = "%s timer[id=%" PRIuPTR " , fp=%p, param=%p] is executing and cannot be stopped." 447: tmrDebug(fmt, timer->ctrl->label, timer->id, timer->fp, timer->param); 448: return false; 449: } 450:	
	污染路径: expanded from macro 'false'	
168	缺陷发生位置: /source/os/src/osEnv.c	行号: 102
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 100: bool osDataSpaceAvailable() { return tsDataSpace.size.avail > 0; } 101: 102: bool osLogSpaceSufficient() { return tsLogSpace.size.avail > tsLogSpace.reserved; } 103: 104: bool osDataSpaceSufficient() { return tsDataSpace.size.avail > tsDataSpace.reserved; }	

	污染路径: 无	
169	缺陷发生位置: /source/util/src/tlruccache.c	行号: 91
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 89: ASSERT(entry->refs > 0); 90: --entry->refs; 91: return entry->refs == 0; 92: } 93:	
	污染路径: 无	
170	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 725
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 723: bool getSumFuncEnv(SFunctionNode* UNUSED_PARAM(pFunc), SFuncExecEnv* pEnv) { 724: pEnv->calcMemSize = sizeof(SSumRes); 725: return true; 726: } 727:	
	污染路径: expanded from macro 'true'	
171	缺陷发生位置: /source/util/src/tcompression.c	行号: 754
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 752: if (input[0] == 0) { 753: memcpy(output, input + 1, nelements * LONG_BYTES); 754: return nelements * LONG_BYTES; 755: } else if (input[0] == 1) { // Decompress 756: int64_t *ostream = (int64_t *)output;	
	污染路径: 无	
172	缺陷发生位置:	行号:

	/source/libs/parser/src/parAstCreator.c	183
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 181: return false; 182: } 183: return true; 184: } 185:	
	污染路径: expanded from macro 'true'	
173	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 141
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 139: 140: if (isNull pkey->isNull) { 141: return false; 142: } 143:	
	污染路径: expanded from macro 'false'	
174	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 361
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 359: static bool isWstartColumnExist(SFillOperatorInfo* pInfo) { 360: if (pInfo->noFillExprSupp.numOfExprs == 0) { 361: return false; 362: } 363:	
	污染路径: expanded from macro 'false'	
175	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 53
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	

	代码片段: <pre> 51: } else if (pSchema[i].type == TSDB_DATA_TYPE_NCHAR) { 52: if (pSchema[i].bytes > TSDB_MAX_NCHAR_LEN) { 53: return false; 54: } 55: } else if (pSchema[i].type == TSDB_DATA_TYPE_GEOMETRY) { </pre>	
	污染路径: expanded from macro 'false'	
176	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 1213
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 1211: releaseUdfFuncHandle(pCtx->udfName, handle); 1212: freeUdfInterBuf(&buf); 1213: return true; 1214: } 1215: </pre>	
	污染路径: expanded from macro 'true'	
177	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 99
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 97: m = word[i] - FIRST_ASCII; 98: if (m < 0 m > CHAR_CNT) { 99: return false; 100: } 101: </pre>	
	污染路径: expanded from macro 'false'	
178	缺陷发生位置: /source/libs/index/src/indexFstDfa.c	行号: 158
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 156: *result = sz; </pre>	

	157: } 158: return true; 159: } 160:	
	污染路径: expanded from macro 'true'	
179	缺陷发生位置: /source/common/src/tdatablock.c	行号: 771
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 769: static bool colDataIsNNull(const SColumnInfoData* pColumnInfoData, int32_t startIndex, uint32_t nRows) { 770: if (!pColumnInfoData->hasNull) { 771: return false; 772: } 773:	
	污染路径: expanded from macro 'false'	
180	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 240
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 238: } 239: 240: return false; 241: } 242:	
	污染路径: expanded from macro 'false'	
181	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'uint64_t'.	
	代码片段: 106: gettimeofday(&tv, NULL); 107: 108: return tv.tv_sec * 1000000 + tv.tv_usec; 109: }	

	110:	
	污染路径: 无	
182	缺陷发生位置:	行号:
	/source/util/src/tjson.c	346
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 344: bool tjsonValidateJson(const char* jIn) { 345: if (!jIn) { 346: return false; 347: } 348:	
183	污染路径: expanded from macro 'false'	
	缺陷发生位置:	行号:
	/source/libs/executor/src/sysscanoperator.c	415
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
184	代码片段: 413: static bool sysTableIsCondOnOneTable(SNode* pCond, char* condTable) { 414: if (pCond == NULL) { 415: return false; 416: } 417: if (nodeType(pCond) == QUERY_NODE_LOGIC_CONDITION) {	
	污染路径: expanded from macro 'false'	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tq/tqSink.c	125
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 123: } 124: 125: return false; 126: } 127:	

	污染路径: expanded from macro 'false'	
185	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 232
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 230: } c; 231: c.i = 1; 232: return c.ch[0] == 1; 233: } 234:	
	污染路径: 无	
186	缺陷发生位置: /source/libs/sync/src/syncUtil.c	行号: 74
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 72: sInfo("vgld:%d, sync addr:%" PRIu64 " , dnode:%d cluster:%" PRId64 " fqdn:%s ip:%s port:%u ipv4:%u", vgld, 73: raftId->addr, pInfo->nodeId, pInfo->clusterId, pInfo->nodeFqdn, ipbuf, pInfo->nodePort, ipv4); 74: return true; 75: } 76:	
	污染路径: expanded from macro 'true'	
187	缺陷发生位置: /source/libs/scalar/src/scVector.c	行号: 545
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 543: } else if (tTagsJson(data)) { 544: terrno = TSDB_CODE_QRY_JSON_NOT_SUPPORT_ERROR; 545: return 0; 546: } else { 547: convertNumberToNumber(data + CHAR_BYTES, &out, *data, TSDB_DATA_TYPE_DOUBLE);	

	污染路径: 无	
188	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 254
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 252: static bool is_same_file(const STFile *f1, const STFile f2) { 253: if (f1->type != f2.type) return false; 254: if (f1->did.level != f2.did.level) return false; 255: if (f1->did.id != f2.did.id) return false; 256: if (f1->cid != f2.cid) return false;	
	污染路径: expanded from macro 'false'	
189	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 1344
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1342: 1343: static bool tsortIsForceUsePQSort(SSortHandle* pHandle) { 1344: return pHandle->forceUsePQSort == true; 1345: } 1346:	
	污染路径: 无	
190	缺陷发生位置: /source/libs/stream/src/streamMeta.c	行号: 760
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 758: // reset the counter 759: pInfo->tickCounter = 0; 760: return true; 761: } 762: return false;	
	污染路径: expanded from macro 'true'	

191	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 1702
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1700: bool asc = ASCENDING_TRAVERSE(order); 1701: if (asc && pBlockInfo->tbBlockIdx >= taosArrayGetSize(pTableBlockScanInfo->pBlockList) - 1) { 1702: return false; 1703: } 1704:	
	污染路径: expanded from macro 'false'	
192	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 410
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 408: } 409: } 410: return true; 411: } 412:	
	污染路径: expanded from macro 'true'	
193	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 914
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 912: } 913: tSimpleHashPut(pFillSup->pResMap, &key, sizeof(SWinKey), NULL, 0); 914: return true; 915: } 916:	
	污染路径: expanded from macro 'true'	
194	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 1813

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1811: 1812: bool mndIsDbReady(SMnode *pMnode, SDbObj *pDb) { 1813: if (pDb->cfg.replications == 1) return true; 1814: 1815: SSdb *pSdb = pMnode->pSdb;	
	污染路径: expanded from macro 'true'	
195	缺陷发生位置: /source/util/src/tsimplehash.c	行号: 401
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 399: size_t tSimpleHashGetMemSize(const SSHashObj *pHashObj) { 400: if (!pHashObj) { 401: return 0; 402: } 403:	
	污染路径: 无	
196	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 168
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 166: } 167: 168: bool needClearDiskBuff(SSStreamFileState* pFileState) { return pFileState->flushMark > 0; } 169: 170: void popUsedBufs(SSStreamFileState* pFileState, SSStreamSnapshot* pFlushList, uint64_t max, bool used) {	
	污染路径: 无	
197	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 119
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: <pre> 117: struct timeval systemTime; 118: rtGetTimeOfDay(&systemTime); 119: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 120: } 121: </pre>	
	污染路径: 无	
198	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 110
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 108: } 109: 110: return NULL != pBuf->pData; 111: } 112: </pre>	
	污染路径: expanded from macro 'NULL'	
199	缺陷发生位置: /source/util/src/tref.c	行号: 168
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 166: if (pNode == NULL) { 167: terrno = TSDB_CODE_OUT_OF_MEMORY; 168: return -1; 169: } 170: </pre>	
	污染路径: 无	
200	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 85
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 83: int64_t syncNodeAdd(SSyncNode *pNode) { </pre>	

	84: pNode->rid = taosAddRef(gNodeRefId, pNode); 85: if (pNode->rid < 0) return -1; 86: 87: sDebug("vgId:%d, sync rid:%" PRIu64 " is added to rsetId:%d", pNode->vgId, pNode->rid, gNodeRefId);	
	污染路径: 无	
201	缺陷发生位置: /source/util/src/tarray.c	行号: 426
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned int'，定义类型为'int32_t'.	
	代码片段: 424: int32_t taosArraySearchIdx(const SArray* pArray, const void* key, __compar_fn_t comparFn, int32_t flags) { 425: void* item = taosArraySearch(pArray, key, comparFn, flags); 426: return item == NULL ? -1 : (int32_t)((char*)item - (char*)pArray->pData) / pArray->elemSize; 427: } 428:	
	污染路径: 无	
202	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 677
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 675: taosArrayPush(pArray, pNode); 676: } 677: return true; 678: } 679:	
	污染路径: expanded from macro 'true'	
203	缺陷发生位置: /source/os/src/osFile.c	行号: 403
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 401: taosThreadRwlockUnlock(&(pFile->rwlock));	

	402: #endif 403: return -1; 404: } 405: } else if (readbytes == 0) {	
	污染路径: 无	
204	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 149
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 147: if (NULL == pEp) { 148: pCxt->errCode = TSDB_CODE_PAR_SYNTAX_ERROR; 149: return false; 150: } 151:	
	污染路径: expanded from macro 'false'	
205	缺陷发生位置: /utlis/test/c/tmqDemo.c	行号: 203
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 201: if ((pDir = taosOpenDir(dir)) == NULL) { 202: fprintf(stderr, "Cannot open dir: %s\n", dir); 203: return -1; 204: } 205:	
	污染路径: 无	
206	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 532
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint64_t'.	
	代码片段: 530: sError("failed to resolve ipv4 addr, host:%s", host); 531: terrno = TSDB_CODE_TSC_INVALID_FQDN; 532: return -1; 533: } 534:	

	污染路径: 无	
207	缺陷发生位置: /source/libs/wal/src/walRead.c	行号: 165
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 163: wError("vgld:%d, failed to seek log file, index:%" PRId64 ", pos:%" PRId64 ", since %s", pReader->pWal->cfg.vgld, 164: ver, entry.offset, terrstr()); 165: return -1; 166: } 167: return ret;	
	污染路径: 无	
208	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1106
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 1104: p = taosDecodeFixedl64(p, &key->key.ts); 1105: p = taosDecodeFixedl64(p, &key->opNum); 1106: return p - buf; 1107: } 1108:	
	污染路径: 无	
209	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 100
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 98: qError("SinkNode queue is full, no capacity, max:%d, current:%d, no capacity", capacity, 99: taosQueueItemSize(pDeleter->pDataBlocks)); 100: return false; 101: } 102:	

	污染路径: expanded from macro 'false'	
210	缺陷发生位置: /include/common/tdatablock.h	行号: 81
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 79: } 80: 81: return colDataIsNull_f(pColumnInfoData->nullbitmap, row); 82: } 83: }	
	污染路径: expanded from macro 'colDataIsNull_f'	
211	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 81
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 79: return (val % ((uint32_t)(1/plInfo->sampleRatio))) == 0; 80: #else 81: return true; 82: #endif 83: }	
	污染路径: expanded from macro 'true'	
212	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 187
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 185: bool hasRemainResults(SGroupResInfo* pGroupResInfo) { 186: if (pGroupResInfo->pRows == NULL) { 187: return false; 188: } 189:	
	污染路径: expanded from macro 'false'	
213	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tsdb/tsdbReadUtil.c	476
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 474: int32_t step = asc ? 1 : -1; 475: if ((pBlockIter->index >= pBlockIter->numOfBlocks - 1 && asc) (pBlockIter->index <= 0 && (!asc))) { 476: return false; 477: } 478:	
214	污染路径: expanded from macro 'false'	
	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 710
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
215	代码片段: 708: if (leftEmpty) { 709: setMergeJoinDone(pOperator); 710: return false; 711: } 712: }	
	污染路径: expanded from macro 'false'	
	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 729
216	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int *'.	
	代码片段: 727: 728: SSDataBlock* getNextBlockFromDownstream(struct SOperatorInfo* pOperator, int32_t idx) { 729: return getNextBlockFromDownstreamImpl(pOperator, idx, true); 730: } 731:	
	污染路径: 无	
216	缺陷发生位置: /source/common/src/ttszip.c	行号: 709

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 707: return false; 708: } else { 709: return true; 710: } 711:	
	污染路径: expanded from macro 'true'	
217	缺陷发生位置: /source/libs/executor/src/dynqueryctrloperator.c	行号: 96
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 94: static FORCE_INLINE bool tableNeedCache(int64_t uid, SStbJoinPrevJoinCtx* pPrev, SStbJoinPostJoinCtx* pPost, bool rightTable, bool batchFetch) { 95: if (batchFetch) { 96: return true; 97: } 98:	
	污染路径: expanded from macro 'true'	
218	缺陷发生位置: /source/util/src/tlrucahce.c	行号: 563
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 561: taosThreadMutexUnlock(&shard->mutex); 562: 563: return true; 564: } 565:	
	污染路径: expanded from macro 'true'	
219	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 99
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	

	代码片段: <pre> 97: int32_t old = atomic_val_compare_exchange_32(&mqReblnExecCnt, 0, 1); 98: mDebug("tq timer, rebalance counter old val:%d", old); 99: return old == 0; 100: } 101: </pre>	
	污染路径: 无	
220	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 216
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 214: tSimpleHashRemove(pInfo-> aggSup.pResultRowHashTable, &key, sizeof(SWinKey)); 215: pAPI->stateStore.streamStateDel(pInfo->pState, &key); 216: return true; 217: } 218: </pre>	
	污染路径: expanded from macro 'true'	
221	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 215
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 213: if (NULL == pAllocator) { 214: nodesError("allocator id %" PRIx64 " weak reference failed", allocatorId); 215: return -1; 216: } 217: return pAllocator->self; </pre>	
	污染路径: 无	
222	缺陷发生位置: /include/common/tdatablock.h	行号: 75
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段:	

	73: 74: if (IS_VAR_DATA_TYPE(pColumnInfoData->info.type)) { 75: return colDataIsNull_var(pColumnInfoData, row); 76: } else { 77: if (pColumnInfoData->nullbitmap == NULL) {		
	污染路径: expanded from macro 'colDataIsNull_var'		
223	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 377	
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint32_t'.		
	代码片段: 375: } 376: *tokenId = TK_NK_MINUS; 377: return 1; 378: } 379: case '(': {		
	污染路径: 无		
224	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 762	
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: 760: cmp = strcmp(pCol->tableAlias, pTable->tableAlias); 761: } 762: return (0 == cmp); 763: } 764:		
	污染路径: 无		
225	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 1794	
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: 1792: bool nodesIsExprNode(const SNode* pNode) { 1793: ENodeType type = nodeType(pNode); 1794: return (QUERY_NODE_COLUMN == type QUERY_NODE_VALUE == type QUERY_NODE_OPERATOR ==		

	<pre> type 1795: QUERY_NODE_FUNCTION == type QUERY_NODE_LOGIC_CONDITION == type QUERY_NODE_CASE_WHEN == type); 1796: }</pre>	
	污染路径: 无	
226	缺陷发生位置: /source/util/src/tcompression.c	行号: 959
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: <pre> 957: } 958: 959: return nelements * DOUBLE_BYTES; 960: } 961:</pre>	
	污染路径: 无	
227	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 106
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 104: } 105: 106: return NULL != pBuf->pData; 107: } 108:</pre>	
	污染路径: expanded from macro 'NULL'	
228	缺陷发生位置: /source/libs/wal/src/walSeek.c	行号: 97
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 95: if (code != 0) { 96: terrno = TAOS_SYSTEM_ERROR(errno); 97: return -1; 98: }</pre>	

	99: code = taosCloseFile(&pWal->pIdxFile);	
	污染路径: 无	
229	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 115
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 113: ASSERT(w.ekey > pBlockInfo->>window.ekey); 114: if (TMAX(w.skey, pBlockInfo->>window.skey) <= pBlockInfo->>window.ekey) { 115: return true; 116: } 117: }	
	污染路径: expanded from macro 'true'	
230	缺陷发生位置: /source/util/src/thash.c	行号: 851
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 849: void taosHashRelease(SHashObj *pHashObj, void *p) { taosHashCancelIterate(pHashObj, p); } 850: 851: int64_t taosHashGetCompTimes(SHashObj *pHashObj) { return 0 /*atomic_load_64(&pHashObj->compTimes)*/; }	
	污染路径: 无	
231	缺陷发生位置: /source/util/src/talgo.c	行号: 190
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'uint8_t*', 定义类型为'void*'.	
	代码片段: 188: return c ? NULL : p; 189: } else if (flags == TD_GE) { 190: return (c <= 0) ? p : (midx + 1 < nmemb ? p + size : NULL); 191: } else if (flags == TD_LE) { 192: return (c >= 0) ? p : (midx > 0 ? p - size : NULL);	

	污染路径: 无	
232	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 248
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 246: static bool walNeedFsync(SWal *pWal) { 247: if (pWal->cfg.fsyncPeriod <= 0 pWal->cfg.level != TAOS_WAL_FSYNC) { 248: return false; 249: } 250:	
	污染路径: expanded from macro 'false'	
233	缺陷发生位置: /source/client/src/clientImpl.c	行号: 43
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 41: } 42: 43: return true; 44: } 45:	
	污染路径: expanded from macro 'true'	
234	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 693
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 691: } 692: 693: return false; 694: } 695:	
	污染路径: expanded from macro 'false'	
235	缺陷发生位置:	行号:

	/source/os/src/osSocket.c	753
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 751: } else if (fd <= 2) { // in, out, err 752: taosCloseSocketNoCheck1(fd); 753: return false; 754: } 755:	
	污染路径: expanded from macro 'false'	
236	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 729
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 727: *pRightTs = *(int64_t*)pRightVal; 728: 729: return true; 730: } 731:	
	污染路径: expanded from macro 'true'	
237	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 385
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 383: 384: if (i >= TARRAY2_SIZE(pStatisBlkArray)) { 385: return false; 386: } 387:	
	污染路径: expanded from macro 'false'	
238	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 386
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段:	

	384: doTimeWindowInterpolation(pInfo->pPrevValues, pDataBlock, actualEndKey, endRowIndex, nextKey, nextRowIndex, key, 385: RESULT_ROW_END_INTERP, pSup); 386: return true; 387: } 388:	
	污染路径: expanded from macro 'true'	
239	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 377
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 375: // todo always retry to avoid read failure caused by wal file deletion 376: if (walNextValidMsg(pWalReader) < 0) { 377: return false; 378: } 379:	
	污染路径: expanded from macro 'false'	
240	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 341
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 339: return true; 340: } 341: return false; 342: _RETURN: 343: return false;	
	污染路径: expanded from macro 'false'	
241	缺陷发生位置: /source/common/src/tname.c	行号: 137
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 135:	

	136: if (name->type == TSDB_DB_NAME_T) { 137: return strlen(name->dbname) > 0; 138: } else { 139: return strlen(name->dbname) > 0 && strlen(name->tname) > 0;	
	污染路径: 无	
242	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 667
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 665: "related stream task currentVer:%" PRIu64, 666: pTask->id.idStr, latestVer); 667: return true; 668: } else { 669: // 2. do secondary scan of the history data, the time window remain, and the version range is updated to	
	污染路径: expanded from macro 'true'	
243	缺陷发生位置: /source/libs/wal/src/walWrite.c	行号: 564
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 562: terrno = TSDB_CODE_WAL_INVALID_VER; 563: taosThreadMutexUnlock(&pWal->mutex); 564: return -1; 565: } 566:	
	污染路径: 无	
244	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 1166
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段: 1164: p = taosDecodeFixedU64(p, &sess->key.groupId); 1165: p = taosDecodeFixedI64(p, &sess->opNum); 1166: return p - buf;	

	1167: } 1168: int stateSessionKeyToString(void* k, char* buf) {	
	污染路径: 无	
245	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 170
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 168: return false; 169: } 170: return true; 171: } 172:	
	污染路径: expanded from macro 'true'	
246	缺陷发生位置: /source/util/src/tconfig.c	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'size_t', 定义类型为'int32_t'.	
	代码片段: 106: } 107: 108: int32_t cfgGetSize(SConfig *pCfg) { return taosArrayGetSize(pCfg->array); } 109: 110: static int32_t cfgCheckAndSetTimezone(SConfigItem *pltem, const char *timezone) {	
	污染路径: 无	
247	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 84
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 82: } 83: 84: return true; 85: } 86:	

	污染路径: expanded from macro 'true'	
248	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 706
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 704: if (pJoinInfo->pRight == NULL) { 705: setMergeJoinDone(pOperator); 706: return false; 707: } else { 708: if (leftEmpty) {	
	污染路径: expanded from macro 'false'	
249	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 1175
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1173: } 1174: pEnv->calcMemSize = sizeof(SUdfAggRes) + pFunc->node.resType.bytes + pFunc->udfBufSize; 1175: return true; 1176: } 1177:	
	污染路径: expanded from macro 'true'	
250	缺陷发生位置: /source/libs/stream/src/streamMeta.c	行号: 762
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 760: return true; 761: } 762: return false; 763: } 764:	
	污染路径: expanded from macro 'false'	

251	缺陷发生位置: /source/util/src/tdigest.c	行号: 241
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段: 239: 240: double tdigestQuantile(TDigest *t, double q) { 241: if (t == NULL) return 0; 242: 243: int32_t i;	
	污染路径: 无	
252	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 40
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 38: } else if (reti == REG_NOMATCH) { 39: regfree(®ex); 40: return false; 41: } else { 42: regerror(reti, ®ex, msgbuf, sizeof(msgbuf));	
	污染路径: expanded from macro 'false'	
253	缺陷发生位置: /include/common/trow.h	行号: 95
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 93: static FORCE_INLINE bool tdValTypeIsNone(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NONE; } 94: static FORCE_INLINE bool tdValTypeIsNull(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NULL; } 95: static FORCE_INLINE bool tdValTypeIsNorm(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NORM; } 96: #endif 97:	
	污染路径: 无	
254	缺陷发生位置:	行号:

	/source/libs/executor/src/streamtimewindowoperator.c	499
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 497: return false; 498: } 499: return false; 500: } 501:	
	污染路径: expanded from macro 'false'	
255	缺陷发生位置: /source/util/src/tcompression.c	行号: 809
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: 807: prev_value = prev_value + prev_delta; 808: ostream[opos++] = prev_value; 809: if (opos == nelements) return nelements * LONG_BYTES; 810: } 811:	
	污染路径: 无	
256	缺陷发生位置: /source/libs/function/src/detail/tavgfunction.c	行号: 354
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 352: bool getAvgFuncEnv(SFunctionNode* UNUSED_PARAM(pFunc), SFuncExecEnv* pEnv) { 353: pEnv->calcMemSize = sizeof(SAvgRes); 354: return true; 355: } 356:	
	污染路径: expanded from macro 'true'	
257	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 662
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 660: static bool hasNextNextWindow(SSstreamFillSupporter* pFillSup) { 661: return pFillSup->nextNext.key != INT64_MIN; 662: return false; 663: } 664: </pre>	
	污染路径: expanded from macro 'false'	
258	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 643
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 641: int32_t getNumOfInMemBufPages(const SDiskbasedBuf* pBuf) { return pBuf->inMemPages; } 642: 643: bool isAllDataInMemBuf(const SDiskbasedBuf* pBuf) { return pBuf->fileSize == 0; } 644: 645: void setBufPageDirty(void* pPage, bool dirty) { </pre>	
	污染路径: 无	
259	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 51
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'double'，定义类型为'float'.	
	代码片段: <pre> 49: r &= 0x007fffff; 50: r = 0x40800000; 51: return *(float*)&r - 6.0; 52: } 53: </pre>	
	污染路径: 无	
260	缺陷发生位置: /source/util/src/tsimplehash.c	行号: 274
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	

	代码片段: 272: } 273: 274: static FORCE_INLINE bool taosHashTableEmpty(const SHashObj *pHashObj) { return tSimpleHashGetSize(pHashObj) == 0; } 275: 276: void *tSimpleHashGet(SHashObj *pHashObj, const void *key, size_t keyLen) {	
	污染路径: 无	
261	缺陷发生位置: /source/util/src/tref.c	行号: 153
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 151: uTrace("rsetId:%d p:%p failed to add, rsetId not valid", rsetId, p); 152: terrno = TSDB_CODE_REF_INVALID_ID; 153: return -1; 154: } 155:	
	污染路径: 无	
262	缺陷发生位置: /include/util/tutil.h	行号: 102
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'uint32_t'，定义类型为'int32_t'.	
	代码片段: 100: return MurmurHash3_32(tbname, tblen); 101: } else if (prefix > 0 suffix > 0) { 102: return MurmurHash3_32(tbname + prefix, tblen - prefix - suffix); 103: } else { 104: char tbName[TSDB_TABLE_FNAME_LEN];	
	污染路径: 无	
263	缺陷发生位置: /source/util/src/tcache.c	行号: 935
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 933: 934: if (plter->entryIndex + 1 >= pCacheObj->capacity) { 935: return false; 936: } 937:	
	污染路径: expanded from macro 'false'	
264	缺陷发生位置: /source/libs/scheduler/src/schJob.c	行号: 74
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 72: } 73: 74: return false; 75: } 76:	
	污染路径: expanded from macro 'false'	
265	缺陷发生位置: /source/libs/function/src/detail/tavgfunction.c	行号: 364
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 362: SAvgRes* pRes = GET_ROWCELL_INTERBUF(pResultInfo); 363: memset(pRes, 0, sizeof(SAvgRes)); 364: return true; 365: } 366:	
	污染路径: expanded from macro 'true'	
266	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 185
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 183: int64_t currentTs = *(int64_t*)colDataGetData(pTsCol,	

	curlIndex); 184: if (currentTs > pSliceInfo->win.ekey) { 185: return false; 186: } 187:	
	污染路径: expanded from macro 'false'	
267	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 92
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 90: } 91: 92: return false; 93: } 94:	
	污染路径: expanded from macro 'false'	
268	缺陷发生位置: /source/libs/parser/src/parCalcConst.c	行号: 403
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 401: break; 402: } 403: return false; 404: } 405:	
	污染路径: expanded from macro 'false'	
269	缺陷发生位置: /source/util/src/thashutil.c	行号: 153
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint32_t'.	
	代码片段: 151: 152: if (FLT_EQUAL(f, 0.0)) { 153: return 0; 154: }	

	155: if (fabs(f) < FLT_MAX / BASE - DLT) {	
	污染路径: 无	
270	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 406
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 404: char* value = nodesGetValueFromNode(pValue); 405: strncpy(condTable, varDataVal(value), TSDB_TABLE_NAME_LEN); 406: return true; 407: } 408: }	
	污染路径: expanded from macro 'true'	
271	缺陷发生位置: /source/os/src/osFile.c	行号: 384
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 382: taosThreadRwlockUnlock(&(pFile->rwlock)); 383: #endif 384: return -1; 385: } 386: int64_t leftbytes = count;	
	污染路径: 无	
272	缺陷发生位置: /source/client/src/clientSml.c	行号: 164
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: 162: return TSDB_TIME_PRECISION_MILLI; 163: } else { 164: return -1; 165: } 166: }	
	污染路径:	

	无	
273	缺陷发生位置： /source/util/src/tskiplist.c	行号： 480
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 478: } 479: 480: return (compare == 0); 481: } 482:	
	污染路径： 无	
274	缺陷发生位置： /source/dnode/vnode/src/tq/tqSink.c	行号： 378
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 376: tqError("vgId:%d, failed to write into %s, since table type:%d incorrect", vgId, ctbName, pReader->me.type); 377: terrno = TSDB_CODE_TDB_INVALID_TABLE_TYPE; 378: return false; 379: } 380:	
	污染路径： expanded from macro 'false'	
275	缺陷发生位置： /source/dnode/vnode/src/tq/tqRead.c	行号： 21
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 19: bool isValValidForTable(STqHandle* pHandle, SWalCont* pHead) { 20: if (pHandle->execHandle.subType != TOPIC_SUB_TYPE__TABLE) { 21: return true; 22: } 23:	
	污染路径： expanded from macro 'true'	

276	缺陷发生位置: /source/libs/scheduler/src/schJob.c	行号: 71
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 69: if ((*pJob->chkKillFp)(pJob->chkKillParam)) { 70: schUpdateJobErrCode(pJob, TSDB_CODE_TSC_QUERY_KILLED); 71: return true; 72: } 73:	
	污染路径: expanded from macro 'true'	
277	缺陷发生位置: /source/os/src/osSemaphore.c	行号: 175
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 173: #include <unistd.h> 174: 175: bool taosCheckPthreadValid(TdThread thread) { return thread != 0; } 176: 177: int64_t taosGetSelfPthreadId() {	
	污染路径: 无	
278	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 1067
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1065: } 1066: 1067: return false; 1068: } 1069:	
	污染路径: expanded from macro 'false'	
279	缺陷发生位置: /source/common/src/tname.c	行号: 133

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 131: bool tNameIsValid(const SName* name) { 132: if (!VALID_NAME_TYPE(name->type)) { 133: return false; 134: } 135:	
	污染路径: expanded from macro 'false'	
280	缺陷发生位置:	行号:
	/source/libs/parser/src/parCalcConst.c	421
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 419: } 420: 421: return true; 422: } 423:	
281	污染路径: expanded from macro 'true'	
	缺陷发生位置:	行号:
	/source/client/src/clientSmlJson.c	837
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
282	代码片段: 835: if (unlikely(smlDoubleToInt64Overflow(timeDouble))) { 836: smlBuildInvalidDataMsg(&info->msgBuf, "timestamp is too large", NULL); 837: return -1; 838: } 839:	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/qcom/src/queryUtil.c	75
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段:	

	73: } 74: 75: return rowLen <= maxLen; 76: } 77:	
	污染路径: 无	
283	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 410
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 408: } 409: } 410: return false; 411: } 412:	
	污染路径: expanded from macro 'false'	
284	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 97
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 95: int32_t rowIndex) { 96: if (!pCol->notFillCol) { 97: return false; 98: } 99: if (pCol->pExpr->pExpr->nodeType == QUERY_NODE_COLUMN) {	
	污染路径: expanded from macro 'false'	
285	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 433
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 431: 432: initResultRowEntry(pResultInfo, pCtx->resDataInfo.interBufSize);	

	433: return true; 434: } 435:	
	污染路径: expanded from macro 'true'	
286	缺陷发生位置: /source/os/src/osSocket.c	行号: 759
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 757: if (pSocket == NULL) { 758: taosCloseSocketNoCheck1(fd); 759: return false; 760: } 761: pSocket->refId = 0;	
	污染路径: expanded from macro 'false'	
287	缺陷发生位置: /source/os/src/osMemory.c	行号: 354
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 352: 353: int64_t taosMemorySize(void *ptr) { 354: if (ptr == NULL) return 0; 355: 356: #ifdef USE_TD_MEMORY	
	污染路径: 无	
288	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 700
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 698: while (*p != 0 && *p++ != c) { 699: } 700: return p - a; 701: } 702: static int idxCacheJsonTermCompareImpl(char* a, char* b) {	

	污染路径: 无	
289	缺陷发生位置: /source/os/src/osString.c	行号: 242
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 240: } 241: 242: return true; 243: #endif 244: }	
	污染路径: expanded from macro 'true'	
290	缺陷发生位置: /source/common/src/tmsg.c	行号: 7454
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 7452: return pLeft->version == pRight->version; 7453: } else if (pLeft->type == TMQ_OFFSET__SNAPSHOT_DATA) { 7454: return pLeft->uid == pRight->uid && pLeft->ts == pRight->ts; 7455: } else if (pLeft->type == TMQ_OFFSET__SNAPSHOT_META) { 7456: return pLeft->uid == pRight->uid;	
	污染路径: 无	
291	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 1190
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1188: // it is the last block in current file, no chance to overlap with neighbor blocks. 1189: if (ASCENDING_TRAVERSE(order)) { 1190: return pBlock->record.lastKey == pRec->firstKey; 1191: } else { 1192: return pBlock->record.firstKey == pRec->lastKey;	

	污染路径: 无	
292	缺陷发生位置: /source/libs/command/src/command.c	行号: 332
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 330: // check have leader on cloumn v*_status on 4 6 8 10 331: if (CHECK_LEADER(4) CHECK_LEADER(6) CHECK_LEADER(8) CHECK_LEADER(10)) { 332: return true; 333: } 334:	
	污染路径: expanded from macro 'true'	
293	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 389
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 387: 388: terrno = 0; 389: return true; 390: } 391:	
	污染路径: expanded from macro 'true'	
294	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 277
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.	
	代码片段: 275: if (code != TSDB_CODE_SUCCESS) { 276: terrno = code; 277: return NULL; 278: } 279:	
	污染路径: expanded from macro 'NULL'	

295	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 340
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 338: return false; 339: } 340: return true; 341: } 342:	
	污染路径: expanded from macro 'true'	
296	缺陷发生位置: /source/os/src/osDir.c	行号: 470
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 468: return (pDirEntry->findFileData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) != 0; 469: #else 470: return (((dirent *)pDirEntry)->d_type & DT_DIR) != 0; 471: #endif 472: }	
	污染路径: 无	
297	缺陷发生位置: /source/util/src/tenv.c	行号: 80
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 78: } 79: } 80: return strlen(cfgStr); 81: }	
	污染路径: 无	
298	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 40
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	

	代码片段: 38: t = tStrGetToken((char*)pStr, &index, false, NULL); 39: if (TK_USING == t.type TK_VALUES == t.type TK_FILE == t.type) { 40: return true; 41: } else if (TK_SELECT == t.type) { 42: return false;	
	污染路径: expanded from macro 'true'	
299	缺陷发生位置: /utils/tsim/src/simSystem.c	行号: 44
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 42: simInitsimCmdList(); 43: memset(simScriptList, 0, sizeof(SScript *) * MAX_MAIN_SCRIPT_NUM); 44: return true; 45: } 46:	
	污染路径: expanded from macro 'true'	
300	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 374
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint32_t'.	
	代码片段: 372: } else if (z[1] == '>') { 373: *tokenId = TK_NK_ARROW; 374: return 2; 375: } 376: *tokenId = TK_NK_MINUS;	
	污染路径: 无	
301	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 2029
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 2027: const SEp *p = &(pCurrent->eps[i]);	

	2028: if (pEp->port == p->port && strncmp(pEp->fqdn, p->fqdn, TSDB_FQDN_LEN) == 0) { 2029: return false; 2030: } 2031: }	
	污染路径: expanded from macro 'false'	
302	缺陷发生位置: /source/util/src/tutil.c	行号: 300
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 298: dig = (num[i] - '0'); 299: } else { 300: return 0; 301: } 302: ret += dig * base;	
	污染路径: 无	
303	缺陷发生位置: /source/os/src/osString.c	行号: 231
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 229: if (iconv(conv, (char **)&mbs, &ucs4_input_len, (char **)&ucs4, &outLeft) == -1) { 230: taosReleaseConv(idx, conv, M2C); 231: return false; 232: } 233:	
	污染路径: expanded from macro 'false'	
304	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 73
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 71: if (unlikely(ts == -1)) { 72: smlBuildInvalidDataMsg(&info->msgBuf, "invalid timestamp", data);	

	73: return -1; 74: } 75: return ts;	
	污染路径: 无	
305	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 202
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 200: } 201: 202: return false; 203: } 204:	
	污染路径: expanded from macro 'false'	
306	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 255
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 253: } 254: 255: return false; 256: } 257:	
	污染路径: expanded from macro 'false'	
307	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 337
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 335: return true; 336: } 337: return false; 338: } 339:	

	污染路径: expanded from macro 'false'	
308	缺陷发生位置: /source/util/src/tbloomfilter.c	行号: 30
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 28: uint64_t old = buf[unitIndex]; 29: buf[unitIndex] = mask; 30: return buf[unitIndex] != old; 31: } 32:	
	污染路径: 无	
309	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 133
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 131: } 132: } 133: return true; 134: } 135:	
	污染路径: expanded from macro 'true'	
310	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 243
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 241: if (QUERY_NODE_LOGIC_PLAN_PARTITION == nodeType(pChild)) { 242: if (1 != LIST_LENGTH(((SLogicNode*)pChild)- >pChildren)) { 243: return false; 244: } 245: pChild = nodesListGetNode(((SLogicNode*)pChild)- >pChildren, 0);	
	污染路径:	

	expanded from macro 'false'	
311	缺陷发生位置： /source/libs/stream/src/streamBackendRocksdb.c	行号： 1300
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'long'，定义类型为'int'.	
	代码片段： 1298: 1299: p = taosDecodeFixedl64(p, groupid); 1300: return p - buf; 1301: } 1302: int parKeyToString(void* k, char* buf) {	
	污染路径： 无	
312	缺陷发生位置： /source/libs/scalar/src/scivector.c	行号： 145
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段： 143: double out = 0; 144: if (*data == TSDB_DATA_TYPE_NULL) { 145: return 0; 146: } else if (*data == TSDB_DATA_TYPE_NCHAR) { // json inner type can not be BINARY 147: convertNcharToDouble(data + CHAR_BYTES, &out);	
	污染路径： 无	
313	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbMemTable.c	行号： 282
	步骤描述： 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段： 280: plter->pNode = SL_GET_NODE_BACKWARD(plter->pNode, 0); 281: if (plter->pNode == plter->pTbData->sl.pHead) { 282: return false; 283: } 284: } else {	
	污染路径： expanded from macro 'false'	
314	缺陷发生位置：	行号：

	/source/libs/wal/src/walWrite.c		569
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.		
	代码片段: 567: if (walCheckAndRoll(pWal) < 0) { 568: taosThreadMutexUnlock(&pWal->mutex); 569: return -1; 570: } 571:		
	污染路径: 无		
315	缺陷发生位置:		行号:
	/source/os/src/osFile.c		141
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.		
	代码片段: 139: 140: if (code != 0) { 141: return -1; 142: } 143: return size;		
	污染路径: 无		
316	缺陷发生位置:		行号:
	/source/libs/parser/src/parAstCreator.c		94
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: 92: int32_t res = regexec(®ex, pPassword, 0, NULL, 0); 93: regfree(®ex); 94: return 0 == res; 95: } 96:		
	污染路径: 无		
317	缺陷发生位置:		行号:
	/source/dnode/vnode/src/tsdb/tsdbMemTable.c		297
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段:		

	295: } 296: 297: return true; 298: } 299:	
	污染路径: expanded from macro 'true'	
318	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 150
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 148: 149: static bool syncNodeCheckNewConfig(SSyncNode* pSyncNode, const SSyncCfg* pCfg) { 150: if (!syncNodeInConfig(pSyncNode, pCfg)) return false; 151: return abs(pCfg->replicaNum - pSyncNode->replicaNum) <= 1; 152: }	
	污染路径: expanded from macro 'false'	
319	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 1129
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 1127: if (pipe->inputLen == pipe->inputCap && pipe->inputTotal == pipe->inputCap) { 1128: fnDebug("receive request complete. length %d", pipe->inputLen); 1129: return true; 1130: } 1131: return false;	
	污染路径: expanded from macro 'true'	
320	缺陷发生位置: /source/util/src/thashutil.c	行号: 165
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段:	

	163: double f = GET_DOUBLE_VAL(key); 164: if (isnan(f)) { 165: return 0x7fc00000; 166: } 167:	
	污染路径: 无	
321	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 189
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 187: 188: if ((pSliceInfo->prevTsSet == true) && (currentTs == pSliceInfo->prevTs)) { 189: return true; 190: } 191:	
	污染路径: expanded from macro 'true'	
322	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 1416
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1414: } 1415: if (1 != LIST_LENGTH(pProject->node.pChildren)) { 1416: return true; 1417: } 1418: SLogicNode* pChild = (SLogicNode*)nodesListGetNode(pProject->node.pChildren, 0);	
	污染路径: expanded from macro 'true'	
323	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 825
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 823: if (unlikely(!cJSON_IsNumber(value))) { 824: smlBuildInvalidDataMsg(&info->msgBuf, "invalidate	

	json", NULL); 825: return -1; 826: } 827:	
	污染路径: 无	
324	缺陷发生位置: /include/util/tutil.h	行号: 100
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'uint32_t'，定义类型为'int32_t'.	
	代码片段: 98: if ((prefix == 0 && suffix == 0) (tblen <= (prefix + suffix)) (tblen <= -1 * (prefix + suffix)) 99: prefix * suffix < 0) { 100: return MurmurHash3_32(tblname, tblen); 101: } else if (prefix > 0 suffix > 0) { 102: return MurmurHash3_32(tblname + prefix, tblen - prefix - suffix);	
	污染路径: 无	
325	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 364
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段: 362: case ':': { 363: *tokenId = TK_NK_COLON; 364: return 1; 365: } 366: case '-': {	
	污染路径: 无	
326	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 71
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 69: static bool splHasScan(SLogicNode* pNode) { 70: if (QUERY_NODE_LOGIC_PLAN_SCAN == nodeType(pNode)) {	

	71: return true; 72: } 73:	
	污染路径: expanded from macro 'true'	
327	缺陷发生位置: /source/util/src/tjson.c	行号: 375
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 373: } 374: } 375: return true; 376: } 377:	
	污染路径: expanded from macro 'true'	
328	缺陷发生位置: /source/client/src/clientTmq.c	行号: 2294
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int32_t'，定义类型为'int64_t'.	
	代码片段: 2292: 2293: // data from tsdb, no valid offset info 2294: return TSDB_CODE_TMQ_SNAPSHOT_ERROR; 2295: } 2296:	
	污染路径: expanded from macro 'TSDB_CODE_TMQ_SNAPSHOT_ERROR' expanded from macro 'TAOS_DEF_ERROR_CODE'	
329	缺陷发生位置: /source/dnode/mnode/impl/src/mndShow.c	行号: 347
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 345: static bool mndCheckRetrieveFinished(SShowObj *pShow) { 346: if (pShow->plter == NULL && pShow->numOfRows != 0) { 347: return true;	

	348: } 349: return false;	
	污染路径: expanded from macro 'true'	
330	缺陷发生位置: /source/libs/parser/src/sql.c	行号: 4479
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'unsigned short'.	
	代码片段: 4477: ** empty, causing the main parser loop to exit. So the return value 4478: ** is never used and does not matter. */ 4479: return 0; 4480: } 4481: yymssp = yypParser->yynos;	
	污染路径: 无	
331	缺陷发生位置: /source/libs/wal/src/walSeek.c	行号: 90
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: 88: if (code != 0) { 89: errno = TAOS_SYSTEM_ERROR(errno); 90: return -1; 91: } 92: }	
	污染路径: 无	
332	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 545
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 543: if (fset->farr[ftype] != NULL) return false; 544: } 545: return TARRAY2_SIZE(fset->lvlArr) == 0; 546: }	
	污染路径:	

	expanded from macro 'TARRAY2_SIZE'	
333	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 439
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 437: } 438: 439: return true; 440: } 441:	
	污染路径: expanded from macro 'true'	
334	缺陷发生位置: /source/libs/wal/src/walWrite.c	行号: 581
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 579: if (walWriteImpl(pWal, index, msgType, syncMeta, body, bodyLen) < 0) { 580: taosThreadMutexUnlock(&pWal->mutex); 581: return -1; 582: } 583:	
	污染路径: 无	
335	缺陷发生位置: /source/util/src/tcache.c	行号: 979
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 977: 978: plter->index += 1; 979: return true; 980: } 981:	
	污染路径: expanded from macro 'true'	
336	缺陷发生位置: /source/os/src/osDir.c	行号: 363

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 361: return true; 362: } 363: return false; 364: } 365:	
	污染路径: expanded from macro 'false'	
337	缺陷发生位置: /source/client/src/clientMain.c	行号: 325
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.	
	代码片段: 323: } 324: } else if (TD_RES_TMQ_META(res)) { 325: return NULL; 326: } else { 327: // assert to avoid un-initialization error	
	污染路径: expanded from macro 'NULL'	
338	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 1113
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1111: 1112: if (!asc && pBlockInfo->tbBlockIdx == 0) { 1113: return false; 1114: } 1115:	
	污染路径: expanded from macro 'false'	
339	缺陷发生位置: /source/util/src/ttimer.c	行号: 433
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 431: if (state != TIMER_STATE_EXPIRED) {	

	432: // timer already stopped or cancelled, has nothing to do in this case 433: return false; 434: } 435:	
	污染路径: expanded from macro 'false'	
340	缺陷发生位置: /source/util/src/tcompression.c	行号: 789
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int32_t'.	
	代码片段: 787: ostream[opos++] = prev_value; 788: } 789: if (opos == nelements) return nelements * LONG_BYTES; 790: 791: // Decode dd2	
	污染路径: 无	
341	缺陷发生位置: /include/common/trow.h	行号: 218
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int16_t'.	
	代码片段: 216: ASSERT(idx >= 0); 217: if (idx == 0) { 218: return PRIMARYKEY_TIMESTAMP_COL_ID; 219: } 220:	
	污染路径: expanded from macro 'PRIMARYKEY_TIMESTAMP_COL_ID'	
342	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 343
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 341: return false; 342: _RETURN: 343: return false;	

	344: } 345: void cliHandleResp(SCliConn* conn) {	
	污染路径: expanded from macro 'false'	
343	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 673
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 671: pRange->minVer = nextStartVer; 672: pRange->maxVer = latestVer - 1; 673: return false; 674: } 675: }	
	污染路径: expanded from macro 'false'	
344	缺陷发生位置: /source/os/src/osMemory.c	行号: 248
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned int', 定义类型为'int32_t'.	
	代码片段: 246: int ret = mallopt(M_MMAP_THRESHOLD, 128 * 1024); 247: if (0 == ret) { 248: return TAOS_SYSTEM_ERROR(errno); 249: } 250:	
	污染路径: expanded from macro 'TAOS_SYSTEM_ERROR'	
345	缺陷发生位置: /source/util/src/tenv.c	行号: 20
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char'.	
	代码片段: 18: #include "tconfig.h" 19: 20: static char toLowChar(char c) { return (c > 'Z' c < 'A' ? c : (c - 'A' + 'a')); } 21: 22: int32_t taosEnvNameToCfgName(const char *envNameStr, char *cfgNameStr, int32_t cfgNameMaxLen) {	

	污染路径: 无	
346	缺陷发生位置: /source/libs/wal/src/walSeek.c	行号: 119
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 117: errno = TAOS_SYSTEM_ERROR(errno); 118: pWal->pIdxFile = NULL; 119: return -1; 120: } 121: walBuildLogName(pWal, fileFirstVer, fnameStr);	
	污染路径: 无	
347	缺陷发生位置: /include/util/tencode.h	行号: 505
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: 503: static FORCE_INLINE int32_t tPutl16(uint8_t* p, int16_t v) { 504: if (p) ((int16_t*)p)[0] = v; 505: return sizeof(int16_t); 506: } 507:	
	污染路径: 无	
348	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 79
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 77: if (tdbTbGet(pVnodeObj->pMeta->pUidIdx, &uid, sizeof(uid), NULL, NULL) < 0) { 78: metaULock(pVnodeObj->pMeta); 79: return false; 80: } 81:	
	污染路径:	

	expanded from macro 'false'	
349	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 49
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 47: if (pSchema[i].type == TSDB_DATA_TYPE_BINARY pSchema[i].type == TSDB_DATA_TYPE_VARBINARY) { 48: if (pSchema[i].bytes > TSDB_MAX_BINARY_LEN) { 49: return false; 50: } 51: } else if (pSchema[i].type == TSDB_DATA_TYPE_NCHAR) { </pre>	
	污染路径: expanded from macro 'false'	
350	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 113
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'long long'，定义类型为'int64_t'.	
	代码片段: <pre> 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: } 115: </pre>	
	污染路径: 无	
351	缺陷发生位置: /source/client/src/clientImpl.c	行号: 1597
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int *'.	
	代码片段: <pre> 1595: strncpy(passStr, pass, TMIN(TSDB_PASSWORD_LEN - 1, passLen)); 1596: strncpy(dbStr, db, TMIN(TSDB_DB_NAME_LEN - 1, dbLen)); 1597: return taos_connect(ipStr, userStr, passStr, dbStr, port); 1598: } 1599: </pre>	

	污染路径: 无	
352	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 733
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int *'.	
	代码片段: 731: 732: SSDataBlock* getNextBlockFromDownstreamRemain(struct SOperatorInfo* pOperator, int32_t idx) { 733: return getNextBlockFromDownstreamImpl(pOperator, idx, false); 734: } 735:	
	污染路径: 无	
353	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 1219
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 1217: if (IS_VAR_DATA_TYPE(pKey->type)) { 1218: if (varDataLen(v) != varDataLen(pKey->pData)) { 1219: return false; 1220: } else { 1221: return memcmp(varDataVal(v), varDataVal(pKey->pData), varDataLen(v)) == 0; 1222: } 1223: }	
	污染路径: expanded from macro 'false'	
354	缺陷发生位置: /source/os/src/osEnv.c	行号: 96
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 94: void osCleanup() {} 95: 96: bool osLogSpaceAvailable() { return tsLogSpace.size.avail > 0; } 97:	

	98: bool osTempSpaceAvailable() { return tsTempSpace.size.avail > 0; }	
	污染路径: 无	
355	缺陷发生位置: /source/util/src/thashutil.c	行号: 149
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'uint32_t'.	
	代码片段: 147: float f = GET_FLOAT_VAL(key); 148: if (isnan(f)) { 149: return 0x7fc00000; 150: } 151:	
	污染路径: 无	
356	缺陷发生位置: /source/common/src/ttime.c	行号: 74
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 72: for (int32_t i = 0; i < seg_len; ++i) { 73: if (*c == 'Z' *c == 'z' *c == '+' *c == '-') { 74: return true; 75: } 76: c--;	
	污染路径: expanded from macro 'true'	
357	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 105
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 103: if (pCol->pExpr->base.pParam[0].pCol->colType == COLUMN_TYPE_WINDOW_START) { 104: colDataSetVal(pDstColInfoData, rowIndex, (const char*)&pFillInfo->currentKey, false); 105: return true; 106: } else if (pCol->pExpr->base.pParam[0].pCol->colType == COLUMN_TYPE_WINDOW_END) {	

	107: // TODO: include endpoint	
	污染路径: expanded from macro 'true'	
358	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 303
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 301: STableNode* pTable = taosArrayGetP(pTables, i); 302: if (0 == strcmp(pTable0->tableAlias, pTable->tableAlias)) { 303: return true; 304: } 305: }	
	污染路径: expanded from macro 'true'	
359	缺陷发生位置: /source/util/src/tlog.c	行号: 236
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 234: int32_t ret = taosLockFile(pFile); 235: if (ret == 0) { 236: return true; 237: } 238: }	
	污染路径: expanded from macro 'true'	
360	缺陷发生位置: /source/util/src/tlog.c	行号: 346
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 344: if (pFile == NULL) { 345: if (errno == ENOENT) { 346: return false; 347: } else { 348: printf("\nfailed to open log file:%s, reason:%s\n", logFileName, strerror(errno));	

	污染路径: expanded from macro 'false'	
361	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 2033
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 2031: } 2032: 2033: return true; 2034: } 2035:	
	污染路径: expanded from macro 'true'	
362	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 658
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 656: 657: if (exceptDnodeId == pDnode->id) { 658: return true; 659: } 660:	
	污染路径: expanded from macro 'true'	
363	缺陷发生位置: /include/common/trow.h	行号: 93
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 91: 92: #ifdef TD_SUPPORT_BITMAP 93: static FORCE_INLINE bool tdValTypeIsNone(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NONE; } 94: static FORCE_INLINE bool tdValTypeIsNull(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NULL; } 95: static FORCE_INLINE bool tdValTypeIsNorm(TDRowValT valType) { return (valType & 0x03U) == TD_VTYPE_NORM; }	
	污染路径:	

	无	
364	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 63
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int64_t'.	
	代码片段: <pre> 61: wError("vgld:%d, failed to open file due to %s. file:%s", pWal->cfg.vgld, strerror(errno), fnameStr); 62: terrno = TAOS_SYSTEM_ERROR(errno); 63: return -1; 64: } 65: </pre>	
	污染路径: 无	
365	缺陷发生位置: /source/dnode/mnode/impl/src/mndShow.c	行号: 349
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 347: return true; 348: } 349: return false; 350: } 351: </pre>	
	污染路径: expanded from macro 'false'	
366	缺陷发生位置: /source/common/src/trow.c	行号: 169
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: <pre> 167: if (colId == PRIMARYKEY_TIMESTAMP_COL_ID) { 168: tdRowSetVal(pVal, TD_VTYPE_NORM, TD_ROW_KEY_ADDR(pRow)); 169: return true; 170: } 171: int16_t nCols = tdRowGetNCols(pRow) - 1; </pre>	
	污染路径: expanded from macro 'true'	
367	缺陷发生位置:	行号:

	/source/libs/parser/src/parTranslator.c	781
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 779: return TIME_LINE_GLOBAL == ((SSetOperator*)pStmt)->timeLineResMode; 780: } else { 781: return false; 782: } 783: }	
368	污染路径: expanded from macro 'false'	
	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 278
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
369	代码片段: 276: bool tsdbIsSameTFile(const STFile *f1, const STFile *f2) { 277: if (f1->type != f2->type) return false; 278: if (f1->did.level != f2->did.level) return false; 279: if (f1->did.id != f2->did.id) return false; 280: if (f1->fid != f2->fid) return false;	
	污染路径: expanded from macro 'false'	
	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 688
370	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 686: } else { 687: setMergeJoinDone(pOperator); 688: return false; 689: } 690: }	
	污染路径: expanded from macro 'false'	
370	缺陷发生位置: /source/os/src/osMemory.c	行号: 235
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned int', 定义类型为'int32_t'.	

	代码片段: 233: int ret = malloc(M_MMAP_THRESHOLD, 0); 234: if (0 == ret) { 235: return TAOS_SYSTEM_ERROR(errno); 236: } 237:	
	污染路径: expanded from macro 'TAOS_SYSTEM_ERROR'	
371	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 226
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 224: bool fmlsInterpFunc(int32_t funcId) { 225: if (funcId < 0 funcId >= funcMgtBuiltinNum) { 226: return false; 227: } 228: return FUNCTION_TYPE_INTERP == funcMgtBuiltin[funcId].type;	
	污染路径: expanded from macro 'false'	
372	缺陷发生位置: /source/os/src/osEnv.c	行号: 100
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 98: bool osTempSpaceAvailable() { return tsTempSpace.size.avail > 0; } 99: 100: bool osDataSpaceAvailable() { return tsDataSpace.size.avail > 0; } 101: 102: bool osLogSpaceSufficient() { return tsLogSpace.size.avail > tsLogSpace.reserved; }	
	污染路径: 无	
373	缺陷发生位置: /source/client/src/clientMain.c	行号: 297
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'void *', 定义类型为'int'.	

	代码片段: <pre> 295: if (pRequest->type == TSDB_SQL_RETRIEVE_EMPTY_RESULT pRequest->type == TSDB_SQL_INSERT 296: pRequest->code != TSDB_CODE_SUCCESS taos_num_fields(res) == 0 pRequest->killed) { 297: return NULL; 298: } 299: </pre>	
	污染路径: expanded from macro 'NULL'	
374	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMemTable.c	行号: 293
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 291: plter->pNode = SL_GET_NODE_FORWARD(plter- >pNode, 0); 292: if (plter->pNode == plter->pTbData->sl.pTail) { 293: return false; 294: } 295: } </pre>	
375	污染路径: expanded from macro 'false'	
	缺陷发生位置: /source/os/src/osString.c	行号: 76
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long long', 定义类型为'int64_t'.	
376	代码片段: <pre> 74: int64_t taosStr2int64(const char *str) { 75: char *endptr = NULL; 76: return strtoll(str, &endptr, 10); 77: } 78: </pre>	
	污染路径: 无	
376	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 57
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	

	代码片段: 55: } else if (pSchema[i].type == TSDB_DATA_TYPE_GEOMETRY) { 56: if (pSchema[i].bytes > TSDB_MAX_GEOMETRY_LEN) { 57: return false; 58: } 59: } else {
	污染路径: expanded from macro 'false'

15.2.1.3 WK4733 具有返回值的函数,其返回值如果不被使用,

调用是应有 void 说明

缺陷数量： 1002 个		
缺陷描述: 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见: 无		
1	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 77
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 75: 76: memcpy(varDataVal(pReader->transferBuf[slotId]), pVal->value.pData, pVal->value.nData); 77: colDataSetVal(pCollInfoData, numOfRows, pReader->transferBuf[slotId], false); 78: } 79: } else {	
	污染路径: 函数'colDataSetVal'的定义点.	
2	缺陷发生位置: /tests/script/api/stopquery.c	行号: 231
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	

	代码片段: 229: sqExecSQL(taos, sql); 230: 231: sprintf(sql, "use %s", dbName); 232: sqExecSQL(taos, sql); 233:	
	污染路径: 无	
3	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 27
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 25: 26: static int32_t sdbDeployData(SSdb *pSdb) { 27: mInfo("start to deploy sdb"); 28: 29: for (int32_t i = SDB_MAX - 1; i >= 0; --i) {	
	污染路径: 函数'mInfo'的定义点.	
4	缺陷发生位置: /utils/test/c/createTable.c	行号: 158
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 156: taos_free_result(pRes); 157: 158: sprintf(qstr, "show tables"); 159: pRes = taos_query(con, qstr); 160: code = taos_errno(pRes);	
	污染路径: 无	
5	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 28
	步骤描述: 不使用返回值的函数'vsprintf'应使用'void'说明.	
	代码片段: 26: va_list vArgList; 27: va_start(vArgList, pFormat); 28: vsprintf(pErrBuf, len, pFormat, vArgList);	

	29: va_end(vArgList); 30: return errCode;	
	污染路径: 无	
6	缺陷发生位置: /tools/shell/src/shellArguments.c	行号: 306
	步骤描述: 不使用返回值的函数'taosSetConsoleEcho'应使用'void'说明.	
	代码片段: 304: if (strlen(argv[i]) == 2) { 305: printf("Enter password: "); 306: taosSetConsoleEcho(false); 307: if (scanf("%20s", shell.args.password) > 1) { 308: fprintf(stderr, "password reading error\n");	
	污染路径: 函数'taosSetConsoleEcho'的定义点.	
7	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 305
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 303: } 304: } else { 305: mInfo("vgld:1, sync restore finished"); 306: } 307:	
	污染路径: 函数'mInfo'的定义点.	
8	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 247
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 245: 246: dInfo("thread:%d, start to open %d vnodes", pThread->threadIndex, pThread->vnodeNum); 247: setThreadName("open-vnodes"); 248: 249: for (int32_t v = 0; v < pThread->vnodeNum; ++v) {	

	污染路径: 函数'setThreadName'的定义点.	
9	缺陷发生位置: /utils/test/c/createTable.c	行号: 88
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 86: taos_free_result(pRes); 87: 88: sprintf(qstr, "create table if not exists %s (ts timestamp, i int) tags (j bigint)", stbName); 89: pRes = taos_query(con, qstr); 90: code = taos_errno(pRes);	
	污染路径: 无	
10	缺陷发生位置: /source/libs/stream/src/streamMeta.c	行号: 61
	步骤描述: 不使用返回值的函数'taosThreadOnce'应使用'void'说明.	
	代码片段: 59: } 60: 61: void streamMetaInit() { taosThreadOnce(&streamMetaModuleInit, streamMetaEnvInit); } 62: void streamMetaCleanup() { 63: taosCloseRef(streamBackendId);	
	污染路径: 函数'taosThreadOnce'的定义点.	
11	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 147
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 145: 146: int32_t dmProcessGrantRsp(SDnodeMgmt *pMgmt, SRpcMsg *pMsg) { 147: dError("grant rsp is received, but not supported yet"); 148: return 0; 149: }	

	污染路径: 函数'dError'的定义点.	
12	缺陷发生位置: /source/common/src/ttszip.c	行号: 108
	步骤描述: 不使用返回值的函数'tsBufDestroy'应使用'void'说明.	
	代码片段: 106: // invalid file 107: if (header.magic != TS_COMP_FILE_MAGIC) { 108: tsBufDestroy(pTSBuf); 109: return NULL; 110: }	
	污染路径: 函数'tsBufDestroy'的定义点.	
13	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 974
	步骤描述: 不使用返回值的函数'taosDecodeFixedU64'应使用'void'说明.	
	代码片段: 972: } 973: 974: taosDecodeFixedU64(buf, &tMagicNumber); 975: return tMagicNumber == FILE_MAGIC_NUMBER ? 0 : -1; 976: }	
	污染路径: 函数'taosDecodeFixedU64'的定义点.	
14	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 85
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 83: dlInfo("shut down signal is %d", signum); 84: #ifndef WINDOWS 85: dlInfo("sender PID:%d cmdline:%s", ((siginfo_t *)sigInfo)->si_pid, 86: taosGetCmdlineByPID(((siginfo_t *)sigInfo)->si_pid)); 87: #endif	
	污染路径: 函数'dlInfo'的定义点.	
15	缺陷发生位置:	行号:

	/tests/script/api/stmtBatchTest.c	226
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 224: taosMemoryFree(v->ts); 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr); 227: taosMemoryFree(v); 228: taosMemoryFree(lb);	
	污染路径: 函数'taosMemoryFree'的定义点.	
16	缺陷发生位置: /source/util/src/ttimer.c	行号: 251
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 249: } 250: 251: taosThreadMutexUnlock(&wheel->mutex); 252: } 253:	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
17	缺陷发生位置: /source/libs/index/test/index_executor_tests.cpp	行号: 75
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 73: idata.info = *collInfo; 74: 75: taosArrayPush(res->pDataBlock, &idata); 76: taosArrayPush(pBlockList, &res); 77:	
	污染路径: 函数'taosArrayPush'的定义点.	
18	缺陷发生位置: /source/client/src/clientTmqConnector.c	行号: 84
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段:	

	82: (*env)->DeleteLocalRef(env, assignment); 83: 84: atomic_store_32(&__init_assignment, 2); 85: jniDebug("tmq method assignment finished"); 86: }	
	污染路径: 函数'atomic_store_32'的定义点.	
19	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 303
	步骤描述: 不使用返回值的函数'initTagColskeyBuf'应使用'void'说明.	
	代码片段: 301: extractEqualOnCondCols(pInfo, pInfo->pColEqualOnConditions, pInfo->leftEqOnCondCols, pInfo->rightEqOnCondCols); 302: initTagColskeyBuf(&pInfo->leftEqOnCondKeyLen, &pInfo->leftEqOnCondKeyBuf, pInfo->leftEqOnCondCols); 303: initTagColskeyBuf(&pInfo->rightEqOnCondKeyLen, &pInfo->rightEqOnCondKeyBuf, pInfo->rightEqOnCondCols); 304: _hash_fn_t hashFn = taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY); 305: pInfo->rightBuildTable = tSimpleHashInit(256, hashFn);	
	污染路径: 函数'initTagColskeyBuf'的定义点.	
20	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 129
	步骤描述: 不使用返回值的函数'taosThreadMutexDestroy'应使用'void'说明.	
	代码片段: 127: pChunk = pTemp; 128: } 129: taosThreadMutexDestroy(&pAllocator->mutex); 130: taosMemoryFree(pAllocator); 131: }	
	污染路径: 函数'taosThreadMutexDestroy'的定义点.	
21	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 473
	步骤描述:	

	不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: <pre> 471: code = tsdbSttFileReadStatisBlk(plter->pReader, (const TStatisBlkArray **)&pBlockLoadInfo->pSttStatisBlkArray); 472: if (code != TSDB_CODE_SUCCESS) { 473: tsdbError("failed to load stt block statistics, code:%s, %s", tstrerror(code), idStr); 474: return code; 475: }</pre>	
	污染路径: 函数'tsdbError'的定义点.	
22	缺陷发生位置: /source/os/src/osSignal.c	行号: 74
	步骤描述: 不使用返回值的函数'signal'应使用'void'说明.	
	代码片段: <pre> 72: void taosIgnSignal(int32_t signum) { signal(signum, SIG_IGN); } 73: 74: void taosDflSignal(int32_t signum) { signal(signum, SIG_DFL); } 75: 76: void taosKillChildOnParentStopped() {</pre>	
	污染路径: 无	
23	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 45
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 43: s = syncUtilPrintBin2((char*)(pEntry->data), pEntry- >dataLen); 44: cJSON_AddStringToObject(pRoot, "data2", s); 45: taosMemoryFree(s); 46: } 47:</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
24	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 168

	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 166: } 167: 168: colDataSetVal(pOutputData, i, output, (output == NULL)); 169: 170: _exit:	
	污染路径: 函数'colDataSetVal'的定义点.	
25	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamTaskSnap.c	行号: 91
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 89: taosArrayDestroy(pReader->tdbTbList); 90: tdbTbcClose(pReader->pCur); 91: taosMemoryFree(pReader); 92: return code; 93: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
26	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 181
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 179: 180: for (int i = 1; i <= nData; i++) { 181: sprintf(key, "key%d", i); 182: sprintf(val, "value%d", i); 183:	
	污染路径: 无	
27	缺陷发生位置: /utils/test/c/createTable.c	行号: 149
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段:	

	147: } 148: 149: snprintf(qstr, 128, "use %s", dbName); 150: TAOS_RES *pRes = taos_query(con, qstr); 151: int code = taos_errno(pRes);	
	污染路径: 无	
28	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 643
	步骤描述: 不使用返回值的函数'SET_RES_WINDOW_KEY'应使用'void'说明.	
	代码片段: 641: static bool isCalculatedWin(SIntervalAggOperatorInfo* pInfo, const STimeWindow* win, uint64_t tableGroupId) { 642: char keyBuf[sizeof(TSKEY) + sizeof(uint64_t)] = {0}; 643: SET_RES_WINDOW_KEY(keyBuf, (char*)&win->skey, sizeof(TSKEY), tableGroupId); 644: return tSimpleHashGet(pInfo->aggSup.pResultRowHashTable, keyBuf, GET_RES_WINDOW_KEY_LEN(sizeof(TSKEY))) != NULL; 645: }	
	污染路径: 函数'SET_RES_WINDOW_KEY'的定义点.	
29	缺陷发生位置: /source/util/src/tcompression.c	行号: 2291
	步骤描述: 不使用返回值的函数'DATA_TYPE_INFO'应使用'void'说明.	
	代码片段: 2289: 2290: if (DATA_TYPE_INFO[type].startFn) { 2291: DATA_TYPE_INFO[type].startFn(pCmprsor, type, cmprAlg); 2292: } 2293:	
	污染路径: 函数'DATA_TYPE_INFO'的定义点.	
30	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 341
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	

	代码片段: 339: rspSize = tSerializeSBatchRsp(NULL, 0, &batchRsp); 340: if (rspSize < 0) { 341: qError("tSerializeSBatchRsp failed"); 342: code = TSDB_CODE_OUT_OF_MEMORY; 343: goto _exit;	
	污染路径: 函数'qError'的定义点.	
31	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 432
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 430: _alloc_finish: 431: if (NULL == pCell) { 432: tdbError("tdb/page-allocate: null ptr pCell."); 433: return -1; 434: }	
	污染路径: 函数'tdbError'的定义点.	
32	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 92
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 90: collInfoDataEnsureCapacity(&idata, rows, true); 91: 92: blockDataAppendCollInfo(res, &idata); 93: 94: res->info.capacity = rows;	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
33	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 875
	步骤描述: 不使用返回值的函数'taosEncodeFixedU64'应使用'void'说明.	
	代码片段: 873: char buf[sizeof(FILE_MAGIC_NUMBER) + 1] = {0}; 874: void* pBuf = (void*)buf; 875: taosEncodeFixedU64((void**)(void*)&pBuf,	

	FILE_MAGIC_NUMBER); 876: int nwrite = write->ctx->write(write->ctx, buf, (int32_t)strlen(buf)); 877:	
	污染路径: 函数'taosEncodeFixedU64'的定义点.	
34	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 238
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: 236: taosMemoryFreeClear(bm->pManifest); 237: bm->pManifest = taosStrdup(name); 238: taosHashPut(bm->pSstTbl[1 - bm->idx], name, strlen(name), &dummy, sizeof(dummy)); 239: continue; 240: }	
	污染路径: 函数'taosHashPut'的定义点.	
35	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable.c	行号: 260
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 258: rc = tdbTbGet(pMeta->pNameIdx, pReq->name, strlen(pReq->name) + 1, &pData, &nData); 259: if (rc < 0 *(tb_uid_t *)pData != pReq->suid) { 260: tdbFree(pData); 261: terrno = TSDB_CODE_TDB_STB_NOT_EXIST; 262: return -1;	
	污染路径: 函数'tdbFree'的定义点.	
36	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 118
	步骤描述: 不使用返回值的函数'QW_SCH_ELOG'应使用'void'说明.	
	代码片段: 116: QW_RET(TSDB_CODE_QRY_SCH_NOT_EXIST); 117: } else { 118: QW_SCH_ELOG("unknown notExistOpt:%d", nOpt);	

	119: QW_ERR_RET(TSDB_CODE_APP_ERROR); 120: }	
	污染路径: 函数'QW_SCH_ELOG'的定义点.	
37	缺陷发生位置: /source/util/src/tworker.c	行号: 105
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 103: if (queue == NULL) return NULL; 104: 105: taosThreadMutexLock(&pool->mutex); 106: taosSetQueueFp(queue, fp, NULL); 107: taosAddIntoQset(pool->qset, queue, ahandle);	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
38	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 385
	步骤描述: 不使用返回值的函数'idxInt2str'应使用'void'说明.	
	代码片段: 383: case TSDB_DATA_TYPE_TIMESTAMP: 384: *dst = taosMemoryCalloc(1, bufSize + 1); 385: idxInt2str(*(int64_t*)src, *dst, -1); 386: tlen = strlen(*dst); 387: break;	
	污染路径: 函数'idxInt2str'的定义点.	
39	缺陷发生位置: /source/util/src/tlruCache.c	行号: 341
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 339: taosLRUCacheShardEvictLRU(shard, 0, lastReferenceList); 340: 341: taosThreadMutexUnlock(&shard->mutex); 342: 343: for (int i = 0; i < taosArrayGetSize(lastReferenceList); + +i) {	

	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
40	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 66
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 64: return -1; 65: } 66: tmsgReportStartup("snode-worker", "initialized"); 67: 68: if (udfcOpen() != 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
41	缺陷发生位置: /source/util/src/texception.c	行号: 55
	步骤描述: 不使用返回值的函数'func'应使用'void'说明.	
	代码片段: 53: static void cleanupWrapper_int_int(SCleanupAction* ca) { 54: int32_t (*func)(int32_t) = ca->func; 55: func(ca->arg1.Int); 56: } 57:	
	污染路径: 函数'func'的定义点.	
42	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 368
	步骤描述: 不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: 366: taosMkDir(root00); 367: taosMkDir(root01); 368: taosMkDir(root10); 369: taosMkDir(root11); 370: taosMkDir(root12);	
	污染路径: 函数'taosMkDir'的定义点.	
43	缺陷发生位置:	行号:

	/source/libs/executor/src/sortoperator.c	141
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 139: } 140: 141: blockDataEnsureCapacity(p, capacity); 142: 143: while (1) {	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
44	缺陷发生位置: /source/libs/scheduler/src/schFlowCtrl.c	行号: 58
	步骤描述: 不使用返回值的函数'SCH_JOB_DLOG'应使用'void'说明.	
	代码片段: 56: 57: if (schMgmt.cfg.maxNodeTableNum <= 0 sum < schMgmt.cfg.maxNodeTableNum) { 58: SCH_JOB_DLOG("job no need flow ctrl, totalTableNum: %" PRId64, sum); 59: return TSDB_CODE_SUCCESS; 60: }	
	污染路径: 函数'SCH_JOB_DLOG'的定义点.	
45	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 384
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 382: 383: if (pTagIdxKey1->type != pTagIdxKey2->type) { 384: metaError("meta/open: incorrect tag idx type."); 385: return TSDB_CODE_FAILED; 386: }	
	污染路径: 函数'metaError'的定义点.	
46	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 348
	步骤描述:	

	不使用返回值的函数'tSerializeSDCreateMnodeReq'应使用'void'说明.	
	代码片段: 346: int32_t contLen = tSerializeSDCreateMnodeReq(NULL, 0, pAlterReq); 347: void *pReq = taosMemoryMalloc(contLen); 348: tSerializeSDCreateMnodeReq(pReq, contLen, pAlterReq); 349: 350: STransAction action = {	
	污染路径: 函数'tSerializeSDCreateMnodeReq'的定义点.	
47	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 182
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 180: SOperatorInfo* extractOperatorInTree(SOperatorInfo* pOperator, int32_t type, const char* id) { 181: if (pOperator == NULL) { 182: qError("invalid operator, failed to find tableScanOperator %s", id); 183: terrno = TSDB_CODE_PAR_INTERNAL_ERROR; 184: return NULL;	
	污染路径: 函数'qError'的定义点.	
48	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 71
	步骤描述: 不使用返回值的函数'tjsonAddItemToObject'应使用'void'说明.	
	代码片段: 69: void *pIter = NULL; 70: SJson *items = tjsonCreateObject(); 71: tjsonAddItemToObject(json, "dbs", items); 72: 73: while (1) {	
	污染路径: 函数'tjsonAddItemToObject'的定义点.	
49	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 216
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 214: 215: if (mndTransAppendRedoAction(pTrans, &action) != 0) { 216: taosMemoryFree(pReq); 217: return -1; 218: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
50	缺陷发生位置: /source/os/src/osFile.c	行号: 137
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 135: int code = taosFsyncFile(pFileTo); 136: 137: taosCloseFile(&pFileFrom); 138: taosCloseFile(&pFileTo); 139:	
	污染路径: 函数'taosCloseFile'的定义点.	
51	缺陷发生位置: /source/libs/nodes/src/nodesCodeFuncs.c	行号: 7374
	步骤描述: 不使用返回值的函数'nodesWarn'应使用'void'说明.	
	代码片段: 7372: break; 7373: } 7374: nodesWarn("jsonToSpecificNode unknown node = %s", nodesNodeName(nodeType(pObj))); 7375: return TSDB_CODE_SUCCESS; 7376: }	
	污染路径: 函数'nodesWarn'的定义点.	
52	缺陷发生位置: /source/dnode/mnode/impl/src/mndGrant.c	行号: 43
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 41: pCollInfo = taosArrayGet(pBlock->pDataBlock, cols);	

	42: src = "false"; 43: STR_WITH_MAXSIZE_TO_VARSTR(tmp, src, 32); 44: colDataSetVal(pColInfo, numOfRows, tmp, false); 45:	
	污染路径: 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
53	缺陷发生位置: /source/os/src/osDir.c	行号: 112
	步骤描述: 不使用返回值的函数'taosCloseDir'应使用'void'说明.	
	代码片段: 110: } 111: 112: taosCloseDir(&pDir); 113: rmdir(dirname); 114:	
	污染路径: 函数'taosCloseDir'的定义点.	
54	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 82
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 80: setThreadName("dnode-crashReport"); 81: char filepath[PATH_MAX] = {0}; 82: snprintf(filepath, sizeof(filepath), "%s%s.taosdCrashLog", tsLogDir, TD_DIRSEP); 83: char *pMsg = NULL; 84: int64_t msgLen = 0;	
	污染路径: 无	
55	缺陷发生位置: /tests/script/api/stopquery.c	行号: 72
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: taos_free_result(pSql); 71: taos_close(taos); 72: taos_cleanup(); 73: exit(EXIT_FAILURE);	

	74: }	
	污染路径: 函数'taos_cleanup'的定义点.	
56	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 341
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 339: taosArrayDestroy(sArray[i]); 340: } 341: taosArrayDestroy(slArray); 342: 343: printf("1h \t %" PRId64 "ms,%" PRId64 "\n", end1h - start1h, end1hCt - start1hCt);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
57	缺陷发生位置: /source/common/src/tdatablock.c	行号: 647
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 645: } else { 646: char* p = colDataGetData(pColData, j); 647: colDataSetVal(pDstCol, j - startIndex, p, false); 648: } 649: }	
	污染路径: 函数'colDataSetVal'的定义点.	
58	缺陷发生位置: /source/common/src/tdatablock.c	行号: 525
	步骤描述: 不使用返回值的函数'colDataMergeCol'应使用'void'说明.	
	代码片段: 523: 524: capacity = pDest->info.capacity; 525: colDataMergeCol(pCol2, pDest->info.rows, &capacity, pCol1, pSrc->info.rows); 526: } 527:	

	污染路径: 函数'colDataMergeCol'的定义点.	
59	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 392
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 390: } 391: 392: taosArrayDestroy(pBuf->pData); 393: } 394:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
60	缺陷发生位置: /source/os/src/osEnv.c	行号: 87
	步骤描述: 不使用返回值的函数'taosGetDiskSize'应使用'void'说明.	
	代码片段: 85: } 86: if (tsDataDir[0] != 0) { 87: taosGetDiskSize(tsDataDir, &tsDataSpace.size); 88: } 89: if (tsTempDir[0] != 0) {	
	污染路径: 函数'taosGetDiskSize'的定义点.	
61	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 930
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 928: int32_t contLen = reqLen + sizeof(SMsgHead); 929: 930: mDebug("start to process ttl timer"); 931: 932: while (1) {	
	污染路径: 函数'mDebug'的定义点.	
62	缺陷发生位置: /source/libs/stream/src/streamQueue.c	行号: 274

	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 272: 273: if (*numOfBlocks >= MAX_STREAM_EXEC_BATCH_NUM) { 274: qDebug("s-task:%s batch size limit:%d reached, start to process blocks", id, MAX_STREAM_EXEC_BATCH_NUM); 275: return TSDB_CODE_SUCCESS; 276: }</pre>	
	污染路径: 函数'qDebug'的定义点.	
63	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 285
	步骤描述: 不使用返回值的函数'blockDataKeepFirstNRows'应使用'void'说明.	
	代码片段: <pre> 283: // limit the output rows 284: int32_t keep = (int32_t)(pLimit->limit - pLimitInfo->numOfOutputRows); 285: blockDataKeepFirstNRows(pBlock, keep); 286: 287: pLimitInfo->numOfOutputRows += pBlock->info.rows;</pre>	
	污染路径: 函数'blockDataKeepFirstNRows'的定义点.	
64	缺陷发生位置: /source/util/src/tcache.c	行号: 137
	步骤描述: 不使用返回值的函数'taosThreadRwlockDestroy'应使用'void'说明.	
	代码片段: <pre> 135: static FORCE_INLINE void __trashcan_lock_destroy(SCacheObj *pCacheObj) { 136: #if defined(LINUX) 137: taosThreadRwlockDestroy(&pCacheObj->lock); 138: #else 139: taosThreadMutexDestroy(&pCacheObj->lock);</pre>	
	污染路径: 函数'taosThreadRwlockDestroy'的定义点.	
65	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 71

	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段: 69: 70: if (tdbPCacheOpenImpl(pCache) < 0) { 71: tdbOsFree(pCache); 72: return -1; 73: }	
	污染路径: 函数'tdbOsFree'的定义点.	
66	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 168
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 166: 167: static void mndPullupGrant(SMnode *pMnode) { 168: mTrace("pullup grant msg"); 169: int32_t contLen = 0; 170: void *pReq = mndBuildTimerMsg(&contLen);	
	污染路径: 函数'mTrace'的定义点.	
67	缺陷发生位置: /source/util/src/tref.c	行号: 110
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 108: } 109: 110: taosThreadMutexUnlock(&tsRefMutex); 111: 112: return rsetId;	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
68	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 27
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 25: for (int i = 0; i < 3; i++) {	

	26: SArray *m = taosArrayInit(10, sizeof(uint64_t)); 27: taosArrayPush(src, &m); 28: } 29:	
	污染路径: 函数'taosArrayPush'的定义点.	
69	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 120
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 118: } 119: 120: taosMemoryFree(fname); 121: return pStore; 122: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
70	缺陷发生位置: /source/libs/parser/test/parInitialCTest.cpp	行号: 195
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 193: expect.pRetentions = taosArrayInit(TARRAY_MIN_SIZE, sizeof(SRetention)); 194: } 195: taosArrayPush(expect.pRetentions, &retention); 196: ++expect.numOfRetentions; 197: };	
	污染路径: 函数'taosArrayPush'的定义点.	
71	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 322
	步骤描述: 不使用返回值的函数'taosThreadOnce'应使用'void'说明.	
	代码片段: 320: 321: static int32_t tKeywordCode(const char* z, int n) { 322: taosThreadOnce(&keywordsHashTableInit, doInitKeywordsTable);	

	323: 324: char key[512] = {0};	
	污染路径: 函数'taosThreadOnce'的定义点.	
72	缺陷发生位置: /source/common/src/tdaformat.c	行号: 1431
	步骤描述: 不使用返回值的函数'tGetTagVal'应使用'void'说明.	
	代码片段: 1429: } 1430: 1431: tGetTagVal(p + offset, &tv, isjson); 1432: if (isjson) { 1433: c = tTagValJsonCmprFn(pTagVal, &tv);	
	污染路径: 函数'tGetTagVal'的定义点.	
73	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 90
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 88: 89: pInserter->submitRes.affectedRows += pInserter->submitRes.pRsp->affectedRows; 90: qDebug("submit rsp received, affectedRows:%d, total:%d" PRId64, pInserter->submitRes.pRsp->affectedRows, pInserter->submitRes.affectedRows); 91: pInserter->submitRes.affectedRows); 92: tDecoderClear(&coder);	
	污染路径: 函数'qDebug'的定义点.	
74	缺陷发生位置: /source/dnode/vnode/test/tsdbSmaTest.cpp	行号: 574
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 572: taosMemoryFreeClear(pDataBlock); 573: } 574: taosArrayDestroy(pDataBlocks); 575: 576: // release meta	

	污染路径: 函数'taosArrayDestroy'的定义点.	
75	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 946
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 944: sdbCancelFetch(pSdb, plter); 945: sdbRelease(pSdb, pVgroup); 946: taosArrayDestroy(pArray); 947: return -1; 948: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
76	缺陷发生位置: /source/common/src/tname.c	行号: 323
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 321: rName->ctbShortName[1] = '_'; 322: for (int i = 0; i < 16; i++) { 323: sprintf(temp, "%02x", context.digest[i]); 324: strcat(rName->ctbShortName, temp); 325: }	
	污染路径: 无	
77	缺陷发生位置: /source/libs/index/test/index_executor_tests.cpp	行号: 48
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: 46: double sifLeftVd = 21.0, sifRightVd = 10.0; 47: 48: void sifFreeDataBlock(void *block) { blockDataDestroy(*(SSDataBlock **)block); } 49: 50: void sifInitLogFile() {	
	污染路径: 函数'blockDataDestroy'的定义点.	

78	缺陷发生位置： /source/dnode/mnode/impl/src/mndQuery.c	行号： 93
	步骤描述： 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段： 91: if (tDeserializeSBatchReq(pMsg->pCont, pMsg->contLen, &batchReq)) { 92: code = TSDB_CODE_OUT_OF_MEMORY; 93: mError("tDeserializeSBatchReq failed"); 94: goto _exit; 95: }	
	污染路径： 函数'mError'的定义点.	
79	缺陷发生位置： /source/util/src/tlog.c	行号： 164
	步骤描述： 不使用返回值的函数'taosUmaskFile'应使用'void'说明.	
	代码片段： 162: if (tsLogObj.slowHandle == NULL) return -1; 163: 164: taosUmaskFile(0); 165: tsLogObj.slowHandle->pFile = taosOpenFile(fullName, TD_FILE_CREATE TD_FILE_WRITE TD_FILE_APPEND); 166: if (tsLogObj.slowHandle->pFile == NULL) {	
	污染路径： 函数'taosUmaskFile'的定义点.	
80	缺陷发生位置： /source/libs/scheduler/src/schTask.c	行号： 116
	步骤描述： 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段： 114: if (taosHashPut(pTask->execNodes, &execId, sizeof(execId), &nodeInfo, sizeof(nodeInfo))) { 115: SCH_TASK_ELOG("taosHashPut nodeInfo to execNodes failed, errno:%d", errno); 116: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 117: } 118:	
	污染路径： 函数'SCH_ERR_RET'的定义点.	

81	缺陷发生位置: /source/libs/scheduler/src/schUtil.c	行号: 35
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 33: } 34: 35: qDebug("sch release jobId:0x%" PRIx64, refId); 36: return taosReleaseRef(schMgmt.jobRef, refId); 37: }	
	污染路径: 函数'qDebug'的定义点.	
82	缺陷发生位置: /tests/script/api/passwdTest.c	行号: 232
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 230: 231: // alter pass for users 232: sprintf(qstr, "alter user %s pass 'taos'", users[i]); 233: queryDB(taos, qstr); 234: }	
	污染路径: 无	
83	缺陷发生位置: /source/os/src/osString.c	行号: 184
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 182: if (used > gConvMaxNum[type]) { 183: used = atomic_sub_fetch_32(&convUsed[type], 1); 184: sched_yield(); 185: continue; 186: }	
	污染路径: 无	
84	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 41
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: 39: s = syncUtilPrintBin((char*)(pEntry->data), pEntry->dataLen); 40: cJSON_AddStringToObject(pRoot, "data", s); 41: taosMemoryFree(s); 42: 43: s = syncUtilPrintBin2((char*)(pEntry->data), pEntry->dataLen);	
	污染路径: 函数'taosMemoryFree'的定义点.	
85	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 377
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 375: qDebug("tagfilter build column info, slotId:%d, colId:%d, type:%d", pSColumnNode->slotId, cInfo.colId, cInfo.type); 376: #endif 377: taosArrayPush(pData->cInfoList, &cInfo); 378: } else { 379: SColumnNode* col = *(SColumnNode**)data;	
	污染路径: 函数'taosArrayPush'的定义点.	
86	缺陷发生位置: /source/libs/parser/test/mockCatalogService.cpp	行号: 143
	步骤描述: 不使用返回值的函数'tNameGetDbName'应使用'void'说明.	
	代码片段: 141: int32_t catalogGetTableDistVgInfo(const SName* pTableName, SArray** vgList) const { 142: char db[TSDB_DB_NAME_LEN] = {0}; 143: tNameGetDbName(pTableName, db); 144: return copyTableVgroup(db, tNameGetTableName(pTableName), vgList); 145: }	
	污染路径: 函数'tNameGetDbName'的定义点.	
87	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 65
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 63: printf("raftStorePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 64: fflush(NULL); 65: taosMemoryFree(serialized); 66: } 67: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
88	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 4295
	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: <pre> 4293: } 4294: if (TSDB_CODE_SUCCESS != code) { 4295: nodesError("msgToSpecificNode error node = %s", nodesNodeName(nodeType(pObj))); 4296: } 4297: return code; </pre>	
	污染路径: 函数'nodesError'的定义点.	
89	缺陷发生位置: /source/libs/transport/test/strBench.c	行号: 74
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: <pre> 72: while (1) { 73: int numOfMsgs = taosReadAllQItemsFromQset(multiQ- >qset[idx], qall, &qinfo); 74: tDebug("%d shell msgs are received", numOfMsgs); 75: if (numOfMsgs <= 0) break; 76: </pre>	
	污染路径: 函数'tDebug'的定义点.	
90	缺陷发生位置: /source/util/src/tarray.c	行号: 395
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	

	代码片段: 393: fp(*(void**)TARRAY_GET_ELEM(pArray, i)); 394: } 395: taosArrayDestroy(pArray); 396: } 397: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
91	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 543
	步骤描述: 不使用返回值的函数'system'应使用'void'说明.	
	代码片段: 541: GTEST_ASSERT_EQ(ret, 0); 542: 543: system("ls -l ./tdb"); 544: } 545:	
	污染路径: 无	
92	缺陷发生位置: /examples/c/demo.c	行号: 107
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 105: 106: // query the records 107: sprintf(qstr, "SELECT * FROM m1"); 108: result = taos_query(taos, qstr); 109: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
93	缺陷发生位置: /source/libs/scheduler/test/schedulerTests.cpp	行号: 505
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 503: pJob = schAcquireJob(queryJobRefId); 504: if (NULL == pJob) { 505: taosArrayDestroy(qnodelist);	

	506: schtFreeQueryDag(&dag); 507: continue;	
	污染路径: 函数'taosArrayDestroy'的定义点.	
94	缺陷发生位置: /source/dnode/vnode/src/sma/smaSnapshot.c	行号: 97
	步骤描述: 不使用返回值的函数'rsmaSnapReaderClose'应使用'void'说明.	
	代码片段: 95: if (code) { 96: smaError("vgId:%d, vnode snapshot rsma read failed since %s", SMA_VID(pReader->pSma), tstrerror(code)); 97: rsmaSnapReaderClose(&pReader); 98: } else { 99: smaInfo("vgId:%d, vnode snapshot rsma read succeed", SMA_VID(pReader->pSma)); 	
	污染路径: 函数'rsmaSnapReaderClose'的定义点.	
95	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 259
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 257: int32_t index = taosArraySearchIdx(chArray, &childId, compareInt32Val, TD_EQ); 258: if (index != -1) { 259: qDebug("===stream===try push delete window %" PRIu64 " chId:%d ,continue", win.skey, childId); 260: getNextTimeWindow(pInterval, &win, TSDB_ORDER_ASC); 261: continue;	
	污染路径: 函数'qDebug'的定义点.	
96	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 214
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 212: } 213: } else {	

	214: ASSERT(0); 215: } 216:	
	污染路径: 函数'ASSERT'的定义点.	
97	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 195
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 193: io->clientRpc = rpcOpen(&rpclnit); 194: if (io->clientRpc == NULL) { 195: sError("failed to initialize RPC"); 196: return -1; 197: }	
	污染路径: 函数'sError'的定义点.	
98	缺陷发生位置: /source/util/src/tmempool.c	行号: 85
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 83: } 84: 85: taosThreadMutexUnlock(&(pool_p->mutex)); 86: 87: if (pos == NULL) uDebug("mempool: out of memory");	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
99	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 216
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 214: } 215: 216: colDataSetVal(pOutputData, i, &res, (res==-1)); 217: 218: return code;	

	污染路径: 函数'colDataSetVal'的定义点.	
100	缺陷发生位置: /source/libs/scheduler/src/schFlowCtrl.c	行号: 66
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 64: if (NULL == pJob->flowCtrl) { 65: SCH_JOB_ELOG("taosHashInit %d flowCtrl failed", pJob->taskNum); 66: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 67: } 68:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
101	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 87
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 85: ++pBlock->info.rows; 86: } else { 87: tsdbError("invalid retrieve type:%d, %s", pReader->type, idStr); 88: return TSDB_CODE_INVALID_PARA; 89: }	
	污染路径: 函数'tsdbError'的定义点.	
102	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 266
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 264: strcpy(rspMsg->dbFName, ctgTestDbname); 265: sprintf(rspMsg->tbName, "%s", ctgTestStablename); 266: sprintf(rspMsg->stbName, "%s", ctgTestStablename); 267: rspMsg->numOfTags = ctgTestTagNum; 268: rspMsg->numOfColumns = ctgTestColNum;	
	污染路径: 无	

103	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 231
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: 229: taosMemoryFreeClear(bm->pCurrent); 230: bm->pCurrent = taosStrdup(name); 231: taosHashPut(bm->pSstTbl[1 - bm->idx], name, strlen(name), &dummy, sizeof(dummy)); 232: continue; 233: }	
	污染路径: 函数'taosHashPut'的定义点.	
104	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 217
	步骤描述: 不使用返回值的函数'collInfoDataEnsureCapacity'应使用'void'说明.	
	代码片段: 215: pColData->info.bytes = sizeof(int64_t); 216: 217: collInfoDataEnsureCapacity(pColData, 5, false); 218: colDataSetInt64(pColData, 0, &pQueryWindow->skey); 219: colDataSetInt64(pColData, 1, &pQueryWindow->ekey);	
	污染路径: 函数'collInfoDataEnsureCapacity'的定义点.	
105	缺陷发生位置: /source/os/src/osTimer.c	行号: 93
	步骤描述: 不使用返回值的函数'sigaddset'应使用'void'说明.	
	代码片段: 91: sigset_t sigset; 92: sigemptyset(&sigset); 93: sigaddset(&sigset, SIGALRM); 94: sigprocmask(SIG_BLOCK, &sigset, NULL); 95: void (*callback)(int) = tharg;	
	污染路径: 无	
106	缺陷发生位置: /source/libs/stream/src/streamDispatch.c	行号: 506
	步骤描述:	

	不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 504: atomic_val_compare_exchange_8(&pTask->outputInfo.status, TASK_OUTPUT_STATUS__NORMAL, TASK_OUTPUT_STATUS__WAIT); 505: if (old != TASK_OUTPUT_STATUS__NORMAL) { 506: qDebug("s-task:%s wait for dispatch rsp, not dispatch now, output status:%d", id, old); 507: return 0; 508: }	
	污染路径: 函数'qDebug'的定义点.	
107	缺陷发生位置: /source/libs/wal/src/walWrite.c	行号: 208
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 206: ((SWalFileInfo *)taosArrayGetLast(pWal->fileInfoSet))->fileSize = entry.offset; 207: taosCloseFile(&pIdxFile); 208: taosCloseFile(&pLogFile); 209: 210: code = walSaveMeta(pWal);	
	污染路径: 函数'taosCloseFile'的定义点.	
108	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 103
	步骤描述: 不使用返回值的函数'strtolower'应使用'void'说明.	
	代码片段: 101: memset(t, 1, tListLen(t)); 102: const char* a2 = "ABC\ABC\D"; 103: strtolower(t, a2); 104: EXPECT_STREQ(t, "abc\ABC\"); 105:	
	污染路径: 函数'strtolower'的定义点.	
109	缺陷发生位置: /tests/script/api/passwdTest.c	行号: 276
	步骤描述:	

	不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: <pre> 274: nPassVerNotified, nConn); 275: if (nPassVerNotified >= nConn) break; 276: sleep(1); 277: } 278: </pre>	
	污染路径: 无	
110	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 72
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 70: void udfdCPluginUdfInitLoadAggFuncs(SUdfCPluginCtx *udfCtx, const char *udfName) { 71: char processFuncName[TSDB_FUNC_NAME_LEN] = {0}; 72: snprintf(processFuncName, sizeof(processFuncName), "%s", udfName); 73: uv_dlsym(&udfCtx->lib, processFuncName, (void **) (&udfCtx->aggProcFunc)); 74: </pre>	
	污染路径: 无	
111	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 75
	步骤描述: 不使用返回值的函数'QW_RET'应使用'void'说明.	
	代码片段: <pre> 73: if (msgSize < 0) { 74: qError("tSerializeSQueryTableRsp failed"); 75: QW_RET(TSDB_CODE_OUT_OF_MEMORY); 76: } 77: </pre>	
	污染路径: 函数'QW_RET'的定义点.	
112	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 37
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	

	代码片段: 35: taosArrayDestroy(m); 36: } 37: taosArrayDestroy(src); 38: } 39:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
113	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 49
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 47: pData->dnodeHash = taosHashInit(4, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), true, HASH_NO_LOCK); 48: if (pData->dnodeHash == NULL) { 49: dError("failed to init dnode hash"); 50: terrno = TSDB_CODE_OUT_OF_MEMORY; 51: return -1;	
	污染路径: 函数'dError'的定义点.	
114	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 116
	步骤描述: 不使用返回值的函数'QW_RET'应使用'void'说明.	
	代码片段: 114: continue; 115: } else if (QW_NOT_EXIST_RET_ERR == nOpt) { 116: QW_RET(TSDB_CODE_QRY_SCH_NOT_EXIST); 117: } else { 118: QW_SCH_ELOG("unknown notExistOpt:%d", nOpt);	
	污染路径: 函数'QW_RET'的定义点.	
115	缺陷发生位置: /source/util/src/tidpool.c	行号: 34
	步骤描述: 不使用返回值的函数'taosThreadMutexInit'应使用'void'说明.	
	代码片段: 32: pIdPool->freeSlot = 0;	

	33: 34: taosThreadMutexInit(&pIdPool->mutex, NULL); 35: 36: uDebug("pool:%p is setup, maxId:%d", pIdPool, pIdPool->maxId);	
	污染路径: 函数'taosThreadMutexInit'的定义点.	
116	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 42
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 40: SCH_TASK_ELOG("rsp msg conflicted with task status, status:%s, rspType:%s", jobTaskStatusStr(taskStatus), 41: TMSG_INFO(msgType)); 42: SCH_ERR_RET(TSDB_CODE_QW_MSG_ERROR); 43: } 44:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
117	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 113
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 111: } 112: 113: dDebug("snode workers are initialized"); 114: return 0; 115: }	
	污染路径: 函数'dDebug'的定义点.	
118	缺陷发生位置: /source/client/src/clientImpl.c	行号: 92
	步骤描述: 不使用返回值的函数'strdequote'应使用'void'说明.	
	代码片段: 90: 91: tstrncpy(localDb, db, sizeof(localDb)); 92: strdequote(localDb);	

	93: } 94:	
	污染路径: 函数'strdequote'的定义点.	
119	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 106
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 104: 105: int32_t dmInitDnode(SDnode *pDnode) { 106: dDebug("start to create dnode"); 107: int32_t code = -1; 108: char path[PATH_MAX + 100] = {0};	
	污染路径: 函数'dDebug'的定义点.	
120	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 149
	步骤描述: 不使用返回值的函数'SET_RES_WINDOW_KEY'应使用'void'说明.	
	代码片段: 147: 148: int64_t buf[2] = {0}; 149: SET_RES_WINDOW_KEY((char*)buf, &groupId, sizeof(groupId), groupId); 150: 151: STableScanInfo* pTableScanInfo = pOperator->info;	
	污染路径: 函数'SET_RES_WINDOW_KEY'的定义点.	
121	缺陷发生位置: /tools/shell/src/shellArguments.c	行号: 248
	步骤描述: 不使用返回值的函数'shellParseSingleOpt'应使用'void'说明.	
	代码片段: 246: if (strcmp(argv[i], "--help") == 0 strcmp(argv[i], "-- usage") == 0 247: strcmp(argv[i], "-?") == 0 strcmp(argv[i], "/?") == 0) { 248: shellParseSingleOpt('?', NULL); 249: return 0;	

	250: }	
	污染路径: 函数'shellParseSingleOpt'的定义点.	
122	缺陷发生位置: /source/common/src/tdataformat.c	行号: 599
	步骤描述: 不使用返回值的函数'taosArrayInsert'应使用'void'说明.	
	代码片段: 597: 598: taosArrayRemoveBatch(aRowP, iStart, nRow, (FDelete)tRowPDestroy); 599: taosArrayInsert(aRowP, iStart, &pRow); 600: 601: _exit:	
	污染路径: 函数'taosArrayInsert'的定义点.	
123	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 42
	步骤描述: 不使用返回值的函数'regerror'应使用'void'说明.	
	代码片段: 40: return false; 41: } else { 42: regerror(reti, ®ex, msgbuf, sizeof(msgbuf)); 43: fprintf(stderr, "Regex match failed: %s\r\n", msgbuf); 44: regfree(®ex);	
	污染路径: 无	
124	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 170
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 168: struct tm tm; 169: taosLocalTime(&tTime, &tm, NULL); 170: sprintf(timeString, "%d-%02d-%02d %02d:%02d: %02d", tm.tm_year + 1900, tm.tm_mon + 1, tm.tm_mday, tm.tm_hour, 171: tm.tm_min, tm.tm_sec); 172:	

	污染路径: 无	
125	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 105
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 103: tEncoderInit(&coder, POINTER_SHIFT(*pBuf, sizeof(SMsgHead)), (*contLen) - sizeof(SMsgHead)); 104: if (tEncodeSVCreateTbBatchReq(&coder, pReqs) < 0) { 105: rpcFreeCont(*pBuf); 106: *pBuf = NULL; 107: *contLen = 0;	
	污染路径: 函数'rpcFreeCont'的定义点.	
126	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 99
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 97: SMnodeOpt option = {0}; 98: if (mmReadFile(pMgmt->path, &option) != 0) { 99: dError("failed to read file since %s", terrstr()); 100: mmClose(pMgmt); 101: return -1;	
	污染路径: 函数'dError'的定义点.	
127	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 132
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 130: SSDataBlock* pDataBlock = NULL; 131: uint64_t ts = 0; 132: tqDebug("tmqsnap task start to execute"); 133: int code = qExecTask(task, &pDataBlock, &ts); 134: if (code != 0) {	
	污染路径: 函数'tqDebug'的定义点.	

128	缺陷发生位置： /source/libs/stream/src/streamState.c	行号： 159
	步骤描述： 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段： 157: 158: char cfgPath[1030]; 159: sprintf(cfgPath, "%s/cfg", statePath); 160: 161: szPage = szPage < 0 ? 4096 : szPage;	
	污染路径： 无	
129	缺陷发生位置： /source/common/src/ttime.c	行号： 772
	步骤描述： 不使用返回值的函数'taosLocalTime'应使用'void'说明.	
	代码片段： 770: struct tm tm; 771: time_t tt = (time_t)start; 772: taosLocalTime(&tt, &tm, NULL); 773: tm.tm_sec = 0; 774: tm.tm_min = 0;	
	污染路径： 函数'taosLocalTime'的定义点.	
130	缺陷发生位置： /source/util/test/trefTest.c	行号： 102
	步骤描述： 不使用返回值的函数'taosThreadAttrInit'应使用'void'说明.	
	代码片段： 100: 101: TdThreadAttr thattr; 102: taosThreadAttrInit(&thattr); 103: taosThreadAttrSetDetachState(&thattr, PTHREAD_CREATE_JOINABLE); 104:	
	污染路径： 函数'taosThreadAttrInit'的定义点.	
131	缺陷发生位置： /source/libs/parser/test/parTestMain.cpp	行号： 73
	步骤描述：	

	不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: 71: 72: taosRemoveDir(path); 73: taosMkDir(path); 74: tstrncpy(tsLogDir, path, PATH_MAX); 75: if (taosInitLog("taoslog", 1) != 0) {	
	污染路径: 函数'taosMkDir'的定义点.	
132	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 813
	步骤描述: 不使用返回值的函数'filterFinishRange'应使用'void'说明.	
	代码片段: 811: 812: int32_t filterGetRangeNum(void *h, int32_t *num) { 813: filterFinishRange(h); 814: 815: SFilterRangeCtx *ctx = (SFilterRangeCtx *)h;	
	污染路径: 函数'filterFinishRange'的定义点.	
133	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 266
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 264: char *shellFormatTimestamp(char *buf, int64_t val, int32_t precision) { 265: if (shell.args.is_raw_time) { 266: sprintf(buf, "%" PRId64, val); 267: return buf; 268: }	
	污染路径: 无	
134	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataalter.c	行号: 185
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 183: tsdbCloseTombFileDataalter(plter);	

	184: } else { 185: ASSERT(0); 186: } 187: }	
	污染路径: 函数'ASSERT'的定义点.	
135	缺陷发生位置: /source/libs/executor/src/cachescanoperator.c	行号: 301
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: 299: void destroyCacheScanOperator(void* param) { 300: SCacheRowsScanInfo* pInfo = (SCacheRowsScanInfo*)param; 301: blockDataDestroy(pInfo->pRes); 302: blockDataDestroy(pInfo->pBufferedRes); 303: taosMemoryFree(pInfo->pSlotIds);	
	污染路径: 函数'blockDataDestroy'的定义点.	
136	缺陷发生位置: /tools/shell/src/shellWebsocket.c	行号: 212
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 210: ws_fetch_block(wres, &data, &rows); 211: } while (rows && !shell.stop_query); 212: taosCloseFile(&pFile); 213: return numOfRows; 214: }	
	污染路径: 函数'taosCloseFile'的定义点.	
137	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 424
	步骤描述: 不使用返回值的函数'taosCreateMD5Hash'应使用'void'说明.	
	代码片段: 422: char name[TSDB_COL_NAME_LEN + TSDB_POINTER_PRINT_BYTES + TSDB_NAME_DELIMITER_LEN + 1] = {0}; 423: int32_t len = snprintf(name, sizeof(name) - 1, "%s.%s"	

	PRId64 "", pWEnd->functionName, pointer); 424: taosCreateMD5Hash(name, len); 425: strncpy(pWEnd->node.aliasName, name, TSDB_COL_NAME_LEN - 1); 426:	
	污染路径: 函数'taosCreateMD5Hash'的定义点.	
138	缺陷发生位置: /source/os/src/osDir.c	行号: 103
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 101: 102: char filename[1024] = {0}; 103: snprintf(filename, sizeof(filename), "%s%s%s", dirname, TD_DIRSEP, taosGetDirEntryName(de)); 104: if (taosDirEntryIsDir(de)) { 105: taosRemoveDir(filename);	
	污染路径: 无	
139	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 2301
	步骤描述: 不使用返回值的函数'taosCreateMD5Hash'应使用'void'说明.	
	代码片段: 2299: char name[TSDB_FUNC_NAME_LEN + TSDB_POINTER_PRINT_BYTES + TSDB_NAME_DELIMITER_LEN + 1] = {0}; 2300: int32_t len = snprintf(name, sizeof(name) - 1, "%s.%" PRId64 "", pFunc->functionName, pointer); 2301: taosCreateMD5Hash(name, len); 2302: strncpy(pFunc->node.aliasName, name, TSDB_COL_NAME_LEN - 1); 2303: }	
	污染路径: 函数'taosCreateMD5Hash'的定义点.	
140	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 303
	步骤描述: 不使用返回值的函数'dataConverToStr'应使用'void'说明.	

	代码片段: 301: 302: if (!buf) goto end; 303: dataConverToStr(buf, pTagVal->type, pTagVal->pData, pTagVal->nData, NULL); 304: tvalue = cJSON_CreateString(buf); 305: taosMemoryFree(buf);	
	污染路径: 函数'dataConverToStr'的定义点.	
141	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 67
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 65: SScalarParam output = {0}; 66: doCallUdfScalarFunc(handle, &input, 1, &output); 67: taosArrayDestroy(pBlock->pDataBlock); 68: 69: SColumnInfoData *col = output.columnData;	
	污染路径: 函数'taosArrayDestroy'的定义点.	
142	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 313
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 311: if (currTotalLevel > pCxt->currLevel) { 312: SArray* pTables = taosArrayGetP(pCxt->pNsLevel, pCxt->currLevel); 313: taosArrayPush(pTables, &pTable); 314: if (hasSameTableAlias(pTables)) { 315: return generateSyntaxErrMsgExt(&pCxt->msgBuf, TSDB_CODE_PAR_NOT_UNIQUE_TABLE_ALIAS,	
	污染路径: 函数'taosArrayPush'的定义点.	
143	缺陷发生位置: /source/libs/wal/src/walSeek.c	行号: 124
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段:	

	122: pLogTFile = taosOpenFile(fnameStr, TD_FILE_CREATE TD_FILE_WRITE TD_FILE_APPEND); 123: if (pLogTFile == NULL) { 124: taosCloseFile(&pIdxTFile); 125: terrno = TAOS_SYSTEM_ERROR(errno); 126: pWal->pLogFile = NULL;	
	污染路径: 函数'taosCloseFile'的定义点.	
144	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 92
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 90: 91: if (pOperator->numOfDownstream > 1) { // not handle this in join query 92: qError("join not supported for stream block scan, %s" PRIx64, id); 93: return TSDB_CODE_APP_ERROR; 94: }	
	污染路径: 函数'qError'的定义点.	
145	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 227
	步骤描述: 不使用返回值的函数'SCL_ERR_RET'应使用'void'说明.	
	代码片段: 225: if (NULL == (*res)) { 226: sclError("malloc %d failed", pNode->node.resType.bytes); 227: SCL_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 228: } 229:	
	污染路径: 函数'SCL_ERR_RET'的定义点.	
146	缺陷发生位置: /source/os/src/osTimezone.c	行号: 819
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段:	

	817: *tsTimezone = tz; 818: tz += daylight; 819: sprintf(outTimezoneStr, "%s (%s, %s%02d00)", buf, tzname[daylight], tz >= 0 ? "+" : "-", abs(tz)); 820: *outDaylight = daylight; 821:	
	污染路径: 无	
147	缺陷发生位置: /source/client/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 76
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 74: LOAD_FP(Windows_run, Windows_runFP, "Windows_run"); 75: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP");	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
148	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 63
	步骤描述: 不使用返回值的函数'CTG_RET'应使用'void'说明.	
	代码片段: 61: *pInfo = DbOut.dbVgroup; 62: 63: CTG_RET(code); 64: } 65:	
	污染路径: 函数'CTG_RET'的定义点.	
149	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 105

	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: <pre> 103: if (fname[i]) { 104: code = tsdbOpenFile(fname[i], config->szPage, TD_FILE_READ, &reader[0]->fd[i]); 105: TSDB_CHECK_CODE(code, lino, _exit); 106: } 107: } </pre>	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
150	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 187
	步骤描述: 不使用返回值的函数'CTG_ERR_JRET'应使用'void'说明.	
	代码片段: <pre> 185: code = ctgUpdateTbMetaEnqueue(pCtg, output, syncReq); 186: output = NULL; 187: CTG_ERR_JRET(code); 188: 189: return TSDB_CODE_SUCCESS; </pre>	
	污染路径: 函数'CTG_ERR_JRET'的定义点.	
151	缺陷发生位置: /source/libs/stream/src/streamDispatch.c	行号: 612
	步骤描述: 不使用返回值的函数'tmsgSendRsp'应使用'void'说明.	
	代码片段: <pre> 610: SStreamChkptReadyInfo* pInfo = taosArrayGet(pTask->pReadyMsgList, 0); 611: 612: tmsgSendRsp(&pInfo->msg); 613: 614: taosArrayClear(pTask->pReadyMsgList); </pre>	
	污染路径: 函数'tmsgSendRsp'的定义点.	
152	缺陷发生位置: /source/util/src/thash.c	行号: 191
	步骤描述:	

	不使用返回值的函数'atomic_sub_fetch_16'应使用'void'说明.	
	代码片段: <pre> 189: static FORCE_INLINE void doUpdateHashNode(SHashObj *pHashObj, SHashEntry *pe, SHashNode *prev, SHashNode *pNode, 190: SHashNode *pNewNode) { 191: atomic_sub_fetch_16(&pNode->refCount, 1); 192: if (prev != NULL) { 193: prev->next = pNewNode; </pre>	
	污染路径: 函数'atomic_sub_fetch_16'的定义点.	
153	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 269
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 267: 268: for (int iData = 1; iData <= nData; iData++) { 269: sprintf(key, "key%d", iData); 270: sprintf(val, "value%d", iData); 271: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn); </pre>	
	污染路径: 无	
154	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 108
	步骤描述: 不使用返回值的函数'shellAutoInit'应使用'void'说明.	
	代码片段: <pre> 106: 107: // support port feature 108: shellAutoInit(); 109: int32_t ret = shellExecute(); 110: shellAutoExit(); </pre>	
	污染路径: 函数'shellAutoInit'的定义点.	
155	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 54
	步骤描述: 不使用返回值的函数'taosMkDir'应使用'void'说明.	

	代码片段: 52: strcpy(tsLogDir, logDir); 53: taosRemoveDir(tsLogDir); 54: taosMkDir(tsLogDir); 55: 56: if (taosInitLog(defaultLogFileNamePrefix, maxLogFileNum) < 0) {	
	污染路径: 函数'taosMkDir'的定义点.	
156	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 115
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 113: SColumnInfoData *pColumn = (SColumnInfoData *)taosArrayGetLast(res->pDataBlock); 114: for (int32_t i = 0; i < rowNum; ++i) { 115: colDataSetVal(pColumn, i, (const char *)value, false); 116: if (IS_VAR_DATA_TYPE(dataType)) { 117: value = (char *)value + varDataTLen(value);	
	污染路径: 函数'colDataSetVal'的定义点.	
157	缺陷发生位置: /utils/tsim/src/simMain.c	行号: 52
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 50: } 51: 52: simInfo("simulator is running ..."); 53: taosSetSignal(SIGINT, simHandleSignal); 54:	
	污染路径: 函数'simInfo'的定义点.	
158	缺陷发生位置: /source/common/src/tdatablock.c	行号: 717
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 715: int32_t blockDataFromBuf(SSDataBlock* pBlock, const	

	char* buf) { 716: int32_t numOfRows = *(int32_t*)buf; 717: blockDataEnsureCapacity(pBlock, numOfRows); 718: 719: pBlock->info.rows = numOfRows;	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
159	缺陷发生位置: /source/common/src/tglobal.c	行号: 888
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 886: tstrncpy(tsLocalFqdn, cfgGetItem(pCfg, "fqdn")->str, TSDB_FQDN_LEN); 887: tsServerPort = (uint16_t)cfgGetItem(pCfg, "serverPort")->i32; 888: snprintf(tsLocalEp, sizeof(tsLocalEp), "%s:%u", tsLocalFqdn, tsServerPort); 889: 890: char defaultFirstEp[TSDB_EP_LEN] = {0};	
	污染路径: 无	
160	缺陷发生位置: /utls/tsim/src/simMain.c	行号: 24
	步骤描述: 不使用返回值的函数'simSystemCleanUp'应使用'void'说明.	
	代码片段: 22: 23: void simHandleSignal(int32_t signo, void *sigInfo, void *context) { 24: simSystemCleanUp(); 25: abortExecution = true; 26: }	
	污染路径: 函数'simSystemCleanUp'的定义点.	
161	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 482
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段:	

	<pre> 480: if (!osTempSpaceAvailable()) { 481: code = TSDB_CODE_NO_DISKSPACE; 482: qError("Init stream agg supporter failed since %s, key: %s, tempDir:%s", terrstr(code), pKey, tsTempDir); 483: return code; 484: }</pre>	
	污染路径: 函数'qError'的定义点.	
162	缺陷发生位置: /tests/script/api/stmt_function.c	行号: 74
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 72: // stmt = taos_stmt_init(taos); 73: assert(stmt != NULL); 74: sprintf(stmt_sql, "select from ?"); 75: assert(taos_stmt_prepare(stmt, stmt_sql, 0) != 0); 76: assert(taos_stmt_close(stmt) == 0);</pre>	
	污染路径: 无	
163	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 48
	步骤描述: 不使用返回值的函数'QW_TASK_DLOG_E'应使用'void'说明.	
	代码片段: <pre> 46: if (QW_QUERY_RUNNING(ctx)) { 47: qwKillTaskHandle(ctx, TSDB_CODE_VND_STOPPED); 48: QW_TASK_DLOG_E("task running, async killed"); 49: } else if (QW_FETCH_RUNNING(ctx)) { 50: QW_UPDATE_RSP_CODE(ctx, TSDB_CODE_VND_STOPPED);</pre>	
	污染路径: 函数'QW_TASK_DLOG_E'的定义点.	
164	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 89
	步骤描述: 不使用返回值的函数'SCL_ERR_RET'应使用'void'说明.	
	代码片段: <pre> 87: if (NULL == pLeft) { 88: sclError("calloc %d failed",</pre>	

	<pre> (int32_t)sizeof(SScalarParam)); 89: SCL_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 90: } 91: </pre>	
	污染路径: 函数'SCL_ERR_RET'的定义点.	
165	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 140
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 138: for (int32_t i = 0; i < 10000; i += 20) { 139: char sql[1024] = {0}; 140: sprintf(sql, 141: "insert into tu%d values(now+%da, %d)(now+ %da, %d)(now+%da, %d)(now+%da, %d)" 142: "(now+%da, %d)(now+%da, %d)(now+%da, %d) (now+%da, %d)(now+%da, %d)(now+%da, %d)" </pre>	
	污染路径: 无	
166	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 191
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 189: tmsgSendRsp(&rpcRsp); 190: 191: qDebug("response %s msg, code: %s", TMSG_INFO(rspType), tstrerror(code)); 192: 193: return TSDB_CODE_SUCCESS; </pre>	
	污染路径: 函数'qDebug'的定义点.	
167	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 358
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 356: 357: if (mndTransAppendRedoAction(pTrans, &action) != 0) { </pre>	

	358: taosMemoryFree(pReq); 359: return -1; 360: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
168	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 270
	步骤描述: 不使用返回值的函数'metaInfo'应使用'void'说明.	
	代码片段: 268: } 269: 270: metaInfo("ttlMgr convert end."); 271: return ret; 272: }	
	污染路径: 函数'metaInfo'的定义点.	
169	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 19
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 17: (SRetrieveTableRsp *) (rpcMalloc ? rpcReallocCont(*rsp, msgSize) : taosMemoryRealloc(*rsp, msgSize)); 18: if (NULL == pRsp) { 19: qError("rpcMallocCont %d failed", msgSize); 20: QW_RET(TSDB_CODE_OUT_OF_MEMORY); 21: }	
	污染路径: 函数'qError'的定义点.	
170	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 211
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 209: ASSERT(fobj->ref > 0 && fobj->state == TSDB_FSTATE_LIVE); 210: nRef = ++fobj->ref; 211: taosThreadMutexUnlock(&fobj->mutex); 212: tsdbTrace("ref file %s, fobj:%p ref %d", fobj->fname,	

	fobj, nRef); 213: return 0;	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
171	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 586
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 584: pTrans->pRpcArray == NULL) { 585: terrno = TSDB_CODE_OUT_OF_MEMORY; 586: mError("failed to create transaction since %s", terrstr()); 587: mndTransDrop(pTrans); 588: return NULL;	
	污染路径: 函数'mError'的定义点.	
172	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 156
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 154: 155: if (tp == &vnodeGlobal.tp[0]) { 156: setThreadName("vnode-commit"); 157: } else if (tp == &vnodeGlobal.tp[1]) { 158: setThreadName("vnode-merge");	
	污染路径: 函数'setThreadName'的定义点.	
173	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 327
	步骤描述: 不使用返回值的函数'tSerializeSDCreateMnodeReq'应使用'void'说明.	
	代码片段: 325: int32_t contLen = tSerializeSDCreateMnodeReq(NULL, 0, pAlterMnodeTypeReq); 326: void *pReq = taosMemoryMalloc(contLen); 327: tSerializeSDCreateMnodeReq(pReq, contLen, pAlterMnodeTypeReq); 328:	

	329: STransAction action = {	
	污染路径: 函数'tSerializeSDCreateMnodeReq'的定义点.	
174	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 89
	步骤描述: 不使用返回值的函数'recoverSnapshot'应使用'void'说明.	
	代码片段: 87: pFileState->id = taosStrdup(taskId); 88: 89: recoverSnapshot(pFileState, checkpointId); 90: return pFileState; 91:	
	污染路径: 函数'recoverSnapshot'的定义点.	
175	缺陷发生位置: /utils/test/c/sml_test.c	行号: 1295
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 1293: memset(tmp, 'a', 26374); 1294: char sql[102400] = {0}; 1295: sprintf(sql,"lujixfvqor,t0=t c0=f,c1=\"%s\",c2=\"%s\",c3=\"%s\",c4=\"wthvqxcsrpls\" 1626006833639000000", tmp, tmp, tmp); 1296: 1297: pRes = taos_query(taos, "use line_23881");	
	污染路径: 无	
176	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 69
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 67: tDecoderInit(&decoder, pMemBuf, size); 68: if (tDecodeSTqOffset(&decoder, &offset) < 0) { 69: taosMemoryFree(pMemBuf); 70: tDecoderClear(&decoder); 71: return code;	

	污染路径: 函数'taosMemoryFree'的定义点.	
177	缺陷发生位置: /source/libs/parser/src/parTokenizer.c	行号: 315
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: <pre> 313: keywordTable[i].len = (uint8_t)strlen(keywordTable[i].name); 314: void* ptr = &keywordTable[i]; 315: taosHashPut(keywordHashTable, keywordTable[i].name, keywordTable[i].len, (void*)&ptr, POINTER_BYTES); 316: } 317: } </pre>	
	污染路径: 函数'taosHashPut'的定义点.	
178	缺陷发生位置: /source/util/src/tcompare.c	行号: 1195
	步骤描述: 不使用返回值的函数'regerror'应使用'void'说明.	
	代码片段: <pre> 1193: int32_t cflags = REG_EXTENDED; 1194: if ((ret = regcomp(&regex, pPattern, cflags)) != 0) { 1195: regerror(ret, &regex, msgbuf, tListLen(msgbuf)); 1196: 1197: uError("Failed to compile regex pattern %s. reason %s", pPattern, msgbuf); </pre>	
	污染路径: 无	
179	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 117
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: <pre> 115: } 116: 117: taosCloseFile(&fp); 118: 119: _exit: </pre>	

	污染路径: 函数'taosCloseFile'的定义点.	
180	缺陷发生位置: /source/util/src/tlog.c	行号: 129
	步骤描述: 不使用返回值的函数'taosGetTimeOfDay'应使用'void'说明.	
	代码片段: 127: struct tm Tm, *ptm; 128: struct timeval timeSecs; 129: taosGetTimeOfDay(&timeSecs); 130: time_t curTime = timeSecs.tv_sec; 131: ptm = taosLocalTime(&curTime, &Tm, NULL);	
	污染路径: 函数'taosGetTimeOfDay'的定义点.	
181	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 641
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 639: } 640: 641: taosThreadMutexLock(&tiresMutex); 642: tires[type] = tire; 643: taosThreadMutexUnlock(&tiresMutex);	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
182	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10001.c	行号: 67
	步骤描述: 不使用返回值的函数'printRaftState'应使用'void'说明.	
	代码片段: 65: struct raft *r = getRaft(&raftEnv, 100); 66: assert(r != NULL); 67: printRaftState(r); 68: 69: // submit value	
	污染路径: 函数'printRaftState'的定义点.	
183	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 30

	步骤描述: 不使用返回值的函数'sscanf'应使用'void'说明.	
	代码片段: 28: token = strtok_r(NULL, separator, &context); 29: if (token) { 30: sscanf(token, "%u", port); 31: } 32:	
	污染路径: 无	
184	缺陷发生位置:	行号:
	/source/libs/executor/src/groupcacheoperator.c	204
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.	
	代码片段: 202: taosThreadMutexInit(&pFileFd->mutex, NULL); 203: 204: qTrace("file path %s created", filename); 205: 206: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'qTrace'的定义点.	
185	缺陷发生位置:	行号:
	/contrib/test/craft/simulate_vnode.c	204
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir); 205: system(cmd); 206:	
	污染路径: 无	
186	缺陷发生位置:	行号:
	/source/dnode/vnode/src/meta/metaCache.c	538
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 536: STagFilterResEntry** pEntry = taosHashGet(pTableMap,	

	<pre> &suid, sizeof(uint64_t)); 537: if (NULL == pEntry) { 538: metaError("meta/cache: pEntry should not be NULL."); 539: return TSDB_CODE_FAILED; 540: }</pre>	
	污染路径: 函数'metaError'的定义点.	
187	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 366
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: <pre> 364: } 365: 366: blockDataDestroy(*ppBlock); 367: *ppBlock = NULL; 368: }</pre>	
	污染路径: 函数'blockDataDestroy'的定义点.	
188	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 99
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 97: pData->dnodeEps = taosArrayInit(1, sizeof(SDnodeEp)); 98: if (pData->dnodeEps == NULL) { 99: dError("failed to calloc dnodeEp array since %s", strerror(errno)); 100: goto _OVER; 101: }</pre>	
	污染路径: 函数'dError'的定义点.	
189	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 97
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 95: 96: char* fullname = taosMemoryCalloc(1, strlen(path) + 32);</pre>	

	97: sprintf(fullname, "%s%s%s", path, TD_DIRSEP, name); 98: 99: ret = taosStatFile(fullname, sz, NULL, NULL);	
	污染路径: 无	
190	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 212
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 210: char subdir[1024]; 211: char* fileName = taosGetDirEntryName(pDirEntry); 212: sprintf(subdir, "%s/%s", dir, fileName); 213: 214: // printf("===d_name: %s\n", entry->d_name);	
	污染路径: 无	
191	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 124
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 122: } 123: SSlotIdInfo info = {.slotId = slotId, .set = false}; 124: taosArrayPush(index.pSlotIdsInfo, &info); 125: return taosHashPut(pHash, pName, len, &index, sizeof(SSlotIndex)); 126: }	
	污染路径: 函数'taosArrayPush'的定义点.	
192	缺陷发生位置: /tests/script/api/apitest.c	行号: 101
	步骤描述: 不使用返回值的函数'taos_print_row'应使用'void'说明.	
	代码片段: 99: while ((row = taos_fetch_row(res))) { 100: char temp[256] = {0}; 101: taos_print_row(temp, row, fields, num_fields); 102: puts(temp); 103: nRows++;	

	污染路径: 函数'taos_print_row'的定义点.	
193	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 117
	步骤描述: 不使用返回值的函数'tqCleanUp'应使用'void'说明.	
	代码片段: <pre> 115: 116: walCleanUp(); 117: tqCleanUp(); 118: smaCleanUp(); 119: s3CleanUp(); </pre>	
	污染路径: 函数'tqCleanUp'的定义点.	
194	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmHandle.c	行号: 63
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 61: if (pInput->pData->dnodeId != 0 && dropReq.dnodeId != pInput->pData->dnodeId) { 62: terrno = TSDB_CODE_INVALID_OPTION; 63: dError("failed to drop qnode since %s", terrstr()); 64: return -1; 65: } </pre>	
	污染路径: 函数'dError'的定义点.	
195	缺陷发生位置: /source/client/src/clientMain.c	行号: 53
	步骤描述: 不使用返回值的函数'tscDebug'应使用'void'说明.	
	代码片段: <pre> 51: // this function may be called by user or system, or by both simultaneously. 52: void taos_cleanup(void) { 53: tscDebug("start to cleanup client environment"); 54: if (atomic_val_compare_exchange_32(&sentinel, TSC_VAR_NOT_RELEASE, TSC_VAR_RELEASED) != TSC_VAR_NOT_RELEASE) { 55: return; </pre>	

	污染路径: 函数'tscDebug'的定义点.	
196	缺陷发生位置: /source/client/src/clientSml.c	行号: 289
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 287: meta->tags = taosArrayInit(32, sizeof(SSmlKv)); 288: if (meta->tags == NULL) { 289: uError("SML:smlBuildSTableMeta failed to allocate memory"); 290: goto cleanup; 291: }	
	污染路径: 函数'uError'的定义点.	
197	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 118
	步骤描述: 不使用返回值的函数'tscDebug'应使用'void'说明.	
	代码片段: 116: SEp* pOrigEp = &pOrig->eps[pOrig->inUse]; 117: SEp* pNewEp = &connectRsp.epSet.eps[connectRsp.epSet.inUse]; 118: tscDebug("mnode epset updated from %d/%d=>%s: %d to %d/%d=>%s: %d in connRsp", pOrig->inUse, pOrig->numOfEps, 119: pOrigEp->fqdn, pOrigEp->port, connectRsp.epSet.inUse, connectRsp.epSet.numOfEps, pNewEp->fqdn, 120: pNewEp->port);	
	污染路径: 函数'tscDebug'的定义点.	
198	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 30
	步骤描述: 不使用返回值的函数'QW_GET_QTID'应使用'void'说明.	
	代码片段: 28: SQWTaskCtx *ctx = (SQWTaskCtx *)plter; 29: void *key = taosHashGetKey(plter, NULL); 30: QW_GET_QTID(key, qld, tld, eld);	

	31: 32: QW_LOCK(QW_WRITE, &ctx->lock);	
	污染路径: 函数'QW_GET_QTID'的定义点.	
199	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 135
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 133: 134: SColumnInfoData collInfo = createColumnInfoData(TSDB_DATA_TYPE_TIMESTAMP, sizeof(int64_t), 1); 135: blockDataAppendCollInfo(pInfo->pBlock, &collInfo); 136: 137: SColumnInfoData collInfo1 = createColumnInfoData(TSDB_DATA_TYPE_INT, 4, 2);	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
200	缺陷发生位置: /source/libs/scheduler/src/schFlowCtrl.c	行号: 104
	步骤描述: 不使用返回值的函数'SCH_RET'应使用'void'说明.	
	代码片段: 102: SCH_UNLOCK(SCH_WRITE, &ctrl->lock); 103: 104: SCH_RET(code); 105: } 106:	
	污染路径: 函数'SCH_RET'的定义点.	
201	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 182
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 180: for (int i = 1; i <= nData; i++) { 181: sprintf(key, "key%d", i); 182: sprintf(val, "value%d", i); 183:	

	184: ret = tdbTbGet(pDb, key, strlen(key), &pVal, &vLen);	
	污染路径: 无	
202	缺陷发生位置: /source/client/src/clientJniConnector.c	行号: 129
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段: 127: (*env)->DeleteLocalRef(env, consumerClass); 128: 129: atomic_store_32(&__init, 2); 130: jniDebug("native method register finished"); 131: }	
	污染路径: 函数'atomic_store_32'的定义点.	
203	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 23
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 21: SDnodeMgmt *pMgmt = param; 22: int64_t lastTime = taosGetTimestampMs(); 23: setThreadName("dnode-status"); 24: 25: const static int16_t TRIM_FREQ = 30;	
	污染路径: expanded from macro 'setThreadName'	
204	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 60
	步骤描述: 不使用返回值的函数'fnInfo'应使用'void'说明.	
	代码片段: 58: 59: void udfUdfdExit(uv_process_t *process, int64_t exitStatus, int termSignal) { 60: fnInfo("udfd process exited with status %" PRIu64 " signal %d", exitStatus, termSignal); 61: SUdfdData *pData = process->data; 62: if (exitStatus == 0 && termSignal == 0	

	atomic_load_32(&pData->stopCalled)) {	
	污染路径: 函数'fnInfo'的定义点.	
205	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 409
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 407: sprintf(total_string, "%s%s", shell.info.promptHeader, cmd->command); 408: } else { 409: sprintf(total_string, "%s%s", shell.info.promptContinue, cmd->command); 410: } 411: int32_t remain_column = ws_col;	
	污染路径: 无	
206	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 83
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 81: fdTo = taosOpenFile(fname, TD_FILE_WRITE TD_FILE_CREATE TD_FILE_TRUNC); 82: if (fdTo == NULL) code = terrno; 83: TSDB_CHECK_CODE(code, lino, _exit); 84: 85: int64_t n = taosFSendFile(fdTo, fdFrom, 0, tsdbLogicToFileSize(from->f->size, rtner->szPage));	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
207	缺陷发生位置: /source/dnode/vnode/src/sma/smaEnv.c	行号: 75
	步骤描述: 不使用返回值的函数'smaError'应使用'void'说明.	
	代码片段: 73: } 74: atomic_store_8(&smaMgmt.inited, 0); 75: smaError("failed to init sma tmr handle since %s", terrstr());	

	76: return TSDB_CODE_FAILED; 77: }	
	污染路径: 函数'smaError'的定义点.	
208	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 704
	步骤描述: 不使用返回值的函数'filterAddRangeOptr'应使用'void'说明.	
	代码片段: 702: bool all = false; 703: FREE_FROM_RANGE(ctx, ctx->rs); 704: filterAddRangeOptr(h, OP_TYPE_IS_NOT_NULL, optr, NULL, &all); 705: if (all) { 706: FILTER_SET_FLAG(ctx->status, MR_ST_ALL);	
	污染路径: 函数'filterAddRangeOptr'的定义点.	
209	缺陷发生位置: /source/os/src/osDir.c	行号: 303
	步骤描述: 不使用返回值的函数'rmdir'应使用'void'说明.	
	代码片段: 301: 302: taosCloseDir(&pDir); 303: rmdir(dirname); 304: } 305:	
	污染路径: 无	
210	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 47
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 45: void qwFreeFetchRsp(void *msg) { 46: if (msg) { 47: rpcFreeCont(msg); 48: } 49: }	

	污染路径: 函数'rpcFreeCont'的定义点.	
211	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffsetSnapshot.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 63: if (taosStatFile(fname, &sz, NULL, NULL) < 0) { 64: taosCloseFile(&pFile); 65: taosMemoryFree(fname); 66: return -1; 67: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
212	缺陷发生位置: /source/client/src/clientTmq.c	行号: 485
	步骤描述: 不使用返回值的函数'tEncodeMqVgOffset'应使用'void'说明.	
	代码片段: 483: SEncoder encoder; 484: tEncoderInit(&encoder, abuf, len); 485: tEncodeMqVgOffset(&encoder, &pOffset); 486: tEncoderClear(&encoder); 487:	
	污染路径: 函数'tEncodeMqVgOffset'的定义点.	
213	缺陷发生位置: /docs/examples/c/async_query_example.c	行号: 141
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 139: taos_free_result(res); 140: taos_close(_taos); 141: taos_cleanup(); 142: } 143: }	
	污染路径: 函数'taos_cleanup'的定义点.	
214	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 560

	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 558: if (pTrans == NULL) { 559: terrno = TSDB_CODE_OUT_OF_MEMORY; 560: mError("failed to create transaction since %s", terrstr()); 561: return NULL; 562: }</pre>	
	污染路径: 函数'mError'的定义点.	
215	缺陷发生位置: /source/common/src/trow.c	行号: 252
	步骤描述: 不使用返回值的函数'tdSTSRowIterGetKvVal'应使用'void'说明.	
	代码片段: <pre> 250: tdSTSRowIterGetTpVal(plter, pCol->type, pCol->offset, pVal); 251: } else if (TD_IS_KV_ROW(plter->pRow)) { 252: tdSTSRowIterGetKvVal(plter, pCol->colId, &plter- >kvIdx, pVal); 253: } else { 254: ASSERT(0);</pre>	
	污染路径: 函数'tdSTSRowIterGetKvVal'的定义点.	
216	缺陷发生位置: /contrib/test/cos/main.c	行号: 104
	步骤描述: 不使用返回值的函数'cos_get_hmac_sha1_hexdigest'应使用'void'说明.	
	代码片段: <pre> 102: fmt_str = cos_create_buf(p, 1024); 103: 104: cos_get_hmac_sha1_hexdigest(sign_key, secret_key, sizeof(secret_key) - 1, time_str, sizeof(time_str) - 1); 105: char *pstr = apr_pstrndup(p, (char *)sign_key, sizeof(sign_key)); 106: cos_warn_log("sign_key: %s", pstr);</pre>	
	污染路径: 函数'cos_get_hmac_sha1_hexdigest'的定义点.	
217	缺陷发生位置:	行号:

	/source/dnode/vnode/src/meta/metaTable.c	209
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: <pre> 207: if (tdbTbGet(pMeta->pNameldx, pReq->name, strlen(pReq->name) + 1, &pData, &nData) == 0) { 208: tb_uid_t uid = *(tb_uid_t *)pData; 209: tdbFree(pData); 210: SMetaInfo info; 211: if (metaGetInfo(pMeta, uid, &info, NULL) == TSDB_CODE_NOT_FOUND) { </pre>	
218	污染路径: 函数'tdbFree'的定义点.	
	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmFile.c	行号: 127
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
219	代码片段: <pre> 125: if (taosReadFile(pFile, pData, size) != size) { 126: terrno = TAOS_SYSTEM_ERROR(errno); 127: dError("failed to read vnode file:%s since %s", file, terrstr()); 128: goto _OVER; 129: } </pre>	
	污染路径: 函数'dError'的定义点.	
	缺陷发生位置: /source/util/src/tidpool.c	行号: 59
220	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: <pre> 57: } 58: 59: taosThreadMutexUnlock(&pIdPool->mutex); 60: return slot + 1; 61: } </pre>	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
220	缺陷发生位置: /source/dnode/mnode/impl/src/mndIndex.c	行号: 329

	步骤描述: 不使用返回值的函数'tNameGetFullDbName'应使用'void'说明.	
	代码片段: 327: 328: char db[TSDB_TABLE_FNAME_LEN] = {0}; 329: tNameGetFullDbName(&name, db); 330: 331: return mndAcquireDb(pMnode, db);	
	污染路径: 函数'tNameGetFullDbName'的定义点.	
221	缺陷发生位置: /source/util/src/tidpool.c	行号: 66
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 64: if (pIdPool == NULL) return; 65: 66: taosThreadMutexLock(&pIdPool->mutex); 67: 68: int32_t slot = (id - 1) % pIdPool->maxId;	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
222	缺陷发生位置: /docs/examples/c/telnet_line_example.c	行号: 50
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 48: taos_free_result(res); 49: taos_close(taos); 50: taos_cleanup(); 51: } 52: // output:	
	污染路径: 函数'taos_cleanup'的定义点.	
223	缺陷发生位置: /source/common/src/tdataformat.c	行号: 1482
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 1480: }	

	1481: tGetTagVal(p + offset, &tv, pTag->flags & TD_TAG_JSON); 1482: taosArrayPush(*ppArray, &tv); 1483: } 1484:	
	污染路径: 函数'taosArrayPush'的定义点.	
224	缺陷发生位置: /source/libs/parser/test/parInitialCTest.cpp	行号: 674
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 672: expect.pTags = taosArrayInit(TARRAY_MIN_SIZE, sizeof(SField)); 673: } 674: taosArrayPush(expect.pTags, &field); 675: expect.numOfTags += 1; 676: }	
	污染路径: 函数'taosArrayPush'的定义点.	
225	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 60
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 58: 59: if (pData->dropped) { 60: dError("dnode will not start since its already dropped"); 61: return -1; 62: }	
	污染路径: 函数'dError'的定义点.	
226	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 643
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 641: taosThreadMutexLock(&tiresMutex); 642: tires[type] = tire; 643: taosThreadMutexUnlock(&tiresMutex);	

	644: } 645:	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
227	缺陷发生位置: /source/os/src/osString.c	行号: 162
	步骤描述: 不使用返回值的函数'iconv_close'应使用'void'说明.	
	代码片段: 160: } 161: for (int32_t i = 0; i < gConvMaxNum[1 - M2C]; ++i) { 162: iconv_close(gConv[1 - M2C][i].conv); 163: } 164: taosMemoryFreeClear(gConv[M2C]);	
	污染路径: 无	
228	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 394
	步骤描述: 不使用返回值的函数'taosCreateMD5Hash'应使用'void'说明.	
	代码片段: 392: char name[TSDB_COL_NAME_LEN + TSDB_POINTER_PRINT_BYTES + TSDB_NAME_DELIMITER_LEN + 1] = {0}; 393: int32_t len = snprintf(name, sizeof(name) - 1, "%s.%" PRId64 "", pWStart->functionName, pointer); 394: taosCreateMD5Hash(name, len); 395: strncpy(pWStart->node.aliasName, name, TSDB_COL_NAME_LEN - 1); 396:	
	污染路径: 函数'taosCreateMD5Hash'的定义点.	
229	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 42
	步骤描述: 不使用返回值的函数'vnodeApplyWMsg'应使用'void'说明.	
	代码片段: 40: 41: SVnode* tmp_vnodes = (SVnode*)(fsm->data); 42: vnodeApplyWMsg(&tmp_vnodes[vid], value, NULL);	

	43: 44: return 0;	
	污染路径: 函数'vnodeApplyWMsg'的定义点.	
230	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 222
	步骤描述: 不使用返回值的函数'smlStrReplace'应使用'void'说明.	
	代码片段: 220: memcpy(measure, currElement->measure, currElement->measureLen); 221: PROCESS_SLASH_IN_MEASUREMENT(measure, measureLen); 222: smlStrReplace(measure, measureLen); 223: } 224: STableMeta *pTableMeta = smlGetMeta(info, measure, measureLen);	
	污染路径: 函数'smlStrReplace'的定义点.	
231	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 45
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 43: if (pOperator->operatorType != QUERY_NODE_PHYSICAL_PLAN_STREAM_SCAN) { 44: if (pOperator->numOfDownstream == 0) { 45: qError("failed to find stream scan operator to set the input data block, %s" PRIx64, id); 46: return TSDB_CODE_APP_ERROR; 47: }	
	污染路径: 函数'qError'的定义点.	
232	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 340
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 338: createReq.changeVersion = pVgroup->syncConfChangeVer;	

	339: 340: mInfo("vgld:%d, build create vnode req, replica:%d selfIndex:%d learnerReplica:%d learnerSelfIndex:%d strict:%d " 341: "changeVersion:%d", 342: createReq.vgld, createReq.replica, createReq.selfIndex, createReq.learnerReplica,	
	污染路径: 函数'mInfo'的定义点.	
233	缺陷发生位置: /source/os/src/osSocket.c	行号: 758
	步骤描述: 不使用返回值的函数'taosCloseSocketNoCheck1'应使用'void'说明.	
	代码片段: 756: TdSocketPtr pSocket = (TdSocketPtr)taosMemoryMalloc(sizeof(TdSocket)); 757: if (pSocket == NULL) { 758: taosCloseSocketNoCheck1(fd); 759: return false; 760: }	
	污染路径: 函数'taosCloseSocketNoCheck1'的定义点.	
234	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 136
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 134: snprintf(tsProcMemFile, sizeof(tsProcMemFile), "/proc/%d/status", tsProclId); 135: snprintf(tsProcCpuFile, sizeof(tsProcCpuFile), "/proc/%d/stat", tsProclId); 136: snprintf(tsProcIOFile, sizeof(tsProcIOFile), "/proc/%d/io", tsProclId); 137: } 138: #endif	
	污染路径: 无	
235	缺陷发生位置: /source/util/src/tqueue.c	行号: 70
	步骤描述:	

	不使用返回值的函数'taosThreadMutexDestroy'应使用'void'说明.	
	代码片段: 68: } 69: 70: taosThreadMutexDestroy(&queue->mutex); 71: taosMemoryFree(queue); 72:	
	污染路径: 函数'taosThreadMutexDestroy'的定义点.	
236	缺陷发生位置: /source/util/src/tsched.c	行号: 123
	步骤描述: 不使用返回值的函数'taosTmrReset'应使用'void'说明.	
	代码片段: 121: if (tmrCtrl != NULL && pSched != NULL) { 122: pSched->pTmrCtrl = tmrCtrl; 123: taosTmrReset(taosDumpSchedulerStatus, DUMP_SCHEDULER_TIME_WINDOW, pSched, pSched->pTmrCtrl, &pSched->pTimer); 124: } 125:	
	污染路径: 函数'taosTmrReset'的定义点.	
237	缺陷发生位置: /source/os/src/osTimer.c	行号: 99
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 97: struct sigevent sevent = {{0}}; 98: 99: setThreadName("tmr"); 100: 101: #ifdef _ALPINE	
	污染路径: expanded from macro 'setThreadName'	
238	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 106
	步骤描述: 不使用返回值的函数'QW_GET_QTID'应使用'void'说明.	
	代码片段:	

	104: pTask = (SQWTaskStatus *)plter; 105: void *key = taosHashGetKey(plter, NULL); 106: QW_GET_QTID(key, qld, tld, eld); 107: 108: QW_TASK_DLOG("job refld:%" PRIx64 ", code:%x, task status:%d", pTask->refld, pTask->code, pTask->status);	
	污染路径: 函数'QW_GET_QTID'的定义点.	
239	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 198
	步骤描述: 不使用返回值的函数'taosThreadMutexInit'应使用'void'说明.	
	代码片段: 196: if (!fobj[0]) return TSDB_CODE_OUT_OF_MEMORY; 197: 198: taosThreadMutexInit(&fobj[0]->mutex, NULL); 199: fobj[0]->f[0] = f[0]; 200: fobj[0]->state = TSDB_FSTATE_LIVE;	
	污染路径: 函数'taosThreadMutexInit'的定义点.	
240	缺陷发生位置: /source/util/src/tref.c	行号: 66
	步骤描述: 不使用返回值的函数'taosThreadOnce'应使用'void'说明.	
	代码片段: 64: int32_t i, rsetId; 65: 66: taosThreadOnce(&tsRefModuleInit, taosInitRefModule); 67: 68: nodeList = taosMemoryCalloc(sizeof(SRefNode *), (size_t)max);	
	污染路径: 函数'taosThreadOnce'的定义点.	
241	缺陷发生位置: /source/dnode/mnode/impl/src/mndSma.c	行号: 261
	步骤描述: 不使用返回值的函数'tNameFromString'应使用'void'说明.	
	代码片段: 259: SDbObj *mndAcquireDbBySma(SMnode *pMnode, const char *smaName) {	

	260: SName name = {0}; 261: tNameFromString(&name, smaName, T_NAME_ACCT T_NAME_DB T_NAME_TABLE); 262: 263: char db[TSDB_TABLE_FNAME_LEN] = {0};	
	污染路径: 函数'tNameFromString'的定义点.	
242	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 158
	步骤描述: 不使用返回值的函数'ASSERTS'应使用'void'说明.	
	代码片段: 156: p->len = 0; 157: } else { 158: ASSERTS(0, "invalid read from sock buf"); 159: return -1; 160: }	
	污染路径: 函数'ASSERTS'的定义点.	
243	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmHandle.c	行号: 41
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 39: if (pInput->pData->dnodeId != 0 && createReq.dnodeId != pInput->pData->dnodeId) { 40: terrno = TSDB_CODE_INVALID_OPTION; 41: dError("failed to create qnode since %s", terrstr()); 42: return -1; 43: }	
	污染路径: 函数'dError'的定义点.	
244	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 25
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 23: char* token = strtok_r(tmp, separator, &context); 24: if (token) { 25: snprintf(host, len, "%s", token);	

	26: } 27:	
	污染路径: 无	
245	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 405
	步骤描述: 不使用返回值的函数'destroyGeometry'应使用'void'说明.	
	代码片段: 403: _exit; 404: destroyGeometry(&geom1, &preparedGeom1); 405: destroyGeometry(&geom2, NULL); 406: 407: return code;	
	污染路径: 函数'destroyGeometry'的定义点.	
246	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 324
	步骤描述: 不使用返回值的函数'tmsgSendRsp'应使用'void'说明.	
	代码片段: 322: SRpcMsg rspMsg = {.code = 0, .pCont = buf, .contLen = sizeof(SMsgHead) + len, .info = *pRpcInfo}; 323: 324: tmsgSendRsp(&rspMsg); 325: return 0; 326: }	
	污染路径: 函数'tmsgSendRsp'的定义点.	
247	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 120
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 118: : (blockNums < pDeleter->pManager- >cfg.maxDataBlockNumPerQuery ? DS_BUF_LOW : DS_BUF_FULL)); 119: pDeleter->status = status; 120: taosThreadMutexUnlock(&pDeleter->mutex); 121: return status;	

	122: }	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
248	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 89
	步骤描述: 不使用返回值的函数'operator='应使用'void'说明.	
	代码片段: 87: 88: SColumnInfoData idata = {0}; 89: idata.info = *collInfo; 90: collInfoDataEnsureCapacity(&idata, rows, true); 91:	
	污染路径: 函数'operator='的定义点.	
249	缺陷发生位置: /source/libs/index/src/index.c	行号: 199
	步骤描述: 不使用返回值的函数'taosReleaseRef'应使用'void'说明.	
	代码片段: 197: void idxReleaseRef(int64_t ref) { 198: // impl 199: taosReleaseRef(indexRefMgt, ref); 200: } 201:	
	污染路径: 函数'taosReleaseRef'的定义点.	
250	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 203
	步骤描述: 不使用返回值的函数'sem_post'应使用'void'说明.	
	代码片段: 201: taosWUnLockLatch(&qwtTestQueryQueueLock); 202: 203: tsem_post(&qwtTestQuerySem); 204: 205: return 0;	
	污染路径: expanded from macro 'tsem_post'	

251	缺陷发生位置: /source/client/src/clientJniConnector.c	行号: 130
	步骤描述: 不使用返回值的函数'jniDebug'应使用'void'说明.	
	代码片段: 128: 129: atomic_store_32(&__init, 2); 130: jniDebug("native method register finished"); 131: } 132:	
	污染路径: 函数'jniDebug'的定义点.	
252	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 55
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 53: pHead->comp = 0; 54: } 55: taosMemoryFree(buf); 56: return ret; 57: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
253	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 29
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 27: int32_t schedulerInit() { 28: if (schMgmt.jobRef >= 0) { 29: qError("scheduler already initialized"); 30: return TSDB_CODE_QRY_INVALID_INPUT; 31: }	
	污染路径: 函数'qError'的定义点.	
254	缺陷发生位置: /source/libs/stream/src/streamState.c	行号: 181
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	

	代码片段: 179: } 180: } 181: taosCloseFile(&pCfgFile); 182: 183: if (tdbOpen(statePath, szPage, pages, &pState->pTdbState->db, 1) < 0) {	
	污染路径: 函数'taosCloseFile'的定义点.	
255	缺陷发生位置: /contrib/test/traft/cluster/node10002.c	行号: 112
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 110: // for test: submit a value every second 111: while (1) { 112: sleep(1); 113: submitValue(); 114: }	
	污染路径: 无	
256	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 993
	步骤描述: 不使用返回值的函数'blockDataUpdateTsWindow'应使用'void'说明.	
	代码片段: 991: pInfo->binfo.pRes->info.scanFlag = pBlock->info.scanFlag; 992: setInputDataBlock(pSup, pBlock, order, MAIN_SCAN, true); 993: blockDataUpdateTsWindow(pBlock, pInfo->tsSlotId); 994: 995: // there is an scalar expression that needs to be calculated right before apply the group aggregation.	
	污染路径: 函数'blockDataUpdateTsWindow'的定义点.	
257	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 110
	步骤描述: 不使用返回值的函数'SCL_ERR_RET'应使用'void'说明.	

	代码片段: 108: if (NULL == pObj) { 109: sclError("taosHashInit failed, size:%d", 256); 110: SCL_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 111: } 112:	
	污染路径: 函数'SCL_ERR_RET'的定义点.	
258	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 39
	步骤描述: 不使用返回值的函数'SERIALIZE_STR_VAR_TO_BUF'应使用'void'说明.	
	代码片段: 37: static FORCE_INLINE void idxGenLRUKey(char* buf, const char* path, int32_t blockId) { 38: char* p = buf; 39: SERIALIZE_STR_VAR_TO_BUF(p, path, strlen(path)); 40: SERIALIZE_VAR_TO_BUF(p, '_', char); 41: idxInt2str(blockId, p, 0);	
	污染路径: 函数'SERIALIZE_STR_VAR_TO_BUF'的定义点.	
259	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 114
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 112: 113: static int32_t updateStatus(SDataDeleterHandle* pDeleter) { 114: taosThreadMutexLock(&pDeleter->mutex); 115: int32_t blockNums = taosQueueItemSize(pDeleter->pDataBlocks); 116: int32_t status =	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
260	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 135
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段:	

	133: if (tdbTbGet(pMeta->pNameldx, name, strlen(name) + 1, &pData, &nData) == 0) { 134: uid = *(tb_uid_t *)pData; 135: tdbFree(pData); 136: } 137:	
	污染路径: 函数'tdbFree'的定义点.	
261	缺陷发生位置: /utls/test/c/varbinary_test.c	行号: 642
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 640: } 641: cnt++; 642: tmq_free_json_meta(result); 643: } 644: taos_free_result(tmqmessage);	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
262	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 219
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 217: if (tEncodeSTqHandle(&encoder, pHandle) < 0) { 218: tEncoderClear(&encoder); 219: taosMemoryFree(buf); 220: return -1; 221: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
263	缺陷发生位置: /source/dnode/mnode/impl/src/mndTelem.c	行号: 77
	步骤描述: 不使用返回值的函数'tjsonAddDoubleToObject'应使用'void'说明.	
	代码片段: 75: tjsonAddDoubleToObject(pJson, "numOfVgroup", mstat.numOfVgroup); 76: tjsonAddDoubleToObject(pJson, "numOfDatabase",	

	mstat.numOfDatabase); 77: tjsonAddDoubleToObject(pJson, "numOfSuperTable", mstat.numOfSuperTable); 78: tjsonAddDoubleToObject(pJson, "numOfChildTable", mstat.numOfChildTable); 79: tjsonAddDoubleToObject(pJson, "numOfColumn", mstat.numOfColumnn);	
	污染路径: 函数'tjsonAddDoubleToObject'的定义点.	
264	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 45
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 43: code = tdbTbcOpen(pTq->pExecStore, &pReader->pCur, NULL); 44: if (code) { 45: taosMemoryFree(pReader); 46: goto _err; 47: } 	
	污染路径: 函数'taosMemoryFree'的定义点.	
265	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 392
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 390: } 391: 392: taosArrayPush(pDnodeInfo, &dInfo); 393: } 394: } 	
	污染路径: 函数'taosArrayPush'的定义点.	
266	缺陷发生位置: /source/dnode/mnode/impl/src/mndPerfSchema.c	行号: 103
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 101: STableMetaRsp *pMeta = taosHashGet(pMnode-	

	<pre>>perfsMeta, tbName, strlen(tbName)); 102: if (NULL == pMeta) { 103: mError("invalid performance schema table name:%s", tbName); 104: terrno = TSDB_CODE_MND_INVALID_SYS_TABLENAME; 105: return -1;</pre>	
	污染路径: 函数'mError'的定义点.	
267	缺陷发生位置: /testes/script/api/batchprepare.c	行号: 996
	步骤描述: 不使用返回值的函数'prepareColData'应使用'void'说明.	
	代码片段: <pre>994: for (int b = 0; b < (allRowNum/gCurCase- >bindRowNum); b++) { 995: for (int c = 0; c < gCurCase->bindColNum; ++c) { 996: prepareColData(BP_BIND_COL, data, b*gCurCase- >bindColNum+c, b*gCurCase->bindRowNum, c); 997: } 998: }</pre>	
	污染路径: 函数'prepareColData'的定义点.	
268	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 157
	步骤描述: 不使用返回值的函数'streamMetaInit'应使用'void'说明.	
	代码片段: <pre>155: 156: indexInit(tsNumOfCommitThreads); 157: streamMetaInit(); 158: 159: dmReportStartup("dnode-transport", "initialized");</pre>	
	污染路径: 函数'streamMetaInit'的定义点.	
269	缺陷发生位置: /source/libs/command/src/command.c	行号: 119
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre>117: }</pre>	

	118: if (pBlock->info.rows <= 0) { 119: qError("no permission to view any columns"); 120: return TSDB_CODE_PAR_PERMISSION_DENIED; 121: }	
	污染路径: 函数'qError'的定义点.	
270	缺陷发生位置: /source/util/src/tqueue.c	行号: 84
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 82: empty = true; 83: } 84: taosThreadMutexUnlock(&queue->mutex); 85: 86: return empty;	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
271	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffsetSnapshot.c	行号: 58
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 56: TdFilePtr pFile = taosOpenFile(fname, TD_FILE_READ); 57: if (pFile == NULL) { 58: taosMemoryFree(fname); 59: return 0; 60: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
272	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamStateSnap.c	行号: 79
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 77: tqDebug("vgId:%d, vnode %s snapshot reader closed", TD_VID(pReader->pTq->pVnode), STREAM_STATE_TRANSFER); 78: streamSnapReaderClose(pReader->pReaderImpl); 79: taosMemoryFree(pReader); 80: return code;	

	81: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
273	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 430
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 428: tAutoQWorkerCleanup(&pMgmt->streamPool); 429: tWWorkerCleanup(&pMgmt->fetchPool); 430: dDebug("vnode workers are closed"); 431: }	
	污染路径: 函数'dDebug'的定义点.	
274	缺陷发生位置: /utils/tsim/src/simExe.c	行号: 265
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 263: } 264: } else if (op2[0] == '.') { 265: snprintf(t3, sizeof(t3), "%s%s", t1, t2); 266: } 267: } else {	
	污染路径: 函数'snprintf'的定义点.	
275	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 129
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 127: printf("logStorePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 128: fflush(NULL); 129: taosMemoryFree(serialized); 130: } 131:	
	污染路径: 函数'taosMemoryFree'的定义点.	

276	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 220
	步骤描述: 不使用返回值的函数'kvDeserial'应使用'void'说明.	
	代码片段: 218: const char *value = rocksdb_iter_value(iter, &vlen); 219: KV kv; 220: kvDeserial(&kv, (char *)key); 221: printf("kv1: %d\t kv2: %d, len:%d, value = %s\n", (int)(kv.k1), (int)(kv.k2), (int)(klen), value); 222: }	
	污染路径: 函数'kvDeserial'的定义点.	
277	缺陷发生位置: /source/libs/executor/src/projectoperator.c	行号: 339
	步骤描述: 不使用返回值的函数'blockDataMerge'应使用'void'说明.	
	代码片段: 337: 338: // continue merge data, ignore the group id 339: blockDataMerge(pFinalRes, pRes); 340: if (pFinalRes->info.rows + pRes->info.rows <= pOperator->resultInfo.threshold) { 341: continue;	
	污染路径: 函数'blockDataMerge'的定义点.	
278	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 126
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 124: return -1; 125: } 126: tmsgReportStartup("mnode-worker", "initialized"); 127: 128: if (option.numOfTotalReplicas > 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
279	缺陷发生位置:	行号:

	/source/libs/catalog/src/ctgUtil.c		869
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.		
	代码片段: 867: pCtx->target = taosStrdup(target); 868: if (NULL == pCtx->target) { 869: CTG_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 870: } 871: } else {		
	污染路径: 函数'CTG_ERR_RET'的定义点.		
280	缺陷发生位置:		行号:
	/source/libs/index/src/indexCache.c		337
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.		
	代码片段: 335: IndexCache* cache = taosMemoryCalloc(1, sizeof(IndexCache)); 336: if (cache == NULL) { 337: indexError("failed to create index cache"); 338: return NULL; 339: };		
	污染路径: 函数'indexError'的定义点.		
281	缺陷发生位置:		行号:
	/contrib/test/traft/join_into_vgroup/node_follower10001.c		108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.		
	代码片段: 106: // for test: submit value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }		
	污染路径: 无		
282	缺陷发生位置:		行号:
	/source/dnode/vnode/src/tsdb/tsdbRead2.c		295
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.		

	代码片段: 293: static int32_t tsdbInitReaderLock(STsdbReader* pReader) { 294: int32_t code = -1; 295: qTrace("tsdb/read: %p, pre-init read mutex: %p, code: %d", pReader, &pReader->readerMutex, code); 296: 297: code = taosThreadMutexInit(&pReader->readerMutex, NULL);	
	污染路径: 函数'qTrace'的定义点.	
283	缺陷发生位置: /source/util/src/tlog.c	行号: 152
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 150: snprintf(logFileName, 64, "%sSlowLog", CUS_PROMPT); 151: #else 152: snprintf(logFileName, 64, "taosSlowLog"); 153: #endif 154:	
	污染路径: 无	
284	缺陷发生位置: /tests/script/api/stmt_function.c	行号: 86
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 84: stmt = taos_stmt_init(taos); 85: assert(stmt != NULL); 86: sprintf(stmt_sql, "insert into super values (?,?,?,?,?,?,?,?,?,?)"); 87: assert(taos_stmt_prepare(stmt, stmt_sql, 0) != 0); 88: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
285	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 134
	步骤描述: 不使用返回值的函数'tdListFreeP'应使用'void'说明.	

	代码片段: 132: 133: taosMemoryFree(pFileState->id); 134: tdListFreeP(pFileState->usedBufs, destroyRowBuffAllPosPtr); 135: tdListFreeP(pFileState->freeBufs, destroyRowBuff); 136: tSimpleHashCleanup(pFileState->rowBuffMap);	
	污染路径: 函数'tdListFreeP'的定义点.	
286	缺陷发生位置: /source/libs/function/src/tpcentile.c	行号: 401
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 399: } 400: pSlot->info.pageld = pageld; 401: taosArrayPush(pPageldList, &pageld); 402: } 403:	
	污染路径: 函数'taosArrayPush'的定义点.	
287	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 233
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 231: printf("syncPingPrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 232: fflush(NULL); 233: taosMemoryFree(serialized); 234: } 235:	
	污染路径: 函数'taosMemoryFree'的定义点.	
288	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_follower10000.c	行号: 67
	步骤描述: 不使用返回值的函数'printRaftState'应使用'void'说明.	
	代码片段: 65: struct raft *r = getRaft(&raftEnv, 100);	

	66: assert(r != NULL); 67: printRaftState(r); 68: 69: // submit value	
	污染路径: 函数'printRaftState'的定义点.	
289	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 77
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 75: char *serialized = raftStore2Str(pObj); 76: sTrace("raftStoreLog len:%d %s", (int32_t)strlen(serialized), serialized); 77: taosMemoryFree(serialized); 78: } 79:	
	污染路径: 函数'taosMemoryFree'的定义点.	
290	缺陷发生位置: /source/client/src/clientMsgHandler.c	行号: 90
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 88: if (delta > timestampDeltaLimit) { 89: code = TSDB_CODE_TIME_UNSYNCED; 90: tscError("time diff:%ds is too big", delta); 91: setErrno(pRequest, code); 92: tsem_post(&pRequest->body.rspSem);	
	污染路径: 函数'tscError'的定义点.	
291	缺陷发生位置: /source/util/src/tmempool.c	行号: 116
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 114: pool_p->numOfFree++; 115: 116: taosThreadMutexUnlock(&pool_p->mutex); 117: }	

	118:	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
292	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamTaskSnap.c	行号: 218
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 216: taosWUnLockLatch(&pTq->pStreamMeta->lock); 217: 218: taosMemoryFree(pWriter); 219: return code; 220:	
	污染路径: 函数'taosMemoryFree'的定义点.	
293	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 371
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 369: for (i = 0; i < 100; i++) { 370: char newStr[100]; 371: sprintf(newStr, "%s-%d", ranStr, i); 372: int len = strlen(newStr); 373: code = walWrite(pWal, i, 0, newStr, len);	
	污染路径: 无	
294	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 98
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 96: 97: for (int32_t i = 0; i < rtTables; ++i) { 98: sprintf(sql, "create table tb%d (ts timestamp, f1 int)", i); 99: rtExecSQL(taos, sql); 100: }	
	污染路径: 无	

295	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 92
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.	
	代码片段: 90: } 91: 92: CTG_ERR_RET(code); 93: } 94:	
	污染路径: 函数'CTG_ERR_RET'的定义点.	
296	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 227
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 225: pConn->status = ConnAcquire; 226: transRefSrvHandle(pConn); 227: tDebug("conn %p acquired by server app", pConn); 228: } 229: }	
	污染路径: 函数'tDebug'的定义点.	
297	缺陷发生位置: /include/common/tmsg.h	行号: 2327
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 2325: char* name; 2326: buf = taosDecodeString(buf, &name); 2327: taosArrayPush(pReq->topicNames, &name); 2328: } 2329:	
	污染路径: 函数'taosArrayPush'的定义点.	
298	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmWorker.c	行号: 135
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	

	代码片段: 133: tSingleWorkerCleanup(&pMgmt->queryWorker); 134: tSingleWorkerCleanup(&pMgmt->fetchWorker); 135: dDebug("qnode workers are closed"); 136: }	
	污染路径: 函数'dDebug'的定义点.	
299	缺陷发生位置: /source/util/src/tarray.c	行号: 413
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 411: } 412: 413: taosArrayDestroy(pArray); 414: } 415:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
300	缺陷发生位置: /source/dnode/vnode/src/sma/smaTimeRange.c	行号: 340
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 338: 339: if (code) { 340: rpcFreeCont(pBuf); 341: taosArrayDestroy(pDeleteReq->deleteReqs); 342: smaWarn("vgId:%d, failed at line %d since %s", TD_VID(pVnode), lino, tsterror(code));	
	污染路径: 函数'rpcFreeCont'的定义点.	
301	缺陷发生位置: /source/client/src/clientTmq.c	行号: 564
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 562: SMqClientTopic* pTopic = getTopicByName(tmq, pTopicName); 563: if (pTopic == NULL) {	

	564: tscError("consumer:0x%" PRIx64 " invalid topic name: %s", tmq->consumerId, pTopicName); 565: return TSDB_CODE_TMQ_INVALID_TOPIC; 566: }	
	污染路径: 函数'tscError'的定义点.	
302	缺陷发生位置: /utls/tsim/src/simSystem.c	行号: 42
	步骤描述: 不使用返回值的函数'simInitsimCmdList'应使用'void'说明.	
	代码片段: 40: bool simSystemInit() { 41: simInitCfg(); 42: simInitsimCmdList(); 43: memset(simScriptList, 0, sizeof(SScript *) * MAX_MAIN_SCRIPT_NUM); 44: return true;	
	污染路径: 函数'simInitsimCmdList'的定义点.	
303	缺陷发生位置: /tools/shell/src/shellWebsocket.c	行号: 69
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 67: if(curDBName[0] !=0) { 68: char command[256]; 69: sprintf(command, "use %s;", curDBName); 70: shellRunSingleCommandWebsocketImp(command); 71: }	
	污染路径: 无	
304	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 61
	步骤描述: 不使用返回值的函数'tsdbTFileObjRef'应使用'void'说明.	
	代码片段: 59: STFileObj *fobj1; 60: TARRAY2_FOREACH(lvl1->fobjArr, fobj1) { 61: tsdbTFileObjRef(fobj1); 62: code = TARRAY2_APPEND(lvl[0]->fobjArr, fobj1);	

	63: if (code) return code;	
	污染路径: 函数'tsdbTFileObjRef'的定义点.	
305	缺陷发生位置: /source/util/src/ttimer.c	行号: 141
	步骤描述: 不使用返回值的函数'atomic_add_fetch_8'应使用'void'说明.	
	代码片段: 139: } 140: 141: static void timerAddRef(tmr_obj_t* timer) { atomic_add_fetch_8(&timer->refCount, 1); } 142: 143: static void timerDecRef(tmr_obj_t* timer) {	
	污染路径: 函数'atomic_add_fetch_8'的定义点.	
306	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 79
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 77: 78: if (syncInit() != 0) { 79: dError("failed to init sync since %s", terrstr()); 80: return -1; 81: }	
	污染路径: 函数'dError'的定义点.	
307	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 110
	步骤描述: 不使用返回值的函数'blockDataAppendColInfo'应使用'void'说明.	
	代码片段: 108: 109: SCColumnInfoData idata = createColumnInfoData(dataType, dataBytes, 3); 110: blockDataAppendColInfo(res, &idata); 111: blockDataEnsureCapacity(res, rowNum); 112:	

	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
308	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 41
	步骤描述: 不使用返回值的函数'idxInt2str'应使用'void'说明.	
	代码片段: 39: SERIALIZE_STR_VAR_TO_BUF(p, path, strlen(path)); 40: SERIALIZE_VAR_TO_BUF(p, '_', char); 41: idxInt2str(blockId, p, 0); 42: return; 43: }	
	污染路径: 函数'idxInt2str'的定义点.	
309	缺陷发生位置: /source/client/src/clientStmt.c	行号: 96
	步骤描述: 不使用返回值的函数'STMT_ERR_RET'应使用'void'说明.	
	代码片段: 94: } 95: 96: STMT_ERR_RET(code); 97: 98: pStmt->sql.status = newStatus;	
	污染路径: 函数'STMT_ERR_RET'的定义点.	
310	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 176
	步骤描述: 不使用返回值的函数'kvSerial'应使用'void'说明.	
	代码片段: 174: char buf[128] = {0}; 175: KV kv = {.k1 = 0, .k2 = 0}; 176: kvSerial(&kv, buf); 177: char *v = rocksdb_get_cf(db, rOpt, cfHandle[1], buf, sizeof(kv), &vlen, &err); 178: printf("Get value %s, and len = %d, xxxx\n", v, (int)vlen);	
	污染路径: 函数'kvSerial'的定义点.	

311	缺陷发生位置: /source/libs/stream/src/streamExec.c	行号: 486
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 484: if (type == TASK_OUTPUT__FIXED_DISPATCH type == TASK_OUTPUT__SHUFFLE_DISPATCH) { 485: if (level == TASK_LEVEL__SOURCE) { 486: qDebug("s-task:%s add transfer-state block into outputQ", id); 487: } else { 488: qDebug("s-task:%s all upstream tasks send transfer- state block, add transfer-state block into outputQ", id); </pre>	
	污染路径: 函数'qDebug'的定义点.	
312	缺陷发生位置: /source/libs/executor/src/executor.c	行号: 50
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 48: 49: if (pOperator->numOfDownstream > 1) { // not handle this in join query 50: qError("join not supported for stream block scan, %s" PRIx64, id); 51: return TSDB_CODE_APP_ERROR; 52: } </pre>	
	污染路径: 函数'qError'的定义点.	
313	缺陷发生位置: /source/util/src/tutil.c	行号: 427
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 425: 426: void taosIpPort2String(uint32_t ip, uint16_t port, char *str) { 427: sprintf(str, "%u.%u.%u.%u:%u", ip & 0xFF, (ip >> 8) & 0xFF, (ip >> 16) & 0xFF, (uint8_t)(ip >> 24), port); 428: } 429: </pre>	

	污染路径: 无	
314	缺陷发生位置: /source/libs/parser/src/parlInsertSml.c	行号: 164
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 162: } 163: } 164: taosArrayDestroy(pTagArray); 165: return code; 166: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
315	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmHandle.c	行号: 88
	步骤描述: 不使用返回值的函数'dGError'应使用'void'说明.	
	代码片段: 86: SMnodeOpt option = {.deploy = false}; 87: if (mmWriteFile(pInput->path, &option) != 0) { 88: dGError("failed to write mnode file since %s", terrstr()); 89: return -1; 90: }	
	污染路径: 函数'dGError'的定义点.	
316	缺陷发生位置: /source/libs/stream/src/streamExec.c	行号: 200
	步骤描述: 不使用返回值的函数'qSetStreamOpOpen'应使用'void'说明.	
	代码片段: 198: int32_t outputBatchSize = 100; 199: 200: qSetStreamOpOpen(exec); 201: 202: while (!finished) {	
	污染路径: 函数'qSetStreamOpOpen'的定义点.	
317	缺陷发生位置:	行号:

	/source/client/src/clientSmlJson.c	488
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 486: static int32_t smlConvertJSONBool(SSmlKv *pVal, char *typeStr, cJSON *value) { 487: if (strcasecmp(typeStr, "bool") != 0) { 488: uError("OTD:invalid type(%s) for JSON Bool", typeStr); 489: return TSDB_CODE_TSC_INVALID_JSON_TYPE; 490: } </pre>	
318	污染路径: 函数'uError'的定义点.	
	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 235
	步骤描述: 不使用返回值的函数'rocksdb_free'应使用'void'说明.	
	代码片段: <pre> 233: tsdbError("vgld:%d, %s failed at line %d, count: %d since %s", TD_VID(pTsd->pVnode), __func__, __LINE__, count, 234: err); 235: rocksdb_free(err); 236: // pTsd->flushState.flush_count = 0; 237: } </pre>	
	污染路径: 函数'rocksdb_free'的定义点.	
319	缺陷发生位置: /source/util/src/tlog.c	行号: 142
	步骤描述: 不使用返回值的函数'taosThreadAttrDestroy'应使用'void'说明.	
	代码片段: <pre> 140: return -1; 141: } 142: taosThreadAttrDestroy(&threadAttr); 143: return 0; 144: } </pre>	
320	污染路径: 函数'taosThreadAttrDestroy'的定义点.	
	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 301
	步骤描述:	

	不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 299: int64_t fetchUs = 0, fetchNum = 0; 300: tsortClearOrderdSource(pSortHandle->pOrderedSource, &fetchUs, &fetchNum); 301: qDebug("all source fetch time: %" PRIu64 "us num:%" PRIu64 " %s", fetchUs, fetchNum, pSortHandle->idStr); 302: 303: taosArrayDestroy(pSortHandle->pOrderedSource); </pre>	
	污染路径: 函数'qDebug'的定义点.	
321	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 143
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 141: char* serialized = logStore2Str(pLogStore); 142: sLTrace("logStoreLog len:%d %s", (int32_t)strlen(serialized), serialized); 143: taosMemoryFree(serialized); 144: } 145: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
322	缺陷发生位置: /source/libs/parser/src/parInsertUtil.c	行号: 467
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 465: if (NULL == pVgroupHash NULL == pVgroupList) { 466: taosHashCleanup(pVgroupHash); 467: taosArrayDestroy(pVgroupList); 468: return TSDB_CODE_OUT_OF_MEMORY; 469: } </pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
323	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftEntryDebug.c	行号: 77
	步骤描述: 不使用返回值的函数'sTrace'应使用'void'说明.	

	代码片段: <pre> 75: void syncEntryLog(const SSyncRaftEntry* pObj) { 76: char* serialized = syncEntry2Str(pObj); 77: sTrace("syncEntryLog len:%zu %s", strlen(serialized), serialized); 78: taosMemoryFree(serialized); 79: }</pre>	
	污染路径: 函数'sTrace'的定义点.	
324	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 63
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 61: for (int i = 0; i < batch; i++) { 62: v++; 63: sprintf(tsql, "insert into t values (%d)", v); 64: rc = sqlite3_exec(db, tsql, 0, 0, &err_msg); 65:</pre>	
	污染路径: 无	
325	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 130
	步骤描述: 不使用返回值的函数'raft_free'应使用'void'说明.	
	代码片段: <pre> 128: 129: void putValueCb(struct raft_apply *req, int status, void *result) { 130: raft_free(req); 131: struct raft *r = req->data; 132: if (status != 0) {</pre>	
	污染路径: 函数'raft_free'的定义点.	
326	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 191
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 189: };</pre>	

	<pre> 190: if (tSingleWorkerInit(&pMgmt->queryWorker, &qCfg) != 0) { 191: dError("failed to start mnode-query worker since %s", terrstr()); 192: return -1; 193: } </pre>	
	污染路径: 函数'dError'的定义点.	
327	缺陷发生位置: /source/libs/index/src/index.c	行号: 74
	步骤描述: 不使用返回值的函数'taosCloseRef'应使用'void'说明.	
	代码片段: <pre> 72: taosCleanUpScheduler(indexQhandle); 73: taosMemoryFreeClear(indexQhandle); 74: taosCloseRef(indexRefMgt); 75: } 76: </pre>	
	污染路径: 函数'taosCloseRef'的定义点.	
328	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 458
	步骤描述: 不使用返回值的函数'transUnrefCliHandle'应使用'void'说明.	
	代码片段: <pre> 456: tTrace("%s conn %p handle except, persist:0", CONN_GET_INST_LABEL(pConn), pConn); 457: if (T_REF_VAL_GET(pConn) > 1) transUnrefCliHandle(pConn); 458: transUnrefCliHandle(pConn); 459: return; 460: } </pre>	
	污染路径: 函数'transUnrefCliHandle'的定义点.	
329	缺陷发生位置: /source/client/src/clientEnv.c	行号: 181
	步骤描述: 不使用返回值的函数'taosVersionStrToInt'应使用'void'说明.	
	代码片段: <pre> 179: rpclnit.timeToGetConn = tsTimeToGetAvailableConn; </pre>	

	180: 181: taosVersionStrToInt(version, &(rpcInit.compatibilityVer)); 182: 183: void *pDnodeConn = rpcOpen(&rpcInit);	
	污染路径: 函数'taosVersionStrToInt'的定义点.	
330	缺陷发生位置: /source/os/src/osSocket.c	行号: 774
	步骤描述: 不使用返回值的函数'taosCloseSocket'应使用'void'说明.	
	代码片段: 772: if (bind(pSocket->fd, (struct sockaddr *)&serverAdd, sizeof(serverAdd)) < 0) { 773: // printf("bind tcp server socket failed, 0x%x: %hu(%s)", ip, port, strerror(errno)); 774: taosCloseSocket(&pSocket); 775: return false; 776: }	
	污染路径: 函数'taosCloseSocket'的定义点.	
331	缺陷发生位置: /source/libs/scheduler/src/schStatus.c	行号: 56
	步骤描述: 不使用返回值的函数'SCH_RET'应使用'void'说明.	
	代码片段: 54: default: { 55: SCH_JOB_ELOG("unknown job status %d", status); 56: SCH_RET(TSDB_CODE_SCH_STATUS_ERROR); 57: } 58: }	
	污染路径: 函数'SCH_RET'的定义点.	
332	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 148
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 146: } 147: 148: colDataSetVal(pOutputData, TMAX(iLeft, iRight), output,	

	(output == NULL)); 149: 150: _exit:	
	污染路径: 函数'colDataSetVal'的定义点.	
333	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamStateSnap.c	行号: 176
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 174: } 175: tqDebug("vgId:%d, vnode %s succ to rebuild stream-state", TD_VID(pWriter->pTq->pVnode), STREAM_STATE_TRANSFER); 176: taosMemoryFree(pWriter); 177: return code; 178: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
334	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 399
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 397: 398: for (int iData = 1; iData <= nData; iData++) { 399: sprintf(key, "key0"); 400: sprintf(val, "value%d", iData); 401:	
	污染路径: 无	
335	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 830
	步骤描述: 不使用返回值的函数'filterFinishRange'应使用'void'说明.	
	代码片段: 828: 829: int32_t filterGetRangeRes(void *h, SFilterRange *ra) { 830: filterFinishRange(h); 831:	

	832: SFilterRangeCtx *ctx = (SFilterRangeCtx *)h;	
	污染路径: 函数'filterFinishRange'的定义点.	
336	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCommit2.c	行号: 212
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 210: _next: 211: code = tsdbIterMergerNext(committer->tombIterMerger); 212: TSDB_CHECK_CODE(code, lino, _exit); 213: } 214:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
337	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 472
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 470: TDB_PAGE_FCELL_SET(pPage, pCell - pPage->pData); 471: } else { 472: tdbError("tdb/page-free: invalid cell size: %d", szCell); 473: return -1; 474: }	
	污染路径: 函数'tdbError'的定义点.	
338	缺陷发生位置: /source/client/src/clientTmq.c	行号: 540
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 538: } 539: 540: tscError("consumer:0x%" PRIx64 " , total:%d, failed to find topic:%s", tmq->consumerId, numOfTopics, pTopicName); 541: return NULL; 542: }	

	污染路径: 函数'tscError'的定义点.	
339	缺陷发生位置: /source/libs/parser/src/parlInsertStmt.c	行号: 185
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 183: } 184: } 185: taosArrayDestroy(pTagArray); 186: taosArrayDestroy(tagName); 187:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
340	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 424
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 422: pBufInfo->pBuf = taosMemoryMalloc(bufSize); 423: if (NULL == pBufInfo->pBuf) { 424: qError("group cache add block to cache failed, size:%" PRId64, bufSize); 425: return TSDB_CODE_OUT_OF_MEMORY; 426: }	
	污染路径: 函数'qError'的定义点.	
341	缺陷发生位置: /utils/tsim/src/simExe.c	行号: 412
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 410: } 411: 412: simInfo("system cmd is %s", buf); 413: return replaced; 414: }	
	污染路径: 函数'simInfo'的定义点.	

342	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 91
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 89: }; 90: 91: mInfo("consumer:0x%" PRIx64 " drop from sdb", consumerId); 92: tmsgPutToQueue(&pMnode->msgCb, WRITE_QUEUE, &rpcMsg); 93: return;	
	污染路径: 函数'mInfo'的定义点.	
343	缺陷发生位置: /source/libs/parser/test/parAlterToBalanceTest.cpp	行号: 123
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 121: 122: auto initAlterDb = [&](const char* pDb) { 123: snprintf(expect.db, sizeof(expect.db), "0.%s", pDb); 124: expect.buffer = -1; 125: expect.pageSize = -1;	
	污染路径: 无	
344	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 682
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 680: coj->Get(&query, colId, 10000, ret, &valType); 681: assert(taosArrayGetSize(ret) == 1); 682: taosArrayDestroy(ret); 683: 684: indexTermDestroy(term);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
345	缺陷发生位置: /source/libs/parser/src/parInsertStmt.c	行号: 318

	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 316: tColDataAddValueByBind(pCol, pBind, IS_VAR_DATA_TYPE(pColSchema->type) ? pColSchema->bytes - VARSTR_HEADER_SIZE: -1); 317: 318: qDebug("stmt col %d bind %d rows data", colIdx, rowNum); 319: 320: _return:	
	污染路径: 函数'qDebug'的定义点.	
346	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c	行号: 987
	步骤描述: 不使用返回值的函数'STR_TO_VARSTR'应使用'void'说明.	
	代码片段: 985: } 986: }else{ 987: STR_TO_VARSTR(schemaJson, "NULL"); 988: } 989:	
	污染路径: 函数'STR_TO_VARSTR'的定义点.	
347	缺陷发生位置: /source/client/src/clientHb.c	行号: 58
	步骤描述: 不使用返回值的函数'catalogUpdateUserAuthInfo'应使用'void'说明.	
	代码片段: 56: tscDebug("hb user auth rsp, user:%s, version:%d", rsp->user, rsp->version); 57: 58: catalogUpdateUserAuthInfo(pCatalog, rsp); 59: } 60:	
	污染路径: 函数'catalogUpdateUserAuthInfo'的定义点.	
348	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 49

	步骤描述: 不使用返回值的函数'tsdbInfo'应使用'void'说明.	
	代码片段: 47: tsdbError("file:%s remove failed", fname); 48: } else { 49: tsdbInfo("file:%s is removed", fname); 50: } 51: }	
	污染路径: 函数'tsdbInfo'的定义点.	
349	缺陷发生位置: /source/dnode/mnode/impl/src/mndAcct.c	行号: 224
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 222: 223: terrno = TSDB_CODE_OPS_NOT_SUPPORT; 224: mError("failed to process create acct request since %s", terrstr()); 225: return -1; 226: }	
	污染路径: 函数'mError'的定义点.	
350	缺陷发生位置: /docs/examples/c/error_handle_example.c	行号: 23
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 21: taos_close(taos); 22: } 23: taos_cleanup(); 24: }	
	污染路径: 函数'taos_cleanup'的定义点.	
351	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 421
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 419:	

	<pre> 420: for (int i = 1; i <= nData; i++) { 421: sprintf(key, "key%d", i); 422: // sprintf(val, "value%d", i); 423: </pre>	
	污染路径: 无	
352	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 80
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: <pre> 78: taosMemoryFree(name); 79: 80: taosArrayPush(deleteReq->deleteReqs, &req); 81: } 82: </pre>	
	污染路径: 函数'taosArrayPush'的定义点.	
353	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 112
	步骤描述: 不使用返回值的函数'rtGetTimeOfDay'应使用'void'说明.	
	代码片段: <pre> 110: static int64_t rtGetTimestampMs() { 111: struct timeval systemTime; 112: rtGetTimeOfDay(&systemTime); 113: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec/1000; 114: } </pre>	
	污染路径: 函数'rtGetTimeOfDay'的定义点.	
354	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 1526
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 1524: tFree(pReader->aBuf[iBuf]); 1525: } 1526: taosMemoryFree(pReader); 1527: } </pre>	

	1528:	
	污染路径: 函数'taosMemoryFree'的定义点.	
355	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 113
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 111: int32_t val = atomic_load_32(&mqRebInExecCnt); 112: if (val <= 0) { 113: mError("rebalance trans end, rebalance counter:%d should not be less equalled than 0, ignore counter desc", val); 114: break; 115: }	
	污染路径: 函数'mError'的定义点.	
356	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 20
	步骤描述: 不使用返回值的函数'QW_RET'应使用'void'说明.	
	代码片段: 18: if (NULL == pRsp) { 19: qError("rpcMallocCont %d failed", msgSize); 20: QW_RET(TSDB_CODE_OUT_OF_MEMORY); 21: } 22:	
	污染路径: 函数'QW_RET'的定义点.	
357	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 126
	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段: 124: if (ret < 0) { 125: tdbAbort(pEnv, txn); 126: tdbOsFree(pBt); 127: return -1; 128: }	
	污染路径:	

	函数'tdbOsFree'的定义点.	
358	缺陷发生位置: /source/util/src/tlog.c	行号: 138
	步骤描述: 不使用返回值的函数'taosThreadAttrInit'应使用'void'说明.	
	代码片段: 136: static int32_t taosStartLog() { 137: TdThreadAttr threadAttr; 138: taosThreadAttrInit(&threadAttr); 139: if (taosThreadCreate(&(tsLogObj.logHandle->asyncThread), &threadAttr, taosAsyncOutputLog, tsLogObj.logHandle) != 0) { 140: return -1;	
	污染路径: 函数'taosThreadAttrInit'的定义点.	
359	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 194
	步骤描述: 不使用返回值的函数'rpcFreeCont'应使用'void'说明.	
	代码片段: 192: pContNew->vgId = pContOld->vgId; 193: 194: rpcFreeCont(pContOld); 195: pMsg->pCont = pContNew; 196: pMsg->contLen = contLenNew;	
	污染路径: 函数'rpcFreeCont'的定义点.	
360	缺陷发生位置: /tests/script/api/insertSameTs.c	行号: 111
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 109: 110: // query the records 111: sprintf(qstr, "SELECT * FROM m1 order by ts desc"); 112: result = taos_query(taos, qstr); 113: if (result == NULL taos_errno(result) != 0) {	
	污染路径: 无	
361	缺陷发生位置:	行号:

	/source/client/src/clientSmlTelnet.c	115
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: <pre> 113: const char *sql = data; 114: while (sql < sqlEnd) { 115: JUMP_SPACE(sql, sqlEnd) 116: if (unlikely(*sql == '\0')) break; 117: </pre>	
	污染路径: 函数'JUMP_SPACE'的定义点.	
362	缺陷发生位置: /source/libs/parser/test/parInitialDTest.cpp	行号: 64
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 62: 63: auto setDropCgroupReq = [&](const char* pTopicName, const char* pCGroupName, int8_t igNotExists = 0) { 64: snprintf(expect.topic, sizeof(expect.topic), "0.%s", pTopicName); 65: strcpy(expect.cgroup, pCGroupName); 66: expect.igNotExists = igNotExists; </pre>	
	污染路径: 无	
363	缺陷发生位置: /source/dnode/vnode/src/sma/smaSnapshot.c	行号: 116
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 114: smaInfo("vgId:%d, vnode snapshot rsma reader closed", SMA_VID(pReader->pSma)); 115: 116: taosMemoryFreeClear(*ppReader); 117: return code; 118: } </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
364	缺陷发生位置: /source/client/src/clientHb.c	行号: 56

	步骤描述: 不使用返回值的函数'tscDebug'应使用'void'说明.	
	代码片段: <pre> 54: for (int32_t i = 0; i < numOfBatchs; ++i) { 55: SGetUserAuthRsp *rsp = taosArrayGet(batchRsp.pArray, i); 56: tscDebug("hb user auth rsp, user:%s, version:%d", rsp- >user, rsp->version); 57: 58: catalogUpdateUserAuthInfo(pCatalog, rsp); </pre>	
	污染路径: 函数'tscDebug'的定义点.	
365	缺陷发生位置: /utils/tsim/src/simSystem.c	行号: 41
	步骤描述: 不使用返回值的函数'simInitCfg'应使用'void'说明.	
	代码片段: <pre> 39: 40: bool simSystemInit() { 41: simInitCfg(); 42: simInitsimCmdList(); 43: memset(simScriptList, 0, sizeof(SScript *) * MAX_MAIN_SCRIPT_NUM); </pre>	
	污染路径: 函数'simInitCfg'的定义点.	
366	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 111
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: <pre> 109: SColumnInfoData idata = createColumnInfoData(dataType, dataBytes, 3); 110: blockDataAppendColInfo(res, &idata); 111: blockDataEnsureCapacity(res, rowNum); 112: 113: SColumnInfoData *pColumn = (SColumnInfoData *)taosArrayGetLast(res->pDataBlock); </pre>	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
367	缺陷发生位置:	行号:

	/source/libs/executor/src/hashjoinoperator.c	740
	步骤描述: 不使用返回值的函数'setOperatorCompleted'应使用'void'说明.	
	代码片段: 738: 739: static void setHJoinDone(struct SOperatorInfo* pOperator) { 740: setOperatorCompleted(pOperator); 741: 742: SHJoinOperatorInfo* pInfo = pOperator->info;	
	污染路径: 函数'setOperatorCompleted'的定义点.	
368	缺陷发生位置: /source/libs/wal/src/walRead.c	行号: 480
	步骤描述: 不使用返回值的函数'wError'应使用'void'说明.	
	代码片段: 478: uint32_t readCkSum = walCalcBodyCksum(pReader->pHead->head.body, pReader->pHead->head.bodyLen); 479: uint32_t logCkSum = pReader->pHead->cksumBody; 480: wError("checksum written into log:%u, checksum calculated:%u", logCkSum, readCkSum); 481: // pReader->curlInvalid = 1; 482: terrno = TSDB_CODE_WAL_FILE_CORRUPTED;	
	污染路径: 函数'wError'的定义点.	
369	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmFile.c	行号: 115
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 113: if (taosFStatFile(pFile, &size, NULL) < 0) { 114: terrno = TAOS_SYSTEM_ERROR(errno); 115: dError("failed to fstat mnode file:%s since %s", file, terrstr()); 116: goto _OVER; 117: }	
	污染路径: 函数'dError'的定义点.	
370	缺陷发生位置:	行号:

	/source/libs/sync/src/syncMain.c	287
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 285: return 0; 286: } else { 287: sError("no message handle to send timeout response, seq:%" PRId64, seq); 288: return -1; 289: }	
	污染路径: 函数'sError'的定义点.	
371	缺陷发生位置: /tests/script/api/dbTableRoute.c	行号: 62
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 60: taos_free_result(pSql); 61: taos_close(taos); 62: taos_cleanup(); 63: exit(EXIT_FAILURE); 64: }	
	污染路径: 函数'taos_cleanup'的定义点.	
372	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 431
	步骤描述: 不使用返回值的函数'tNameGetFullDbName'应使用'void'说明.	
	代码片段: 429: 430: char db[TSDB_TABLE_FNAME_LEN] = {0}; 431: tNameGetFullDbName(&name, db); 432: 433: return mndAcquireDb(pMnode, db);	
	污染路径: 函数'tNameGetFullDbName'的定义点.	
373	缺陷发生位置: /source/libs/stream/src/streamQueue.c	行号: 224
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	

	代码片段: 222: if (qlItem == NULL) { 223: if (pTask->info.taskLevel == TASK_LEVEL__SOURCE && (++retryTimes) < MAX_RETRY_TIMES) { 224: taosMsleep(10); 225: qDebug("===stream===try again batchSize:%d, retry:%d, %s", *numOfBlocks, retryTimes, id); 226: continue;	
	污染路径: 函数'taosMsleep'的定义点.	
374	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 81
	步骤描述: 不使用返回值的函数'tdbPCacheCloseImpl'应使用'void'说明.	
	代码片段: 79: int tdbPCacheClose(SPCache *pCache) { 80: if (pCache) { 81: tdbPCacheCloseImpl(pCache); 82: tdbOsFree(pCache->aPage); 83: tdbOsFree(pCache);	
	污染路径: 函数'tdbPCacheCloseImpl'的定义点.	
375	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 3629
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: 3627: nodesNodeToString((SNode*)pSubplan, false, &pStr, NULL); 3628: if (NULL == pRuleName) { 3629: qDebugL("before optimize, JsonPlan: %s", pStr); 3630: } else { 3631: qDebugL("apply optimize %s rule, JsonPlan: %s", pRuleName, pStr);	
	污染路径: 函数'qDebugL'的定义点.	
376	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 59
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	

	代码片段: 57: } 58: 59: tmsgReportStartup("snode-impl", "initialized"); 60: 61: if (smStartWorker(pMgmt) != 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
377	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 182
	步骤描述: 不使用返回值的函数'tdListPopNode'应使用'void'说明.	
	代码片段: 180: pFileState->flushMark = TMAX(pFileState->flushMark, pFileState->getTs(pPos->pKey)); 181: tSimpleHashRemove(pFileState->rowBuffMap, pPos->pKey, pFileState->keyLen); 182: tdListPopNode(pFileState->usedBufs, pNode); 183: taosMemoryFreeClear(pNode); 184: i++;	
	污染路径: 函数'tdListPopNode'的定义点.	
378	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 156
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 154: for (int iData = 1; iData <= nData; iData++) { 155: sprintf(key, "key%d", iData); 156: sprintf(val, "value%d", iData); 157: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn); 158: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
379	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 593
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段:	

	591: 592: blockDecode(pBlock, pStart); 593: blockDataEnsureCapacity(pRes, pBlock->info.rows); 594: 595: // data from mnode	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
380	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 523
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 521: 522: if (0 != taosHashPut((*pDst)->vgHash, vgId, sizeof(*vgId), vgInfo, sizeof(*vgInfo))) { 523: qError("taosHashPut failed, vgId:%d", vgInfo->vgId); 524: taosHashCancelIterate(pSrc->vgHash, plter); 525: freeVgInfo(*pDst);	
	污染路径: 函数'qError'的定义点.	
381	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 98
	步骤描述: 不使用返回值的函数'strtolower'应使用'void'说明.	
	代码片段: 96: 97: const char* a1 = "ABC"; 98: strtolower(t, a1); 99: EXPECT_STREQ(t, "abc"); 100:	
	污染路径: 函数'strtolower'的定义点.	
382	缺陷发生位置: /source/libs/stream/src/streamQueue.c	行号: 239
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 237: 238: if (*pInput == NULL) { 239: qDebug("s-task:%s %s msg extracted, start to	

	process immediately", id, p); 240: 241: *numOfBlocks = 1;	
	污染路径: 函数'qDebug'的定义点.	
383	缺陷发生位置: /tests/script/api/stopquery.c	行号: 261
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 259: if (taos == NULL) sqExit("taos_connect", taos_errstr(NULL)); 260: 261: sprintf(sql, "reset query cache"); 262: sqExecSQL(taos, sql); 263:	
	污染路径: 无	
384	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 156
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 154: 155: if (p->info.rows > 0) { 156: blockDataEnsureCapacity(pDataBlock, capacity); 157: 158: // todo extract function to handle this	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
385	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 61
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 59: 60: if ((code = taosReadFile(pFile, pMemBuf, size)) != size) { 61: taosMemoryFree(pMemBuf); 62: return -1;	

	63: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
386	缺陷发生位置: /source/dnode/vnode/src/sma/smaEnv.c	行号: 97
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 95: if (old != 2) break; 96: if (++nLoops > 1000) { 97: sched_yield(); 98: nLoops = 0; 99: }	
	污染路径: 函数'sched_yield'的定义点.	
387	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 958
	步骤描述: 不使用返回值的函数'taosDecodeFixedU64'应使用'void'说明.	
	代码片段: 956: 957: uint64_t version; 958: taosDecodeFixedU64(buf, &version); 959: skip += sizeof(version); 960: if (version == 0 version > VERSION) {	
	污染路径: 函数'taosDecodeFixedU64'的定义点.	
388	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 228
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 226: 0) { 227: tEncoderClear(&encoder); 228: taosMemoryFree(buf); 229: return -1; 230: }	
	污染路径: 函数'taosMemoryFree'的定义点.	

389	缺陷发生位置： /source/libs/tdb/src/db/tdbPCache.c	行号： 66
	步骤描述： 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段： <pre> 64: pCache->aPage = (SPage **)tdbOsCalloc(cacheSize, sizeof(SPage *)); 65: if (pCache->aPage == NULL) { 66: tdbOsFree(pCache); 67: return -1; 68: } </pre>	
	污染路径： 函数'tdbOsFree'的定义点.	
390	缺陷发生位置： /source/libs/executor/src/sysscanoperator.c	行号： 459
	步骤描述： 不使用返回值的函数'tNameGetDbName'应使用'void'说明.	
	代码片段： <pre> 457: tNameFromString(&sn, db, T_NAME_ACCT T_NAME_DB); 458: 459: tNameGetDbName(&sn, varDataVal(dbname)); 460: varDataSetLen(dbname, strlen(varDataVal(dbname))); 461: </pre>	
	污染路径： 函数'tNameGetDbName'的定义点.	
391	缺陷发生位置： /source/libs/executor/src/exchangeoperator.c	行号： 494
	步骤描述： 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段： <pre> 492: if (pDataInfo->pSrcUidList) { 493: int32_t code = buildTableScanOperatorParam(&req.pOpParam, pDataInfo- >pSrcUidList, pDataInfo->srcOpType, pDataInfo->tableSeq); 494: taosArrayDestroy(pDataInfo->pSrcUidList); 495: pDataInfo->pSrcUidList = NULL; 496: if (TSDB_CODE_SUCCESS != code) { </pre>	
	污染路径： 函数'taosArrayDestroy'的定义点.	

392	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 301
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 299: 300: if (precision == TSDB_TIME_PRECISION_NANO) { 301: sprintf(buf + pos, "%.09d", ms); 302: } else if (precision == TSDB_TIME_PRECISION_MICRO) { 303: sprintf(buf + pos, "%.06d", ms);	
	污染路径: 无	
393	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCommit2.c	行号: 96
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 94: 95: code = tsdbFSetWriterOpen(&config, &committer->writer); 96: TSDB_CHECK_CODE(code, lino, _exit); 97: 98: _exit:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
394	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 1020
	步骤描述: 不使用返回值的函数'taosCloseDir'应使用'void'说明.	
	代码片段: 1018: taosArrayPush(files, &buf); 1019: } 1020: taosCloseDir(&pDir); 1021: 1022: taosArraySort(files, tfileCompare);	
	污染路径: 函数'taosCloseDir'的定义点.	
395	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 298
	步骤描述:	

	不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 296: } 297: 298: taosArrayPush(pReq->aSubmitTbData, &tbData); 299: 300: _end:	
	污染路径: 函数'taosArrayPush'的定义点.	
396	缺陷发生位置: /source/util/src/tlockfree.c	行号: 83
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 81: nLoops++; 82: if (nLoops > 1000) { 83: sched_yield(); 84: nLoops = 0; 85: }	
	污染路径: 无	
397	缺陷发生位置: /source/libs/executor/test/tSimpleHashTests.cpp	行号: 71
	步骤描述: 不使用返回值的函数'tSimpleHashRemove'应使用'void'说明.	
	代码片段: 69: 70: for (int64_t i = 1; i <= 100; ++i) { 71: tSimpleHashRemove(pHashObj, (const void *)&i, keyLen); 72: ASSERT_EQ(100 - i, tSimpleHashGetSize(pHashObj)); 73: }	
	污染路径: 函数'tSimpleHashRemove'的定义点.	
398	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 47
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 45: int32_t code = taosRemoveFile(fname);	

	46: if (code) { 47: tsdbError("file:%s remove failed", fname); 48: } else { 49: tsdbInfo("file:%s is removed", fname);	
	污染路径: 函数'tsdbError'的定义点.	
399	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmFile.c	行号: 83
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 81: if (taosFStatFile(pFile, &size, NULL) < 0) { 82: terrno = TAOS_SYSTEM_ERROR(errno); 83: dError("failed to fstat mnode file:%s since %s", file, terrstr()); 84: goto _OVER; 85: }	
	污染路径: 函数'dError'的定义点.	
400	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 963
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 961: int32_t startIndex; 962: if (numOfParams != 4) { 963: snprintf(errMsg, msgLen, "%s", msg1); 964: cJSON_Delete(binDesc); 965: return false;	
	污染路径: 无	
401	缺陷发生位置: /source/libs/sync/src/syncRaftStore.c	行号: 121
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 119: SRaftStore *pStore = &pNode->raftStore; 120: char file[PATH_MAX] = {0}; 121: snprintf(file, sizeof(file), "%s.bak", realfile); 122:	

	123: terrno = TSDB_CODE_OUT_OF_MEMORY;	
	污染路径: 无	
402	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffsetSnapshot.c	行号: 68
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 66: return -1; 67: } 68: taosMemoryFree(fname); 69: 70: SSnapDataHdr* buf = taosMemoryCalloc(1, sz + sizeof(SSnapDataHdr));	
	污染路径: 函数'taosMemoryFree'的定义点.	
403	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 281
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 279: STR_WITH_SIZE_TO_VARSTR(tmp, pFuncParam->param.pz, pFuncParam->param.nLen); 280: for (int32_t i = 0; i < numOfRows; ++i) { 281: colDataSetVal(pCollInfo, i, tmp, false); 282: } 283: taosMemoryFree(tmp);	
	污染路径: 函数'colDataSetVal'的定义点.	
404	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUtil.c	行号: 999
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 997: _clear; 998: taosArrayDestroy(aTmpSkyline); 999: taosArrayDestroy(pSkyline); 1000: 1001: return code;	

	污染路径: 函数'taosArrayDestroy'的定义点.	
405	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmWorker.c	行号: 422
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: <pre> 420: if (tSingleWorkerInit(&pMgmt->mgmtWorker, &mgmtCfg) != 0) return -1; 421: 422: dDebug("vnode workers are initialized"); 423: return 0; 424: }</pre>	
	污染路径: 函数'dDebug'的定义点.	
406	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 708
	步骤描述: 不使用返回值的函数'transUnrefCliHandle'应使用'void'说明.	
	代码片段: <pre> 706: } 707: if (T_REF_VAL_GET(conn) > 1) { 708: transUnrefCliHandle(conn); 709: } 710:</pre>	
	污染路径: 函数'transUnrefCliHandle'的定义点.	
407	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 202
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 200: 201: // free info binary 202: taosMemoryFree(data); 203: 204: vInfo("vgId:%d, vnode info is saved, fname:%s replica: %d selfIndex:%d changeVersion:%d",</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	

	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: <pre> 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79: 80: LOAD_FP(SetCaretUpdate, SetCaretUpdateFP, "setCaretUpdateFP"); </pre>	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
412	缺陷发生位置: /utils/test/c/createTable.c	行号: 69
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 67: } 68: 69: sprintf(qstr, "create database if not exists %s vgroups %d", dbName, numOfVgroups); 70: TAOS_RES *pRes = taos_query(con, qstr); 71: int32_t code = taos_errno(pRes); </pre>	
	污染路径: 无	
413	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 310
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 308: pTaskInfo->code = code; 309: tableListDestroy(pTableListInfo); 310: qError("failed to createScanTableListInfo, code:%s, %s", tstrerror(code), idstr); 311: return NULL; 312: } </pre>	

	污染路径: 函数'qError'的定义点.	
414	缺陷发生位置: /examples/c/demo.c	行号: 87
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 85: int i = 0; 86: for (i = 0; i < 10; ++i) { 87: sprintf(qstr, "insert into m1 values (%d PRId64 ", %d, %d, %d, %d, %f, %lf, '%s')", (uint64_t)(1546300800000 + i * 1000), i, i, i, i*10000000, i*1.0, i*2.0, "hello"); 88: printf("qstr: %s\n", qstr); 89: </pre>	
	污染路径: 无	
415	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 333
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 331: pTaskInfo->code = code; 332: tableListDestroy(pTableListInfo); 333: qError("failed to createScanTableListInfo, code: %s", tstrerror(code)); 334: return NULL; 335: } </pre>	
	污染路径: 函数'qError'的定义点.	
416	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 260
	步骤描述: 不使用返回值的函数'getNextTimeWindow'应使用'void'说明.	
	代码片段: <pre> 258: if (index != -1) { 259: qDebug("===stream===try push delete window %d PRId64 "chId:%d ,continue", win.skey, childId); 260: getNextTimeWindow(pInterval, &win, TSDB_ORDER_ASC); 261: continue; 262: } </pre>	

	污染路径: 函数'getNextTimeWindow'的定义点.	
417	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 98
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 96: uint32_t capacity = pDeleter->pManager-> cfg.maxDataBlockNumPerQuery; 97: if (taosQueueItemSize(pDeleter->pDataBlocks) > capacity) { 98: qError("SinkNode queue is full, no capacity, max:%d, current:%d, no capacity", capacity, 99: taosQueueItemSize(pDeleter->pDataBlocks)); 100: return false; </pre>	
	污染路径: 函数'qError'的定义点.	
418	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 65
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 63: char destroyFuncName[TSDB_FUNC_NAME_LEN + 9] = {0}; 64: char *destroySuffix = "_destroy"; 65: snprintf(destroyFuncName, sizeof(destroyFuncName), "%s%s", udfName, destroySuffix); 66: uv_dlsym(&udfCtx->lib, destroyFuncName, (void **) (&udfCtx->destroyFunc)); 67: return udfName; </pre>	
	污染路径: 无	
419	缺陷发生位置: /source/util/src/terror.c	行号: 686
	步骤描述: 不使用返回值的函数'taosThreadOnce'应使用'void'说明.	
	代码片段: <pre> 684: 685: const char* tsterror(int32_t err) { 686: taosThreadOnce(&tsErrorInit, tsSortError); </pre>	

	687: 688: // this is a system errno	
	污染路径: 函数'taosThreadOnce'的定义点.	
420	缺陷发生位置: /tools/shell/src/shellArguments.c	行号: 280
	步骤描述: 不使用返回值的函数'shellParseSingleOpt'应使用'void'说明.	
	代码片段: <pre> 278: return -1; 279: } 280: shellParseSingleOpt(key[1], val); 281: i++; 282: } else if (key[1] == 'p' key[1] == 'A' key[1] == 'C' </pre>	
	污染路径: 函数'shellParseSingleOpt'的定义点.	
421	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 50
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 48: code = tdbTbcMoveTo(pReader->pTbc, &(STbDbKey) {.version = sver, .uid = INT64_MIN}, sizeof(STbDbKey), &c); 49: if (code) { 50: taosMemoryFree(pReader); 51: goto _err; 52: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
422	缺陷发生位置: /source/client/src/clientSmlLine.c	行号: 114
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 112: memcpy(tmp, pVal->value + NCHAR_ADD_LEN - 1, pVal->length - NCHAR_ADD_LEN); 113: code = doGeomFromText(tmp, (unsigned char **) &pVal->value, &pVal->length); 114: taosMemoryFree(tmp); 115: if (code != TSDB_CODE_SUCCESS) { </pre>	

	116: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
423	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 41
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.	
	代码片段: 39: if (NULL == ctx->pName) { 40: taosMemoryFree(task.taskCtx); 41: CTG_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 42: } 43:	
	污染路径: 函数'CTG_ERR_RET'的定义点.	
424	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 123
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 121: taosThreadMutexLock(&pDispatcher->mutex); 122: int32_t status = pDispatcher->status; 123: taosThreadMutexUnlock(&pDispatcher->mutex); 124: return status; 125: }	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
425	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 183
	步骤描述: 不使用返回值的函数'tsdbDataFileReaderClose'应使用'void'说明.	
	代码片段: 181: while (1) { 182: if (pReader->pFileReader != NULL) { 183: tsdbDataFileReaderClose(&pReader->pFileReader); 184: } 185:	
	污染路径: 函数'tsdbDataFileReaderClose'的定义点.	

426	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 165
	步骤描述: 不使用返回值的函数'colDataAssign'应使用'void'说明.	
	代码片段: 163: SColumnInfoData* pSrc = taosArrayGet(p->pDataBlock, pmlInfo->srcSlotId); 164: SColumnInfoData* pDst = taosArrayGet(pDataBlock->pDataBlock, pmlInfo->dstSlotId); 165: colDataAssign(pDst, pSrc, p->info.rows, &pDataBlock->info); 166: } 167:	
	污染路径: 函数'colDataAssign'的定义点.	
427	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 433
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 431: } 432: static int32_t sifExecFunction(SFunctionNode *node, SIFCtx *ctx, SIFParam *output) { 433: indexError("index-filter not support buildin function"); 434: return TSDB_CODE_QRY_INVALID_INPUT; 435: }	
	污染路径: 函数'indexError'的定义点.	
428	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 63
	步骤描述: 不使用返回值的函数'fnInfo'应使用'void'说明.	
	代码片段: 61: SUdfdData *pData = process->data; 62: if (exitStatus == 0 && termSignal == 0 atomic_load_32(&pData->stopCalled)) { 63: fnInfo("udfd process exit due to SIGINT or dnode-mgmt called stop"); 64: } else { 65: fnInfo("udfd process restart");	

	污染路径: 函数'fnInfo'的定义点.	
429	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 118
	步骤描述: 不使用返回值的函数'smaCleanUp'应使用'void'说明.	
	代码片段: 116: walCleanUp(); 117: tqCleanUp(); 118: smaCleanUp(); 119: s3CleanUp(); 120: }	
	污染路径: 函数'smaCleanUp'的定义点.	
430	缺陷发生位置: /contrib/test/traft/single_node/singleNode.c	行号: 106
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 104: // for test: submit a value every second 105: while (1) { 106: sleep(1); 107: submitValue(); 108: }	
	污染路径: 无	
431	缺陷发生位置: /contrib/test/traft/cluster/node10002_restart.c	行号: 108
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 106: // for test: submit a value every second 107: while (1) { 108: sleep(1); 109: submitValue(); 110: }	
	污染路径: 无	
432	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c	行号: 61

	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 59: 60: if (udfcOpen() != 0) { 61: dError("qnode can not open udfc"); 62: qmClose(pMgmt); 63: return -1;	
	污染路径: 函数'dError'的定义点.	
433	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 290
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 288: if (tDeserializeSBatchReq(pMsg->pCont, pMsg->contLen, &batchReq)) { 289: code = TSDB_CODE_OUT_OF_MEMORY; 290: qError("tDeserializeSBatchReq failed"); 291: goto _exit; 292: }	
	污染路径: 函数'qError'的定义点.	
434	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 380
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 378: 379: for (int iData = 1; iData <= nData; iData++) { 380: sprintf(key, "key0"); 381: sprintf(val, "value%d", iData); 382:	
	污染路径: 无	
435	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 55
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段:	

	53: tsdbTFileName(config->tsdb, config->file, fname1); 54: code = tsdbOpenFile(fname1, config->szPage, TD_FILE_READ, &reader[0]->fd); 55: TSDB_CHECK_CODE(code, lino, _exit); 56: } 57:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
436	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 70
	步骤描述: 不使用返回值的函数'taosIgnSIGPIPE'应使用'void'说明.	
	代码片段: 68: 69: static int32_t dmInitSystem() { 70: taosIgnSIGPIPE(); 71: taosBlockSIGPIPE(); 72: taosResolveCRC();	
	污染路径: 函数'taosIgnSIGPIPE'的定义点.	
437	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 97
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 95: pData = getBufPage(pResultBuf, *currentPageId); 96: if (pData == NULL) { 97: qError("failed to get buffer, code:%s", tstrerror(errno)); 98: return NULL; 99: }	
	污染路径: 函数'qError'的定义点.	
438	缺陷发生位置: /tests/script/api/stmtBatchTest.c	行号: 227
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr);	

	227: taosMemoryFree(v); 228: taosMemoryFree(lb); 229: taosMemoryFree(params);	
	污染路径: 函数'taosMemoryFree'的定义点.	
439	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 181
	步骤描述: 不使用返回值的函数'tNameGetFullDbName'应使用'void'说明.	
	代码片段: 179: 180: char db[TSDB_DB_FNAME_LEN] = {0}; 181: tNameGetFullDbName(&cn, db); 182: 183: strcpy(output->dbFName, db);	
	污染路径: 函数'tNameGetFullDbName'的定义点.	
440	缺陷发生位置: /source/client/src/clientStmt.c	行号: 110
	步骤描述: 不使用返回值的函数'STMT_ERR_RET'应使用'void'说明.	
	代码片段: 108: if ('\0' == pStmt->bInfo.tbName[0]) { 109: tscError("no table name set"); 110: STMT_ERR_RET(TSDB_CODE_TSC_STMT_TBNAME_ERROR); 111: } 112:	
	污染路径: 函数'STMT_ERR_RET'的定义点.	
441	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.c	行号: 83
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 81: 82: code = tsdbDataFileWriteRow(writer->dataWriter, &row); 83: TSDB_CHECK_CODE(code, lino, _exit); 84: }	

	85:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
442	缺陷发生位置: /source/util/src/tconfig.c	行号: 103
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 101: taosMemoryFreeClear(pltem->name); 102: } 103: taosArrayDestroy(pCfg->array); 104: taosMemoryFree(pCfg); 105: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
443	缺陷发生位置: /source/libs/executor/src/dataInserter.c	行号: 234
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 232: case TSDB_DATA_TYPE_JSON: 233: case TSDB_DATA_TYPE_MEDIUMBLOB: 234: qError("the column type %" PRIi16 " is defined but not implemented yet", pCollInfoData->info.type); 235: terrno = TSDB_CODE_APP_ERROR; 236: goto _end;	
	污染路径: 函数'qError'的定义点.	
444	缺陷发生位置: /source/util/src/tlruccache.c	行号: 335
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 333: SArray *lastReferenceList = taosArrayInit(16, POINTER_BYTES); 334: 335: taosThreadMutexLock(&shard->mutex); 336: 337: shard->capacity = capacity;	

	污染路径: 函数'taosThreadMutexLock'的定义点.	
445	缺陷发生位置: /source/os/src/osFile.c	行号: 99
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 97: sprintf(rand, "%" PRIu64, atomic_add_fetch_64(&seqId, 1)); 98: 99: snprintf(dstPath, PATH_MAX, tmpPath, getpid(), rand); 100: 101: #endif </pre>	
	污染路径: 无	
446	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 588
	步骤描述: 不使用返回值的函数'fError'应使用'void'说明.	
	代码片段: <pre> 586: static int32_t tfsCheck(STfs *pTfs) { 587: if (TFS_PRIMARY_DISK(pTfs) == NULL) { 588: fError("no primary disk is set"); 589: terrno = TSDB_CODE_FS_NO_PRIMARY_DISK; 590: return -1; </pre>	
	污染路径: 函数'fError'的定义点.	
447	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 83
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 81: char *serialized = raftStore2Str(pObj); 82: sTrace("raftStoreLog2 len:%d %s %s", (int32_t)strlen(serialized), s, serialized); 83: taosMemoryFree(serialized); 84: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
448	缺陷发生位置:	行号:

	/source/dnode/mgmt/exe/dmMain.c	212
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 210: } 211: 212: dlInfo("startup path:%s args:%s", path, args); 213: } 214:	
	污染路径: 函数'dlInfo'的定义点.	
449	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 407
	步骤描述: 不使用返回值的函数'tSerializeSAlterVnodeConfigReq'应使用'void'说明.	
	代码片段: 405: pHead->vgId = htonl(pVgroup->vgId); 406: 407: tSerializeSAlterVnodeConfigReq((char *)pReq + sizeof(SMsgHead), contLen, &alterReq); 408: *pContLen = contLen; 409: return pReq;	
	污染路径: 函数'tSerializeSAlterVnodeConfigReq'的定义点.	
450	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 169
	步骤描述: 不使用返回值的函数'taosLocalTime'应使用'void'说明.	
	代码片段: 167: time_t tTime = taosGetTimestampSec(); 168: struct tm tm; 169: taosLocalTime(&tTime, &tm, NULL); 170: sprintf(timeString, "%d-%02d-%02d %02d:%02d: %02d", tm.tm_year + 1900, tm.tm_mon + 1, tm.tm_mday, tm.tm_hour, 171: tm.tm_min, tm.tm_sec);	
	污染路径: 函数'taosLocalTime'的定义点.	
451	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 83

	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 81: } 82: 83: tDebug("thread:%d, it is over", pInfo->index); 84: tcount++; 85:	
	污染路径: 函数'tDebug'的定义点.	
452	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 178
	步骤描述: 不使用返回值的函数'rpcInit'应使用'void'说明.	
	代码片段: 176: taosBlockSIGPIPE(); 177: 178: rpcInit(); 179: 180: // client rpc init	
	污染路径: 函数'rpcInit'的定义点.	
453	缺陷发生位置: /source/util/src/tskiplist.c	行号: 97
	步骤描述: 不使用返回值的函数'tSkipListWLock'应使用'void'说明.	
	代码片段: 95: if (pSkipList == NULL) return; 96: 97: tSkipListWLock(pSkipList); 98: 99: SSkipListNode *pNode = SL_NODE_GET_FORWARD_POINTER(pSkipList->pHead, 0);	
	污染路径: 函数'tSkipListWLock'的定义点.	
454	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftStoreDebug.c	行号: 72
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	<pre> 70: printf("raftStorePrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 71: fflush(NULL); 72: taosMemoryFree(serialized); 73: } 74: void raftStoreLog(SRaftStore *pObj) { </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
455	缺陷发生位置: /contrib/test/cos/main.c	行号: 132
	步骤描述: 不使用返回值的函数'cos_warn_log'应使用'void'说明.	
	代码片段: <pre> 130: 131: char *pstr2 = apr_pstrndup(p, (char *)fmt_str_hex, sizeof(fmt_str_hex)); 132: cos_warn_log("Format string sha1hash: %s", pstr2); 133: 134: cos_pool_destroy(p); </pre>	
	污染路径: 函数'cos_warn_log'的定义点.	
456	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 367
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: <pre> 365: void freeBlock(void* pParam) { 366: SSDataBlock* pBlock = *(SSDataBlock**)pParam; 367: blockDataDestroy(pBlock); 368: } 369: </pre>	
	污染路径: 函数'blockDataDestroy'的定义点.	
457	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 154
	步骤描述: 不使用返回值的函数'rocksdb_free'应使用'void'说明.	
	代码片段: <pre> 152: if (NULL == db) { 153: tsdbError("vgId:%d, %s failed at line %d since %s", </pre>	

	TD_VID(pTsdB->pVnode), __func__, __LINE__, err); 154: rocksdb_free(err); 155: 156: code = TSDB_CODE_OUT_OF_MEMORY;	
	污染路径: 函数'rocksdb_free'的定义点.	
458	缺陷发生位置: /source/libs/executor/src/dynqueryctrloperator.c	行号: 33
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 31: 32: void freeVgTableList(void* ptr) { 33: taosArrayDestroy(*(SArray**)ptr); 34: } 35:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
459	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 88
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.	
	代码片段: 86: ctgDebug("db is dropping, dbId:0x%" PRIx64, dbCache->dbId); 87: CTG_UNLOCK(CTG_WRITE, &dbCache->vgCache.vgLock); 88: CTG_ERR_RET(TSDB_CODE_CTG_DB_DROPPED); 89: } 90:	
	污染路径: 函数'CTG_ERR_RET'的定义点.	
460	缺陷发生位置: /source/libs/executor/src/hashjoinoperator.c	行号: 745
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 743: destroyHJoinKeyHash(&pInfo->pKeyHash); 744: 745: qError("hash join done");	

	746: } 747:	
	污染路径: 函数'qError'的定义点.	
461	缺陷发生位置: /source/libs/executor/src/tlinearhash.c	行号: 104
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 102: } 103: 104: taosArrayPush(pBucket->pPagelIdList, &newPagelId); 105: 106: doCopyObject(pNewPage->data, key, keyLen, data, size);	
	污染路径: 函数'taosArrayPush'的定义点.	
462	缺陷发生位置: /source/util/src/rbtree.c	行号: 415
	步骤描述: 不使用返回值的函数'rbtree_delete'应使用'void'说明.	
	代码片段: 413: } 414: 415: rbtree_delete(pTree, z); 416: 417: pTree->n--;	
	污染路径: 函数'rbtree_delete'的定义点.	
463	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 85
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 83: uint64_t v = 0; 84: GET_TYPED_DATA(v, uint64_t, pVar->nType, &pVar->u); 85: colDataSetVal(pDst, rowIndex, (char*)&v, isNull); 86: } else if (pDst->info.type == TSDB_DATA_TYPE_TIMESTAMP) { 87: colDataSetVal(pDst, rowIndex, (const	

	char*)¤tKey, isNull);	
	污染路径: 函数'colDataSetVal'的定义点.	
464	缺陷发生位置: /source/libs/executor/test/executorTests.cpp	行号: 77
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 75: 76: SColumnInfoData collInfo = createColumnInfoData(TSDB_DATA_TYPE_INT, sizeof(int32_t), 1); 77: blockDataAppendCollInfo(pInfo->pBlock, &collInfo); 78: blockDataEnsureCapacity(pInfo->pBlock, pInfo->numOfRowsPerPage); 79:	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
465	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 78
	步骤描述: 不使用返回值的函数'taosGetQitem'应使用'void'说明.	
	代码片段: 76: 77: for (int i = 0; i < numOfMsgs; ++i) { 78: taosGetQitem(qall, (void **)&pRpcMsg); 79: 80: if (pDataFile != NULL) {	
	污染路径: 函数'taosGetQitem'的定义点.	
466	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 284
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 282: 283: if (numOfSources == 0) { 284: qError("%s invalid number: %d of sources in exchange operator", id, (int32_t)numOfSources); 285: return TSDB_CODE_INVALID_PARA;	

	286: }	
	污染路径: 函数'qError'的定义点.	
467	缺陷发生位置: /source/util/src/tworker.c	行号: 64
	步骤描述: 不使用返回值的函数'taosThreadMutexDestroy'应使用'void'说明.	
	代码片段: 62: taosMemoryFreeClear(pool->workers); 63: taosCloseQset(pool->qset); 64: taosThreadMutexDestroy(&pool->mutex); 65: 66: uInfo("worker:%s is closed", pool->name);	
	污染路径: 函数'taosThreadMutexDestroy'的定义点.	
468	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 144
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 142: if (pIter->pLastBlockReader == NULL) { 143: int32_t code = TSDB_CODE_OUT_OF_MEMORY; 144: tsdbError("failed to prepare the last block iterator, since:%s %s", tstrerror(code), pReader->idStr); 145: return code; 146: }	
	污染路径: 函数'tsdbError'的定义点.	
469	缺陷发生位置: /source/common/src/tmisc.c	行号: 78
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 76: 77: void updateEpSet_s(SCorEpSet* pEpSet, SEpSet* pNewEpSet) { 78: taosCorBeginWrite(&pEpSet->version); 79: pEpSet->epSet = *pNewEpSet; 80: taosCorEndWrite(&pEpSet->version);	

	污染路径: expanded from macro 'taosCorBeginWrite' expanded from macro 'taosCorBeginRead'	
470	缺陷发生位置: /source/libs/function/src/udfd.c	行号: 82
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 80: char finishFuncName[TSDB_FUNC_NAME_LEN + 8] = {0}; 81: char *finishSuffix = "_finish"; 82: snprintf(finishFuncName, sizeof(finishFuncName), "%s%s", processFuncName, finishSuffix); 83: uv_dlsym(&udfCtx->lib, finishFuncName, (void **)(&udfCtx->aggFinishFunc)); 84:	
	污染路径: 无	
471	缺陷发生位置: /source/dnode/mnode/impl/src/mndScheduler.c	行号: 289
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 287: static SArray* addNewTaskList(SArray* pTasksList) { 288: SArray* pTaskList = taosArrayInit(0, POINTER_BYTES); 289: taosArrayPush(pTasksList, &pTaskList); 290: return pTaskList; 291: }	
	污染路径: 函数'taosArrayPush'的定义点.	
472	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 2358
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 2356: colDataSetNULL(pOutputData, 0); 2357: } else { 2358: colDataSetVal(pOutputData, 0, (char *)&val, false); 2359: } 2360:	

	污染路径: 函数'colDataSetVal'的定义点.	
473	缺陷发生位置: /source/dnode/mnode/impl/src/mndSma.c	行号: 264
	步骤描述: 不使用返回值的函数'tNameGetFullDbName'应使用'void'说明.	
	代码片段: 262: 263: char db[TSDB_TABLE_FNAME_LEN] = {0}; 264: tNameGetFullDbName(&name, db); 265: 266: return mndAcquireDb(pMnode, db);	
	污染路径: 函数'tNameGetFullDbName'的定义点.	
474	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 113
	步骤描述: 不使用返回值的函数'strtolower'应使用'void'说明.	
	代码片段: 111: memset(t, 1, tListLen(t)); 112: const char* a4 = "\"AbcDEF\""; 113: strtolower(t, a4); 114: EXPECT_STREQ(t, a4); 115:	
	污染路径: 函数'strtolower'的定义点.	
475	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 133
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 131: int32_t idx = taosArrayGetSize(res->pDataBlock); 132: SColumnInfoData idata = createColumnInfoData(dataType, dataBytes, 1 + idx); 133: blockDataAppendCollInfo(res, &idata); 134: blockDataEnsureCapacity(res, rowNum); 135:	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
476	缺陷发生位置:	行号:

	/source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	246
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 244: char* serialized = syncPing2Str(pMsg); 245: sTrace("syncPingLog len:%d %s", (int32_t)strlen(serialized), serialized); 246: taosMemoryFree(serialized); 247: } 248:	
	污染路径: 函数'taosMemoryFree'的定义点.	
477	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 69
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 67: 68: tdbTbcClose((*ppReader)->pTbc); 69: taosMemoryFree(*ppReader); 70: *ppReader = NULL; 71:	
	污染路径: 函数'taosMemoryFree'的定义点.	
478	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 965
	步骤描述: 不使用返回值的函数'taosDecodeFixedU64'应使用'void'说明.	
	代码片段: 963: 964: uint64_t type; 965: taosDecodeFixedU64(buf + skip, &type); 966: skip += sizeof(type); 967:	
	污染路径: 函数'taosDecodeFixedU64'的定义点.	
479	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 194
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	

	代码片段: 192: // merger 193: code = tsdblterMergerOpen(reader->datalterArr, &reader->datalterMerger, false); 194: TSDB_CHECK_CODE(code, lino, _exit); 195: 196: code = tsdblterMergerOpen(reader->tombilterArr, &reader->tombilterMerger, true);	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
480	缺陷发生位置: /source/libs/stream/src/streamMeta.c	行号: 65
	步骤描述: 不使用返回值的函数'taosCloseRef'应使用'void'说明.	
	代码片段: 63: taosCloseRef(streamBackendId); 64: taosCloseRef(streamBackendCfWrapperId); 65: taosCloseRef(streamMetaId); 66: 67: metaRefMgtCleanup();	
	污染路径: 函数'taosCloseRef'的定义点.	
481	缺陷发生位置: /source/libs/executor/src/cachescanoperator.c	行号: 117
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 115: 116: pInfo->pBufferedRes = createOneDataBlock(pInfo->pRes, false); 117: blockDataEnsureCapacity(pInfo->pBufferedRes, capacity); 118: } else { // by tags 119: pInfo->retrieveType = CACHESCAN_RETRIEVE_TYPE_SINGLE SCAN_ROW_TYPE(pScanNode->ignoreNull);	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
482	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 69

	步骤描述: 不使用返回值的函数'system'应使用'void'说明.	
	代码片段: 67: #pragma GCC diagnostic push 68: #pragma GCC diagnostic ignored "-Wunused-result" 69: system("clear"); 70: #pragma GCC diagnostic pop 71: return 0;	
	污染路径: 无	
483	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 108
	步骤描述: 不使用返回值的函数'strtolower'应使用'void'说明.	
	代码片段: 106: memset(t, 1, tListLen(t)); 107: const char* a3 = ""; 108: strtolower(t, a3); 109: EXPECT_STREQ(t, ""); 110:	
	污染路径: 函数'strtolower'的定义点.	
484	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 202
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 200: 201: char cmd[128]; 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir);	
	污染路径: 无	
485	缺陷发生位置: /source/os/test/osTests.cpp	行号: 116
	步骤描述: 不使用返回值的函数'taosThreadJoin'应使用'void'说明.	
	代码片段: 114: taosThreadAttrDestroy(&thattr);	

	115: 116: taosThreadJoin(thread1, NULL); 117: taosThreadClear(&thread1); 118:	
	污染路径: 函数'taosThreadJoin'的定义点.	
486	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 27
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 25: if (pSdb == NULL) { 26: terrno = TSDB_CODE_OUT_OF_MEMORY; 27: mError("failed to init sdb since %s", terrstr()); 28: return NULL; 29: }	
	污染路径: 函数'mError'的定义点.	
487	缺陷发生位置: /source/common/src/trow.c	行号: 250
	步骤描述: 不使用返回值的函数'tdSTSTRowIterGetTpVal'应使用'void'说明.	
	代码片段: 248: 249: if (TD_IS_TP_ROW(plter->pRow)) { 250: tdSTSTRowIterGetTpVal(plter, pCol->type, pCol->offset, pVal); 251: } else if (TD_IS_KV_ROW(plter->pRow)) { 252: tdSTSTRowIterGetKvVal(plter, pCol->colId, &plter-> kvIdx, pVal);	
	污染路径: 函数'tdSTSTRowIterGetTpVal'的定义点.	
488	缺陷发生位置: /include/common/tmsg.h	行号: 2564
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 2562: taosMemoryFreeClear(req->ctb.pTag); 2563: taosMemoryFreeClear(req->ctb.stbName); 2564: taosArrayDestroy(req->ctb.tagName);	

	2565: req->ctb.tagName = NULL; 2566: } else if (req->type == TSDB_NORMAL_TABLE) {	
	污染路径: 函数'taosArrayDestroy'的定义点.	
489	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 289
	步骤描述: 不使用返回值的函数'tsortClose'应使用'void'说明.	
	代码片段: 287: return; 288: } 289: tsortClose(pSortHandle); 290: if (pSortHandle->pMergeTree != NULL) { 291: tMergeTreeDestroy(&pSortHandle->pMergeTree);	
	污染路径: 函数'tsortClose'的定义点.	
490	缺陷发生位置: /source/client/src/clientSmlTelnet.c	行号: 281
	步骤描述: 不使用返回值的函数'JUMP_SPACE'应使用'void'说明.	
	代码片段: 279: } 280: 281: JUMP_SPACE(sql, sqlEnd) 282: 283: elements->tags = sql;	
	污染路径: 函数'JUMP_SPACE'的定义点.	
491	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 57
	步骤描述: 不使用返回值的函数'sem_init'应使用'void'说明.	
	代码片段: 55: 56: fs[0]->tsdb = pTsdb; 57: tsem_init(&fs[0]->canEdit, 0, 1); 58: fs[0]->state = TSDB_FS_STATE_NONE; 59: fs[0]->neid = 0;	
	污染路径:	

	expanded from macro 'tsem_init'	
492	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 126
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 124: size_t vallen = 0; 125: char *val = rocksdb_get(db, readoptions, "key", 3, &vallen, &err); 126: snprintf(buf, vallen + 5, "val:%s", val); 127: printf("%ld %ld %s\n", strlen(val), vallen, buf); 128:	
	污染路径: 无	
493	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 240
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 238: printf("syncPingPrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 239: fflush(NULL); 240: taosMemoryFree(serialized); 241: } 242:	
	污染路径: 函数'taosMemoryFree'的定义点.	
494	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 57
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 55: default: 56: SCH_TASK_ELOG("unknown rsp msg, type:%s, status:%s", TMSG_INFO(msgType), jobTaskStatusStr(taskStatus)); 57: SCH_ERR_RET(TSDB_CODE_INVALID_MSG); 58: } 59:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	

495	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 131
	步骤描述: 不使用返回值的函数'osUpdate'应使用'void'说明.	
	代码片段: 129: 130: static bool dmCheckDiskSpace() { 131: osUpdate(); 132: // availability 133: bool ret = true;	
	污染路径: 函数'osUpdate'的定义点.	
496	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 584
	步骤描述: 不使用返回值的函数'setBufPageDirty'应使用'void'说明.	
	代码片段: 582: (*rows) += 1; 583: 584: setBufPageDirty(pPage, true); 585: releaseBufPage(pInfo->pBuf, pPage); 586: }	
	污染路径: 函数'setBufPageDirty'的定义点.	
497	缺陷发生位置: /source/libs/stream/src/streamTask.c	行号: 330
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 328: 329: if (pTask->pState) { 330: qDebug("s-task:0x%x start to free task state", taskId); 331: streamStateClose(pTask->pState, status == TASK_STATUS__DROPPING); 332: }	
	污染路径: 函数'qDebug'的定义点.	
498	缺陷发生位置: /source/client/src/clientTmqConnector.c	行号: 85
	步骤描述:	

	不使用返回值的函数'jniDebug'应使用'void'说明.	
	代码片段: 83: 84: atomic_store_32(&__init_assignment, 2); 85: jniDebug("tmq method assignment finished"); 86: } 87:	
	污染路径: 函数'jniDebug'的定义点.	
499	缺陷发生位置: /source/util/src/tlruccache.c	行号: 328
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 326: shard->usage -= old->totalCharge; 327: 328: taosArrayPush(deleted, &old); 329: } 330: }	
	污染路径: 函数'taosArrayPush'的定义点.	
500	缺陷发生位置: /source/util/src/tref.c	行号: 81
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 79: } 80: 81: taosThreadMutexLock(&tsRefMutex); 82: 83: for (i = 0; i < TSDB_REF_OBJECTS; ++i) {	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
501	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 252
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 250: if (0 == strcasecmp(option, "meta")) { 251: gCTGDebug.metaEnable = enable;	

	252: qDebug("catalog meta debug set to %d", enable); 253: return TSDB_CODE_SUCCESS; 254: }	
	污染路径: 函数'qDebug'的定义点.	
502	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 79
	步骤描述: 不使用返回值的函数'tjsonAddItemToObject'应使用'void'说明.	
	代码片段: 77: 78: Sjson *item = tjsonCreateObject(); 79: tjsonAddItemToObject(items, "db", item); 80: 81: tjsonAddStringToObject(item, "name", mndGetDbStr(pObj->name));	
	污染路径: 函数'tjsonAddItemToObject'的定义点.	
503	缺陷发生位置: /source/dnode/mnode/impl/src/mndInfoSchema.c	行号: 106
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 104: STableMetaRsp *pMeta = taosHashGet(pMnode->infosMeta, tbName, strlen(tbName)); 105: if (NULL == pMeta) { 106: mError("invalid information schema table name:%s", tbName); 107: terrno = TSDB_CODE_MND_INVALID_SYS_TABLENAME; 108: return -1;	
	污染路径: 函数'mError'的定义点.	
504	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 400
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 398: metaULock(pMeta); 399: } 400: tdbFree(pData);	

	401: return pSchema; 402:	
	污染路径: 函数'tdbFree'的定义点.	
505	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 142
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 140: extractEqualOnCondColsFromOper(pInfo, pOperNode, &left, &right); 141: taosArrayPush(leftTagEqCols, &left); 142: taosArrayPush(rightTagEqCols, &right); 143: } 144: }	
	污染路径: 函数'taosArrayPush'的定义点.	
506	缺陷发生位置: /source/dnode/vnode/src/meta/metatable.c	行号: 311
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 309: _exit; 310: tdbFree(pKey); 311: tdbFree(pData); 312: metaDebug("vgId:%d, super table %s uid:%" PRIu64 " is dropped", TD_VID(pMeta->pVnode), pReq->name, pReq->suid); 313: return 0;	
	污染路径: 函数'tdbFree'的定义点.	
507	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmFile.c	行号: 101
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 99: 100: if (taosStatFile(file, NULL, NULL, NULL) < 0) { 101: dlInfo("vnode file:%s not exist", file); 102: return 0;	

	103: }	
	污染路径: 函数'dInfo'的定义点.	
508	缺陷发生位置: /source/libs/command/src/command.c	行号: 92
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 90: 91: static int32_t setDescResultIntoDataBlock(bool sysInfoUser, SSDataBlock* pBlock, int32_t numRows, STableMeta* pMeta) { 92: blockDataEnsureCapacity(pBlock, numRows); 93: pBlock->info.rows = 0; 94:	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
509	缺陷发生位置: /contrib/test/craft/raftServer.c	行号: 162
	步骤描述: 不使用返回值的函数'splitString'应使用'void'说明.	
	代码片段: 160: 161: char arr[2][MAX_TOKEN_LEN]; 162: splitString(msg, "--", arr, 2); 163: putKV(arr[0], arr[1]); 164:	
	污染路径: 函数'splitString'的定义点.	
510	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 85
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 83: if (pMgmt->connCache == NULL) { 84: terrno = TSDB_CODE_OUT_OF_MEMORY; 85: mError("failed to alloc profile cache since %s", terrstr()); 86: return -1; 87: }	

	污染路径: 函数'mError'的定义点.	
511	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmHandle.c	行号: 69
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 67: bool deployed = false; 68: if (dmWriteFile(pInput->path, pInput->name, deployed) != 0) { 69: dError("failed to write qnode file since %s", terrstr()); 70: return -1; 71: }	
	污染路径: 函数'dError'的定义点.	
512	缺陷发生位置: /source/libs/scheduler/src/schStatus.c	行号: 71
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 69: if (NULL == pJob) { 70: qDebug("Acquire sch job failed, may be dropped, jobId:0x%" PRIx64, jobId); 71: SCH_ERR_RET(TSDB_CODE_SCH_STATUS_ERROR); 72: } 73:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
513	缺陷发生位置: /source/libs/scheduler/src/schJob.c	行号: 142
	步骤描述: 不使用返回值的函数'SCH_ERR_JRET'应使用'void'说明.	
	代码片段: 140: default: 141: SCH_JOB_ELOG("invalid job status:%s", jobTaskStatusStr(oriStatus)); 142: SCH_ERR_JRET(TSDB_CODE_APP_ERROR); 143: } 144:	
	污染路径:	

	函数'SCH_ERR_JRET'的定义点.	
514	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 115
	步骤描述: 不使用返回值的函数'SCH_TASK_ELOG'应使用'void'说明.	
	代码片段: 113: 114: if (taosHashPut(pTask->execNodes, &execId, sizeof(execId), &nodeInfo, sizeof(nodeInfo))) { 115: SCH_TASK_ELOG("taosHashPut nodeInfo to execNodes failed, errno:%d", errno); 116: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 117: }	
	污染路径: 函数'SCH_TASK_ELOG'的定义点.	
515	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 64
	步骤描述: 不使用返回值的函数'taosThreadOnce'应使用'void'说明.	
	代码片段: 62: 63: int32_t fmFuncMgtInit() { 64: taosThreadOnce(&functionHashTableInit, doInitFunctionTable); 65: return initFunctionCode; 66: }	
	污染路径: 函数'taosThreadOnce'的定义点.	
516	缺陷发生位置: /source/util/src/theap.c	行号: 257
	步骤描述: 不使用返回值的函数'pqHeapify'应使用'void'说明.	
	代码片段: 255: pqHeapify(pq, i, pqContainerSize(pq)); 256: } 257: pqHeapify(pq, 0, pqContainerSize(pq)); 258: } 259: }	
	污染路径: 函数'pqHeapify'的定义点.	

517	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 35
	步骤描述: 不使用返回值的函数'SCH_TASK_ELOG'应使用'void'说明.	
	代码片段: <pre> 33: case TDMT_SCH_MERGE_FETCH_RSP: 34: if (lastMsgType != reqMsgType) { 35: SCH_TASK_ELOG("rsp msg type mis-match, last sent msgType:%s, rspType:%s", TMSG_INFO(lastMsgType), 36: TMSG_INFO(msgType)); 37: SCH_ERR_RET(TSDB_CODE_QW_MSG_ERROR); </pre>	
	污染路径: 函数'SCH_TASK_ELOG'的定义点.	
518	缺陷发生位置: /source/client/src/clientEnv.c	行号: 106
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: <pre> 104: pRequest->metric.planCostUs, pRequest- >metric.execCostUs); 105: 106: atomic_add_fetch_64((int64_t *)&pActivity- >queryElapsedTime, duration); 107: reqType = SLOW_LOG_TYPE_QUERY; 108: } </pre>	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
519	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 75
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: <pre> 73: code = tsdbReadFile(reader->fd[ftype], reader->config- >files[ftype].file.size - sizeof(STombFooter), 74: (uint8_t *)reader->tombFooter, sizeof(STombFooter)); 75: TSDB_CHECK_CODE(code, lino, _exit); 76: } 77: reader->ctx->tombFooterLoaded = true; </pre>	
	污染路径:	

	函数'TSDB_CHECK_CODE'的定义点.	
520	缺陷发生位置: /source/libs/index/src/index.c	行号: 248
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 246: SArray* trslt = NULL; 247: idxTermSearch(index, qterm, &trslt); 248: taosArrayPush(iRslts, (void*)&trslt); 249: } 250: idxMergeFinalResults(iRslts, opera, result);	
	污染路径: 函数'taosArrayPush'的定义点.	
521	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 471
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 469: } 470: varDataSetLen(output, dataLen); 471: colDataSetVal(pOutputData, k, output, false); 472: output += varDataTLen(output); 473: }	
	污染路径: 函数'colDataSetVal'的定义点.	
522	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 55
	步骤描述: 不使用返回值的函数'CTG_ERR_JRET'应使用'void'说明.	
	代码片段: 53: if (NULL == pBatchs) { 54: ctgError("taosHashInit %d batch failed", taskNum); 55: CTG_ERR_JRET(TSDB_CODE_OUT_OF_MEMORY); 56: } 57:	
	污染路径: 函数'CTG_ERR_JRET'的定义点.	
523	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.c	行号: 114
	步骤描述:	

	不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 112: tsdbTFileName(config->tsdb, &config->files[i].file, fname1); 113: code = tsdbOpenFile(fname1, config->szPage, TD_FILE_READ, &reader[0]->fd[i]); 114: TSDB_CHECK_CODE(code, lino, _exit); 115: } 116: }	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
524	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSvr.c	行号: 190
	步骤描述: 不使用返回值的函数'tSerializeSVDropTtlTableReq'应使用'void'说明.	
	代码片段: 188: } 189: 190: tSerializeSVDropTtlTableReq((char *)pContNew + sizeof(SMsgHead), reqLenNew, &ttlReq); 191: pContNew->contLen = htonl(reqLenNew); 192: pContNew->vgId = pContOld->vgId;	
	污染路径: 函数'tSerializeSVDropTtlTableReq'的定义点.	
525	缺陷发生位置: /tests/script/api/apitest.c	行号: 27
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 25: result = taos_query(taos, "create database test precision 'us';"); 26: taos_free_result(result); 27: usleep(100000); 28: taos_select_db(taos, "test"); 29:	
	污染路径: 无	
526	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 295
	步骤描述:	

	不使用返回值的函数'planDebug'应使用'void'说明. 代码片段: <pre> 293: return; 294: } 295: planDebug("%s", pName); 296: void* plt = taosHashIterate(pHash, NULL); 297: while (NULL != plt) { </pre>	
	污染路径: 函数'planDebug'的定义点.	
527	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 498
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 496: destroyTFileValue(taosArrayGetP(data, i)); 497: } 498: taosArrayDestroy(data); 499: 500: std::string colName("voltage"); </pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
528	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 563
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: <pre> 561: 562: tdbFree(pStbCur->pKey); 563: tdbFree(pStbCur->pVal); 564: } 565: </pre>	
	污染路径: 函数'tdbFree'的定义点.	
529	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 115
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: <pre> 113: 114: static void mndPullupTtl(SMnode *pMnode) { </pre>	

	406: if (length < 0) { 407: qError("charset:%s to %s. val:%s convert json value failed.", DEFAULT_UNICODE_ENCODEC, tsCharset, 408: pTagVal->pData); 409: taosMemoryFree(tagJsonValue);	
	污染路径: 函数'qError'的定义点.	
533	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 201
	步骤描述: 不使用返回值的函数'ctgDebug'应使用'void'说明.	
	代码片段: 199: ctgAcquireDBCACHE(pCtg, dbFName, &dbCache); 200: if (NULL == dbCache) { 201: ctgDebug("db %s not in cache", dbFName); 202: goto _return; 203: }	
	污染路径: 函数'ctgDebug'的定义点.	
534	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 111
	步骤描述: 不使用返回值的函数'tSerializeSUseDbReq'应使用'void'说明.	
	代码片段: 109: int32_t bufLen = tSerializeSUseDbReq(NULL, 0, &usedbReq); 110: void *pBuf = (*mallocFp)(bufLen); 111: tSerializeSUseDbReq(pBuf, bufLen, &usedbReq); 112: 113: *msg = pBuf;	
	污染路径: 函数'tSerializeSUseDbReq'的定义点.	
535	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.c	行号: 105
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 103: if (writer->blockData[pidx].nRow > 0) { 104: code = tsdbDataFileWriteBlockData(writer->dataWriter, &writer->blockData[pidx]);	

	105: TSDB_CHECK_CODE(code, lino, _exit); 106: } 107:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
536	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 101
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 99: 100: code = tsdbSttFileReaderOpen(fobj->fname, &config, &sttReader); 101: TSDB_CHECK_CODE(code, lino, _exit); 102: 103: code = TARRAY2_APPEND(reader->sttReaderArr, sttReader);	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
537	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 55
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 53: 54: code = TARRAY2_APPEND(rtnr->fopArr, op); 55: TSDB_CHECK_CODE(code, lino, _exit); 56: 57: const char *object_name = taosDirEntryBaseName((char *)fobj->fname);	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
538	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 208
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 206: int32_t tsdbTFileObjRef(STFileObj *fobj) { 207: int32_t nRef; 208: taosThreadMutexLock(&fobj->mutex);	

	209: ASSERT(fobj->ref > 0 && fobj->state == TSDB_FSTATE_LIVE); 210: nRef = ++fobj->ref;	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
539	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 116
	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段: 114: ret = tdbBegin(pEnv, &txn, tdbDefaultMalloc, tdbDefaultFree, NULL, TDB_TXN_WRITE TDB_TXN_READ_UNCOMMITTED); 115: if (ret < 0) { 116: tdbOsFree(pBt); 117: return -1; 118: }	
	污染路径: 函数'tdbOsFree'的定义点.	
540	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 721
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 719: int32_t tagNum = cJSON_GetArraySize(tags); 720: if (unlikely(tagNum == 0)) { 721: uError("SML:Tag should not be empty"); 722: return TSDB_CODE_TSC_INVALID_JSON; 723: }	
	污染路径: 函数'uError'的定义点.	
541	缺陷发生位置: /source/util/src/thash.c	行号: 207
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 205: pNewNode->next = pNode; 206: pe->num++; 207: atomic_add_fetch_64(&pHashObj->size, 1); 208: }	

	209: }	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
542	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 369
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 367: } 368: 369: taosArrayPush(pBuf->pData, &p); 370: } 371:	
	污染路径: 函数'taosArrayPush'的定义点.	
543	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 881
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 879: colDataSetNULL(pDstCol, rowIndex); 880: } else { 881: colDataSetVal(pDstCol, rowIndex, pStart, false); 882: } 883: pStart += pDstCol->info.bytes;	
	污染路径: 函数'colDataSetVal'的定义点.	
544	缺陷发生位置: /tools/shell/src/shellWebsocket.c	行号: 181
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 179: } 180: if (!rows) { 181: taosCloseFile(&pFile); 182: return 0; 183: }	
	污染路径: 函数'taosCloseFile'的定义点.	

545	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 364
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 362: 363: if (updateVnodesList && vmWriteVnodeListToFile(pMgmt) != 0) { 364: dError("failed to write vnode list since %s", terrstr()); 365: return -1; 366: }	
	污染路径: 函数'dError'的定义点.	
546	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 82
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 80: SNode* pPrimKeyCond = pJoinNode->pPrimKeyCond; 81: if (nodeType(pPrimKeyCond) != QUERY_NODE_OPERATOR) { 82: qError("not support this in join operator, %s", idStr); 83: return; // do not handle this 84: }	
	污染路径: 函数'qError'的定义点.	
547	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 181
	步骤描述: 不使用返回值的函数'tDebug'应使用'void'说明.	
	代码片段: 179: static void uvHandleActivityTimeout(uv_timer_t* handle) { 180: SSvrConn* conn = handle->data; 181: tDebug("%p timeout since no activity", conn); 182: } 183:	
	污染路径: 函数'tDebug'的定义点.	
548	缺陷发生位置:	行号:

	/source/dnode/vnode/src/meta/metaQuery.c	407
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 405: metaULock(pMeta); 406: } 407: tdbFree(pData); 408: return NULL; 409: }	
	污染路径: 函数'tdbFree'的定义点.	
549	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 324
	步骤描述: 不使用返回值的函数'COMMON_INDEX'应使用'void'说明.	
	代码片段: 322: 323: uint8_t val; 324: COMMON_INDEX(inp, 0b111111, val); 325: s->val = (s->val & fstStateDict[s->state].val) val; 326: }	
	污染路径: 函数'COMMON_INDEX'的定义点.	
550	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 89
	步骤描述: 不使用返回值的函数'dmStop'应使用'void'说明.	
	代码片段: 87: #endif 88: 89: dmStop(); 90: } 91:	
	污染路径: 函数'dmStop'的定义点.	
551	缺陷发生位置: /source/libs/executor/src/tlinearhash.c	行号: 304
	步骤描述: 不使用返回值的函数'doAddToBucket'应使用'void'说明.	
	代码片段:	

	<pre> 302: if (pHashObj->bits == 0) { 303: SLHashBucket* pBucket = pHashObj->pBucket[0]; 304: doAddToBucket(pHashObj, pBucket, 0, key, keyLen, data, size); 305: } else { 306: int32_t hashVal = pHashObj->hashFn(key, keyLen); </pre>	
	污染路径: 函数'doAddToBucket'的定义点.	
552	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 82
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: <pre> 80: if (pDataFile != NULL) { 81: if (taosWriteFile(pDataFile, pRpcMsg->pCont, pRpcMsg->contLen) < 0) { 82: tlInfo("failed to write data file, reason:%s", strerror(errno)); 83: } 84: } </pre>	
	污染路径: 函数'tlInfo'的定义点.	
553	缺陷发生位置: /source/libs/parser/src/parInsertStmt.c	行号: 186
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 184: } 185: taosArrayDestroy(pTagArray); 186: taosArrayDestroy(tagName); 187: 188: return code; </pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
554	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 1607
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: <pre> 1605: nodesNodeToString((SNode*)pSubplan, false, &pStr, </pre>	

	NULL); 1606: if (NULL == pRuleName) { 1607: qDebugL("before split, JsonPlan: %s", pStr); 1608: } else { 1609: qDebugL("apply split %s rule, JsonPlan: %s", pRuleName, pStr);	
	污染路径: 函数'qDebugL'的定义点.	
555	缺陷发生位置: /source/libs/tdb/src/db/tdbTable.c	行号: 258
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 256: } 257: tdbFree(pKey); 258: tdbFree(pValue); 259: tdbTbcClose(pCur); 260:	
	污染路径: 函数'tdbFree'的定义点.	
556	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 161
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 159: return NULL; 160: } else { 161: mTrace("conn:%u, is created, data:%p user:%s", pConn->id, pConn, user); 162: return pConn; 163: }	
	污染路径: 函数'mTrace'的定义点.	
557	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 4141
	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: 4139: } 4140: if (TSDB_CODE_SUCCESS != code) {	

	4141: nodesError("specificNodeToMsg error node = %s", nodesNodeName(nodeType(pObj))); 4142: } 4143: // nodesWarn("specificNodeToMsg node = %s, after tlv count = %d", nodesNodeName(nodeType(pObj)), pEncoder->tlvCount);	
	污染路径: 函数'nodesError'的定义点.	
558	缺陷发生位置: /include/common/tmsg.h	行号: 2425
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 2423: _err: 2424: taosArrayDestroy(pRebInfo->removedConsumers); 2425: taosArrayDestroy(pRebInfo->newConsumers); 2426: taosMemoryFreeClear(pRebInfo); 2427: return NULL;	
	污染路径: 函数'taosArrayDestroy'的定义点.	
559	缺陷发生位置: /source/libs/stream/src/streamTask.c	行号: 362
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 360: taosMemoryFree(pTask); 361: 362: qDebug("s-task:0x%x free task completed", taskId); 363: } 364:	
	污染路径: 函数'qDebug'的定义点.	
560	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 297
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 295: if (pMgmt->hash == NULL) { 296: terrno = TSDB_CODE_OUT_OF_MEMORY; 297: dError("failed to init vnode hash since %s", terrstr());	

	298: return -1; 299: }	
	污染路径: 函数'dError'的定义点.	
561	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 294
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 292: 293: if (true == pThrd->quit) { 294: tlInfo("work thread received quit msg, destroy conn"); 295: destroyConn(conn, true); 296: return;	
	污染路径: 函数'tlInfo'的定义点.	
562	缺陷发生位置: /source/libs/scheduler/test/schedulerTests.cpp	行号: 630
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 628: strcpy(qnodeAddr.fqdn, "qnode0.ep"); 629: qnodeAddr.port = 6031; 630: taosArrayPush(qnodeList, &qnodeAddr); 631: 632: int32_t code = schedulerInit();	
	污染路径: 函数'taosArrayPush'的定义点.	
563	缺陷发生位置: /source/libs/scalar/src/scVector.c	行号: 418
	步骤描述: 不使用返回值的函数'geosFreeBuffer'应使用'void'说明.	
	代码片段: 416: 417: taosMemoryFree(output); 418: geosFreeBuffer(t); 419: 420: return;	
	污染路径:	

	函数'geosFreeBuffer'的定义点.	
564	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 63: } 64: cJSON *pIndex = cJSON_CreateIntArray(arr, pSyncIndexMgr->replicaNum); 65: taosMemoryFree(arr); 66: cJSON_AddItemToObject(pRoot, "index", pIndex); 67: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
565	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 68
	步骤描述: 不使用返回值的函数'tsdbSttFileReaderClose'应使用'void'说明.	
	代码片段: 66: if (code) { 67: TSDB_ERROR_LOG(TD_VID(config->tsdb->pVnode), lino, code); 68: tsdbSttFileReaderClose(reader); 69: } 70: return code;	
	污染路径: 函数'tsdbSttFileReaderClose'的定义点.	
566	缺陷发生位置: /utils/test/c/createTable.c	行号: 79
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 77: taos_free_result(pRes); 78: 79: sprintf(qstr, "use %s", dbName); 80: pRes = taos_query(con, qstr); 81: code = taos_errno(pRes);	
	污染路径: 无	
567	缺陷发生位置:	行号:

	/tests/script/api/apitest.c		79
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.		
	代码片段: 77: taos_free_result(result); 78: // super tables subscription 79: usleep(1000000); 80: } 81:		
	污染路径: 无		
568	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp		行号: 418
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.		
	代码片段: 416: 417: for (int i = 1; i <= nData; i++) { 418: sprintf(key, "key%d", i); 419: // sprintf(val, "value%d", i); 420:		
	污染路径: 无		
569	缺陷发生位置: /source/dnode/mnode/impl/src/mndTopic.c		行号: 929
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.		
	代码片段: 927: STR_TO_VARSTR(schemaJson, string); 928: }else{ 929: mError("mndRetrieveTopic build schema error json:%p, json len:%zu", string, len); 930: } 931: taosMemoryFree(string);		
	污染路径: 函数'mError'的定义点.		
570	缺陷发生位置: /tests/script/api/demoapi.c		行号: 132
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.		

	代码片段: <pre> 130: int64_t affected; 131: for (; j < g_num_of_rec -2; j++) { 132: sprintf(command, "insert into t%"PRId64" " 133: "values(%" PRId64 ", %f, %"PRId64", " 134: "'%c%d', '%s%c%d', '%c%d')", </pre>	
	污染路径: 无	
571	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 427
	步骤描述: 不使用返回值的函数'tmsgSendRsp'应使用'void'说明.	
	代码片段: <pre> 425: initRpcMsg(&msg, 0, pBuf, sizeof(SMsgHead) + len); 426: 427: tmsgSendRsp(&msg); 428: qDebug("s-task:%s level:%d notify upstream:0x %x(vglD:%d) to continue process data in WAL", pTask- >id.idStr, 429: pTask->info.taskLevel, pReq->upstreamTaskId, pReq->upstreamNodeId); </pre>	
	污染路径: 函数'tmsgSendRsp'的定义点.	
572	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 278
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: <pre> 276: int32_t code = tsdbDataFileReadBrinBlock(plter- >pReader, plter->pCurrentBlk, &plter->block); 277: if (code != TSDB_CODE_SUCCESS) { 278: tsdbError("failed to read brinBlock from file, code: %s", tstrerror(code)); 279: return NULL; 280: } </pre>	
	污染路径: 函数'tsdbError'的定义点.	
573	缺陷发生位置: /source/dnode/vnode/src/sma/smaEnv.c	行号: 48
	步骤描述:	

	不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: <pre> 46: if (old != 2) break; 47: if (++nLoops > 1000) { 48: sched_yield(); 49: nLoops = 0; 50: }</pre>	
	污染路径: 函数'sched_yield'的定义点.	
574	缺陷发生位置: /source/libs/parser/test/parInitialCTest.cpp	行号: 33
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 31: 32: auto setCompactDbReq = [&](const char* pDb, int64_t start = INT64_MIN, int64_t end = INT64_MAX) { 33: snprintf(expect.db, sizeof(expect.db), "0.%s", pDb); 34: expect.timeRange.skey = start; 35: expect.timeRange.ekey = end;</pre>	
	污染路径: 无	
575	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 373
	步骤描述: 不使用返回值的函数'rocksdb_free'应使用'void'说明.	
	代码片段: <pre> 371: if (NULL != err) { 372: tsdbError("vgId:%d, %s failed at line %d since %s", TD_VID(pTsdb->pVnode), __func__, __LINE__, err); 373: rocksdb_free(err); 374: } 375:</pre>	
	污染路径: 函数'rocksdb_free'的定义点.	
576	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 65
	步骤描述: 不使用返回值的函数'fnInfo'应使用'void'说明.	
	代码片段:	

	63: fnInfo("udfd process exit due to SIGINT or dnode-mgmt called stop"); 64: } else { 65: fnInfo("udfd process restart"); 66: udfSpawnUdfd(pData); 67: }	
	污染路径: 函数'fnInfo'的定义点.	
577	缺陷发生位置: /source/common/src/tmisc.c	行号: 80
	步骤描述: 不使用返回值的函数'atomic_add_fetch_32'应使用'void'说明.	
	代码片段: 78: taosCorBeginWrite(&pEpSet->version); 79: pEpSet->epSet = *pNewEpSet; 80: taosCorEndWrite(&pEpSet->version); 81: } 82:	
	污染路径: expanded from macro 'taosCorEndWrite' 函数'atomic_add_fetch_32'的定义点.	
578	缺陷发生位置: /source/libs/transport/src/transSvr.c	行号: 226
	步骤描述: 不使用返回值的函数'transRefSrvHandle'应使用'void'说明.	
	代码片段: 224: if (pHead->persist == 1) { 225: pConn->status = ConnAcquire; 226: transRefSrvHandle(pConn); 227: tDebug("conn %p acquired by server app", pConn); 228: }	
	污染路径: 函数'transRefSrvHandle'的定义点.	
579	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 130
	步骤描述: 不使用返回值的函数'strtrim'应使用'void'说明.	
	代码片段: 128: COPY_STRING_FORM_ID_TOKEN(ep, pEp); 129: strdequote(ep);	

	130: strtrim(ep); 131: if (NULL == pPort) { 132: strcpy(pFqdn, ep);	
	污染路径: 函数'strtrim'的定义点.	
580	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 124
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 122: terrno = TSDB_CODE_INVALID_PARA; 123: int32_t size = tSimpleHashGetSize(pTableMap); 124: tsdbError("failed to locate the uid:%" PRIu64 " in query table uid list, total tables:%d, %s", uid, size, id); 125: return NULL; 126: }	
	污染路径: 函数'tsdbError'的定义点.	
581	缺陷发生位置: /source/util/src/tworker.c	行号: 107
	步骤描述: 不使用返回值的函数'taosAddIntoQset'应使用'void'说明.	
	代码片段: 105: taosThreadMutexLock(&pool->mutex); 106: taosSetQueueFp(queue, fp, NULL); 107: taosAddIntoQset(pool->qset, queue, ahandle); 108: 109: // spawn a thread to process queue	
	污染路径: 函数'taosAddIntoQset'的定义点.	
582	缺陷发生位置: /source/libs/index/src/indexFstDfa.c	行号: 77
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 75: SArray *states = taosArrayInit(0, sizeof(uint32_t)); 76: if (dfaBuilderCacheState(builder, cur, &result)) { 77: taosArrayPush(states, &result); 78: } 79: SHashObj *seen = taosHashInit(12,	

	taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), false, HASH_NO_LOCK);	
	污染路径: 函数'taosArrayPush'的定义点.	
583	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 69
	步骤描述: 不使用返回值的函数'sqlite3_free'应使用'void'说明.	
	代码片段: 67: fprintf(stderr, "SQL error: %s\n", err_msg); 68: 69: sqlite3_free(err_msg); 70: sqlite3_close(db); 71:	
	污染路径: 函数'sqlite3_free'的定义点.	
584	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 334
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 332: rpcClose(pTrans->clientRpc); 333: pTrans->clientRpc = NULL; 334: dDebug("dnode rpc client is closed"); 335: } 336: }	
	污染路径: 函数'dDebug'的定义点.	
585	缺陷发生位置: /source/common/src/tmisce.c	行号: 97
	步骤描述: 不使用返回值的函数'tjsonAddDoubleToObject'应使用'void'说明.	
	代码片段: 95: char tmp[4096] = {0}; 96: 97: tjsonAddDoubleToObject(pJson, "reportVersion", 1); 98: 99: tjsonAddIntegerToObject(pJson, "clusterId", clusterId);	
	污染路径:	

	函数'tjsonAddDoubleToObject'的定义点.	
586	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 97
	步骤描述: 不使用返回值的函数'metaInfo'应使用'void'说明.	
	代码片段: 95: bool needUpgrade = tdbTbExist(ttlTbname, pEnv); 96: if (needUpgrade) { 97: metaInfo("find ttl idx in old version , will convert"); 98: } 99: return needUpgrade;	
	污染路径: 函数'metaInfo'的定义点.	
587	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c	行号: 54
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 52: pMgmt->pQnode = qndOpen(&option); 53: if (pMgmt->pQnode == NULL) { 54: dError("failed to open qnode since %s", terrstr()); 55: qmClose(pMgmt); 56: return -1;	
	污染路径: 函数'dError'的定义点.	
588	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 68
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: 66: double scltLeftVd = 21.0, scltRightVd = 10.0; 67: 68: void scltFreeDataBlock(void *block) { blockDataDestroy(*(SSDataBlock **)block); } 69: 70: void scltInitLogFile() {	
	污染路径: 函数'blockDataDestroy'的定义点.	
589	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 278

	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: <pre> 276: taosHashPut(pOutput->pSub->consumerHash, &consumerId, sizeof(int64_t), &newConsumerEp, sizeof(SMqConsumerEp)); 277: taosArrayPush(pOutput->newConsumers, &consumerId); 278: mInfo("sub:%s mq rebalance add new consumer:0x%" PRIx64, pSubKey, consumerId); 279: } 280: }</pre>	
	污染路径: 函数'mInfo'的定义点.	
590	缺陷发生位置: /source/libs/executor/src/projectoperator.c	行号: 58
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: <pre> 56: taosArrayDestroy(pInfo->pPseudoCollInfo); 57: 58: blockDataDestroy(pInfo->pFinalRes); 59: taosMemoryFreeClear(param); 60: }</pre>	
	污染路径: 函数'blockDataDestroy'的定义点.	
591	缺陷发生位置: /source/client/src/clientHb.c	行号: 148
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: <pre> 146: SVgroupInfo *pInfo = taosArrayGet(rsp- >pVgroupInfos, j); 147: if (taosHashPut(vgInfo->vgHash, &pInfo->vgId, sizeof(int32_t), pInfo, sizeof(SVgroupInfo)) != 0) { 148: tscError("hash push failed, errno:%d", errno); 149: code = TSDB_CODE_OUT_OF_MEMORY; 150: goto _return; </pre>	
	污染路径: 函数'tscError'的定义点.	

592	缺陷发生位置: /source/util/src/tcache.c	行号: 159
	步骤描述: 不使用返回值的函数'taosThreadAttrInit'应使用'void'说明.	
	代码片段: 157: 158: TdThreadAttr thattr; 159: taosThreadAttrInit(&thattr); 160: taosThreadAttrSetDetachState(&thattr, PTHREAD_CREATE_JOINABLE); 161:	
	污染路径: 函数'taosThreadAttrInit'的定义点.	
593	缺陷发生位置: /source/libs/scheduler/src/schStatus.c	行号: 84
	步骤描述: 不使用返回值的函数'SCH_RET'应使用'void'说明.	
	代码片段: 82: if (NULL == pJob) { 83: schDirectPostJobRes(pReq, errCode); 84: SCH_RET(code); 85: } 86:	
	污染路径: 函数'SCH_RET'的定义点.	
594	缺陷发生位置: /source/dnode/mnode/impl/src/mndShow.c	行号: 38
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 36: if (pMgmt->cache == NULL) { 37: terrno = TSDB_CODE_OUT_OF_MEMORY; 38: mError("failed to alloc show cache since %s", terrstr()); 39: return -1; 40: }	
	污染路径: 函数'mError'的定义点.	
595	缺陷发生位置: /tests/script/api/stmtTest.c	行号: 220
	步骤描述:	

	不使用返回值的函数'taos_stmt_add_batch'应使用'void'说明.	
	代码片段: <pre> 218: c10len=strlen(v.c10); 219: taos_stmt_bind_param(stmt, params); 220: taos_stmt_add_batch(stmt); 221: } 222: </pre>	
	污染路径: 函数'taos_stmt_add_batch'的定义点.	
596	缺陷发生位置: /source/os/src/osTimezone.c	行号: 1000
	步骤描述: 不使用返回值的函数'setenv'应使用'void'说明.	
	代码片段: <pre> 998: // } 999: 1000: setenv("TZ", tz, 1); 1001: tzset(); 1002: } </pre>	
	污染路径: 无	
597	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 50
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: <pre> 48: if (NULL == schMgmt.hbConnections) { 49: qError("taosHashInit hb connections failed"); 50: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 51: } 52: </pre>	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
598	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 172
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: <pre> 170: } 171: </pre>	

	172: dDebug("reset dnode list on startup"); 173: dmResetEps(pData, pData->dnodeEps); 174:	
	污染路径: 函数'dDebug'的定义点.	
599	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 314
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 312: char *total = (char *)taosMemoryCalloc(1, SHELL_MAX_COMMAND_SIZE); 313: memset(cmd->command + cmd->commandSize, 0, SHELL_MAX_COMMAND_SIZE - cmd->commandSize); 314: sprintf(total, "%s%s", cmd->buffer, cmd->command); 315: 316: char *reg_str =	
	污染路径: 无	
600	缺陷发生位置: /source/util/src/tutil.c	行号: 418
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 416: // sprintf(ipStr, "0x%x:%u.%u.%u", ipInt, ipInt & 0xFF, (ipInt >> 8) & 0xFF, (ipInt >> 16) & 0xFF, (uint8_t)(ipInt 417: // >> 24)); 418: sprintf(ipStr, "%u.%u.%u.%u", ipInt & 0xFF, (ipInt >> 8) & 0xFF, (ipInt >> 16) & 0xFF, (uint8_t)(ipInt >> 24)); 419: return ipStr; 420: }	
	污染路径: 无	
601	缺陷发生位置: /source/util/src/tworker.c	行号: 76
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 74: 75: taosBlockSIGPIPE();	

	76: setThreadName(pool->name); 77: worker->pid = taosGetSelfPthreadId(); 78: ulInfo("worker:%s:%d is running, thread:%08" PRIu64, pool->name, worker->id, worker->pid);	
	污染路径: expanded from macro 'setThreadName'	
602	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 93
	步骤描述: 不使用返回值的函数'taosFillResultDataBlock'应使用'void'说明.	
	代码片段: 91: 92: int32_t numOfResultRows = pResultInfo->capacity - pResBlock->info.rows; 93: taosFillResultDataBlock(pInfo->pFillInfo, pResBlock, numOfResultRows); 94: 95: pInfo->curGroupId = pInfo->existNewGroupBlock->info.id.groupId;	
	污染路径: 函数'taosFillResultDataBlock'的定义点.	
603	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 99
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 97: ASSERT(p->pData != NULL && p->>nullbitmap != NULL); 98: 99: taosArrayPush(pBlockList, &res); 100: *dataBlockId = taosArrayGetSize(pBlockList) - 1; 101: res->info.id.blockId = *dataBlockId;	
	污染路径: 函数'taosArrayPush'的定义点.	
604	缺陷发生位置: /Utils/tsim/src/simSystem.c	行号: 32
	步骤描述: 不使用返回值的函数'taosInitCfg'应使用'void'说明.	
	代码片段: 30: int32_t simInitCfg() { 31: taosCreateLog("simlog", 1, configDir, NULL, NULL, NULL,	

	NULL, 1); 32: taosInitCfg(configDir, NULL, NULL, NULL, NULL, 1); 33: 34: SConfig *pCfg = taosGetCfg();	
	污染路径: 函数'taosInitCfg'的定义点.	
605	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 165
	步骤描述: 不使用返回值的函数'tNameFromString'应使用'void'说明.	
	代码片段: 163: SName sn = {0}; 164: char dbname[TSDB_DB_FNAME_LEN + VARSTR_HEADER_SIZE] = {0}; 165: tNameFromString(&sn, db, T_NAME_ACCT T_NAME_DB); 166: 167: tNameGetDbName(&sn, varDataVal(dbname));	
	污染路径: 函数'tNameFromString'的定义点.	
606	缺陷发生位置: /source/util/src/tidpool.c	行号: 74
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 72: } 73: 74: taosThreadMutexUnlock(&pIdPool->mutex); 75: } 76:	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
607	缺陷发生位置: /source/client/src/clientStmt.c	行号: 145
	步骤描述: 不使用返回值的函数'STMT_ERR_RET'应使用'void'说明.	
	代码片段: 143: pStmt->exec.pRequest->body.resInfo.fields = taosMemoryMalloc(size); 144: if (NULL == pStmt->exec.pRequest-	

	<pre>>body.resInfo.fields) { 145: STMT_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 146: } 147: memcpy(pStmt->exec.pRequest->body.resInfo.fields, pRes->fields, size);</pre>	
	污染路径: 函数'STMT_ERR_RET'的定义点.	
608	缺陷发生位置: /source/dnode/qnode/src/qnode.c	行号: 78
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.	
	代码片段: <pre>76: int32_t code = -1; 77: SReadHandle handle = {.pMsgCb = &pQnode->msgCb}; 78: qTrace("message in qnode queue is processing"); 79: 80: switch (pMsg->msgType) {</pre>	
	污染路径: 函数'qTrace'的定义点.	
609	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smHandle.c	行号: 52
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre>50: if (pInput->pData->dnodeld != 0 && dropReq.dnodeld != pInput->pData->dnodeld) { 51: terrno = TSDB_CODE_INVALID_OPTION; 52: dError("failed to drop snode since %s", terrstr()); 53: return -1; 54: }</pre>	
	污染路径: 函数'dError'的定义点.	
610	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 130
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre>128: void createNewTable(TAOS* pConn, int32_t index) { 129: char str[1024] = {0}; 130: sprintf(str, "create table tu%d using st2 tags(%d)",</pre>	

	index, index); 131: 132: TAOS_RES* pRes = taos_query(pConn, str);	
	污染路径: 无	
611	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 330
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: 328: char *t = taosMemoryCalloc(1, inputLen); 329: if (t == NULL) { 330: sclError("Out of memory"); 331: terrno = TSDB_CODE_OUT_OF_MEMORY; 332: taosMemoryFree(data);	
	污染路径: 函数'sclError'的定义点.	
612	缺陷发生位置: /examples/c/asyncdemo.c	行号: 115
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 113: queryDB(taos, sql); 114: 115: sprintf(sql, "use %s", db); 116: queryDB(taos, sql); 117:	
	污染路径: 无	
613	缺陷发生位置: /source/util/src/tlruache.c	行号: 347
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 345: taosLRUEntryFree(entry); 346: } 347: taosArrayDestroy(lastReferenceList); 348: } 349:	

	污染路径: 函数'taosArrayDestroy'的定义点.	
614	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 134
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 132: 133: // restore value is not enough buffer available 134: atomic_add_fetch_64(&tsQueryBufferSizeBytes, t); 135: } 136:	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
615	缺陷发生位置: /source/os/src/osSignal.c	行号: 72
	步骤描述: 不使用返回值的函数'signal'应使用'void'说明.	
	代码片段: 70: } 71: 72: void taosIgnSignal(int32_t signum) { signal(signum, SIG_IGN); } 73: 74: void taosDflSignal(int32_t signum) { signal(signum, SIG_DFL); }	
	污染路径: 无	
616	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 143
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 141: s = syncUtilPrintBin((char*)(pMsg->data), pMsg->dataLen); 142: cJSON_AddStringToObject(pRoot, "data", s); 143: taosMemoryFree(s); 144: s = syncUtilPrintBin2((char*)(pMsg->data), pMsg->dataLen); 145: cJSON_AddStringToObject(pRoot, "data2", s);	

	污染路径: 函数'taosMemoryFree'的定义点.	
617	缺陷发生位置: /source/libs/planner/test/planTestMain.cpp	行号: 68
	步骤描述: 不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: 66: 67: taosRemoveDir(path); 68: taosMkDir(path); 69: tstrncpy(tsLogDir, path, PATH_MAX); 70: if (taosInitLog("taoslog", 1) != 0) {	
	污染路径: 函数'taosMkDir'的定义点.	
618	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 33
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 31: static int32_t initLog() { 32: char logName[12] = {0}; 33: snprintf(logName, sizeof(logName), "%slog", "udfc"); 34: return taosCreateLog(logName, 1, configDir, NULL, NULL, NULL, NULL, 0); 35: }	
	污染路径: 无	
619	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 355
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 353: pTrans->serverRpc = rpcOpen(&rpclnit); 354: if (pTrans->serverRpc == NULL) { 355: dError("failed to init dnode rpc server"); 356: return -1; 357: }	
	污染路径: 函数'dError'的定义点.	
620	缺陷发生位置:	行号:

	/source/libs/executor/src/cachescanoperator.c	306
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 304: taosMemoryFree(pInfo->pDstSlotIds); 305: taosArrayDestroy(pInfo->pCidList); 306: taosArrayDestroy(pInfo->pUidList); 307: taosArrayDestroy(pInfo->matchInfo.pList); 308: tableListDestroy(pInfo->pTableList);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
621	缺陷发生位置: /source/os/src/osString.c	行号: 215
	步骤描述: 不使用返回值的函数'atomic_sub_fetch_32'应使用'void'说明.	
	代码片段: 213: 214: atomic_store_8(&gConv[type][idx].inUse, 0); 215: atomic_sub_fetch_32(&convUsed[type], 1); 216: } 217:	
	污染路径: 函数'atomic_sub_fetch_32'的定义点.	
622	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 90
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.	
	代码片段: 88: task.taskCtx = taosMemoryCalloc(1, sizeof(SCtgDbVgCtx)); 89: if (NULL == task.taskCtx) { 90: CTG_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 91: } 92:	
	污染路径: 函数'CTG_ERR_RET'的定义点.	
623	缺陷发生位置: /source/util/src/tcache.c	行号: 113
	步骤描述: 不使用返回值的函数'taosThreadRwlockWrlock'应使用'void'说明.	

	代码片段: 111: static FORCE_INLINE void __trashcan_wr_lock(SCacheObj *pCacheObj) { 112: #if defined(LINUX) 113: taosThreadRwlockWrlock(&pCacheObj->lock); 114: #else 115: taosThreadMutexLock(&pCacheObj->lock);	
	污染路径: 函数'taosThreadRwlockWrlock'的定义点.	
624	缺陷发生位置: /contrib/test/rocksdb/main.c	行号: 167
	步骤描述: 不使用返回值的函数'kvSerial'应使用'void'说明.	
	代码片段: 165: char buf[128] = {0}; 166: KV kv = {.k1 = (100 - i) % 26, .k2 = i % 26}; 167: kvSerial(&kv, buf); 168: rocksdb_writebatch_put_cf(wBatch, cfHandle[1], buf, sizeof(kv), "value", strlen("value")); 169: }	
625	污染路径: 函数'kvSerial'的定义点.	
	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmFile.c	行号: 39
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
626	代码片段: 37: Sjson *pjson = NULL; 38: char file[PATH_MAX] = {0}; 39: snprintf(file, sizeof(file), "%s%s%s.json", path, TD_DIRSEP, name); 40: 41: if (taosStatFile(file, NULL, NULL, NULL) < 0) {	
	污染路径: 无	
	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 377
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段:	

	375: 376: dInfo("thread:%d, start to close %d vnodes", pThread->threadIndex, pThread->vnodeNum); 377: setThreadName("close-vnodes"); 378: 379: for (int32_t v = 0; v < pThread->vnodeNum; ++v) {	
	污染路径: 函数'setThreadName'的定义点.	
627	缺陷发生位置:	行号:
	/source/common/src/tmsg.c	396
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 394: SQueryNodeLoad load = {0}; 395: if (tDecodeSQueryNodeLoad(pDecoder, &load) < 0) return -1; 396: taosArrayPush(pRsp->query->pQnodeList, &load); 397: } 398: }	
	污染路径: 函数'taosArrayPush'的定义点.	
628	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndUser.c	1357
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 1355: int32_t numOfWriteTbs = taosHashGetSize(pUser->writeTbs); 1356: if (numOfRows + numOfReadDbs + numOfWriteDbs + numOfTopics + numOfReadTbs + numOfWriteTbs >= rows) { 1357: mInfo("will restore. current num of rows: %d, read dbs %d, write dbs %d, topics %d, read tables %d, write tables %d", 1358: numOfRows, numOfReadDbs, numOfWriteDbs, numOfTopics, numOfReadTbs, numOfWriteTbs); 1359: pShow->restore = true;	
	污染路径: 函数'mInfo'的定义点.	
629	缺陷发生位置:	行号:

	/source/libs/scheduler/src/schUtil.c	26
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 24: 25: FORCE_INLINE SSchJob *schAcquireJob(int64_t refId) { 26: qDebug("sch acquire jobId:0x%" PRIx64, refId); 27: return (SSchJob *)taosAcquireRef(schMgmt.jobRef, refId); 28: }	
	污染路径: 函数'qDebug'的定义点.	
630	缺陷发生位置: /source/util/src/theap.c	行号: 273
	步骤描述: 不使用返回值的函数'pqHeapify'应使用'void'说明.	
	代码片段: 271: if (i == 0 pq->fn(pqContainerGetEle(pq, i)->data, pqContainerGetEle(pq, pqParent(i))->data, pq->param)) { 272: // if value in pos i is smaller than parent, heapify down from i to the end 273: pqHeapify(pq, i, pqContainerSize(pq)); 274: } else { 275: // if value in pos i is big than parent, heapify up from i	
	污染路径: 函数'pqHeapify'的定义点.	
631	缺陷发生位置: /source/util/src/ttimer.c	行号: 154
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 152: while (atomic_val_compare_exchange_64(&(list->lockedBy), 0, tid) != 0) { 153: if (++i % 1000 == 0) { 154: sched_yield(); 155: } 156: }	
	污染路径: 无	
632	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 417

	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 415: for (i = 100; i < 200; i++) { 416: char newStr[100]; 417: sprintf(newStr, "%s-%d", ranStr, i); 418: int len = strlen(newStr); 419: code = walWrite(pWal, i, 0, newStr, len);	
	污染路径: 无	
633	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 512
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 510: for (int iData = 0; iData < nData; ++iData) { 511: sprintf(key, "key%d", iData); 512: sprintf(val, "value%d", iData); 513: 514: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn);	
	污染路径: 无	
634	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 129
	步骤描述: 不使用返回值的函数'syncNodeRemove'应使用'void'说明.	
	代码片段: 127: pSyncNode->isStart = false; 128: syncNodeRelease(pSyncNode); 129: syncNodeRemove(rid); 130: } 131: }	
	污染路径: 函数'syncNodeRemove'的定义点.	
635	缺陷发生位置: /source/libs/geometry/src/geomFunc.c	行号: 188
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段:	

	186: } 187: 188: colDataSetVal(pOutputData, i, output, (output == NULL)); 189: 190: _exit:	
	污染路径: 函数'colDataSetVal'的定义点.	
636	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 110
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 108: 109: static int32_t updateStatus(SDataDispatchHandle* pDispatcher) { 110: taosThreadMutexLock(&pDispatcher->mutex); 111: int32_t blockNums = taosQueueItemSize(pDispatcher->pDataBlocks); 112: int32_t status =	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
637	缺陷发生位置: /source/client/src/clientTmqConnector.c	行号: 52
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段: 50: (*env)->DeleteLocalRef(env, offset); 51: 52: atomic_store_32(&__init_tmq, 2); 53: jniDebug("tmq method register finished"); 54: }	
	污染路径: 函数'atomic_store_32'的定义点.	
638	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 39
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 37: sdbCleanup(pSdb);	

	<pre> 38: terrno = TSDB_CODE_OUT_OF_MEMORY; 39: mError("failed to init sdb since %s", terrstr()); 40: return NULL; 41: } </pre>	
	污染路径: 函数'mError'的定义点.	
639	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSnapshot.c	行号: 84
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 82: } 83: 84: taosMemoryFree(pReader); 85: } 86: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
640	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 299
	步骤描述: 不使用返回值的函数'qTrace'应使用'void'说明.	
	代码片段: <pre> 297: code = taosThreadMutexInit(&pReader->readerMutex, NULL); 298: 299: qTrace("tsdb/read: %p, post-init read mutex: %p, code: %d", pReader, &pReader->readerMutex, code); 300: 301: return code; </pre>	
	污染路径: 函数'qTrace'的定义点.	
641	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCommit2.c	行号: 208
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: <pre> 206: numRecord++; 207: code = tsdbFSetWriteTombRecord(committer->writer, record); </pre>	

	208: TSDB_CHECK_CODE(code, lino, _exit); 209: 210: _next:	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
642	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 141
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 139: SOperatorNode* pOperNode = (SOperatorNode*)pEqualOnCondNode; 140: extractEqualOnCondColsFromOper(pInfo, pOperNode, &left, &right); 141: taosArrayPush(leftTagEqCols, &left); 142: taosArrayPush(rightTagEqCols, &right); 143: }	
	污染路径: 函数'taosArrayPush'的定义点.	
643	缺陷发生位置: /source/dnode/vnode/src/meta/metaQuery.c	行号: 562
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 560: tdbTbcClose(pStbCur->pCur); 561: 562: tdbFree(pStbCur->pKey); 563: tdbFree(pStbCur->pVal); 564: }	
	污染路径: 函数'tdbFree'的定义点.	
644	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 108
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 106: STagFilterResEntry** p = param; 107: tdListEmpty(&(*p)->list); 108: taosMemoryFreeClear(*p); 109: }	

	110:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
645	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 220
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 218: } 219: if (ctx->file.pFile == NULL) { 220: indexError("failed to open file, error %d", errno); 221: goto END; 222: }	
	污染路径: 函数'indexError'的定义点.	
646	缺陷发生位置: /tests/script/api/api_with_reqid_test.c	行号: 191
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 189: result = taos_query_with_reqid(taos, "drop database if exists test;", genReqid()); 190: taos_free_result(result); 191: usleep(100000); 192: result = taos_query_with_reqid(taos, "create database test precision 'us' update 1;", genReqid()); 193: taos_free_result(result);	
	污染路径: 无	
647	缺陷发生位置: /source/util/src/tref.c	行号: 127
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: 125: pSet = tsRefSetList + rsetId; 126: 127: taosThreadMutexLock(&tsRefMutex); 128: 129: if (pSet->state == TSDB_REF_STATE_ACTIVE) {	

	污染路径: 函数'taosThreadMutexLock'的定义点.	
648	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 35
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 33: for (int i = 0; i < taosArrayGetSize(src); i++) { 34: SArray *m = (SArray *)taosArrayGetP(src, i); 35: taosArrayDestroy(m); 36: } 37: taosArrayDestroy(src); </pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
649	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 76
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: <pre> 74: collInfo.info.colId = 1; 75: 76: blockDataAppendCollInfo(pBlock, &collInfo); 77: blockDataEnsureCapacity(pBlock, pInfo->pageRows); 78: </pre>	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
650	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 773
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: <pre> 771: } 772: assert(result == nullptr); 773: taosHashPut(phash, &v, sizeof(v), NULL, 0); 774: } 775: </pre>	
	污染路径: 函数'taosHashPut'的定义点.	
651	缺陷发生位置: /source/libs/executor/src/dataDeleter.c	行号: 91

	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 89: pBuf->useSize += pEntry->dataLen; 90: 91: atomic_add_fetch_64(&pHandle->cachedSize, pEntry->dataLen); 92: atomic_add_fetch_64(&gDataSinkStat.cachedSize, pEntry->dataLen); 93: }	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
652	缺陷发生位置: /source/libs/nodes/src/nodesCodeFuncs.c	行号: 7047
	步骤描述: 不使用返回值的函数'nodesWarn'应使用'void'说明.	
	代码片段: 7045: break; 7046: } 7047: nodesWarn("specificNodeToJson unknown node = %s", nodesNodeName(nodeType(pObj))); 7048: return TSDB_CODE_SUCCESS; 7049: }	
	污染路径: 函数'nodesWarn'的定义点.	
653	缺陷发生位置: /source/libs/geometry/src/geosWrapper.c	行号: 31
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 29: void geosErrMsgHandler(const char *errMsg, void *userData) { 30: char* targetErrMsg = userData; 31: snprintf(targetErrMsg, 512, "%s", errMsg); 32: } 33:	
	污染路径: 无	
654	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 482

	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre> 480: type == TSDB_DATA_TYPE_BINARY type == TSDB_DATA_TYPE_VARBINARY 481: type == TSDB_DATA_TYPE_NCHAR type == TSDB_DATA_TYPE_GEOMETRY) { 482: qError("not supported range type:%d", type); 483: return NULL; 484: }</pre>	
	污染路径: 函数'qError'的定义点.	
655	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamTaskSnap.c	行号: 63
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 61: tqInfo("vgId:%d, vnode stream-task snapshot reader failed to open, reason: %s", TD_VID(pTq->pVnode), 62: tsterror(code)); 63: taosMemoryFree(pReader); 64: goto _err; 65: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
656	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 107
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: <pre> 105: } else if (pInfo->type == TSDB_DATA_TYPE_DOUBLE pInfo->type == TSDB_DATA_TYPE_FLOAT) { 106: double v = rand_f2(); 107: colDataSetVal(pCollInfo, i, reinterpret_cast<const char*>(&v), false); 108: printf("float: %f\n", v); 109: } else {</pre>	
	污染路径: 函数'colDataSetVal'的定义点.	
657	缺陷发生位置:	行号:

	/source/dnode/mgmt/mgmt_mnode/src/mmFile.c	69
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 67: 68: if (taosStatFile(file, NULL, NULL, NULL) < 0) { 69: dlInfo("mnode file:%s not exist", file); 70: return 0; 71: }	
	污染路径: 函数'dlInfo'的定义点.	
658	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 331
	步骤描述: 不使用返回值的函数'taosMbsToWchars'应使用'void'说明.	
	代码片段: 329: TdWchar *wc = (TdWchar *)taosMemoryCalloc(sizeof(TdWchar), SHELL_MAX_COMMAND_SIZE); 330: *size = strlen(str); 331: taosMbsToWchars(wc, str, SHELL_MAX_COMMAND_SIZE); 332: *width = taosWcharsWidth(wc, SHELL_MAX_COMMAND_SIZE); 333: taosMemoryFree(wc);	
	污染路径: 函数'taosMbsToWchars'的定义点.	
659	缺陷发生位置: /source/client/src/clientEnv.c	行号: 112
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 110: 111: if (duration >= (tsSlowLogThreshold * 1000000UL)) { 112: atomic_add_fetch_64((int64_t *)&pActivity->numOfSlowQueries, 1); 113: if (tsSlowLogScope & reqType) { 114: taosPrintSlowLog("PID:%d, Conn:%u, QID:0x%" PRIx64 " , Start:%" PRIu64 " , Duration:%" PRIu64 "us, SQL:%s",	
	污染路径:	

	函数'atomic_add_fetch_64'的定义点.	
660	缺陷发生位置: /source/libs/parser/src/parlInsertSql.c	行号: 838
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 836: } 837: } 838: taosArrayDestroy(pTagVals); 839: taosArrayDestroy(pTagName); 840: tTagFree(pTag);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
661	缺陷发生位置: /examples/c/schemaless.c	行号: 40
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 38: result = taos_query(taos, "drop database if exists db;"); 39: taos_free_result(result); 40: usleep(100000); 41: result = taos_query(taos, "create database db precision 'ms;"); 42: taos_free_result(result);	
	污染路径: 无	
662	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 209
	步骤描述: 不使用返回值的函数'taosCloseDir'应使用'void'说明.	
	代码片段: 207: } 208: 209: taosCloseDir(&pDir); 210: 211: if (pFile->pCurrent == NULL) {	
	污染路径: 函数'taosCloseDir'的定义点.	
663	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 64

	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 62: pSdb->pMnode = pOption->pMnode; 63: taosThreadMutexInit(&pSdb->filelock, NULL); 64: mInfo("sdb init success"); 65: return pSdb; 66: }	
	污染路径: 函数'mInfo'的定义点.	
664	缺陷发生位置: /docs/examples/c/async_query_example.c	行号: 163
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 161: taos_free_result(res); 162: taos_close(_taos); 163: taos_cleanup(); 164: exit(EXIT_FAILURE); 165: }	
	污染路径: 函数'taos_cleanup'的定义点.	
665	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 176
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 174: static void* httpThread(void* arg) { 175: SHttpModule* http = (SHttpModule*)arg; 176: setThreadName("http-cli-send-thread"); 177: uv_run(http->loop, UV_RUN_DEFAULT); 178: return NULL;	
	污染路径: expanded from macro 'setThreadName'	
666	缺陷发生位置: /source/common/src/ttszip.c	行号: 116
	步骤描述: 不使用返回值的函数'tsBufDestroy'应使用'void'说明.	
	代码片段: 114: STSGroupBlockInfoEx* tmp =	

	taosMemoryRealloc(pTSCBuf->pData, sizeof(STSGroupBlockInfoEx) * pTSCBuf->numOfAlloc); 115: if (tmp == NULL) { 116: tsBufDestroy(pTSCBuf); 117: return NULL; 118: }	
	污染路径: 函数'tsBufDestroy'的定义点.	
667	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 989
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 987: 988: if (NULL == ctx.pRes) { 989: indexError("index-filter failed to taosHashInit"); 990: return TSDB_CODE_OUT_OF_MEMORY; 991: }	
	污染路径: 函数'indexError'的定义点.	
668	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffsetSnapshot.c	行号: 47
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 45: 46: int32_t tqOffsetReaderClose(STqOffsetReader** ppReader) { 47: taosMemoryFree(*ppReader); 48: *ppReader = NULL; 49: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
669	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmWorker.c	行号: 111
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 109: 110: if (tSingleWorkerInit(&pMgmt->queryWorker,	

	<pre>&queryCfg) != 0) { 111: dError("failed to start qnode-query worker since %s", terrstr()); 112: return -1; 113: }</pre>	
	污染路径: 函数'dError'的定义点.	
670	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 1503
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 1501: code = tsdbOpenFile(fname, pTsdb->pVnode- >config.tsdbPageSize, TD_FILE_READ, &pDelFReader- >pReadH); 1502: if (code) { 1503: taosMemoryFree(pDelFReader); 1504: goto _exit; 1505: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
671	缺陷发生位置: /source/util/src/tutil.c	行号: 423
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 421: 422: void taosIp2String(uint32_t ip, char *str) { 423: sprintf(str, "%u.%u.%u.%u", ip & 0xFF, (ip >> 8) & 0xFF, (ip >> 16) & 0xFF, (uint8_t)(ip >> 24)); 424: } 425:</pre>	
	污染路径: 无	
672	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmHandle.c	行号: 47
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: <pre> 45: bool deployed = true;</pre>	

	46: if (dmWriteFile(plnput->path, plnput->name, deployed) != 0) { 47: dError("failed to write qnode file since %s", terrstr()); 48: return -1; 49: }	
	污染路径: 函数'dError'的定义点.	
673	缺陷发生位置: /tests/script/api/stmt_function.c	行号: 80
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 78: stmt = taos_stmt_init(taos); 79: assert(stmt != NULL); 80: sprintf(stmt_sql, "insert into ? values (?,?,?,?,?,?,?,?,?,?)"); 81: assert(taos_stmt_prepare(stmt, stmt_sql, 0) == 0); 82: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
674	缺陷发生位置: /source/util/src/tlockfree.c	行号: 50
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 48: nLoops++; 49: if (nLoops > 1000) { 50: sched_yield(); 51: nLoops = 0; 52: }	
	污染路径: 无	
675	缺陷发生位置: /source/libs/scalar/src/sclvector.c	行号: 344
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: 342: char *t = taosMemoryCalloc(1, inputLen); 343: if (t == NULL) { 344: sclError("Out of memory");	

	345: terrno = TSDB_CODE_OUT_OF_MEMORY; 346: return;	
	污染路径: 函数'sclError'的定义点.	
676	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 46
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 44: pOut->dbVgroup->stateTs = usedbRsp->stateTs; 45: 46: qDebug("Got %d vgroup for db %s, vgVersion:%d, stateTs:%" PRId64, usedbRsp->vgNum, usedbRsp->db, usedbRsp->vgVersion, usedbRsp->stateTs); 47: 48: if (usedbRsp->vgNum <= 0) {	
	污染路径: 函数'qDebug'的定义点.	
677	缺陷发生位置: /source/libs/index/test/fstTest.cc	行号: 201
	步骤描述: 不使用返回值的函数'Performance_fstWriteRecords'应使用'void'说明.	
	代码片段: 199: void checkMillonWriteAndReadOfFst() { 200: FstWriter* fw = new FstWriter; 201: Performance_fstWriteRecords(fw); 202: delete fw; 203: FstReadMemory* fr = new FstReadMemory(1024 * 64 * 1024);	
	污染路径: 函数'Performance_fstWriteRecords'的定义点.	
678	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 116
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 114: } 115: void rpcClose(void* arg) { 116: tlInfo("start to close rpc"); 117: transRemoveExHandle(transGetInstMgt(), (int64_t)arg);	

	污染路径: 函数'STR_TO_VARSTR'的定义点.	
682	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 57
	步骤描述: 不使用返回值的函数'syncCleanUp'应使用'void'说明.	
	代码片段: 55: if (gHbDataRefId < 0) { 56: sError("failed to init hb-data ref"); 57: syncCleanUp(); 58: return -1; 59: }	
	污染路径: 函数'syncCleanUp'的定义点.	
683	缺陷发生位置: /source/libs/qworker/src/qwMsg.c	行号: 74
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 72: int32_t msgSize = tSerializeSQueryTableRsp(NULL, 0, &rsp); 73: if (msgSize < 0) { 74: qError("tSerializeSQueryTableRsp failed"); 75: QW_RET(TSDB_CODE_OUT_OF_MEMORY); 76: }	
	污染路径: 函数'qError'的定义点.	
684	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 58
	步骤描述: 不使用返回值的函数's3DeleteObjects'应使用'void'说明.	
	代码片段: 56: 57: const char *object_name = taosDirEntryBaseName((char *)fobj->fname); 58: s3DeleteObjects(&object_name, 1); 59: 60: _exit:	
	污染路径:	

	函数's3DeleteObjects'的定义点.	
685	缺陷发生位置: /source/util/src/thash.c	行号: 363
	步骤描述: 不使用返回值的函数'atomic_add_fetch_64'应使用'void'说明.	
	代码片段: 361: // enable resize 362: taosHashRUnlock(pHashObj); 363: atomic_add_fetch_64(&pHashObj->size, 1); 364: 365: return 0;	
	污染路径: 函数'atomic_add_fetch_64'的定义点.	
686	缺陷发生位置: /source/common/src/tglobal.c	行号: 330
	步骤描述: 不使用返回值的函数'taosExpandDir'应使用'void'说明.	
	代码片段: 328: char cfgFile[PATH_MAX + 100] = {0}; 329: 330: taosExpandDir(inputCfgDir, cfgDir, PATH_MAX); 331: if (taosIsDir(cfgDir)) { 332: #ifdef CUS_PROMPT	
	污染路径: 函数'taosExpandDir'的定义点.	
687	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 243
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: 241: if (strlen(name) >= sstLen && strncmp(name + strlen(name) - 4, pSST, sstLen) == 0) { 242: char* p = taosStrdup(name); 243: taosHashPut(bm->pSstTbl[1 - bm->idx], name, strlen(name), &dummy, sizeof(dummy)); 244: continue; 245: }	
	污染路径: 函数'taosHashPut'的定义点.	
688	缺陷发生位置:	行号:

	/utils/test/c/tmqSim.c	193
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 191: sprintf(filename, "/tmp/tmqlog-%d-%s.txt", process_id, getCurrentTimeString(tmpString)); 192: } else { 193: sprintf(filename, "%s/../log/tmqlog-%d-%s.txt", configDir, process_id, getCurrentTimeString(tmpString)); 194: } 195: #ifdef WINDOWS	
	污染路径: 无	
689	缺陷发生位置: /source/libs/scheduler/test/schedulerTests.cpp	行号: 485
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 483: strcpy(qnodeAddr.fqdn, "qnode0.ep"); 484: qnodeAddr.port = 6031; 485: taosArrayPush(qnodeList, &qnodeAddr); 486: 487: queryDone = 0;	
	污染路径: 函数'taosArrayPush'的定义点.	
690	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmHandle.c	行号: 82
	步骤描述: 不使用返回值的函数'dGError'应使用'void'说明.	
	代码片段: 80: if (pInput->pData->dnodeId != 0 && dropReq.dnodeId != pInput->pData->dnodeId) { 81: terrno = TSDB_CODE_INVALID_OPTION; 82: dGError("failed to drop mnode since %s", terrstr()); 83: return -1; 84: }	
	污染路径: 函数'dGError'的定义点.	
691	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 30

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 28: printf("syncIndexMgrPrint2 len:%" PRIu64 " %s %s \n", (uint64_t)strlen(serialized), s, serialized); 29: fflush(NULL); 30: taosMemoryFree(serialized); 31: } 32:	
	污染路径: 函数'taosMemoryFree'的定义点.	
692	缺陷发生位置: /source/client/src/clientSmlJson.c	行号: 565
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 563: 564: // if reach here means type is unsupported 565: uError("OTD:invalid type(%s) for JSON Number", typeStr); 566: return TSDB_CODE_TSC_INVALID_JSON_TYPE; 567: }	
	污染路径: 函数'uError'的定义点.	
693	缺陷发生位置: /source/os/src/osSystem.c	行号: 297
	步骤描述: 不使用返回值的函数'pclose'应使用'void'说明.	
	代码片段: 295: _pclose((FILE*)(*ppCmd)); 296: #else 297: pclose((FILE*)(*ppCmd)); 298: #endif 299: *ppCmd = NULL;	
	污染路径: 无	
694	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 381
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	

	代码片段: 379: taosMemoryFreeClear(pCreateReq->comment); 380: if (pCreateReq->type == TSDB_CHILD_TABLE) { 381: taosArrayDestroy(pCreateReq->ctb.tagName); 382: } 383: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
695	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 460
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 458: int32_t code = tsdbSttFileReadSttBlk(plter->pReader, &pSttBlkArray); 459: if (code != TSDB_CODE_SUCCESS) { 460: tsdbError("load stt blk failed, code:%s, %s", tstrerror(code), idStr); 461: return code; 462: }	
	污染路径: 函数'tsdbError'的定义点.	
696	缺陷发生位置: /source/libs/executor/src/streamtimewindowoperator.c	行号: 271
	步骤描述: 不使用返回值的函数'getNextTimeWindow'应使用'void'说明.	
	代码片段: 269: tSimpleHashRemove(pUpdatedMap, &winRes, sizeof(SWinKey)); 270: } 271: getNextTimeWindow(pInterval, &win, TSDB_ORDER_ASC); 272: } while (win.ekey <= endTsCols[i]); 273: }	
	污染路径: 函数'getNextTimeWindow'的定义点.	
697	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 325
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	

	代码片段: <pre> 323: } 324: if (pCxt->currLevel == currTotalLevel) { 325: taosArrayPush(pTables, &pTable); 326: if (hasSameTableAlias(pTables)) { 327: taosArrayDestroy(pTables); </pre>	
	污染路径: 函数'taosArrayPush'的定义点.	
698	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 44
	步骤描述: 不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: <pre> 42: strcpy(tsLogDir, (char *)logDir); 43: taosRemoveDir(tsLogDir); 44: taosMkDir(tsLogDir); 45: 46: if (taosInitLog(defaultLogFileNamePrefix, maxLogFileNum) < 0) { </pre>	
	污染路径: 函数'taosMkDir'的定义点.	
699	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 364
	步骤描述: 不使用返回值的函数'tSerializeSCreateVnodeReq'应使用'void'说明.	
	代码片段: <pre> 362: } 363: 364: tSerializeSCreateVnodeReq(pReq, contLen, &createReq); 365: *pContLen = contLen; 366: return pReq; </pre>	
	污染路径: 函数'tSerializeSCreateVnodeReq'的定义点.	
700	缺陷发生位置: /tests/script/api/apitest.c	行号: 24
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: <pre> 22: result = taos_query(taos, "drop database if exists test;"); </pre>	

	23: taos_free_result(result); 24: usleep(100000); 25: result = taos_query(taos, "create database test precision 'us';"); 26: taos_free_result(result);	
	污染路径: 无	
701	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamStateSnap.c	行号: 157
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 155: tqError("vgId:%d, vnode %s snapshot writer failed to open since %s", TD_VID(pTq->pVnode), STREAM_STATE_TRANSFER, 156: tsterror(code)); 157: taosMemoryFree(pWriter); 158: *ppWriter = NULL; 159: return -1;	
	污染路径: 函数'taosMemoryFree'的定义点.	
702	缺陷发生位置: /source/util/src/tarray.c	行号: 503
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 501: void* data = taosMemoryCalloc(1, dataSz); 502: buf = decode(buf, data, sver); 503: taosArrayPush(*pArray, &data); 504: } 505: return (void*)buf;	
	污染路径: 函数'taosArrayPush'的定义点.	
703	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmFile.c	行号: 108
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 106: if (pFile == NULL) { 107: terrno = TAOS_SYSTEM_ERROR(errno);	

	108: dError("failed to open vnode file:%s since %s", file, terrstr()); 109: goto _OVER; 110: }	
	污染路径: 函数'dError'的定义点.	
704	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 216
	步骤描述: 不使用返回值的函数'ctgDebug'应使用'void'说明.	
	代码片段: 214: CTG_CACHE_HIT_INC(CTG_CI_DB_VGROUP, 1); 215: 216: ctgDebug("Got db vgInfo from cache, dbFName:%s", dbFName); 217: 218: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'ctgDebug'的定义点.	
705	缺陷发生位置: /examples/c/asyncdemo.c	行号: 109
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 107: printf("success to connect to server\n"); 108: 109: sprintf(sql, "drop database if exists %s", db); 110: queryDB(taos, sql); 111:	
	污染路径: 无	
706	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 50
	步骤描述: 不使用返回值的函数'wError'应使用'void'说明.	
	代码片段: 48: int32_t code = walCreateThread(); 49: if (code != 0) { 50: wError("failed to init wal module since %s", tstrerror(code));	

	51: atomic_store_8(&tsWal.inited, 0); 52: return code;	
	污染路径: 函数'wError'的定义点.	
707	缺陷发生位置: /source/libs/nodes/src/nodesToSQLFuncs.c	行号: 232
	步骤描述: 不使用返回值的函数'NODES_RET'应使用'void'说明.	
	代码片段: 230: 231: nodesError("nodesNodeToSQL unknown node = %s", nodesNodeName(pNode->type)); 232: NODES_RET(TSDB_CODE_APP_ERROR); 233: }	
	污染路径: 函数'NODES_RET'的定义点.	
708	缺陷发生位置: /contrib/test/lucene/main.cpp	行号: 4
	步骤描述: 不使用返回值的函数'operator<<'应使用'void'说明.	
	代码片段: 2: 3: int main(int argc, char const *argv[]) { 4: std::cout << "Hello, this is lucene test" << std::endl; 5: return 0; 6: }	
	污染路径: 函数'operator<<'的定义点.	
709	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 135
	步骤描述: 不使用返回值的函数'tqError'应使用'void'说明.	
	代码片段: 133: SRpcMsg msg = { .msgType = TDMT_VND_CREATE_TABLE, .pCont = buf, .contLen = tlen }; 134: if (tmsgPutToQueue(&pVnode->msgCb, WRITE_QUEUE, &msg) != 0) { 135: tqError("failed to put into write-queue since %s", terrstr()); 136: }	

	137:	
	污染路径: 函数'tqError'的定义点.	
710	缺陷发生位置: /source/libs/scheduler/src/schUtil.c	行号: 139
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 137: } 138: 139: qError("taosHashPut hb trans failed, nodeId:%d, fqdn: %s, port:%d", epld->nodeId, epld->ep.fqdn, epld->ep.port); 140: SCH_ERR_RET(code); 141: }	
	污染路径: 函数'qError'的定义点.	
711	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmFile.c	行号: 42
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 40: 41: if (taosStatFile(file, NULL, NULL, NULL) < 0) { 42: dlInfo("file:%s not exist", file); 43: code = 0; 44: goto _OVER;	
	污染路径: 函数'dlInfo'的定义点.	
712	缺陷发生位置: /source/os/src/osTimer.c	行号: 83
	步骤描述: 不使用返回值的函数'timer_delete'应使用'void'说明.	
	代码片段: 81: static void taosDeleteTimer(void *tharg) { 82: timer_t *pTimer = tharg; 83: timer_delete(*pTimer); 84: } 85:	
	污染路径:	

	无	
713	缺陷发生位置: /source/libs/nodes/src/nodesCodeFuncs.c	行号: 396
	步骤描述: 不使用返回值的函数'nodesWarn'应使用'void'说明.	
	代码片段: <pre> 394: break; 395: } 396: nodesWarn("nodesNodeName unknown node = %d", type); 397: return "UnknownNode"; 398: }</pre>	
	污染路径: 函数'nodesWarn'的定义点.	
714	缺陷发生位置: /source/dnode/mnode/impl/src/mndInfoSchema.c	行号: 80
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 78: STableMetaRsp *pMeta = taosHashGet(pMnode->infosMeta, tbName, strlen(tbName)); 79: if (NULL == pMeta (!sysinfo && pMeta->sysInfo)) { 80: mError("invalid information schema table name:%s", tbName); 81: terrno = TSDB_CODE_MND_INVALID_SYS_TABLENAME; 82: return -1;</pre>	
	污染路径: 函数'mError'的定义点.	
715	缺陷发生位置: /source/libs/executor/src/filloperator.c	行号: 77
	步骤描述: 不使用返回值的函数'taosResetFillInfo'应使用'void'说明.	
	代码片段: <pre> 75: int32_t scanFlag = MAIN_SCAN; 76: // getTableScanInfo(pOperator, &order, &scanFlag, false); 77: taosResetFillInfo(pInfo->pFillInfo, getFillInfoStart(pInfo->pFillInfo)); 78: 79: blockDataCleanup(pInfo->pRes);</pre>	
	污染路径:	

	函数'taosResetFillInfo'的定义点.	
716	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 92
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 90: schema = mer1.me.ntbEntry.schemaRow; 91: } else { 92: ASSERT(0); 93: } 94:	
	污染路径: 函数'ASSERT'的定义点.	
717	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c	行号: 71
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 69: return -1; 70: } 71: tmsgReportStartup("qnode-worker", "initialized"); 72: 73: pOutput->pMgmt = pMgmt;	
	污染路径: 函数'tmsgReportStartup'的定义点.	
718	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 109
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 107: void *p = taosInitScheduler(queueSize, tsNumOfTaskQueueThreads, "tsc", &pTaskQueue); 108: if (NULL == p) { 109: qError("failed to init task queue"); 110: return -1; 111: }	
	污染路径: 函数'qError'的定义点.	
719	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 359

	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 357: } 358: 359: dDebug("dnode rpc server is initialized"); 360: return 0; 361: }	
	污染路径: 函数'dDebug'的定义点.	
720	缺陷发生位置:	行号:
	/source/common/src/tdatablock.c	626
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 624: SColumnInfoData* pSrcCol = taosArrayGet(pBlock->pDataBlock, i); 625: collInfo.info = pSrcCol->info; 626: blockDataAppendCollInfo(pDst, &collInfo); 627: } 628:	
721	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
	缺陷发生位置:	行号:
	/source/dnode/mgmt/mgmt_qnode/src/qmWorker.c	128
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
722	代码片段: 126: } 127: 128: dDebug("qnode workers are initialized"); 129: return 0; 130: }	
	污染路径: 函数'dDebug'的定义点.	
	缺陷发生位置:	行号:
	/source/client/src/clientTmqConnector.c	37
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段:	

	35: case 1: 36: do { 37: taosMsleep(0); 38: } while (atomic_load_32(&__init_tmq) == 1); 39: case 2:	
	污染路径: 函数'taosMsleep'的定义点.	
723	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 274
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 272: } 273: 274: taosMemoryFree(pData); 275: 276: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
724	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 355
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 353: } 354: 355: blockDataEnsureCapacity(pBlock, pBlock->info.rows); 356: *ppBlock = pBlock; 357:	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
725	缺陷发生位置: /utils/tsim/src/simExe.c	行号: 450
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 448: script->linePos++; 449: if (replaced && strstr(buf, "start") != NULL) { 450: simInfo("====> startup is slow in valgrind mode, so sleep 5 seconds after exec.sh -s start");	

	451: taosMsleep(5000); 452: }	
	污染路径: 函数'simInfo'的定义点.	
726	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 134
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 132: 133: void mndCleanupStream(SMnode *pMnode) { 134: taosArrayDestroy(execNodeList.pTaskList); 135: taosHashCleanup(execNodeList.pTaskMap); 136: taosThreadMutexDestroy(&execNodeList.lock);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
727	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 37
	步骤描述: 不使用返回值的函数'taosHashRemove'应使用'void'说明.	
	代码片段: 35: 36: for (int32_t i = 1000; i < 2000; ++i) { 37: taosHashRemove(hashTable, (const char*)&i, sizeof(int32_t)); 38: } 39:	
	污染路径: 函数'taosHashRemove'的定义点.	
728	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 72
	步骤描述: 不使用返回值的函数'colDataSetNULL'应使用'void'说明.	
	代码片段: 70: if (IS_VAR_DATA_TYPE(pColVal->colVal.type)) { 71: if (!COL_VAL_IS_VALUE(&pColVal->colVal)) { 72: colDataSetNULL(pColInfoData, numOfRows); 73: } else { 74: varDataSetLen(pReader->transferBuf[slotId], pVal->value.nData);	

	污染路径: 函数'colDataSetNULL'的定义点.	
729	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 224
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 222: { 223: if (taosThreadCreate(&io->consumerTid, NULL, syncIOConsumerFunc, io) != 0) { 224: sError("failed to create sync consumer thread since %s", strerror(errno)); 225: terrno = TAOS_SYSTEM_ERROR(errno); 226: return -1;	
	污染路径: 函数'sError'的定义点.	
730	缺陷发生位置: /source/libs/tdb/src/db/tdbPager.c	行号: 801
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 799: code = tdbTbcDelete(pCur); 800: if (code < 0) { 801: tdbError("tdb/remove-free-page: tbc delete failed with ret: %d.", code); 802: tdbTbcClose(pCur); 803: return 0;	
	污染路径: 函数'tdbError'的定义点.	
731	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 109
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 107: 108: if (tSingleWorkerInit(&pMgmt->streamWorker, &cfg)) { 109: dError("failed to start snode shared-worker since %s", terrstr()); 110: return -1; 111: }	

	污染路径: 函数'dError'的定义点.	
732	缺陷发生位置: /source/libs/sync/src/syncRaftCfg.c	行号: 113
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 111: if (taosFsyncFile(pFile) < 0) goto _OVER; 112: 113: taosCloseFile(&pFile); 114: if (taosRenameFile(file, realfile) != 0) goto _OVER; 115:	
	污染路径: 函数'taosCloseFile'的定义点.	
733	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmEnv.c	行号: 71
	步骤描述: 不使用返回值的函数'taosBlockSIGPIPE'应使用'void'说明.	
	代码片段: 69: static int32_t dmlInitSystem() { 70: taosIgnSIGPIPE(); 71: taosBlockSIGPIPE(); 72: taosResolveCRC(); 73: return 0;	
	污染路径: 函数'taosBlockSIGPIPE'的定义点.	
734	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 894
	步骤描述: 不使用返回值的函数'taosEncodeFixedU32'应使用'void'说明.	
	代码片段: 892: void* pBuf32 = buf32; 893: uint32_t sum = idxFileMaskedCheckSum(b->wrt); 894: taosEncodeFixedU32(&pBuf32, sum); 895: idxFileWrite(b->wrt, buf32, sizeof(buf32)); 896:	
	污染路径: 函数'taosEncodeFixedU32'的定义点.	
735	缺陷发生位置:	行号:

	/tests/script/api/insertSameTs.c	71
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 69: Test(taos, qstr); 70: taos_close(taos); 71: taos_cleanup(); 72: } 73: void Test(TAOS *taos, char *qstr) {	
	污染路径: 函数'taos_cleanup'的定义点.	
736	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 217
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 215: 216: memset(bm->buf, 0, bm->len); 217: sprintf(bm->buf, "%s%scheckpoint%" PRIu64 "", bm->path, TD_DIRSEP, chkpId); 218: 219: taosArrayClearP(bm->pAdd, taosMemoryFree);	
	污染路径: 无	
737	缺陷发生位置: /tools/shell/src/shellNettest.c	行号: 69
	步骤描述: 不使用返回值的函数'rpcSendRecv'应使用'void'说明.	
	代码片段: 67: 68: printf("request is sent, size:%d\r\n", rpcMsg.contLen); 69: rpcSendRecv(clientRpc, &epSet, &rpcMsg, &rpcRsp); 70: if (rpcRsp.code == 0 && rpcRsp.contLen == rpcMsg.contLen) { 71: printf("response is received, size:%d\r\n", rpcMsg.contLen);	
	污染路径: 函数'rpcSendRecv'的定义点.	
738	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 375

	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 373: 374: taosFtruncateFile(pFile, lastEndOffset); 375: taosCloseFile(&pFile); 376: return 0; 377: }	
	污染路径: 函数'taosCloseFile'的定义点.	
739	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReaderWriter.c	行号: 31
	步骤描述: 不使用返回值的函数's3EvictCache'应使用'void'说明.	
	代码片段: 29: long s3_size = tsS3Enabled ? s3Size(object_name) : 0; 30: if (tsS3Enabled && !strncmp(path + strlen(path) - 5, ".data", 5) && s3_size > 0) { 31: s3EvictCache(path, s3_size); 32: s3Get(object_name, path); 33:	
	污染路径: 函数's3EvictCache'的定义点.	
740	缺陷发生位置: /source/libs/stream/src/tstreamFileState.c	行号: 135
	步骤描述: 不使用返回值的函数'tdListFreeP'应使用'void'说明.	
	代码片段: 133: taosMemoryFree(pFileState->id); 134: tdListFreeP(pFileState->usedBufs, destroyRowBuffAllPosPtr); 135: tdListFreeP(pFileState->freeBufs, destroyRowBuff); 136: tSimpleHashCleanup(pFileState->rowBuffMap); 137: taosMemoryFree(pFileState);	
	污染路径: 函数'tdListFreeP'的定义点.	
741	缺陷发生位置: /source/client/src/clientHb.c	行号: 65
	步骤描述:	

	不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 63: atomic_val_compare_exchange_8(&pAppHbMgr->connHbFlag, 1, 2); 64: 65: taosArrayDestroy(batchRsp.pArray); 66: return TSDB_CODE_SUCCESS; 67: }</pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
742	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCacheRead.c	行号: 80
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: <pre> 78: } 79: } else { 80: colDataSetVal(pCollInfoData, numOfRows, (const char*)&pVal->value.val, !COL_VAL_IS_VALUE(pVal)); 81: } 82: }</pre>	
	污染路径: 函数'colDataSetVal'的定义点.	
743	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmFile.c	行号: 145
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: <pre> 143: 144: code = 0; 145: dlInfo("succceed to read vnode file %s", file); 146: 147: _OVER:</pre>	
	污染路径: 函数'dlInfo'的定义点.	
744	缺陷发生位置: /source/libs/executor/test/tSimpleHashTests.cpp	行号: 98
	步骤描述: 不使用返回值的函数'tSimpleHashPut'应使用'void'说明.	
	代码片段:	

	<pre> 96: combineKey.suid = i; 97: combineKey.uid = i + 1; 98: tSimpleHashPut(pHashObj, (const void *)&combineKey, keyLen, (const void *)&i, dataLen); 99: originDataSum += i; 100: ASSERT_EQ(i, tSimpleHashGetSize(pHashObj)); </pre>	
	污染路径: 函数'tSimpleHashPut'的定义点.	
745	缺陷发生位置: /source/util/src/theap.c	行号: 276
	步骤描述: 不使用返回值的函数'pqReverseHeapify'应使用'void'说明.	
	代码片段: <pre> 274: } else { 275: // if value in pos i is big than parent, heapify up from i 276: pqReverseHeapify(pq, i); 277: } 278: } </pre>	
	污染路径: 函数'pqReverseHeapify'的定义点.	
746	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 520
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: <pre> 518: if (wc == NULL) { 519: terrno = TAOS_SYSTEM_ERROR(errno); 520: indexError("failed to open readonly file: %s, reason: %s", fullname, terrstr()); 521: return NULL; 522: } </pre>	
	污染路径: 函数'indexError'的定义点.	
747	缺陷发生位置: /examples/c/asyncdemo.c	行号: 112
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 110: queryDB(taos, sql); 111: </pre>	

	112: sprintf(sql, "create database %s", db); 113: queryDB(taos, sql); 114:	
	污染路径: 无	
748	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 115
	步骤描述: 不使用返回值的函数'taosMkdir'应使用'void'说明.	
	代码片段: 113: char indexFullPath[128] = {0}; 114: sprintf(indexFullPath, "%s/%s", pMeta->path, "invert"); 115: taosMkdir(indexFullPath); 116: 117: SIndexOpts opts = {.cacheSize = 8 * 1024 * 1024};	
	污染路径: 函数'taosMkdir'的定义点.	
749	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 3631
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: 3629: qDebugL("before optimize, JsonPlan: %s", pStr); 3630: } else { 3631: qDebugL("apply optimize %s rule, JsonPlan: %s", pRuleName, pStr); 3632: } 3633: taosMemoryFree(pStr);	
	污染路径: 函数'qDebugL'的定义点.	
750	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 1398
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 1396: mError("trans:%d, stage keep on redoAction since action:%d code:0x%x not 0x%x, failedTimes:%d", pTrans->id, 1397: pTrans->lastAction, pTrans->code, pAction-> retryCode, pTrans->failedTimes);	

	1398: taosMsleep(1000); 1399: continueExec = true; 1400: return true;	
	污染路径: 函数'taosMsleep'的定义点.	
751	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 140
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 138: 139: if (dmStartNodes(pDnode) != 0) { 140: dError("failed to start nodes since %s", terrstr()); 141: dmSetStatus(pDnode, DND_STAT_STOPPED); 142: dmStopNodes(pDnode);	
	污染路径: 函数'dError'的定义点.	
752	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 84
	步骤描述: 不使用返回值的函数'rpcSendRequest'应使用'void'说明.	
	代码片段: 82: addEpIntoEpSet(&epSet, "127.0.0.1", 7000); 83: 84: rpcSendRequest(this->transCli, &epSet, req, NULL); 85: SemWait(); 86: *resp = this->resp;	
	污染路径: 函数'rpcSendRequest'的定义点.	
753	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 116
	步骤描述: 不使用返回值的函数'taosThreadMutexUnlock'应使用'void'说明.	
	代码片段: 114: : (blockNums < pDispatcher->pManager->cfg.maxDataBlockNumPerQuery ? DS_BUF_LOW : DS_BUF_FULL)); 115: pDispatcher->status = status; 116: taosThreadMutexUnlock(&pDispatcher->mutex); 117: return status;	

	118: }	
	污染路径: 函数'taosThreadMutexUnlock'的定义点.	
754	缺陷发生位置: /source/common/src/tdaformat.c	行号: 1481
	步骤描述: 不使用返回值的函数'tGetTagVal'应使用'void'说明.	
	代码片段: 1479: offset = pTag->idx[iTag]; 1480: } 1481: tGetTagVal(p + offset, &tv, pTag->flags & TD_TAG_JSON); 1482: taosArrayPush(*ppArray, &tv); 1483: }	
	污染路径: 函数'tGetTagVal'的定义点.	
755	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 105
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 103: 104: static void mndPullupTrans(SMnode *pMnode) { 105: mTrace("pullup trans msg"); 106: int32_t contLen = 0; 107: void *pReq = mndBuildTimerMsg(&contLen);	
	污染路径: 函数'mTrace'的定义点.	
756	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 106
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 104: void mndRebCntInc() { 105: int32_t val = atomic_add_fetch_32(&mqRebInExecCnt, 1); 106: mInfo("rebalance trans start, rebalance counter:%d", val); 107: } 108:	

	污染路径: 函数'mInfo'的定义点.	
757	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 173
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 171: printf("syncClientRequestBatchPrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 172: fflush(NULL); 173: taosMemoryFree(serialized); 174: } 175:	
	污染路径: 函数'taosMemoryFree'的定义点.	
758	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smHandle.c	行号: 30
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 28: if (pInput->pData->dnodeId != 0 && createReq.dnodeId != pInput->pData->dnodeId) { 29: terrno = TSDB_CODE_INVALID_OPTION; 30: dError("failed to create snode since %s", terrstr()); 31: return -1; 32: }	
	污染路径: 函数'dError'的定义点.	
759	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 663
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 661: if (IS_VAR_DATA_TYPE(left->colValType)) { 662: if (!IS_VAR_DATA_TYPE(right->colValType)) { 663: NUM_TO_STRING(right->colValType, right->condValue, sizeof(buf) - 2, buf + VARSTR_HEADER_SIZE); 664: varDataSetLen(buf, strlen(buf) + VARSTR_HEADER_SIZE); 665: param.val = buf;	

	污染路径: expanded from macro 'NUM_TO_STRING'	
760	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 261
	步骤描述: 不使用返回值的函数'doDecompressData'应使用'void'说明.	
	代码片段: 259: 260: int32_t fullSize = 0; 261: doDecompressData(pPage, pg->length, &fullSize, pBuf); 262: return 0; 263: }	
	污染路径: 函数'doDecompressData'的定义点.	
761	缺陷发生位置: /source/client/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 77
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 75: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP"); 77: LOAD_FP(SetFocusGained, SetFocusGainedFP, "setFocusGainedFP"); 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79:	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
762	缺陷发生位置: /source/os/src/osTimezone.c	行号: 814
	步骤描述: 不使用返回值的函数'setenv'应使用'void'说明.	
	代码片段: 812: 813: #else 814: setenv("TZ", buf, 1); 815: tzset();	

	816: int32_t tz = (int32_t)((-timezone * MILLISECOND_PER_SECOND) / MILLISECOND_PER_HOUR);	
	污染路径: 无	
763	缺陷发生位置: /source/libs/scheduler/src/schJob.c	行号: 90
	步骤描述: 不使用返回值的函数'SCH_ERR_JRET'应使用'void'说明.	
	代码片段: 88: } 89: 90: SCH_ERR_JRET(TSDB_CODE_SCH_IGNORE_ERROR); 91: } 92:	
	污染路径: 函数'SCH_ERR_JRET'的定义点.	
764	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 45
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 43: 44: static void sdbResetData(SSdb *pSdb) { 45: mInfo("start to reset sdb"); 46: 47: for (ESdbType i = 0; i < SDB_MAX; ++i) {	
	污染路径: 函数'mInfo'的定义点.	
765	缺陷发生位置: /utils/tsim/src/simMain.c	行号: 62
	步骤描述: 不使用返回值的函数'simError'应使用'void'说明.	
	代码片段: 60: 61: if (abortExecution) { 62: simError("execute abort"); 63: return -1; 64: }	
	污染路径:	

	函数'simError'的定义点.	
766	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 106
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 104: for (int32_t i = 0; i < 2; ++i) { 105: SColumnInfoData idata = createColumnInfoData(TSDB_DATA_TYPE_NULL, 10, 1 + i); 106: blockDataAppendCollInfo(res, &idata); 107: } 108:	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
767	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamTaskSnap.c	行号: 152
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 150: pHdr->size = vLen; 151: memcpy(pHdr->data, pVal, vLen); 152: taosMemoryFree(pVal); 153: 154: tqDebug("vgId:%d, vnode stream-task snapshot read data vLen:%d", TD_VID(pReader->pTq->pVnode), vLen);	
	污染路径: 函数'taosMemoryFree'的定义点.	
768	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 203
	步骤描述: 不使用返回值的函数'system'应使用'void'说明.	
	代码片段: 201: char cmd[128]; 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir); 205: system(cmd);	
	污染路径: 无	
769	缺陷发生位置:	行号:

	/source/libs/executor/src/timesliceoperator.c	441
	步骤描述: 不使用返回值的函数'timeSliceEnsureBlockCapacity'应使用'void'说明.	
	代码片段: <pre> 439: static void addCurrentRowToResult(STimeSliceOperatorInfo* pSliceInfo, SEExprSupp* pExprSup, SSDataBlock* pResBlock, 440: SSDataBlock* pSrcBlock, int32_t index) { 441: timeSliceEnsureBlockCapacity(pSliceInfo, pResBlock); 442: for (int32_t j = 0; j < pExprSup->numOfExprs; ++j) { 443: SExprInfo* pExprInfo = &pExprSup->pExprInfo[j]; </pre>	
	污染路径: 函数'timeSliceEnsureBlockCapacity'的定义点.	
770	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 377
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: <pre> 375: return TSDB_CODE_OUT_OF_MEMORY; 376: } 377: taosArrayPush(pBuf->pData, &p); 378: } 379: </pre>	
	污染路径: 函数'taosArrayPush'的定义点.	
771	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 519
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: <pre> 517: 518: // 8. generate logs 519: mInfo("sub:%s mq re-balance calculation completed, re- balanced vg", pSubKey); 520: for (int32_t i = 0; i < taosArrayGetSize(pOutput- >rebVgs); i++) { 521: SMqRebOutputVg *pOutputRebVg = taosArrayGet(pOutput->rebVgs, i); </pre>	
	污染路径:	

	函数'mInfo'的定义点.	
772	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 78
	步骤描述: 不使用返回值的函数'taosMbToWchar'应使用'void'说明.	
	代码片段: 76: } 77: 78: taosMbToWchar(&wc, str + pos, MB_CUR_MAX); 79: // ASSERT(rc == *size); // it will be core, if str is encode by utf8 and taos charset is gbk 80:	
	污染路径: 函数'taosMbToWchar'的定义点.	
773	缺陷发生位置: /source/libs/parser/test/mockCatalogService.cpp	行号: 177
	步骤描述: 不使用返回值的函数'tNameExtractFullName'应使用'void'说明.	
	代码片段: 175: int32_t catalogGetTableIndex(const SName* pTableName, SArray** pIndexes) const { 176: char tbFName[TSDB_TABLE_FNAME_LEN] = {0}; 177: tNameExtractFullName(pTableName, tbFName); 178: auto it = index_.find(tbFName); 179: if (index_.end() == it) {	
	污染路径: 函数'tNameExtractFullName'的定义点.	
774	缺陷发生位置: /source/util/src/tlockfree.c	行号: 33
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 31: nLoops++; 32: if (nLoops > 1000) { 33: sched_yield(); 34: nLoops = 0; 35: }	
	污染路径: 无	
775	缺陷发生位置:	行号:

	/source/libs/executor/src/sysscanoperator.c	614
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: 612: } 613: 614: blockDataDestroy(dataBlock); 615: if (ret != 0) { 616: pAPI->metaFn.closeTableMetaCursor(pInfo->pCur);	
	污染路径: 函数'blockDataDestroy'的定义点.	
776	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 44
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 42: if (schMgmt.jobRef < 0) { 43: qError("init scheduler jobRef failed, num:%u", schMgmt.cfg.maxJobNum); 44: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 45: } 46:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
777	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 35
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 33: mInfo("start to deploy sdb:%s", sdbTableName(i)); 34: if ((*fp)(pSdb->pMnode) != 0) { 35: mError("failed to deploy sdb:%s since %s", sdbTableName(i), terrstr()); 36: return -1; 37: }	
	污染路径: 函数'mError'的定义点.	
778	缺陷发生位置: /source/libs/parser/test/parInitialDTest.cpp	行号: 169
	步骤描述:	

	不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 167: 168: auto setDropSmaReq = [&](const char* pName, int8_t igNotExists = 0) { 169: sprintf(expect.name, "0.test.%s", pName); 170: expect.igNotExists = igNotExists; 171: };	
	污染路径: 无	
779	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndQnode.c	484
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 482: 483: if (tDeserializeSQnodeListReq(pReq->pCont, pReq->contLen, &qlistReq) != 0) { 484: mError("failed to parse qnode list req"); 485: terrno = TSDB_CODE_INVALID_MSG; 486: goto _OVER;	
780	污染路径: 函数'mError'的定义点.	
	缺陷发生位置:	行号:
	/source/libs/tdb/src/db/tdbBtree.c	132
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 130: ret = tdbPagerWrite(pPager, pPage); 131: if (ret < 0) { 132: tdbError("failed to write page since %s", terrstr()); 133: tdbAbort(pEnv, txn); 134: tdbOsFree(pBt);	
781	污染路径: 函数'tdbError'的定义点.	
	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndTelem.c	74
	步骤描述: 不使用返回值的函数'tjsonAddDoubleToObject'应使用'void'说明.	
	代码片段:	

	72: 73: tjsonAddDoubleToObject(pJson, "numOfDnode", mstat.numOfDnode); 74: tjsonAddDoubleToObject(pJson, "numOfMnode", mstat.numOfMnode); 75: tjsonAddDoubleToObject(pJson, "numOfVgroup", mstat.numOfVgroup); 76: tjsonAddDoubleToObject(pJson, "numOfDatabase", mstat.numOfDatabase);	
	污染路径: 函数'tjsonAddDoubleToObject'的定义点.	
782	缺陷发生位置: /source/util/src/tcache.c	行号: 160
	步骤描述: 不使用返回值的函数'taosThreadAttrSetDetachState'应使用'void'说明.	
	代码片段: 158: TdThreadAttr thattr; 159: taosThreadAttrInit(&thattr); 160: taosThreadAttrSetDetachState(&thattr, PTHREAD_CREATE_JOINABLE); 161: 162: taosThreadCreate(&cacheRefreshWorker, &thattr, taosCacheTimedRefresh, NULL);	
	污染路径: 函数'taosThreadAttrSetDetachState'的定义点.	
783	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 158
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 156: if (pConn == NULL) { 157: errno = TSDB_CODE_OUT_OF_MEMORY; 158: mError("conn:%d, failed to put into cache since %s, user:%s", connId, user, terrstr()); 159: return NULL; 160: } else {	
	污染路径: 函数'mError'的定义点.	
784	缺陷发生位置: /source/libs/executor/src/groupoperator.c	行号: 796

	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 794: 795: pOperator->status = OP_RES_TO_RETURN; 796: blockDataEnsureCapacity(pRes, 4096); 797: return buildPartitionResult(pOperator); 798: }	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
785	缺陷发生位置: /source/util/src/tthread.c	行号: 23
	步骤描述: 不使用返回值的函数'taosThreadAttrSetDetachState'应使用'void'说明.	
	代码片段: 21: TdThreadAttr thattr; 22: taosThreadAttrInit(&thattr); 23: taosThreadAttrSetDetachState(&thattr, PTHREAD_CREATE_JOINABLE); 24: int32_t ret = taosThreadCreate(pthread, &thattr, _start_routine, param); 25: taosThreadAttrDestroy(&thattr);	
	污染路径: 函数'taosThreadAttrSetDetachState'的定义点.	
786	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 483
	步骤描述: 不使用返回值的函数'mWarn'应使用'void'说明.	
	代码片段: 481: if (statusReq.clusterId != 0 && statusReq.clusterId != clusterid) { 482: code = TSDB_CODE_MND_DNODE_DIFF_CLUSTER; 483: mWarn("dnode:%d, %s, its clusterid:%" PRIu64 " differ from current cluster:%" PRIu64 " , code:0x%x", 484: statusReq.dnodeId, statusReq.dnodeEp, statusReq.clusterId, clusterid, code); 485: goto _OVER;	
	污染路径: 函数'mWarn'的定义点.	
787	缺陷发生位置:	行号:

	/source/libs/transport/test/transUT.cpp	71
	步骤描述: 不使用返回值的函数'taosVersionStrToInt'应使用'void'说明.	
	代码片段: 69: rpcClose(this->transCli); 70: rpclnit_.cfp = cb; 71: taosVersionStrToInt(version, &(rpclnit_.compatibilityVer)); 72: this->transCli = rpcOpen(&rpclnit_); 73: }	
	污染路径: 函数'taosVersionStrToInt'的定义点.	
788	缺陷发生位置: /source/common/src/tglobal.c	行号: 335
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 333: snprintf(cfgFile, sizeof(cfgFile), "%s" TD_DIRSEP "%s.cfg", cfgDir, CUS_PROMPT); 334: #else 335: snprintf(cfgFile, sizeof(cfgFile), "%s" TD_DIRSEP "taos.cfg", cfgDir); 336: #endif 337: } else {	
	污染路径: 无	
789	缺陷发生位置: /source/os/src/osFile.c	行号: 97
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 95: char rand[32] = {0}; 96: 97: sprintf(rand, "%" PRIu64, atomic_add_fetch_64(&seqId, 1)); 98: 99: snprintf(dstPath, PATH_MAX, tmpPath, getpid(), rand);	
	污染路径: 无	
790	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndConsumer.c	98
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 96: bool mndRebTryStart() { 97: int32_t old = atomic_val_compare_exchange_32(&mqReblnExecCnt, 0, 1); 98: mDebug("tq timer, rebalance counter old val:%d", old); 99: return old == 0; 100: }	
	污染路径: 函数'mDebug'的定义点.	
791	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 84
	步骤描述: 不使用返回值的函数'taos_init'应使用'void'说明.	
	代码片段: 82: shellCheckConnectMode(); 83: #endif 84: taos_init(); 85: 86: // kill heart-beat thread when quit	
	污染路径: 函数'taos_init'的定义点.	
792	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.c	行号: 45
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 43: 44: code = tsdbUpdateSkmTb(writer->config->tsdb, writer->ctx->tbid, writer->skmTb); 45: TSDB_CHECK_CODE(code, lino, _exit); 46: 47: writer->blockDataIdx = 0;	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
793	缺陷发生位置: /source/libs/function/src/tpercentile.c	行号: 307
	步骤描述:	

	不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 305: SArray **p1 = p; 306: p = taosHashIterate(pBucket->groupPagesMap, p); 307: taosArrayDestroy(*p1); 308: } 309:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
794	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 332
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 330: } 331: } 332: taosArrayPush(pCxt->pNsLevel, &pTables); 333: ++currTotalLevel; 334: } while (currTotalLevel <= pCxt->currLevel);	
	污染路径: 函数'taosArrayPush'的定义点.	
795	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 178
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 176: if (code) goto _err; 177: } 178: taosMemoryFree(pWriter); 179: *ppWriter = NULL; 180:	
	污染路径: 函数'taosMemoryFree'的定义点.	
796	缺陷发生位置: /docs/examples/c/subscribe_demo.c	行号: 65
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 63: 64: taos_close(taos);	

	65: taos_cleanup(); 66: }	
	污染路径: 函数'taos_cleanup'的定义点.	
797	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 122
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: <pre> 120: char* pData = tsortGetValue(pTupleHandle, i); 121: if (pData != NULL) { 122: colDataSetVal(pColInfo, pBlock->info.rows, pData, false); 123: } 124: }</pre>	
	污染路径: 函数'colDataSetVal'的定义点.	
798	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smWorker.c	行号: 125
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: <pre> 123: taosArrayDestroy(pMgmt->writeWroker); 124: tSingleWorkerCleanup(&pMgmt->streamWorker); 125: dDebug("snode workers are closed"); 126: } 127:</pre>	
	污染路径: 函数'dDebug'的定义点.	
799	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmFile.c	行号: 66
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: <pre> 64: SJson *pjson = NULL; 65: char file[PATH_MAX] = {0}; 66: snprintf(file, sizeof(file), "%s%snode.json", path, TD_DIRSEP); 67: 68: if (taosStatFile(file, NULL, NULL, NULL) < 0) {</pre>	

	污染路径: 无	
800	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 146
	步骤描述: 不使用返回值的函数'tjsonAddItemToArray'应使用'void'说明.	
	代码片段: 144: 145: SJson *item = tjsonCreateObject(); 146: tjsonAddItemToArray(items, item); 147: tjsonAddStringToObject(item, "name", mndGetStbStr(pObj->name)); 148: tjsonAddStringToObject(item, "db", mndGetDbStr(pObj->db));	
	污染路径: 函数'tjsonAddItemToArray'的定义点.	
801	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 822
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 820: taosWUnLockLatch(&pCtx->grpLock); 821: 822: taosArrayDestroy(pParam->pChildren); 823: pParam->pChildren = NULL; 824: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
802	缺陷发生位置: /source/libs/stream/src/streamSnapshot.c	行号: 223
	步骤描述: 不使用返回值的函数'streamGetFileSize'应使用'void'说明.	
	代码片段: 221: item.name = pFile->pCurrent; 222: item.type = ROCKSDB_CURRENT_TYPE; 223: streamGetFileSize(pFile->path, item.name, &item.size); 224: taosArrayPush(list, &item); 225:	
	污染路径:	

	函数'streamGetFileSize'的定义点.	
803	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 240
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 238: 239: if (!pTxn) { 240: tdbError("tdb/pcache: null ptr pTxn, release failed."); 241: return; 242: }	
	污染路径: 函数'tdbError'的定义点.	
804	缺陷发生位置: /source/libs/executor/src/aggregateoperator.c	行号: 426
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 424: pData = getBufPage(pResultBuf, getPageld(pi)); 425: if (pData == NULL) { 426: qError("failed to get buffer, code:%s", tstrerror(errno)); 427: return errno; 428: }	
	污染路径: 函数'qError'的定义点.	
805	缺陷发生位置: /source/common/src/tname.c	行号: 93
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 91: pName->type = TSDB_TABLE_NAME_T; 92: pName->acctId = acctId; 93: snprintf(pName->dbname, sizeof(pName->dbname), "%s", pDbName); 94: snprintf(pName->tname, sizeof(pName->tname), "%s", pTableName); 95: return pName;	
	污染路径: 无	

806	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 321
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 319: pTrans->clientRpc = rpcOpen(&rpclnit); 320: if (pTrans->clientRpc == NULL) { 321: dError("failed to init dnode rpc client"); 322: return -1; 323: }	
	污染路径: 函数'dError'的定义点.	
807	缺陷发生位置: /source/util/src/tsched.c	行号: 98
	步骤描述: 不使用返回值的函数'taosThreadAttrInit'应使用'void'说明.	
	代码片段: 96: for (int32_t i = 0; i < numOfThreads; ++i) { 97: TdThreadAttr attr; 98: taosThreadAttrInit(&attr); 99: taosThreadAttrSetDetachState(&attr, PTHREAD_CREATE_JOINABLE); 100: int32_t code = taosThreadCreate(pSched->qthread + i, &attr, taosProcessSchedQueue, (void *)pSched);	
	污染路径: 函数'taosThreadAttrInit'的定义点.	
808	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 73
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 71: float v = 0; 72: GET_TYPED_DATA(v, float, pVar->nType, &pVar->f); 73: colDataSetVal(pDst, rowIndex, (char*)&v, isNull); 74: } else if (pDst->info.type == TSDB_DATA_TYPE_DOUBLE) { 75: double v = 0;	
	污染路径: 函数'colDataSetVal'的定义点.	
809	缺陷发生位置:	行号:

	/source/libs/qworker/src/qwUtil.c	81
	步骤描述: 不使用返回值的函数'QW_ERR_RET'应使用'void'说明.	
	代码片段: 79: if (NULL == newSch.tasksHash) { 80: QW_SCH_ELOG("taosHashInit %d failed", mgmt->cfg.maxSchTaskNum); 81: QW_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 82: } 83:	
	污染路径: 函数'QW_ERR_RET'的定义点.	
810	缺陷发生位置: /source/libs/transport/src/tmsgcb.c	行号: 50
	步骤描述: 不使用返回值的函数'rpcSendResponse'应使用'void'说明.	
	代码片段: 48: void tmsgSendRsp(SRpcMsg* pMsg) { 49: #if 1 50: rpcSendResponse(pMsg); 51: #else 52: return (*defaultMsgCb.sendRspFp)(pMsg);	
	污染路径: 函数'rpcSendResponse'的定义点.	
811	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 74
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 72: static int32_t mmOpen(SMgmtInputOpt *pInput, SMgmtOutputOpt *pOutput) { 73: if (wallnit() != 0) { 74: dError("failed to init wal since %s", terrstr()); 75: return -1; 76: }	
	污染路径: 函数'dError'的定义点.	
812	缺陷发生位置: /source/client/src/clientStmt.c	行号: 109
	步骤描述:	

	不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 107: 108: if ('\0' == pStmt->bInfo.tbName[0]) { 109: tscError("no table name set"); 110: STMT_ERR_RET(TSDB_CODE_TSC_STMT_TBNAME_ERROR); 111: }	
	污染路径: 函数'tscError'的定义点.	
813	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 1914
	步骤描述: 不使用返回值的函数'tdListFree'应使用'void'说明.	
	代码片段: 1912: void destroyMergeIntervalOperatorInfo(void* param) { 1913: SMergeIntervalAggOperatorInfo* mialInfo = (SMergeIntervalAggOperatorInfo*)param; 1914: tdListFree(mialInfo->groupIntervals); 1915: destroyIntervalOperatorInfo(&mialInfo->intervalAggOperatorInfo); 1916:	
	污染路径: 函数'tdListFree'的定义点.	
814	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 23
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 21: printf("syncIndexMgrPrint len:%" PRIu64 " %s \n", (uint64_t)strlen(serialized), serialized); 22: fflush(NULL); 23: taosMemoryFree(serialized); 24: } 25:	
	污染路径: 函数'taosMemoryFree'的定义点.	
815	缺陷发生位置: /source/libs/tfs/test/tfsTest.cpp	行号: 367
	步骤描述:	

	不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: 365: taosRemoveDir(root23); 366: taosMkDir(root00); 367: taosMkDir(root01); 368: taosMkDir(root10); 369: taosMkDir(root11);	
	污染路径: 函数'taosMkDir'的定义点.	
816	缺陷发生位置: /source/util/src/tthread.c	行号: 38
	步骤描述: 不使用返回值的函数'taosThreadJoin'应使用'void'说明.	
	代码片段: 36: if (taosThreadRunning(pthread)) { 37: taosThreadCancel(*pthread); 38: taosThreadJoin(*pthread, NULL); 39: } 40:	
	污染路径: 函数'taosThreadJoin'的定义点.	
817	缺陷发生位置: /source/os/src/osSleep.c	行号: 29
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 27: Sleep(1000 * s); 28: #else 29: sleep(s); 30: #endif 31: }	
	污染路径: 无	
818	缺陷发生位置: /source/libs/transport/src/trans.c	行号: 119
	步骤描述: 不使用返回值的函数'tInfo'应使用'void'说明.	
	代码片段: 117: transRemoveExHandle(transGetInstMgt(), (int64_t)arg); 118: transReleaseExHandle(transGetInstMgt(), (int64_t)arg);	

	119: tInfo("end to close rpc"); 120: return; 121: }	
	污染路径: 函数'tInfo'的定义点.	
819	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 306
	步骤描述: 不使用返回值的函数'tSerializeSDCreateMnodeReq'应使用'void'说明.	
	代码片段: 304: int32_t contLen = tSerializeSDCreateMnodeReq(NULL, 0, pCreateReq); 305: void *pReq = taosMemoryMalloc(contLen); 306: tSerializeSDCreateMnodeReq(pReq, contLen, pCreateReq); 307: 308: STransAction action = {	
	污染路径: 函数'tSerializeSDCreateMnodeReq'的定义点.	
820	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmFile.c	行号: 76
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 74: if (pFile == NULL) { 75: terrno = TAOS_SYSTEM_ERROR(errno); 76: dError("failed to open mnode file:%s since %s", file, terrstr()); 77: goto _OVER; 78: }	
	污染路径: 函数'dError'的定义点.	
821	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 124
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 122: ret = smlParseInfluxString(info, sql, sql + strlen(sql), &elements); 123: ASSERT_EQ(ret, 0);	

	124: taosArrayDestroy(elements.colArray); 125: elements.colArray = NULL; 126: smlClearForRerun(info);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
822	缺陷发生位置: /source/common/src/trow.c	行号: 223
	步骤描述: 不使用返回值的函数'tdSTSRolterGetTpVal'应使用'void'说明.	
	代码片段: 221: } 222: } 223: tdSTSRolterGetTpVal(plter, pCol->type, pCol->offset, pVal); 224: ++plter->colIdx; 225: } else if (TD_IS_KV_ROW(plter->pRow)) {	
	污染路径: 函数'tdSTSRolterGetTpVal'的定义点.	
823	缺陷发生位置: /source/libs/executor/src/cachescanoperator.c	行号: 305
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 303: taosMemoryFree(pInfo->pSlotIds); 304: taosMemoryFree(pInfo->pDstSlotIds); 305: taosArrayDestroy(pInfo->pCidList); 306: taosArrayDestroy(pInfo->pUidList); 307: taosArrayDestroy(pInfo->matchInfo.pList);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
824	缺陷发生位置: /source/libs/qworker/src/qwUtil.c	行号: 92
	步骤描述: 不使用返回值的函数'QW_ERR_RET'应使用'void'说明.	
	代码片段: 90: QW_SCH_ELOG("taosHashPut new sch to scheduleHash failed, errno:%d", errno); 91: taosHashCleanup(newSch.tasksHash); 92: QW_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 93: }	

	94:	
	污染路径: 函数'QW_ERR_RET'的定义点.	
825	缺陷发生位置: /source/libs/tdb/src/db/tdbPager.c	行号: 735
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 733: code = tdbTbInsert(pPager->pEnv->pFreeDb, &pgno, sizeof(pgno), NULL, 0, pTxn); 734: if (code < 0) { 735: tdbError("tdb/insert-free-page: tb insert failed with ret: %d.", code); 736: taosArrayDestroy(pPager->frps); 737: pPager->frps = NULL;	
	污染路径: 函数'tdbError'的定义点.	
826	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 87
	步骤描述: 不使用返回值的函数'tSerializeSTableInfoReq'应使用'void'说明.	
	代码片段: 85: int32_t bufLen = tSerializeSTableInfoReq(NULL, 0, &infoReq); 86: void *pBuf = (*mallocFp)(bufLen); 87: tSerializeSTableInfoReq(pBuf, bufLen, &infoReq); 88: 89: *msg = pBuf;	
	污染路径: 函数'tSerializeSTableInfoReq'的定义点.	
827	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号: 161
	步骤描述: 不使用返回值的函数'tBrinBlockDestroy'应使用'void'说明.	
	代码片段: 159: } 160: TARRAY2_DESTROY(ctx->brinBlkArray, NULL); 161: tBrinBlockDestroy(ctx->brinBlock); 162: tBlockDataDestroy(ctx->blockData); 163: tMapDataClear(ctx->mDataBlk);	

	污染路径: 函数'tBrinBlockDestroy'的定义点.	
828	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 119
	步骤描述: 不使用返回值的函数's3CleanUp'应使用'void'说明.	
	代码片段: 117: tqCleanUp(); 118: smaCleanUp(); 119: s3CleanUp(); 120: } 121:	
	污染路径: 函数's3CleanUp'的定义点.	
829	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 65
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 63: ASSERT_EQ(elements.timestamp, sql + elements.measureTagsLen + 1 + elements.colsLen + 1); 64: ASSERT_EQ(elements.timestampLen, strlen("1626006833639000000")); 65: taosArrayDestroy(elements.colArray); 66: elements.colArray = NULL; 67:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
830	缺陷发生位置: /source/libs/parser/test/parShowToUse.cpp	行号: 272
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 270: 271: auto setTrimDbReq = [&](const char* pDb, int32_t maxSpeed = 0) { 272: snprintf(expect.db, sizeof(expect.db), "0.%s", pDb); 273: expect.maxSpeed = maxSpeed; 274: };	

	污染路径: 无	
831	缺陷发生位置: /source/util/src/theap.c	行号: 255
	步骤描述: 不使用返回值的函数'pqHeapify'应使用'void'说明.	
	代码片段: 253: if (pqContainerSize(pq) > 1) { 254: for (size_t i = pqContainerSize(pq) - 1; i > 0; --i) { 255: pqHeapify(pq, i, pqContainerSize(pq)); 256: } 257: pqHeapify(pq, 0, pqContainerSize(pq));	
	污染路径: 函数'pqHeapify'的定义点.	
832	缺陷发生位置: /source/libs/catalog/src/ctgRemote.c	行号: 54
	步骤描述: 不使用返回值的函数'ctgError'应使用'void'说明.	
	代码片段: 52: SHashObj* pBatchs = taosHashInit(taskNum, taosGetDefaultHashFunction(TSDB_DATA_TYPE_INT), false, HASH_NO_LOCK); 53: if (NULL == pBatchs) { 54: ctgError("taosHashInit %d batch failed", taskNum); 55: CTG_ERR_JRET(TSDB_CODE_OUT_OF_MEMORY); 56: }	
	污染路径: 函数'ctgError'的定义点.	
833	缺陷发生位置: /source/client/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 80
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 78: LOAD_FP(SetFocusLost, SetFocusLostFP, "setFocusLostFP"); 79: 80: LOAD_FP(SetCaretUpdate, SetCaretUpdateFP, "setCaretUpdateFP"); 81: 82: LOAD_FP(SetMouseClicked, SetMouseClickedFP, "setMouseClickedFP");	

	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
834	缺陷发生位置: /source/libs/index/test/index_executor_tests.cpp	行号: 76
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 74: 75: taosArrayPush(res->pDataBlock, &idata); 76: taosArrayPush(pBlockList, &res); 77: 78: blockDataEnsureCapacity(res, rows);	
	污染路径: 函数'taosArrayPush'的定义点.	
835	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 377
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 375: 376: for (int iData = 1; iData <= nData; iData++) { 377: sprintf(key, "key0"); 378: sprintf(val, "value%d", iData); 379:	
	污染路径: 无	
836	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 46
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 44: } 45: if (pEp != NULL) { 46: snprintf(pEp, TSDB_EP_LEN, "%s:%u", pDnodeEp->ep.fqdn, pDnodeEp->ep.port); 47: } 48: }	

	污染路径: 无	
837	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 69
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 67: 68: if (udfcOpen() != 0) { 69: dError("failed to open udfc in snode"); 70: smClose(pMgmt); 71: return -1;	
	污染路径: 函数'dError'的定义点.	
838	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 95
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 93: ASSERT_EQ(elements.timestamp, sql + elements.measureTagsLen + 1 + elements.colsLen + 1); 94: ASSERT_EQ(elements.timestampLen, strlen("1626006833639000000")); 95: taosArrayDestroy(elements.colArray); 96: elements.colArray = NULL; 97:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
839	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 116
	步骤描述: 不使用返回值的函数'tNameFromString'应使用'void'说明.	
	代码片段: 114: // cJSON_AddItemToObject(json, "uid", uid); 115: SName name = {0}; 116: tNameFromString(&name, req.name, T_NAME_ACCT T_NAME_DB T_NAME_TABLE); 117: cJSON* tableType = cJSON_CreateString("super"); 118: cJSON_AddItemToObject(json, "tableType", tableType);	
	污染路径:	

	函数'tNameFromString'的定义点.	
840	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 374
	步骤描述: 不使用返回值的函数'taosFtruncateFile'应使用'void'说明.	
	代码片段: 372: lastEndOffset); 373: 374: taosFtruncateFile(pFile, lastEndOffset); 375: taosCloseFile(&pFile); 376: return 0;	
	污染路径: 函数'taosFtruncateFile'的定义点.	
841	缺陷发生位置: /utils/test/c/tmqOffset.c	行号: 56
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 54: } 55: 56: taosCloseFile(&pFile); 57: } 58: return 0;	
	污染路径: 函数'taosCloseFile'的定义点.	
842	缺陷发生位置: /source/dnode/vnode/src/meta/metaSnapshot.c	行号: 44
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 42: code = tdbTbcOpen(pMeta->pTbDb, &pReader->pTbc, NULL); 43: if (code) { 44: taosMemoryFree(pReader); 45: goto _err; 46: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
843	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 99

	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 97: pHead->bodyLen = tlen + sizeof(SMsgHead); 98: taosMemoryFree(buf); 99: taosArrayDestroy(reqNew.pArray); 100: } 101:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
844	缺陷发生位置: /tools/shell/src/shellCommand.c	行号: 407
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 405: memset(total_string, '\0', SHELL_MAX_COMMAND_SIZE); 406: if (strcmp(cmd->buffer, "") == 0) { 407: sprintf(total_string, "%s%s", shell.info.promptHeader, cmd->command); 408: } else { 409: sprintf(total_string, "%s%s", shell.info.promptContinue, cmd->command);	
	污染路径: 无	
845	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffsetSnapshot.c	行号: 142
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 140: } 141: } 142: taosMemoryFree(fname); 143: taosMemoryFree(pWriter->fname); 144: taosMemoryFree(pWriter);	
	污染路径: 函数'taosMemoryFree'的定义点.	
846	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 33
	步骤描述: 不使用返回值的函数'QW_ERR_JRET'应使用'void'说明.	

	代码片段: 31: } 32: 33: QW_ERR_JRET(TSDB_CODE_APP_ERROR); 34: } 35:	
	污染路径: 函数'QW_ERR_JRET'的定义点.	
847	缺陷发生位置: /tests/script/api/passwdTest.c	行号: 213
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 211: // users 212: for (int i = 0; i < nUser; ++i) { 213: sprintf(users[i], "user%d", i); 214: sprintf(qstr, "CREATE USER %s PASS 'taosdata'", users[i]); 215: queryDB(taos, qstr);	
	污染路径: 无	
848	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 239
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 237: 238: if (status == TASK_STATUS__SCAN_HISTORY) { 239: qDebug("s-task:%s enter into scan-history data stage, status:%s", id, str); 240: streamTaskLaunchScanHistory(pTask); 241: } else {	
	污染路径: 函数'qDebug'的定义点.	
849	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 94
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 92:	

	93: SColumnInfoData infoData = createColumnInfoData(TSDB_DATA_TYPE_INT, 4, 1); 94: taosArrayPush(b->pDataBlock, &infoData); 95: blockDataAppendColInfo(b, &infoData); 96:	
	污染路径: 函数'taosArrayPush'的定义点.	
850	缺陷发生位置: /examples/lua/lua51/lua_connector51.c	行号: 63
	步骤描述: 不使用返回值的函数'taos_init'应使用'void'说明.	
	代码片段: 61: lua_settop(L, 0); 62: 63: taos_init(); 64: 65: lua_newtable(L);	
	污染路径: 函数'taos_init'的定义点.	
851	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 160
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 158: printf("logStoreSimplePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 159: fflush(NULL); 160: taosMemoryFree(serialized); 161: } 162:	
	污染路径: 函数'taosMemoryFree'的定义点.	
852	缺陷发生位置: /source/common/src/tmisc.c	行号: 85
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: 83: SEpSet getEpSet_s(SCorEpSet* pEpSet) { 84: SEpSet ep = {0}; 85: taosCorBeginRead(&pEpSet->version);	

	86: ep = pEpSet->epSet; 87: taosCorEndRead(&pEpSet->version);	
	污染路径: expanded from macro 'taosCorBeginRead'	
853	缺陷发生位置: /source/libs/parser/src/parInsertSml.c	行号: 398
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 396: tdDestroySVCreateTbReq(pCreateTbReq); 397: taosMemoryFree(pCreateTbReq); 398: taosArrayDestroy(tagName); 399: return ret; 400: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
854	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 407
	步骤描述: 不使用返回值的函数'getNextTimeWindow'应使用'void'说明.	
	代码片段: 405: 406: int32_t precision = pInterval->precision; 407: getNextTimeWindow(pInterval, pNext, order); 408: 409: // next time window is not in current block	
	污染路径: 函数'getNextTimeWindow'的定义点.	
855	缺陷发生位置: /source/libs/executor/src/dynqueryctrloperator.c	行号: 506
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 504: } 505: 506: qError("dynamic post task begin"); 507: *ppRes = pOperator->pDownstream[1]- >fpSet.getNextExtFn(pOperator->pDownstream[1], pParam); 508: if (*ppRes) {	

	污染路径: 函数'qError'的定义点.	
856	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 262
	步骤描述: 不使用返回值的函数'mFatal'应使用'void'说明.	
	代码片段: <pre> 260: } else { 261: terrno = TSDB_CODE_APP_ERROR; 262: mFatal("dnode:%d, failed to acquire db since %s", dnodeId, terrstr()); 263: } 264: }</pre>	
	污染路径: 函数'mFatal'的定义点.	
857	缺陷发生位置: /source/libs/stream/src/streamRecover.c	行号: 242
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: <pre> 240: streamTaskLaunchScanHistory(pTask); 241: } else { 242: qDebug("s-task:%s downstream tasks are ready, now ready for data from wal, status:%s", id, str); 243: } 244:</pre>	
	污染路径: 函数'qDebug'的定义点.	
858	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 70
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 68: 69: tdbTbcClose((*ppReader)->pCur); 70: taosMemoryFree(*ppReader); 71: *ppReader = NULL; 72:</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	

859	缺陷发生位置： /source/libs/parser/src/parAstCreator.c	行号： 104
	步骤描述： 不使用返回值的函数'strdequote'应使用'void'说明.	
	代码片段： 102: } else { 103: strncpy(pPassword, pPasswordToken->z, pPasswordToken->n); 104: strdequote(pPassword); 105: if (strtrim(pPassword) <= 0) { 106: pCxt->errCode = generateSyntaxErrMsg(&pCxt->msgBuf, TSDB_CODE_PAR_PASSWD_EMPTY);	
	污染路径： 函数'strdequote'的定义点.	
860	缺陷发生位置： /source/libs/index/src/indexFstDfa.c	行号： 96
	步骤描述： 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段： 94: } 95: } 96: taosArrayDestroy(states); 97: taosHashCleanup(seen); 98: return builder->dfa;	
	污染路径： 函数'taosArrayDestroy'的定义点.	
861	缺陷发生位置： /source/libs/index/test/indexTests.cc	行号： 668
	步骤描述： 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段： 666: std::cout << "size : " << taosArrayGetSize(ret) << std::endl; 667: assert(taosArrayGetSize(ret) == 4); 668: taosArrayDestroy(ret); 669: 670: indexTermDestroy(term);	
	污染路径： 函数'taosArrayDestroy'的定义点.	
862	缺陷发生位置：	行号：

	/source/client/src/clientImpl.c	54
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 52: static char* getClusterKey(const char* user, const char* auth, const char* ip, int32_t port) { 53: char key[512] = {0}; 54: snprintf(key, sizeof(key), "%s:%s:%s:%d", user, auth, ip, port); 55: return taosStrdup(key); 56: }	
	污染路径: 无	
863	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 94
	步骤描述: 不使用返回值的函数'tInfo'应使用'void'说明.	
	代码片段: 92: 93: if (num % 10000 == 0) { 94: tInfo("%d request have been written into disk", num); 95: } 96: }	
	污染路径: 函数'tInfo'的定义点.	
864	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 90
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 88: } 89: 90: taosMemoryFree(pMemBuf); 91: } 92:	
	污染路径: 函数'taosMemoryFree'的定义点.	
865	缺陷发生位置: /examples/c/asyncdemo.c	行号: 72
	步骤描述:	

	不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 70: taos_free_result(pSql); 71: taos_close(taos); 72: taos_cleanup(); 73: exit(EXIT_FAILURE); 74: }	
	污染路径: 函数'taos_cleanup'的定义点.	
866	缺陷发生位置: /source/libs/executor/src/groupcacheoperator.c	行号: 84
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 82: offset += sprintf(buf + offset, " %" PRIu64 , pGrpCacheOperator->execInfo.pDownstreamBlkNum[i]); 83: } 84: qDebug("%s", buf); 85: taosMemoryFree(buf); 86: }	
	污染路径: 函数'qDebug'的定义点.	
867	缺陷发生位置: /source/common/src/ttime.c	行号: 746
	步骤描述: 不使用返回值的函数'taosLocalTime'应使用'void'说明.	
	代码片段: 744: 745: t = (time_t)ekey; 746: taosLocalTime(&t, &tm, NULL); 747: int32_t emon = tm.tm_year * 12 + tm.tm_mon; 748:	
	污染路径: 函数'taosLocalTime'的定义点.	
868	缺陷发生位置: /source/libs/executor/test/sortTests.cpp	行号: 77
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 75:	

	76: blockDataAppendCollInfo(pBlock, &collInfo); 77: blockDataEnsureCapacity(pBlock, pInfo->pageRows); 78: 79: for (int32_t i = 0; i < pInfo->pageRows; ++i) {	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
869	缺陷发生位置: /tests/script/api/passwdTest.c	行号: 214
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 212: for (int i = 0; i < nUser; ++i) { 213: sprintf(users[i], "user%d", i); 214: sprintf(qstr, "CREATE USER %s PASS 'taosdata'", users[i]); 215: queryDB(taos, qstr); 216:	
	污染路径: 无	
870	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 87
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 85: SSyncNode* pSyncNode = syncNodeAcquire(rid); 86: if (pSyncNode == NULL) { 87: sError("failed to acquire rid:%" PRIu64 " of tsNodeReftId for pSyncNode", rid); 88: return -1; 89: }	
	污染路径: 函数'sError'的定义点.	
871	缺陷发生位置: /source/libs/executor/src/dynqueryctrloperator.c	行号: 385
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 383: int32_t vgNum = tSimpleHashGetSize(pVg); 384: if (vgNum <= 0 vgNum > 1) { 385: qError("Invalid vgroup num %d to build table scan	

	operator param", vgNum); 386: return TSDB_CODE_QRY_EXECUTOR_INTERNAL_ERROR; 387: }	
	污染路径: 函数'qError'的定义点.	
872	缺陷发生位置: /source/libs/function/src/builtinsimpl.c	行号: 1340
	步骤描述: 不使用返回值的函数'colDataSetVal'应使用'void'说明.	
	代码片段: 1338: SColumnInfoData* pCol = taosArrayGet(pBlock->pDataBlock, slotId); 1339: 1340: colDataSetVal(pCol, pBlock->info.rows, res, false); 1341: 1342: taosMemoryFree(res);	
	污染路径: 函数'colDataSetVal'的定义点.	
873	缺陷发生位置: /tests/script/api/demoapi.c	行号: 93
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 91: 92: char command[1024] = {0}; 93: sprintf(command, "%s", "create table meters(ts timestamp, f float, n int, " 94: "bin1 binary(20), c nchar(20), bin2 binary(20)) tags(area int, " 95: "city binary(20), dist nchar(20), street binary(20));");	
	污染路径: 无	
874	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmHandle.c	行号: 65
	步骤描述: 不使用返回值的函数'dGError'应使用'void'说明.	
	代码片段: 63:	

	<pre> 64: if (mmWriteFile(pInput->path, &option) != 0) { 65: dGError("failed to write mnode file since %s", terrstr()); 66: return -1; 67: } </pre>	
	污染路径: 函数'dGError'的定义点.	
875	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 648
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: <pre> 646: taosMemoryFreeClear(keyBuf); 647: taosHashCleanup(ctx.colHash); 648: taosArrayDestroy(ctx.cInfoList); 649: blockDataDestroy(pResBlock); 650: taosArrayDestroy(pBlockList); </pre>	
	污染路径: 函数'taosArrayDestroy'的定义点.	
876	缺陷发生位置: /source/libs/executor/src/tsort.c	行号: 296
	步骤描述: 不使用返回值的函数'blockDataDestroy'应使用'void'说明.	
	代码片段: <pre> 294: destroyDiskbasedBuf(pSortHandle->pBuf); 295: taosMemoryFreeClear(pSortHandle->idStr); 296: blockDataDestroy(pSortHandle->pDataBlock); 297: if (pSortHandle->pBoundedQueue) destroyBoundedQueue(pSortHandle->pBoundedQueue); 298: </pre>	
	污染路径: 函数'blockDataDestroy'的定义点.	
877	缺陷发生位置: /source/os/src/osSocket.c	行号: 768
	步骤描述: 不使用返回值的函数'taosCloseSocket'应使用'void'说明.	
	代码片段: <pre> 766: if (taosSetSockOpt(pSocket, SOL_SOCKET, SO_REUSEADDR, (void *)&reuse, sizeof(reuse)) < 0) { 767: // printf("setsockopt SO_REUSEADDR failed: %d (%s)", errno, strerror(errno)); </pre>	

	768: taosCloseSocket(&pSocket); 769: return false; 770: }	
	污染路径: 函数'taosCloseSocket'的定义点.	
878	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 431
	步骤描述: 不使用返回值的函数'ctgInfo'应使用'void'说明.	
	代码片段: 429: CTG_STAT_RT_INC(numOfOpClearCache, 1); 430: 431: ctgInfo("handle cleared, clusterId:0x%" PRIx64, clusterId); 432: } 433:	
	污染路径: 函数'ctgInfo'的定义点.	
879	缺陷发生位置: /source/os/src/osDir.c	行号: 302
	步骤描述: 不使用返回值的函数'taosCloseDir'应使用'void'说明.	
	代码片段: 300: } 301: 302: taosCloseDir(&pDir); 303: rmdir(dirname); 304: }	
	污染路径: 函数'taosCloseDir'的定义点.	
880	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 82
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 80: void* buf = taosMemoryMalloc(tlen); 81: if (NULL == buf) { 82: taosArrayDestroy(reqNew.pArray); 83: for (int32_t iReq = 0; iReq < req.nReqs; iReq++) { 84: pCreateReq = req.pReqs + iReq;	

	污染路径: 函数'taosArrayDestroy'的定义点.	
881	缺陷发生位置: /source/libs/executor/src/dataDispatcher.c	行号: 103
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 101: pBuf->pData = taosMemoryMalloc(pBuf->allocSize); 102: if (pBuf->pData == NULL) { 103: qError("SinkNode failed to malloc memory, size:%d, code:%d", pBuf->allocSize, TAOS_SYSTEM_ERROR(errno)); 104: } 105:	
	污染路径: 函数'qError'的定义点.	
882	缺陷发生位置: /contrib/test/cos/main.c	行号: 126
	步骤描述: 不使用返回值的函数'cos_warn_log'应使用'void'说明.	
	代码片段: 124: 125: char *pstr3 = apr_pstrndup(p, (char *)fmt_str->pos, cos_buf_size(fmt_str)); 126: cos_warn_log("Format string: %s", pstr3); 127: 128: // Format-String sha1hash	
	污染路径: 函数'cos_warn_log'的定义点.	
883	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead.c	行号: 338
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 336: return TSDB_CODE_OUT_OF_MEMORY; 337: } 338: taosArrayPush(pBuf->pData, &p); 339: } 340:	
	污染路径:	

	函数'taosArrayPush'的定义点.	
884	缺陷发生位置: /tests/script/api/stmt_function.c	行号: 92
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 90: stmt = taos_stmt_init(taos); 91: assert(stmt != NULL); 92: sprintf(stmt_sql, "insert into ? values (?,?,?,?,?,?,?,1,?,?,?)); 93: assert(taos_stmt_prepare(stmt, stmt_sql, 0) == 0); 94: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
885	缺陷发生位置: /tests/script/api/demoapi.c	行号: 161
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 159: __func__, __LINE__, taos_errstr(res)); 160: } 161: sprintf(command, "insert into t%"PRIu64" " 162: "values(%" PRIu64 " , %f, %"PRIu64" , " 163: "'%c%d', '%s%c%d', '%c%d')",	
	污染路径: 无	
886	缺陷发生位置: /source/libs/command/src/explain.c	行号: 96
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 94: SArray *rows = taosArrayInit(10, sizeof(SQueryExplainRowInfo)); 95: if (NULL == rows) { 96: qError("taosArrayInit SQueryExplainRowInfo failed"); 97: QRY_ERR_JRET(TSDB_CODE_OUT_OF_MEMORY); 98: }	
	污染路径: 函数'qError'的定义点.	
887	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndGrant.c	31
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 29: SColumnInfoData *pCollInfo = taosArrayGet(pBlock->pDataBlock, cols); 30: const char *src = "community"; 31: STR_WITH_MAXSIZE_TO_VARSTR(tmp, src, 32); 32: colDataSetVal(pCollInfo, numOfRows, tmp, false); 33:	
888	污染路径: 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 142
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 140: 141: int32_t dmProcessAuthRsp(SDnodeMgmt *pMgmt, SRpcMsg *pMsg) { 142: dError("auth rsp is received, but not supported yet"); 143: return 0; 144: }	
	污染路径: 函数'dError'的定义点.	
	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmInt.c	行号: 63
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
889	代码片段: 61: 62: if (udfStartUdfd(pMgmt->pData->dnodeId) != 0) { 63: dError("failed to start udfd"); 64: } 65:	
	污染路径: 函数'dError'的定义点.	
890	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 629
	步骤描述:	

	不使用返回值的函数'indexInfo'应使用'void'说明.	
	代码片段: 627: idxMemUnRef(mem); 628: idxMemUnRef(imm); 629: indexInfo("cache search, time cost %" PRIu64 "us", taosGetTimestampUs() - st); 630: 631: return ret;	
	污染路径: 函数'indexInfo'的定义点.	
891	缺陷发生位置: /source/util/src/tarray.c	行号: 405
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 403: 404: if (fp == NULL) { 405: taosArrayDestroy(pArray); 406: return; 407: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
892	缺陷发生位置: /source/client/src/clientRawBlockWrite.c	行号: 318
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 316: end: 317: cJSON_AddItemToObject(json, "tags", tags); 318: taosArrayDestroy(pTagVals); 319: } 320:	
	污染路径: 函数'taosArrayDestroy'的定义点.	
893	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 511
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 509:	

	510: for (int iData = 0; iData < nData; ++iData) { 511: sprintf(key, "key%d", iData); 512: sprintf(val, "value%d", iData); 513:	
	污染路径: 无	
894	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeSnapshot.c	行号: 440
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 438: } else { 439: vInfo("vgId:%d, vnode snapshot writer closed, rollback:%d", TD_VID(pVnode), rollback); 440: taosMemoryFree(pWriter); 441: } 442: tsem_post(&pVnode->canCommit);	
	污染路径: 函数'taosMemoryFree'的定义点.	
895	缺陷发生位置: /source/dnode/mnode/impl/src/mndMain.c	行号: 123
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 121: 122: static void mndPullupTrimDb(SMnode *pMnode) { 123: mTrace("pullup trim"); 124: int32_t contLen = 0; 125: void *pReq = mndBuildTimerMsg(&contLen);	
	污染路径: 函数'mTrace'的定义点.	
896	缺陷发生位置: /source/libs/scheduler/src/schFlowCtrl.c	行号: 84
	步骤描述: 不使用返回值的函数'SCH_TASK_ELOG'应使用'void'说明.	
	代码片段: 82: ctrl = (SSchFlowControl *)taosHashGet(pJob->flowCtrl, ep, sizeof(SEp)); 83: if (NULL == ctrl) { 84: SCH_TASK_ELOG("taosHashGet node from flowCtrl	

	failed, fqdn:%s, port:%d", ep->fqdn, ep->port); 85: SCH_ERR_RET(TSDB_CODE_SCH_INTERNAL_ERROR); 86: }	
	污染路径: 函数'SCH_TASK_ELOG'的定义点.	
897	缺陷发生位置: /testscript/api/api_with_reqid_test.c	行号: 75
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 73: taos_free_result(result); 74: // super tables subscription 75: usleep(1000000); 76: } 77:	
	污染路径: 无	
898	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMergeTree.c	行号: 466
	步骤描述: 不使用返回值的函数'tsdbError'应使用'void'说明.	
	代码片段: 464: code = extractSttBlockInfo(plter, pSttBlkArray, pBlockLoadInfo, suid); 465: if (code != TSDB_CODE_SUCCESS) { 466: tsdbError("load stt block info failed, code:%s, %s", tstrerror(code), idStr); 467: return code; 468: }	
	污染路径: 函数'tsdbError'的定义点.	
899	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 50
	步骤描述: 不使用返回值的函数'blockDataAppendCollInfo'应使用'void'说明.	
	代码片段: 48: for (int32_t i = 0; i < 1; ++i) { 49: SColumnInfoData collInfo = createColumnInfoData(TSDB_DATA_TYPE_INT, sizeof(int32_t), 1);	

	50: blockDataAppendCollInfo(pBlock, &collInfo); 51: } 52:	
	污染路径: 函数'blockDataAppendCollInfo'的定义点.	
900	缺陷发生位置: /source/libs/index/test/index_executor_tests.cpp	行号: 58
	步骤描述: 不使用返回值的函数'taosMkDir'应使用'void'说明.	
	代码片段: 56: strcpy(tsLogDir, TD_TMP_DIR_PATH "sif"); 57: taosRemoveDir(tsLogDir); 58: taosMkDir(tsLogDir); 59: 60: if (taosInitLog(defaultLogFileNamePrefix, maxLogFileNum) < 0) {	
	污染路径: 函数'taosMkDir'的定义点.	
901	缺陷发生位置: /source/libs/scheduler/src/schUtil.c	行号: 105
	步骤描述: 不使用返回值的函数'SCH_TASK_ELOG'应使用'void'说明.	
	代码片段: 103: if (NULL == hb) { 104: SCH_UNLOCK(SCH_WRITE, &schMgmt.hbLock); 105: SCH_TASK_ELOG("nodeId %d fqdn %s port %d not in hb connections", epId->nodeId, epId->ep.fqdn, epId->ep.port); 106: return TSDB_CODE_SUCCESS; 107: }	
	污染路径: 函数'SCH_TASK_ELOG'的定义点.	
902	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 55
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 53: schMgmt.timer = taosTmrInit(0, 0, 0, "scheduler"); 54: if (NULL == schMgmt.timer) { 55: qError("init timer failed, error:%s", tsterror(terrno)); 56: SCH_ERR_RET(TSDB_CODE_OUT_OF_MEMORY);	

	57: }	
	污染路径: 函数'qError'的定义点.	
903	缺陷发生位置: /source/util/src/tsched.c	行号: 101
	步骤描述: 不使用返回值的函数'taosThreadAttrDestroy'应使用'void'说明.	
	代码片段: 99: taosThreadAttrSetDetachState(&attr, PTHREAD_CREATE_JOINABLE); 100: int32_t code = taosThreadCreate(pSched->qthread + i, &attr, taosProcessSchedQueue, (void *)pSched); 101: taosThreadAttrDestroy(&attr); 102: if (code != 0) { 103: uError("%s: failed to create rpc thread(%s)", label, strerror(errno));	
	污染路径: 函数'taosThreadAttrDestroy'的定义点.	
904	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 149
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 147: 148: code = 0; 149: dlInfo("succceed to read dnode file %s", file); 150: 151: _OVER:	
	污染路径: 函数'dlInfo'的定义点.	
905	缺陷发生位置: /source/libs/scheduler/src/schJob.c	行号: 163
	步骤描述: 不使用返回值的函数'SCH_RET'应使用'void'说明.	
	代码片段: 161: SCH_JOB_ELOG("invalid job status update, from %s to %s", jobTaskStatusStr(oriStatus), jobTaskStatusStr(newStatus)); 162: } 163: SCH_RET(code);	

	164: } 165:	
	污染路径: 函数'SCH_RET'的定义点.	
906	缺陷发生位置: /source/common/src/tmsg.c	行号: 312
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 310: if (tDecodeI64(pDecoder, &sDesc.tid) < 0) return -1; 311: if (tDecodeCStrTo(pDecoder, sDesc.status) < 0) return -1; 312: taosArrayPush(desc.subDesc, &sDesc); 313: } 314: }	
	污染路径: 函数'taosArrayPush'的定义点.	
907	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 93
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 91: 92: if (!xFree) { 93: tdbError("tdb/page-destroy: null xFree."); 94: return -1; 95: }	
	污染路径: 函数'tdbError'的定义点.	
908	缺陷发生位置: /source/util/src/tpagedbuf.c	行号: 323
	步骤描述: 不使用返回值的函数'tdListPrepend'应使用'void'说明.	
	代码片段: 321: 322: static void lruListPushFront(SList* pList, SPageInfo* pi) { 323: tdListPrepend(pList, &pi); 324: SListNode* front = tdListGetHead(pList); 325: pi->pn = front;	

	污染路径: 函数'tdListPrepend'的定义点.	
909	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 51
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 49: code = tdbTbcMoveToFirst(pReader->pCur); 50: if (code) { 51: taosMemoryFree(pReader); 52: goto _err; 53: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
910	缺陷发生位置: /source/util/src/tarray.c	行号: 281
	步骤描述: 不使用返回值的函数'taosArrayPop'应使用'void'说明.	
	代码片段: 279: 280: if (index == pArray->size - 1) { 281: taosArrayPop(pArray); 282: return; 283: }	
	污染路径: 函数'taosArrayPop'的定义点.	
911	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 53
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 51: } 52: 53: blockDataEnsureCapacity(pBlock, 1024); 54: pBlock->info.rows = 1024; 55:	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
912	缺陷发生位置:	行号:

	/source/libs/parser/test/parInitialDTest.cpp	280
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 278: SDropUserReq expect = {0}; 279: 280: auto setDropUserReq = [&](const char* pUser) { sprintf(expect.user, "%s", pUser); }; 281: 282: setCheckDdlFunc([&](const SQuery* pQuery, ParserStage stage) {	
	污染路径: 无	
913	缺陷发生位置: /source/util/src/tcompare.c	行号: 1205
	步骤描述: 不使用返回值的函数'regerror'应使用'void'说明.	
	代码片段: 1203: ret = regexec(®ex, pString, 1, pmatch, 0); 1204: if (ret != 0 && ret != REG_NOMATCH) { 1205: regerror(ret, ®ex, msgbuf, sizeof(msgbuf)); 1206: uDebug("Failed to match %s with pattern %s, reason %s", pString, pPattern, msgbuf) 1207: }	
	污染路径: 无	
914	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 156
	步骤描述: 不使用返回值的函数'tsdbDebug'应使用'void'说明.	
	代码片段: 154: pLReader->uid = 0; 155: tMergeTreeClose(&pLReader->mergeTree); 156: tsdbDebug("init fileset iterator, total files:%d %s", plter->numOfFiles, pReader->idStr); 157: return TSDB_CODE_SUCCESS; 158: }	
	污染路径: 函数'tsdbDebug'的定义点.	
915	缺陷发生位置:	行号:

	污染路径: 函数'mError'的定义点.	
918	缺陷发生位置: /source/dnode/vnode/src/tq/tqStreamTaskSnap.c	行号: 71
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 69: tqInfo("vgld:%d, vnode stream-task snapshot reader failed to iterate, reason: %s", TD_VID(pTq->pVnode), 70: tstreerror(code)); 71: taosMemoryFree(pReader); 72: goto _err; 73: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
919	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号: 84
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: 82: 83: code = tsdbTFileObjInit(tsdb, &file, &fset->farr[TSDB_FTYPE_HEAD]); 84: TSDB_CHECK_CODE(code, lino, _exit); 85: 86: // open fd	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
920	缺陷发生位置: /utils/tsim/src/simSystem.c	行号: 31
	步骤描述: 不使用返回值的函数'taosCreateLog'应使用'void'说明.	
	代码片段: 29: 30: int32_t simInitCfg() { 31: taosCreateLog("simlog", 1, configDir, NULL, NULL, NULL, NULL, 1); 32: taosInitCfg(configDir, NULL, NULL, NULL, NULL, 1); 33:	

	污染路径: 函数'taosCreateLog'的定义点.	
921	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 180
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: <pre> 178: *pMetaRsp = *tmp; 179: 180: tqDebug("tmqsnap task get meta"); 181: break; 182: }</pre>	
	污染路径: 函数'tqDebug'的定义点.	
922	缺陷发生位置: /source/client/src/clientMain.c	行号: 58
	步骤描述: 不使用返回值的函数'tscStopCrashReport'应使用'void'说明.	
	代码片段: <pre> 56: } 57: 58: tscStopCrashReport(); 59: 60: int32_t id = clientReqRefPool;</pre>	
	污染路径: 函数'tscStopCrashReport'的定义点.	
923	缺陷发生位置: /source/client/src/clientSml.c	行号: 276
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 274: meta->tagHash = taosHashInit(32, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY), false, HASH_NO_LOCK); 275: if (meta->tagHash == NULL) { 276: uError("SML:smlBuildSTableMeta failed to allocate memory"); 277: goto cleanup; 278: }</pre>	
	污染路径:	

	函数'uError'的定义点.	
924	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 445
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段: 443: end; 444: cJSON_Delete(json); 445: taosArrayDestroy(pTagVals); 446: if (string == NULL) { 447: string = taosStrdup(TSDB_DATA_NULL_STR_L);	
	污染路径: 函数'taosArrayDestroy'的定义点.	
925	缺陷发生位置: /contrib/test/traft/join_into_vgroup/node_leader10002.c	行号: 139
	步骤描述: 不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: 137: // for test: submit value every second 138: while (1) { 139: sleep(1); 140: submitValue(); 141: }	
	污染路径: 无	
926	缺陷发生位置: /source/libs/stream/src/streamBackendRocksdb.c	行号: 192
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 190: if (!taosHashGet(p1, name, len)) { 191: char* p = taosStrdup(name); 192: taosArrayPush(diff, &p); 193: } 194: plter = taosHashIterate(p2, plter);	
	污染路径: 函数'taosArrayPush'的定义点.	
927	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 73
	步骤描述:	

	不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 71: } 72: 73: tqDebug("s-task:%s build delete msg groupId:%s PRId64 ", name:%s, skey:%s PRId64 " ekey:%s PRId64, 74: pIdStr, groupId, name, skey, ekey); 75:	
	污染路径: 函数'tqDebug'的定义点.	
928	缺陷发生位置: /source/os/src/osRand.c	行号: 72
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 70: seed = (int)taosGetTimestampSec(); 71: } 72: taosCloseFile(&pFile); 73: } 74:	
	污染路径: 函数'taosCloseFile'的定义点.	
929	缺陷发生位置: /source/dnode/mnode/impl/src/mndQnode.c	行号: 216
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 214: 215: if (mndTransAppendRedoAction(pTrans, &action) != 0) { 216: taosMemoryFree(pReq); 217: return -1; 218: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
930	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 354
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 352: ASSERT_EQ(pRead->curVersion, ver + 1);	

	353: char newStr[100]; 354: sprintf(newStr, "%s-%d", ranStr, ver); 355: int len = strlen(newStr); 356: ASSERT_EQ(pRead->pHead->head.bodyLen, len);	
	污染路径: 无	
931	缺陷发生位置: /testes/script/api/stmtTest.c	行号: 26
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 24: void check_result(TAOS *taos, int id, int expected) { 25: char sql[256] = {0}; 26: sprintf(sql, "select * from t%d", id); 27: TAOS_RES *result; 28: result = taos_query(taos, sql);	
	污染路径: 无	
932	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 60
	步骤描述: 不使用返回值的函数'taosArrayPush'应使用'void'说明.	
	代码片段: 58: for (int i = 0; i < 10; i++) { 59: uint64_t val = i; 60: taosArrayPush(f, &val); 61: } 62: f = (SArray *)taosArrayGetP(src, 2);	
	污染路径: 函数'taosArrayPush'的定义点.	
933	缺陷发生位置: /source/libs/tdb/src/db/tdbTable.c	行号: 257
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 255: if (ret < 0) break; 256: } 257: tdbFree(pKey); 258: tdbFree(pValue); 259: tdbTbcClose(pCur);	

	污染路径: 函数'tdbFree'的定义点.	
934	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 406
	步骤描述: 不使用返回值的函数'idxInt2str'应使用'void'说明.	
	代码片段: 404: case TSDB_DATA_TYPE_USMALLINT: 405: *dst = taosMemoryCalloc(1, bufSize + 1); 406: idxInt2str(*(uint16_t*)src, *dst, -1); 407: tlen = strlen(*dst); 408: break;	
	污染路径: 函数'idxInt2str'的定义点.	
935	缺陷发生位置: /source/libs/executor/src/timesliceoperator.c	行号: 164
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 162: uint32_t winNum = (pSliceInfo->win.ekey - pSliceInfo->win.skey) / pSliceInfo->interval.interval; 163: uint32_t newRowsNum = pBlock->info.rows + TMIN(winNum / 4 + 1, 1048576); 164: blockDataEnsureCapacity(pBlock, newRowsNum); 165: 166: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
936	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 405
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 403: // do nothing 404: } else { 405: qError("invalid constant type for sma info"); 406: } 407:	
	污染路径:	

	函数'qError'的定义点.	
937	缺陷发生位置: /source/libs/function/src/tudf.c	行号: 66
	步骤描述: 不使用返回值的函数'udfSpawnUdfd'应使用'void'说明.	
	代码片段: 64: } else { 65: fnInfo("udfd process restart"); 66: udfSpawnUdfd(pData); 67: } 68: }	
	污染路径: 函数'udfSpawnUdfd'的定义点.	
938	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 1609
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: 1607: qDebugL("before split, JsonPlan: %s", pStr); 1608: } else { 1609: qDebugL("apply split %s rule, JsonPlan: %s", pRuleName, pStr); 1610: } 1611: taosMemoryFree(pStr);	
	污染路径: 函数'qDebugL'的定义点.	
939	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 369
	步骤描述: 不使用返回值的函数'tSerializeSCreateDropMQSNodeReq'应使用'void'说明.	
	代码片段: 367: int32_t contLen = tSerializeSCreateDropMQSNodeReq(NULL, 0, pDropReq); 368: void *pReq = taosMemoryMalloc(contLen); 369: tSerializeSCreateDropMQSNodeReq(pReq, contLen, pDropReq); 370: 371: STransAction action = {	
	污染路径: 函数'tSerializeSCreateDropMQSNodeReq'的定义点.	

940	缺陷发生位置: /tests/script/api/batchprepare.c	行号: 376
	步骤描述: 不使用返回值的函数'taosGetTimeOfDay'应使用'void'说明.	
	代码片段: <pre> 374: static int64_t taosGetTimestampUs() { 375: struct timeval systemTime; 376: taosGetTimeOfDay(&systemTime); 377: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 378: }</pre>	
	污染路径: 函数'taosGetTimeOfDay'的定义点.	
941	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 139
	步骤描述: 不使用返回值的函数'rpcSendRecv'应使用'void'说明.	
	代码片段: <pre> 137: 138: SRpcMsg rpcRsp = {0}; 139: rpcSendRecv(shandle, pEpSet, &rpcMsg, &rpcRsp); 140: 141: ASSERT_EQ(rpcRsp.code, 0);</pre>	
	污染路径: 函数'rpcSendRecv'的定义点.	
942	缺陷发生位置: /source/client/src/clientMain.c	行号: 43
	步骤描述: 不使用返回值的函数'sched_yield'应使用'void'说明.	
	代码片段: <pre> 41: if (i % 1000 == 0) { 42: tscInfo("haven't acquire lock after spin %d times.", i); 43: sched_yield(); 44: } 45: }</pre>	
	污染路径: 无	
943	缺陷发生位置: /source/dnode/mgmt/node_util/src/dmEps.c	行号: 95
	步骤描述:	

	不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 93: SJson *pjson = NULL; 94: char file[PATH_MAX] = {0}; 95: snprintf(file, sizeof(file), "%s%sdnode%sdnode.json", tsDataDir, TD_DIRSEP, TD_DIRSEP); 96: 97: pData->dnodeEps = taosArrayInit(1, sizeof(SDnodeEp));	
	污染路径: 无	
944	缺陷发生位置:	行号:
	/source/os/src/osSysinfo.c	134
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 132: tsProcId = (pid_t)syscall(SYS_gettid); 133: 134: snprintf(tsProcMemFile, sizeof(tsProcMemFile), "/proc/%d/status", tsProcId); 135: snprintf(tsProcCpuFile, sizeof(tsProcCpuFile), "/proc/%d/stat", tsProcId); 136: snprintf(tsProcIOFile, sizeof(tsProcIOFile), "/proc/%d/io", tsProcId);	
	污染路径: 无	
945	缺陷发生位置:	行号:
	/source/libs/executor/src/scanoperator.c	108
	步骤描述: 不使用返回值的函数'getNextTimeWindow'应使用'void'说明.	
	代码片段: 106: 107: while (1) { 108: getNextTimeWindow(pInterval, &w, order); 109: if (w.skey > pBlockInfo->window.ekey) { 110: break;	
	污染路径: 函数'getNextTimeWindow'的定义点.	
946	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/src/mndQuery.c	172
	步骤描述:	

	不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 170: int32_t mndInitQuery(SMnode *pMnode) { 171: if (qWorkerInit(NODE_TYPE_MNODE, MNODE_HANDLE, (void **)&pMnode->pQuery, &pMnode->msgCb) != 0) { 172: mError("failed to init qworker in mnode since %s", terrstr()); 173: return -1; 174: }</pre>	
	污染路径: 函数'mError'的定义点.	
947	缺陷发生位置: /source/libs/index/src/indexComm.c	行号: 396
	步骤描述: 不使用返回值的函数'idxInt2str'应使用'void'说明.	
	代码片段: <pre> 394: case TSDB_DATA_TYPE_TINYINT: 395: *dst = taosMemoryCalloc(1, bufSize + 1); 396: idxInt2str(*(int8_t*)src, *dst, 1); 397: tlen = strlen(*dst); 398: break;</pre>	
	污染路径: 函数'idxInt2str'的定义点.	
948	缺陷发生位置: /source/util/src/tqueue.c	行号: 80
	步骤描述: 不使用返回值的函数'taosThreadMutexLock'应使用'void'说明.	
	代码片段: <pre> 78: 79: bool empty = false; 80: taosThreadMutexLock(&queue->mutex); 81: if (queue->head == NULL && queue->tail == NULL && queue->numOfItems == 0 /*&& queue->memOfItems == 0*/) { 82: empty = true;</pre>	
	污染路径: 函数'taosThreadMutexLock'的定义点.	
949	缺陷发生位置: /source/util/src/tcache.c	行号: 121
	步骤描述:	

	不使用返回值的函数'taosThreadRwlockUnlock'应使用'void'说明.	
	代码片段: 119: static FORCE_INLINE void __trashcan_unlock(SCacheObj *pCacheObj) { 120: #if defined(LINUX) 121: taosThreadRwlockUnlock(&pCacheObj->lock); 122: #else 123: taosThreadMutexUnlock(&pCacheObj->lock);	
	污染路径: 函数'taosThreadRwlockUnlock'的定义点.	
950	缺陷发生位置: /source/dnode/vnode/src/meta/metaTtl.c	行号: 261
	步骤描述: 不使用返回值的函数'metaInfo'应使用'void'说明.	
	代码片段: 259: SMeta *meta = pMeta; 260: 261: metaInfo("ttlMgr convert start."); 262: 263: SConvertData cvData = {.pNewTtlIdx = pNewTtlIdx, .pMeta = meta};	
	污染路径: 函数'metaInfo'的定义点.	
951	缺陷发生位置: /source/libs/wal/src/walMeta.c	行号: 386
	步骤描述: 不使用返回值的函数'regcomp'应使用'void'说明.	
	代码片段: 384: regex_t idxRegPattern; 385: 386: regcomp(&logRegPattern, logPattern, REG_EXTENDED); 387: regcomp(&idxRegPattern, idxPattern, REG_EXTENDED); 388:	
	污染路径: 无	
952	缺陷发生位置: /include/common/tmsg.h	行号: 2835
	步骤描述: 不使用返回值的函数'taosArrayDestroy'应使用'void'说明.	
	代码片段:	

	2833: SQueryDesc* desc = (SQueryDesc*)pDesc; 2834: if (desc->subDesc) { 2835: taosArrayDestroy(desc->subDesc); 2836: desc->subDesc = NULL; 2837: }	
	污染路径: 函数'taosArrayDestroy'的定义点.	
953	缺陷发生位置: /source/dnode/vnode/src/tq/tqScan.c	行号: 196
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: 194: 195: if (pRsp->blockNum > 0) { 196: tqDebug("tmqsnap task exec exited, get data"); 197: qStreamExtractOffset(task, &pRsp->rspOffset); 198: break;	
	污染路径: 函数'tqDebug'的定义点.	
954	缺陷发生位置: /source/util/test/stringTest.cpp	行号: 118
	步骤描述: 不使用返回值的函数'strtolower'应使用'void'说明.	
	代码片段: 116: memset(t, 1, tListLen(t)); 117: const char* a5 = "1234\"AbcDEF\"456"; 118: strtolower(t, a5); 119: EXPECT_STREQ(t, a5); 120:	
	污染路径: 函数'strtolower'的定义点.	
955	缺陷发生位置: /source/libs/function/src/builtins.c	行号: 975
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 973: 974: if (!cJSON_IsNumber(start) !cJSON_IsNumber(count) !cJSON_IsBool(infinity)) { 975: snprintf(errMsg, msgLen, "%s", msg3);	

	976: cJSON_Delete(binDesc); 977: return false;	
	污染路径: 无	
956	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 177
	步骤描述: 不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 175: &cpuInfo->idle); 176: 177: taosCloseFile(&pFile); 178: #endif 179: return 0;	
	污染路径: 函数'taosCloseFile'的定义点.	
957	缺陷发生位置: /source/libs/scheduler/src/schUtil.c	行号: 140
	步骤描述: 不使用返回值的函数'SCH_ERR_RET'应使用'void'说明.	
	代码片段: 138: 139: qError("taosHashPut hb trans failed, nodeId:%d, fqdn: %s, port:%d", epId->nodeId, epId->ep.fqdn, epId->ep.port); 140: SCH_ERR_RET(code); 141: } 142:	
	污染路径: 函数'SCH_ERR_RET'的定义点.	
958	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 49
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 47: gNodeRefId = taosOpenRef(200, (RefFp)syncNodeClose); 48: if (gNodeRefId < 0) { 49: sError("failed to init node ref"); 50: syncCleanUp(); 51: return -1;	

	污染路径: 函数'sError'的定义点.	
959	缺陷发生位置: /source/dnode/vnode/src/sma/smaEnv.c	行号: 59
	步骤描述: 不使用返回值的函数'smaError'应使用'void'说明.	
	代码片段: <pre> 57: if (smaMgmt.rsetId < 0) { 58: atomic_store_8(&smaMgmt.inited, 0); 59: smaError("failed to init sma rset since %s", terrstr()); 60: return TSDB_CODE_FAILED; 61: }</pre>	
	污染路径: 函数'smaError'的定义点.	
960	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 624
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: <pre> 622: } 623: 624: mInfo("mnode:%d, start to create", createReq.dnodeId); 625: if (mndCheckOperPrivilege(pMnode, pReq->info.conn.user, MND_OPER_CREATE_MNODE) != 0) { 626: goto _OVER;</pre>	
	污染路径: 函数'mInfo'的定义点.	
961	缺陷发生位置: /utils/tsim/src/simExe.c	行号: 268
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 266: } 267: } else { 268: tstrncpy(t3, t1, sizeof(t3)); 269: } 270:</pre>	
	污染路径: 函数'tstrncpy'的定义点.	
962	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tq/tqSnapshot.c	149
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 147: if (tdbBegin(pTq->pMetaDB, &pWriter->txn, tdbDefaultMalloc, tdbDefaultFree, NULL, 0) < 0) { 148: code = -1; 149: taosMemoryFree(pWriter); 150: goto _err; 151: }</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
963	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 75
	步骤描述: 不使用返回值的函数'FREE_FUNC_TAOS_FORBID'应使用'void'说明.	
	代码片段: <pre> 73: ret: 74: return n; 75: free(tmp); 76: } 77:</pre>	
	污染路径: expanded from macro 'free' 函数'FREE_FUNC_TAOS_FORBID'的定义点. expanded from macro 'free'	
964	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.c	行号: 87
	步骤描述: 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段: <pre> 85: 86: code = tsdbDataFileFlush(writer->dataWriter); 87: TSDB_CHECK_CODE(code, lino, _exit); 88: 89: for (int32_t i = numRows; i < writer->blockData[pidx].nRow; i++) {</pre>	
	污染路径: 函数'TSDB_CHECK_CODE'的定义点.	
965	缺陷发生位置:	行号:

	/source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	80
	步骤描述: 不使用返回值的函数'prctl'应使用'void'说明.	
	代码片段: 78: SDnodeMgmt *pMgmt = param; 79: int64_t lastTime = taosGetTimestampMs(); 80: setThreadName("dnode-crashReport"); 81: char filepath[PATH_MAX] = {0}; 82: snprintf(filepath, sizeof(filepath), "%s%s.taosdCrashLog", tsLogDir, TD_DIRSEP);	
	污染路径: expanded from macro 'setThreadName'	
966	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smlnt.c	行号: 54
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 52: pMgmt->pSnode = sndOpen(pMgmt->path, &option); 53: if (pMgmt->pSnode == NULL) { 54: dError("failed to open snode since %s", terrstr()); 55: smClose(pMgmt); 56: return -1;	
	污染路径: 函数'dError'的定义点.	
967	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 113
	步骤描述: 不使用返回值的函数'dmReportStartup'应使用'void'说明.	
	代码片段: 111: 112: dInfo("The daemon initialized successfully"); 113: dmReportStartup("The daemon", "initialized successfully"); 114: return 0; 115: }	
	污染路径: 函数'dmReportStartup'的定义点.	
968	缺陷发生位置: /source/os/src/osFile.c	行号: 138
	步骤描述:	

	不使用返回值的函数'taosCloseFile'应使用'void'说明.	
	代码片段: 136: 137: taosCloseFile(&pFileFrom); 138: taosCloseFile(&pFileTo); 139: 140: if (code != 0) {	
	污染路径: 函数'taosCloseFile'的定义点.	
969	缺陷发生位置: /source/libs/parser/src/parInsertUtil.c	行号: 96
	步骤描述: 不使用返回值的函数'strdequote'应使用'void'说明.	
	代码片段: 94: char tdbname[TSDB_TABLE_FNAME_LEN] = {0}; 95: strncpy(tdbname, p + 1, tbLen); 96: /*tbLen = */ strdequote(tdbname); 97: 98: code = tNameFromString(pName, tdbname, T_NAME_TABLE);	
	污染路径: 函数'strdequote'的定义点.	
970	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 61
	步骤描述: 不使用返回值的函数'sDebug'应使用'void'说明.	
	代码片段: 59: } 60: 61: sDebug("sync rsetId:%d is open", gNodeRefId); 62: return 0; 63: }	
	污染路径: 函数'sDebug'的定义点.	
971	缺陷发生位置: /source/client/src/clientMain.c	行号: 42
	步骤描述: 不使用返回值的函数'tsclInfo'应使用'void'说明.	
	代码片段: 40: for (int i = 1; atomic_val_compare_exchange_32(&lock,	

	<pre> 0, 1) != 0; ++i) { 41: if (i % 1000 == 0) { 42: tscInfo("haven't acquire lock after spin %d times.", i); 43: sched_yield(); 44: } </pre>	
	污染路径: 函数'tscInfo'的定义点.	
972	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 109
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 107: pStore->pHash = taosHashInit(64, MurmurHash3_32, true, HASH_ENTRY_LOCK); 108: if (pStore->pHash == NULL) { 109: taosMemoryFree(pStore); 110: return NULL; 111: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
973	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 331
	步骤描述: 不使用返回值的函数'ctgInfo'应使用'void'说明.	
	代码片段: <pre> 329: taosMemoryFree(pCtg); 330: 331: ctgInfo("handle freed, clusterId:0x%" PRIx64, clusterId); 332: } 333: </pre>	
	污染路径: 函数'ctgInfo'的定义点.	
974	缺陷发生位置: /source/libs/transport/test/transUT.cpp	行号: 60
	步骤描述: 不使用返回值的函数'sem_init'应使用'void'说明.	
	代码片段: <pre> 58: taosVersionStrToInt(version, &(rpcInit_.compatibilityVer)); 59: this->transCli = rpcOpen(&rpcInit_); </pre>	

	60: tsem_init(&this->sem, 0, 0); 61: } 62: void SetResp(SRpcMsg *pMsg) {	
	污染路径: expanded from macro 'tsem_init'	
975	缺陷发生位置: /source/util/src/tjson.c	行号: 46
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 44: int32_t tjsonAddIntegerToObject(SJson* pjson, const char* pName, const uint64_t number) { 45: char tmp[40] = {0}; 46: snprintf(tmp, sizeof(tmp), "%" PRIu64, number); 47: return tjsonAddStringToObject(pjson, pName, tmp); 48: }	
	污染路径: 无	
976	缺陷发生位置: /utls/test/c/tmq_taosx_ci.c	行号: 67
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 65: } 66: } 67: tmq_free_json_meta(result); 68: } 69:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
977	缺陷发生位置: /source/client/src/clientTmq.c	行号: 520
	步骤描述: 不使用返回值的函数'atomic_add_fetch_32'应使用'void'说明.	
	代码片段: 518: 519: atomic_add_fetch_32(&pParamSet->waitingRspNum, 1); 520: atomic_add_fetch_32(&pParamSet->totalRspNum, 1); 521:	

	522: SEp* pEp = GET_ACTIVE_EP(epSet);	
	污染路径: 函数'atomic_add_fetch_32'的定义点.	
978	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 267
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 265: if (*ppEntry) { // update 266: if (pInfo->suid != (*ppEntry)->info.suid) { 267: metaError("meta/cache: suid should be same as the one in cache."); 268: return TSDB_CODE_FAILED; 269: }	
	污染路径: 函数'metaError'的定义点.	
979	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 55
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 53: 54: if (dmReadEps(pData) != 0) { 55: dError("failed to read file since %s", terrstr()); 56: return -1; 57: }	
	污染路径: 函数'dError'的定义点.	
980	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 328
	步骤描述: 不使用返回值的函数'mFatal'应使用'void'说明.	
	代码片段: 326: } else { 327: terrno = TSDB_CODE_APP_ERROR; 328: mFatal("db:%s, failed to acquire db since %s", db, terrstr()); 329: } 330: }	

	污染路径: 函数'mFatal'的定义点.	
981	缺陷发生位置: /source/util/src/tthread.c	行号: 25
	步骤描述: 不使用返回值的函数'taosThreadAttrDestroy'应使用'void'说明.	
	代码片段: <pre> 23: taosThreadAttrSetDetachState(&thattr, PTHREAD_CREATE_JOINABLE); 24: int32_t ret = taosThreadCreate(pthread, &thattr, _start_routine, param); 25: taosThreadAttrDestroy(&thattr); 26: 27: if (ret != 0) { </pre>	
	污染路径: 函数'taosThreadAttrDestroy'的定义点.	
982	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 53
	步骤描述: 不使用返回值的函数'tqDebug'应使用'void'说明.	
	代码片段: <pre> 51: SColumnInfoData* pTbNameCol = taosArrayGet(pDataBlock->pDataBlock, TABLE_NAME_COLUMN_INDEX); 52: 53: tqDebug("s-task:%s build %d rows delete msg for table: %s", pIdStr, totalRows, stbFullName); 54: 55: for (int32_t row = 0; row < totalRows; row++) { </pre>	
	污染路径: 函数'tqDebug'的定义点.	
983	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 163
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: <pre> 161: } 162: 163: taosMsleep(100); 164: } 165: } </pre>	

	污染路径: 函数'taosMsleep'的定义点.	
984	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smHandle.c	行号: 36
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 34: bool deployed = true; 35: if (dmWriteFile(pInput->path, pInput->name, deployed) != 0) { 36: dError("failed to write snode file since %s", terrstr()); 37: return -1; 38: }	
	污染路径: 函数'dError'的定义点.	
985	缺陷发生位置: /source/common/src/tglobal.c	行号: 820
	步骤描述: 不使用返回值的函数'taosExpandDir'应使用'void'说明.	
	代码片段: 818: SConfigItem *pItem = cfgGetItem(pCfg, "logDir"); 819: strncpy(tsLogDir, cfgGetItem(pCfg, "logDir")->str, PATH_MAX); 820: taosExpandDir(tsLogDir, tsLogDir, PATH_MAX); 821: tsLogSpace.reserved = (int64_t) (((double)cfgGetItem(pCfg, "minimalLogDirGB")->fval) * 1024 * 1024 * 1024); 822: tsNumOfLogLines = cfgGetItem(pCfg, "numOfLogLines")->i32;	
	污染路径: 函数'taosExpandDir'的定义点.	
986	缺陷发生位置: /source/util/src/tskiplist.c	行号: 109
	步骤描述: 不使用返回值的函数'taosThreadRwlockDestroy'应使用'void'说明.	
	代码片段: 107: tSkipListUnlock(pSkipList); 108: if (pSkipList->lock != NULL) { 109: taosThreadRwlockDestroy(pSkipList->lock); 110: taosMemoryFreeClear(pSkipList->lock);	

	111: }	
	污染路径: 函数'taosThreadRwlockDestroy'的定义点.	
987	缺陷发生位置: /source/dnode/mnode/impl/src/mndAcct.c	行号: 244
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 242: 243: terrno = TSDB_CODE_OPS_NOT_SUPPORT; 244: mError("failed to process create acct request since %s", terrstr()); 245: return -1; 246: }	
	污染路径: 函数'mError'的定义点.	
988	缺陷发生位置: /source/os/test/osTests.cpp	行号: 113
	步骤描述: 不使用返回值的函数'taosThreadCreate'应使用'void'说明.	
	代码片段: 111: 112: TdThread thread1, thread2; 113: taosThreadCreate(&(thread1), &thattr, fileOperateOnBusyThread, (void *)fname); 114: taosThreadAttrDestroy(&thattr); 115:	
	污染路径: 函数'taosThreadCreate'的定义点.	
989	缺陷发生位置: /source/libs/stream/src/streamState.c	行号: 94
	步骤描述: 不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 92: 93: SStreamState* streamStateOpen(char* path, void* pTask, bool specPath, int32_t szPage, int32_t pages) { 94: qDebug("open stream state, %s", path); 95: SStreamState* pState = taosMemoryCalloc(1, sizeof(SStreamState));	

	96: if (pState == NULL) {	
	污染路径: 函数'qDebug'的定义点.	
990	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 50
	步骤描述: 不使用返回值的函数'syncCleanUp'应使用'void'说明.	
	代码片段: 48: if (gNodeRefId < 0) { 49: sError("failed to init node ref"); 50: syncCleanUp(); 51: return -1; 52: }	
	污染路径: 函数'syncCleanUp'的定义点.	
991	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 115
	步骤描述: 不使用返回值的函数'CTG_ERR_RET'应使用'void'说明.	
	代码片段: 113: task.taskCtx = taosMemoryCalloc(1, sizeof(SCTgDbCfgCtx)); 114: if (NULL == task.taskCtx) { 115: CTG_ERR_RET(TSDB_CODE_OUT_OF_MEMORY); 116: } 117:	
	污染路径: 函数'CTG_ERR_RET'的定义点.	
992	缺陷发生位置: /docs/examples/c/multi_bind_example.c	行号: 143
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 141: insertData(taos); 142: taos_close(taos); 143: taos_cleanup(); 144: } 145:	
	污染路径:	

	函数'taos_cleanup'的定义点.	
993	缺陷发生位置: /source/common/src/tmsg.c	行号: 333
	步骤描述: 不使用返回值的函数'taosHashPut'应使用'void'说明.	
	代码片段: 331: SKv kv = {0}; 332: if (tDecodeSKv(pDecoder, &kv) < 0) return -1; 333: taosHashPut(pReq->info, &kv.key, sizeof(kv.key), &kv, sizeof(kv)); 334: } 335:	
	污染路径: 函数'taosHashPut'的定义点.	
994	缺陷发生位置: /source/util/src/tlruccache.c	行号: 322
	步骤描述: 不使用返回值的函数'taosLRUEntryTableRemove'应使用'void'说明.	
	代码片段: 320: 321: taosLRUCacheShardLRURemove(shard, old); 322: taosLRUEntryTableRemove(&shard->table, old->keyData, old->keyLength, old->hash); 323: 324: TAOS_LRU_ENTRY_SET_IN_CACHE(old, false);	
	污染路径: 函数'taosLRUEntryTableRemove'的定义点.	
995	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmHandle.c	行号: 60
	步骤描述: 不使用返回值的函数'dGError'应使用'void'说明.	
	代码片段: 58: if (option.selfIndex == -1) { 59: terrno = TSDB_CODE_INVALID_OPTION; 60: dGError("failed to create mnode since %s, selfIndex is -1", terrstr()); 61: return -1; 62: }	
	污染路径: 函数'dGError'的定义点.	

996	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 155
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 153: 154: for (int iData = 1; iData <= nData; iData++) { 155: sprintf(key, "key%d", iData); 156: sprintf(val, "value%d", iData); 157: ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), txn);	
	污染路径: 无	
997	缺陷发生位置: /source/libs/transport/src/tmsgcb.c	行号: 72
	步骤描述: 不使用返回值的函数'tmsgUpdateDnodeInfo'应使用'void'说明.	
	代码片段: 70: void tmsgUpdateDnodeEpSet(SEpSet* epset) { 71: for (int32_t i = 0; i < epset->numOfEps; ++i) { 72: tmsgUpdateDnodeInfo(NULL, NULL, epset->eps[i].fqdn, &epset->eps[i].port); 73: } 74: }	
	污染路径: 函数'tmsgUpdateDnodeInfo'的定义点.	
998	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 261
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 259: mError("sub:%s mq re-balance removedNum:%d not matched with actual:%d", pSubKey, numOfRemoved, actualRemoved); 260: } else { 261: mInfo("sub:%s removed %d consumers", pSubKey, numOfRemoved); 262: } 263: }	
	污染路径:	

	函数'mInfo'的定义点.	
999	缺陷发生位置: /tools/shell/src/shellNettest.c	行号: 106
	步骤描述: 不使用返回值的函数'rpcSendResponse'应使用'void'说明.	
	代码片段: 104: rsp.contLen = pRpc->contLen; 105: } 106: rpcSendResponse(&rsp); 107: } 108:	
	污染路径: 函数'rpcSendResponse'的定义点.	
1000	缺陷发生位置: /source/common/src/tdatablock.c	行号: 629
	步骤描述: 不使用返回值的函数'blockDataEnsureCapacity'应使用'void'说明.	
	代码片段: 627: } 628: 629: blockDataEnsureCapacity(pDst, rowCount); 630: 631: for (int32_t i = 0; i < numOfCols; ++i) {	
	污染路径: 函数'blockDataEnsureCapacity'的定义点.	
1001	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 509
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 507: 508: for (int iData = 0; iData < nData; ++iData) { 509: sprintf(key, "key%03d", iData); 510: sprintf(val, "value%03d", iData); 511:	
	污染路径: 无	
1002	缺陷发生位置: /source/libs/command/src/explain.c	行号: 91
	步骤描述:	

	不使用返回值的函数'QRY_ERR_JRET'应使用'void'说明.
	代码片段: 89: if (NULL == ctx) { 90: qError("calloc SExplainCtx failed"); 91: QRY_ERR_JRET(TSDB_CODE_OUT_OF_MEMORY); 92: } 93:
	污染路径: 函数'QRY_ERR_JRET'的定义点.